

Introduction to Computer Architecture

Lecture 1

Technology Trends & Design Challenges

Prof. Simon W. Moore



Overview of the course

- 10 lectures on computer architecture with links to ECAD+Arch labs
 1. Technology Trends and Design Challenges
 2. Digital systems design (with a link to ECAD)
 3. Eight great ideas in computer architecture
 4. RISC (Reduced Instruction Set Computers)
 5. Executable and synthesisable models
 6. Pipelining
 7. More pipelining
 8. Memory hierarchy
 9. OS support
 10. Alternatives to RISC
- 6 lectures on system-on-chip (SoC) design
(lectured last year by Dr Timothy Jones)
 11. Overview of SoC design
 12. Multicore processors 1
 13. Multicore processors 2
 14. Graphics Processing Units 1
 15. Graphics Processing Units 2
 16. Future Directions

Recommended Material

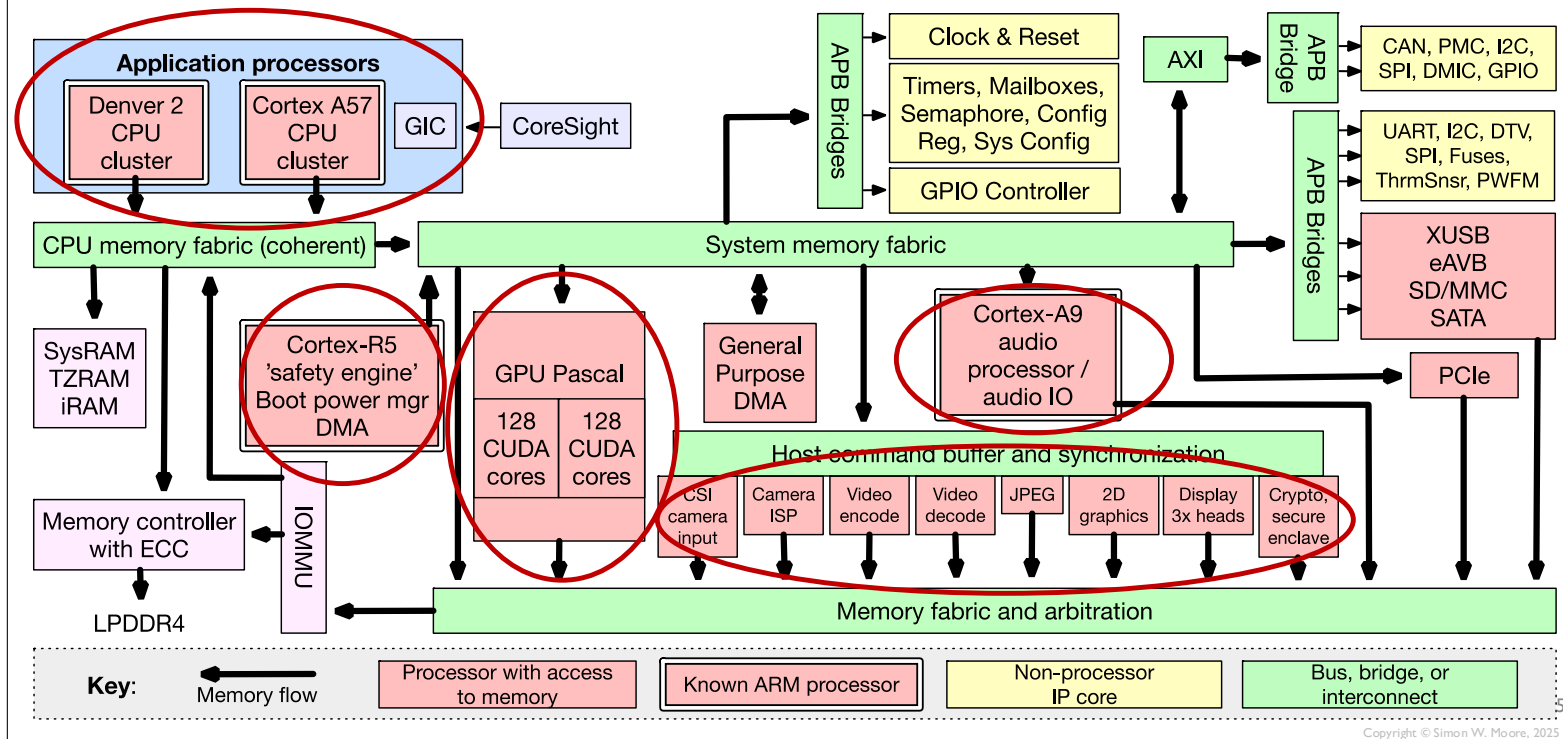
- Recommended textbook
 - Patterson, D.A. & Hennessy, J.L. (2017). *Computer organization and design: The hardware/software interface RISC-V edition*. Morgan Kaufmann. ISBN 978-0-12-812275-4.
- There are many other computer architecture texts:
 - by Patterson and Hennessy of different ages and levels of detail
 - by many others
- And a great deal of material on the web:
 - E.g. from Nand to Tetris: <https://www.nand2tetris.org/>
- Note: previous versions of this course were called “Computer Design” so past exam. papers, etc., are under this name

Computers today: many form factors, many solutions



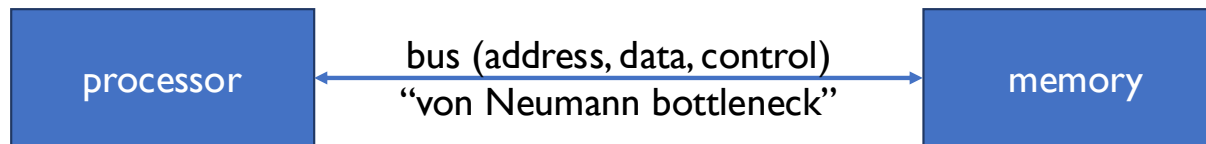
Systems-on-chip have many processors

more in later lectures



The “von Neumann” architecture

- In 1945 John von Neumann wrote the “First Draft Report on EDVAC” controversially incorporating ideas from others without acknowledging them (Eckert and Mauchly)
- Outlines the idea of a linear memory storing the program and data



- ...and control-flow processors that execute instructions sequentially
- This basic model pervades modern processor design despite its limitations

Memory hierarchy

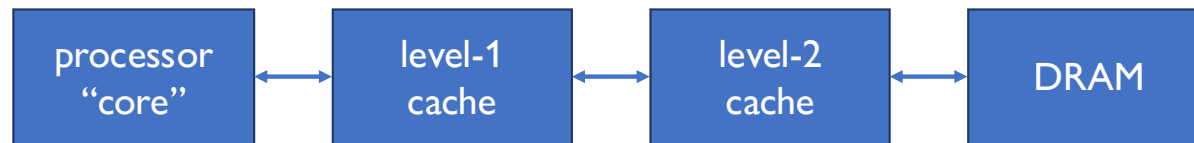
- “Ideally one would desire an indefinitely large memory capacity such that any particular. . . word would be immediately available. . . We are. . . forced to recognize the possibility of constructing a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible.”

A. W. Burks, H. H. Goldstine and J. von Neumann

Preliminary Discussion of the Logical Design of an Electronic Computing Instrument (1946)

more in later
lectures

- Simplified memory hierarchy of a modern machine:



Implementation technology scaling: Moore's law

- In 1965 Gordon Moore predicted that transistor density on silicon would double over 18 to 24 months!

10000 nm – 1971
6000 nm – 1974
3000 nm – 1977
1500 nm – 1981
1000 nm – 1984
800 nm – 1987
600 nm – 1990
350 nm – 1994
250 nm – 1996
180 nm – 1999
130 nm – 2001
90 nm – 2003
65 nm – 2005
45 nm – 2007
32 nm – 2009
22 nm – 2012
14 nm – 2014
10 nm – 2016
7 nm – 2018
5 nm – 2019
3 nm – ~2023

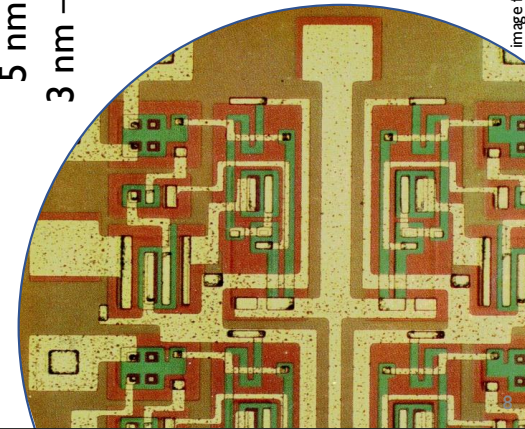


Image from Wikipedia

Moore's law in perspective



In 1978, a commercial flight between New York and Paris cost around \$900 and took seven hours. If the principles of Moore's Law had been applied to the airline industry the way they have to the semiconductor industry since 1978, that flight would now cost about a penny and take less than one second.

Copyright © 2005 Intel Corporation. All rights reserved.



Moore's law in perspective

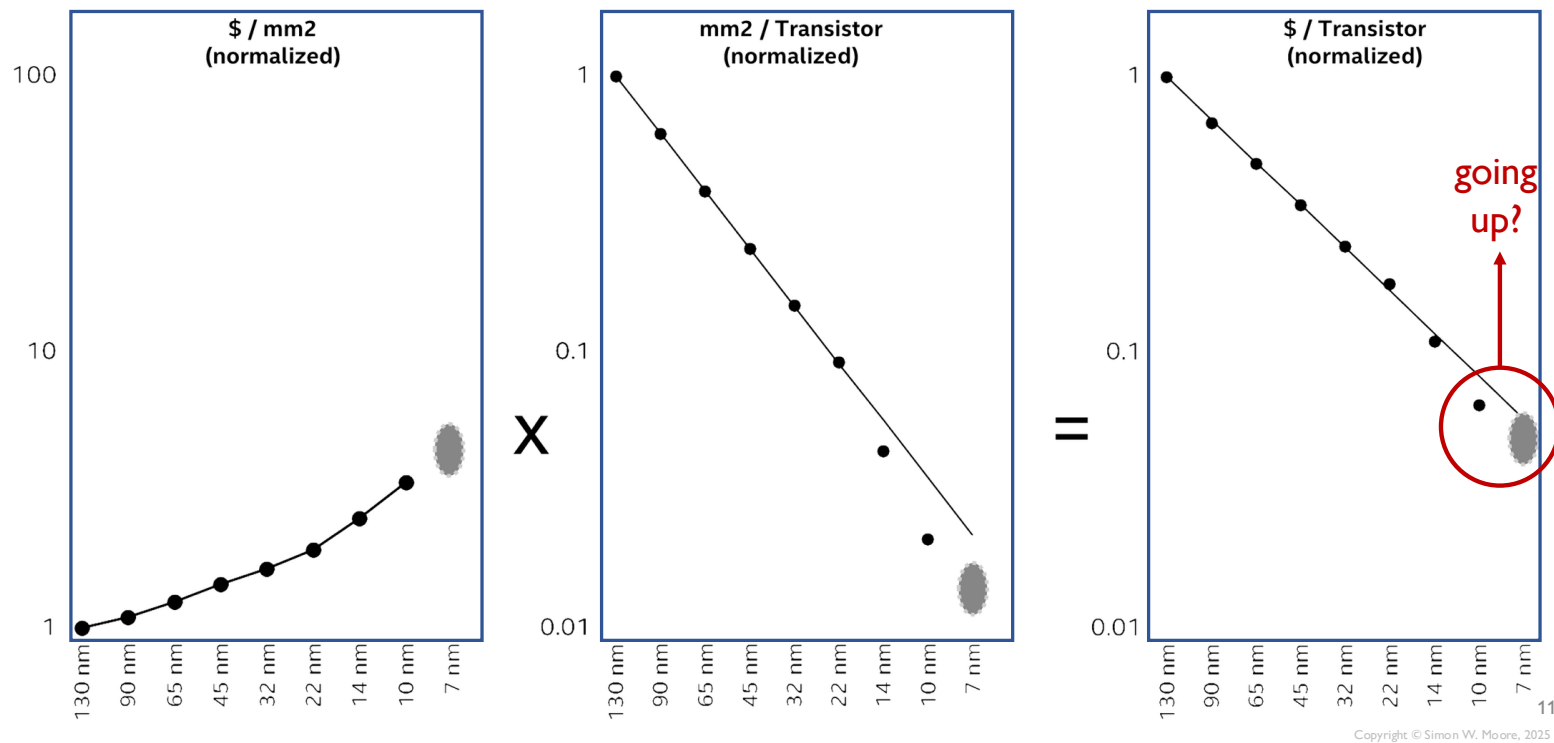
...to such we
als connected to a
or automobiles, a
ipment. The elec
be feasible today
in the pro

The price per transistor

on a chip has dropped dramatically since Intel was founded in 1968. Some people estimate that the price of a transistor is now about the same as that of one printed newspaper character.

Copyright © 2005 Intel Corporation. All rights reserved.

Moore's law for cost?

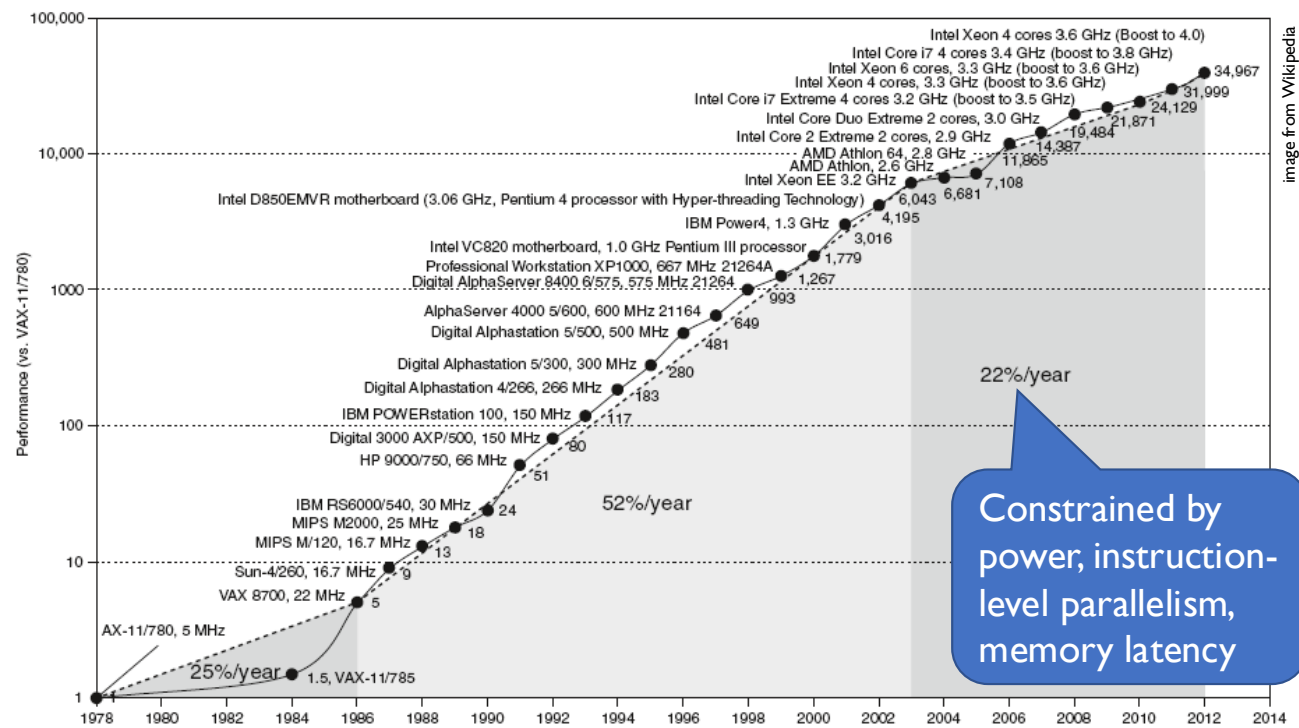




Manufacturing costs

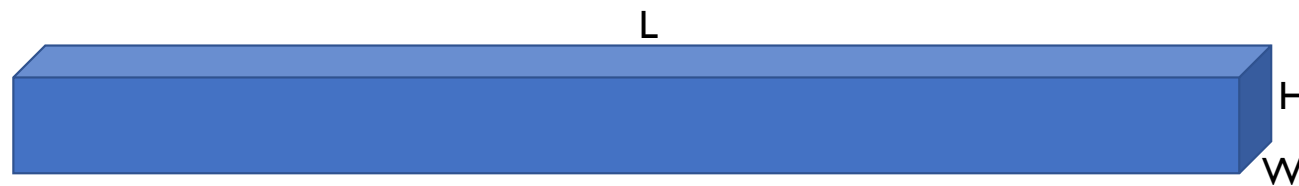
- Yield is critical, i.e. proportion of fully functional chips
 - Yield numbers are a closely guarded secret; reveals profitability of a silicon foundry
 - Many tricks used to increase yield including design time (e.g. redundancy in memory arrays)
 - Number of defects $\propto$ chip area, so yield $\propto$ 1/chip area
- Nonrecurrent (one-off) manufacturing costs for a chip have gone up:
 - e.g. mask costs for a 10nm chip are around \$10m per chip design
 - must be offset against volume of sales
- Cost of semiconductor fabrication foundries has grown exponentially:
 - TSMC's Fab15 cost \$9b in 2010
 - TSMC estimate that a 3nm fab. might cost \$20b
- Indications are that the cost/transistor is starting to rise for the latest technologies
 - Semiconductor industry is bifurcating: cheap chips and high-performance chips

Moore's law for performance?





Moore's law and wire scaling



- Resistance (R), Length (L), Width (W), Height (H):
 $R \propto L / (W \times H)$
- Capacitance (C) to substrate:
 $C \propto W \times L$
- Simple wire driving model: charge time (T):
 $T \propto R \times C \propto L^2 / H$
- So wires don't get any faster for the same chip area
- Halving L makes the wire go 4x faster: add buffering/repeaters helps



Dennard scaling

- Dennard observed in 1974 that:
 - Transistor dimensions are scaled by 30% (0.7x) every technology generation, thus reducing their area by 50% (i.e. Moore's law)
 - This reduces the delay by 30% (0.7x) and therefore increases operating frequency by about 40% (1.4x)
 - To keep the electric field constant, voltage is reduced by 30%, reducing energy by 65% and power (at 1.4x frequency) by 50%
 - Therefore, in every technology generation the transistor density doubles, the circuit becomes 40% faster, and power consumption (with twice the number of transistors) stays the same
- In around 2006, this breaks down
 - Cannot further decrease supply voltage since transistors leak too much
 - i.e. static power dramatically increases compared to dynamic power
 - Wires don't scale for the same chip area
 - Semiconductor technology no longer delivers on Moore's law for performance, so much more emphasis on computer architecture improvements



Power wall and the age of “dark silicon”

- Moore's law is still (just about) delivering on transistor density improvements
 - Needs new transistor structures (beyond these lectures) – see: <https://semiengineering.com/new-transistor-structures-at-3nm-2nm/>
- BUT we have to constrain the number of transistors that are switching (i.e. the dynamic power)
 - Move toward more accelerators, i.e. domain optimised compute which are only turned on when needed
 - Move toward a range of processor cores: small efficient cores when lightly loaded and larger cores for heavy workloads, particularly in phones/tablets
- Race-to-dark (RTD)
 - Turning a core off saves static as well as dynamic power
 - So racing to turn-off (“dark”) is another goal



Voltage and frequency trade-offs

- Active power = $C.V^2.f.A$
 - C = capacitance, proportional to silicon area and thus feature size
 - V = voltage
 - f = clock frequency
 - A = activity factor, i.e. proportion of transistors switching
- Performance
 - Over a limited range of V , $f \propto V$
 - Also, CMOS circuits also get slower as they get hotter
- Dynamic voltage frequency scaling (DVFS)
 - Adjust voltage with frequency to trade off performance and power
 - DVFS marketed as “turbo boost”, “PowerNow!”, etc.
- Overclocking
 - Aggressive cooling can allow the frequency (f) to be increased
 - Forcing the frequency beyond manufacturer recommendations reduces reliability



Fallacy: low power at idle

- Intel i7 power benchmark
 - At 100% load: 258W
 - At 50% load: 170W (66%)
 - At 10% load: 121W (47%)
 - Interesting to compare with a max 10W Raspberry Pi 4 with ARM cores that is > 10% performance of an Intel i7 but substantially lower power
- Google data centre
 - Mostly operates at 10% – 50% load
 - At 100% load less than 1% of the time
- Consider designing processors to make power proportional to load
 - Static power makes this challenging
 - Transistors with lower static power are typically slower



Moore's law for storage?

- Moore's law density improvements also apply to DRAM (volatile) and Flash (non-volatile) memories
- Kryder's law for magnetic hard disks (Kryder, CTO Seagate)
 - hard disk storage densities have risen faster than Moore's law
 - 155 Kbit/mm² in 1990
 - 170 Mbit/mm² in 2005
- Solid state storage taking over for performance, but more expensive
 - SATA (legacy) or NVMe (modern, PCIe-based) storage massively increased bandwidth and reduces latency
 - Spinning magnetic disk is still cost effective for long term storage

Moore's law for networking?

- Latency limited by the speed of light
- Bandwidth is helped by frequency improvements in silicon, but there are other physical factors

Ethernet standard on twisted pair	Bandwidth
10BASE-T from 1990	10Mb/s
100BASE-T from 1995	100Mb/s
1000BASE-T from 1999	1000Mb/s = 1Gb/s
10GBASE-T from 2006	10Gb/s
40GBASE-T from 2016	40Gb/s

- Processor speed is no longer keeping up, making it difficult for a single core to fully use future network technologies



Design and verification gaps

- How does design and verification scale with Moore's law?
 - Exponentially more difficult?
 - Exponentially bigger design teams?
- Some of the solutions
 - Hierarchical decomposition
 - Replication of components (e.g. processor cores, memories, etc.)
 - Libraries of predesigned components
 - Well specified abstract interfaces (e.g. standardised memory bus/switched interconnect)
 - Use of higher-level hardware description languages, e.g. like SystemVerilog used in the ECAD+Arch labs and widely used by industry, or Bluespec (BSV) used in research in the department and elsewhere

Conclusions

- We're still using "von Neumann" style processors
 - Surely we can do better!
 - Really not an efficient general-purpose processor: "Turing tax"
 - Even algorithms mapped to FPGAs are much more efficient even though FPGAs are typically 15x slower per gate
- The golden age of silicon technology improvements is over
 - Ever more need for better computer architectures
- Computer systems are now power limited
 - Power dominates everything, even the size of your mobile phone!
 - Today computers produce ~3.5% of global emissions, and could rise to 20%
 - not sustainable!
 - flip side: how can we use computers to minimise global power consumption?

Introduction to Computer Architecture

Lecture 2

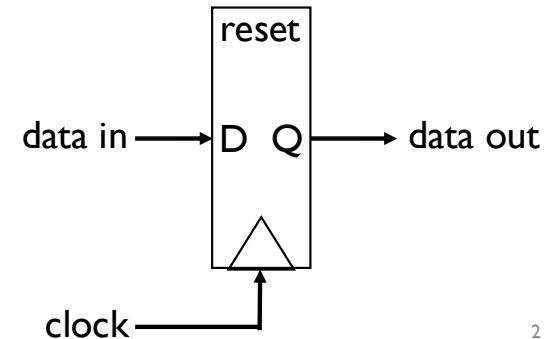
Digital Systems Design

Prof. Simon W. Moore



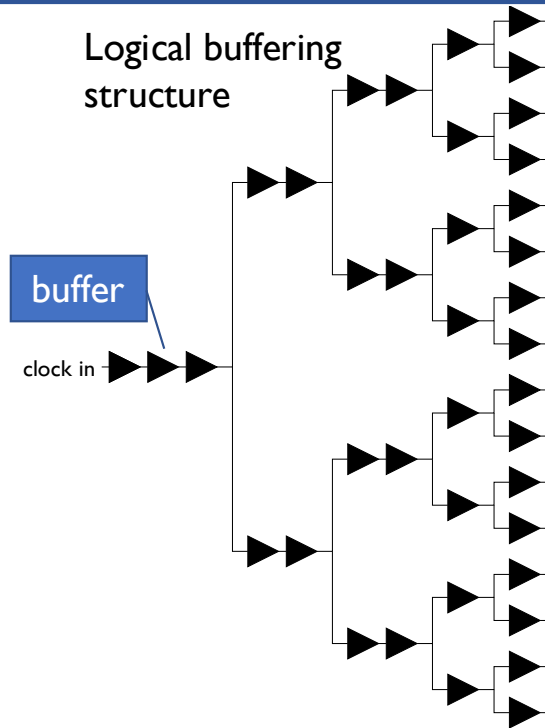
Digital circuit design

- Digital logic levels
 - e.g. 0=false is represented by a low voltage and 1=true as a high voltage
 - so the logic level is discrete
- Clocked circuits
 - time is also made discrete
 - typically D flip-flops used to store state and control the flow of information
- Setup and hold times
 - setup = time D input stable before clock edge
 - setup time is usually positive
 - hold = time D input stable after clock edge
 - hold time is usually negative
 - necessary to tolerate clock jitter



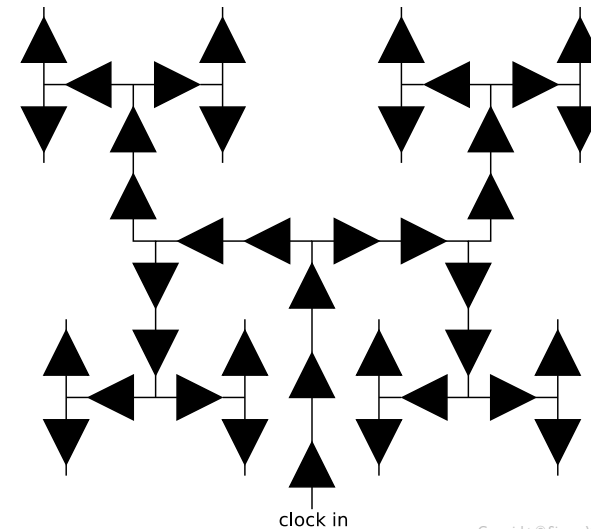
Clock trees

Logical buffering structure



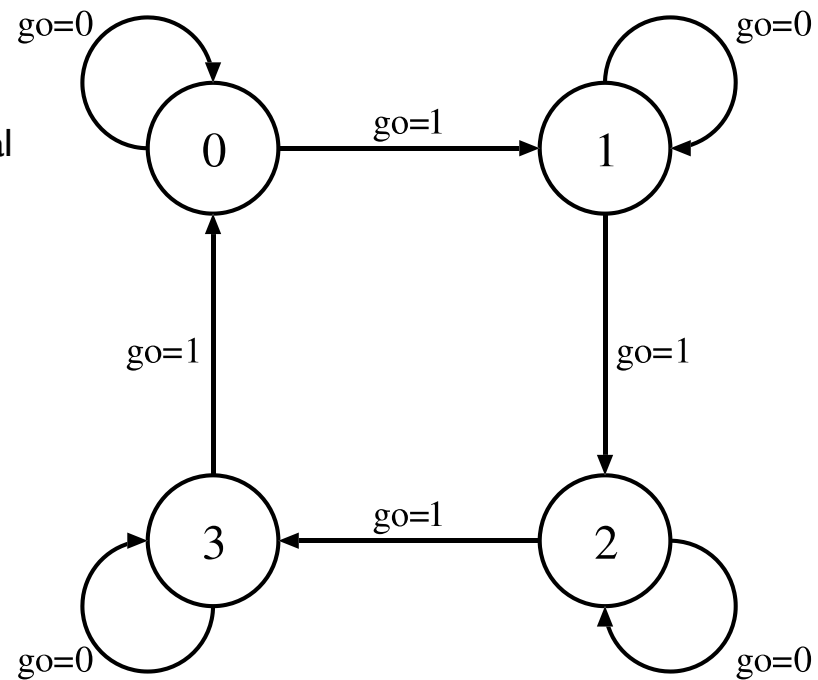
H-tree physical structure (a space filling curve)

Aims to propagate clock to every part of the chip so that the clock edge appears at the same time everywhere (i.e. low clock skew)



State transition graph

- Example of a 2-bit counter with enable (go) input
 - Thinking back to Part 1a Digital Electronics...



Revising state machines: state transition table

input go	current state (n1, n0)	next state (n1', n0')
0	0 0	0 0
0	0 1	0 1
0	1 0	1 0
0	1 1	1 1
1	0 0	0 1
1	0 1	1 0
1	1 0	1 1
1	1 1	0 0

Boolean minimisation using K-maps

$n0' =$

	$n0$		$n1$	
	0	1	1	0
go	1	0	0	1

$$n0' = n0 \oplus go$$

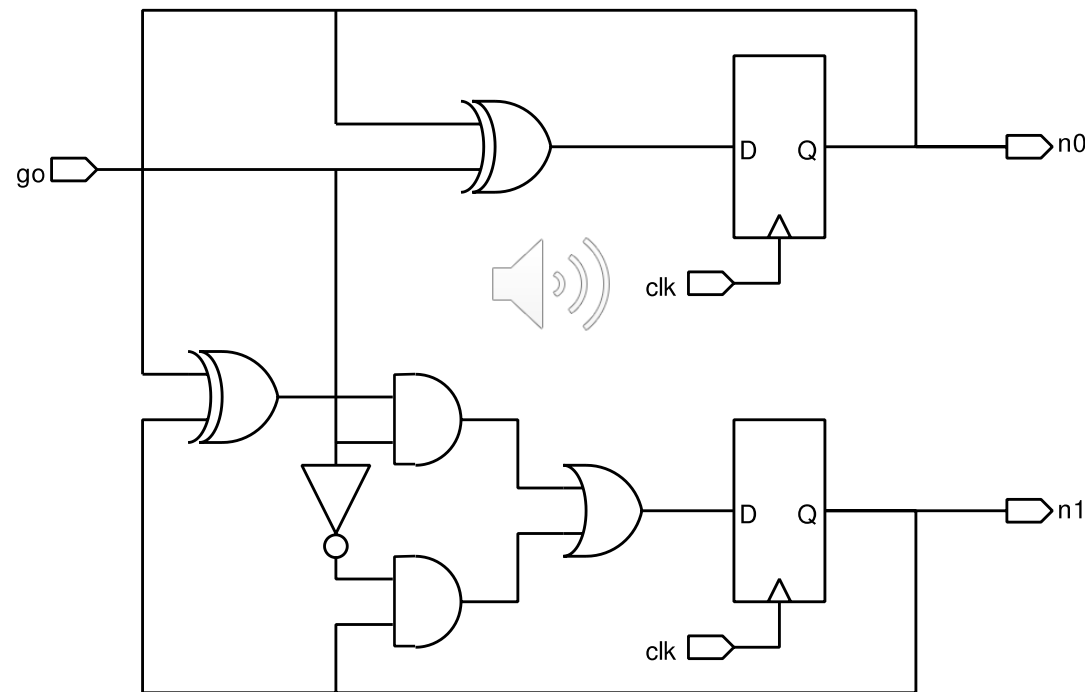
$n1' =$

	$n0$		$n1$	
	0	0	1	1
go	0	1	0	1

$$n1' = \overline{go}.n1 + \overline{n0}.n1 + n0.\overline{n1}.go$$

$$n1' = go.(n0 \oplus n1) + \overline{go}.n1$$

Final design as a schematic

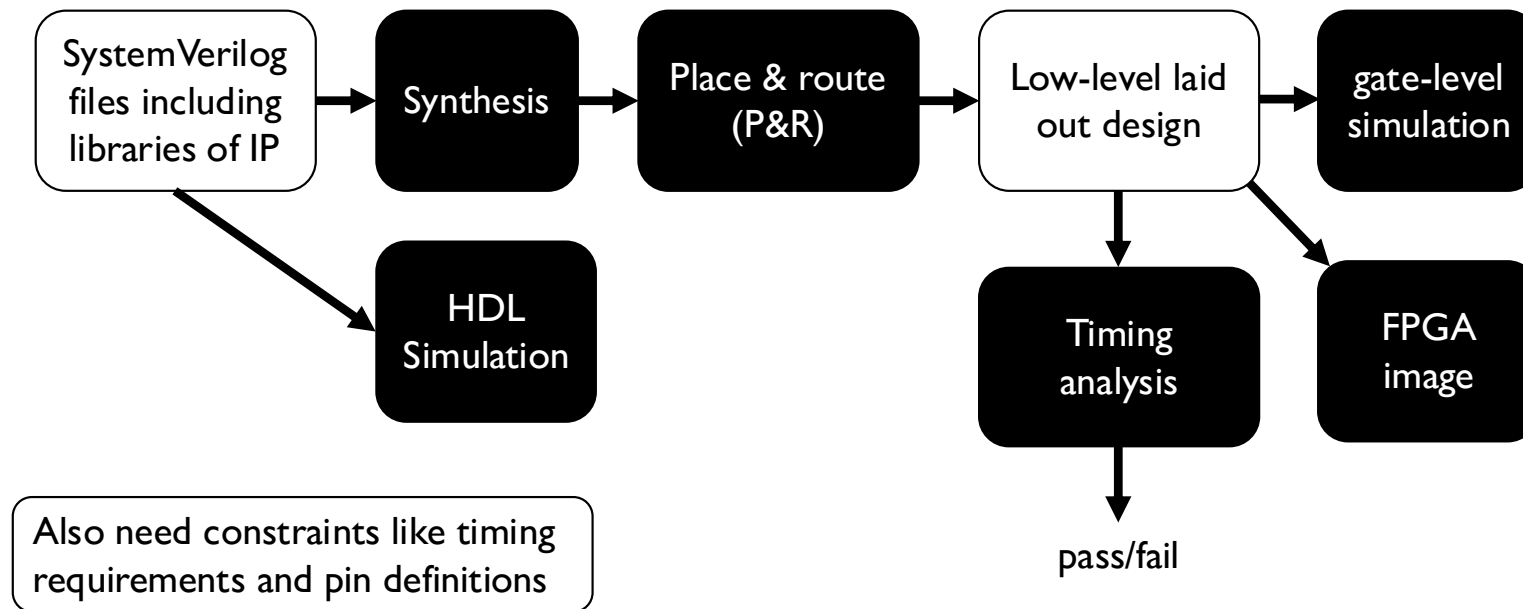


Alternative: hardware description languages (HDLs)

- e.g. SystemVerilog
 - widely used in industry and used in ECAD+Arch labs
- For the counter example, we want to be able to describe the design as:
if(go) n <= n+1;
- Wrapping this up into a SystemVerilog module and including a clock

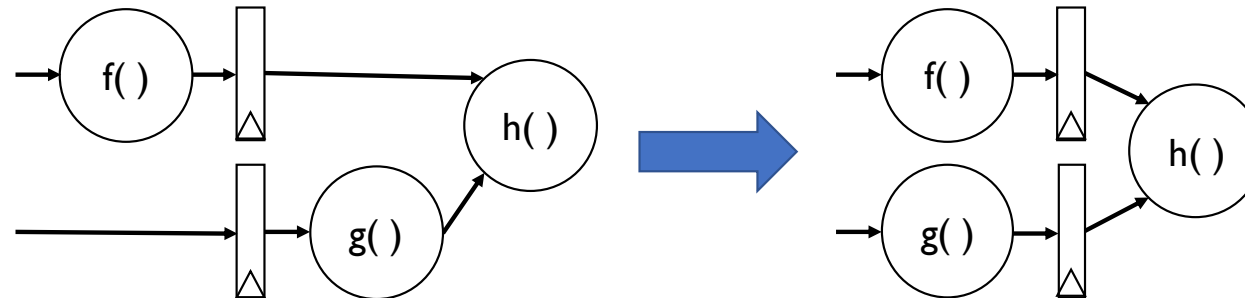
```
module counter2bit(  
    input clk,           // clock input  
    input go,            // go input  
    output logic [1:0] n); // output: 2-bit counter value  
  
    always_ff @(posedge clk) // every clock tick...  
        if(go) n <= n+1;    // ...do the conditional counting  
endmodule
```

From HDL to an implementation



Synthesis and automatic optimisation

- SystemVerilog files are compiled or *synthesised* into a lower-level description (typically a gate-level netlist)
- Optimisations include:
 - Boolean optimisation
 - Optimal state assignment
 - Retiming for timing closure



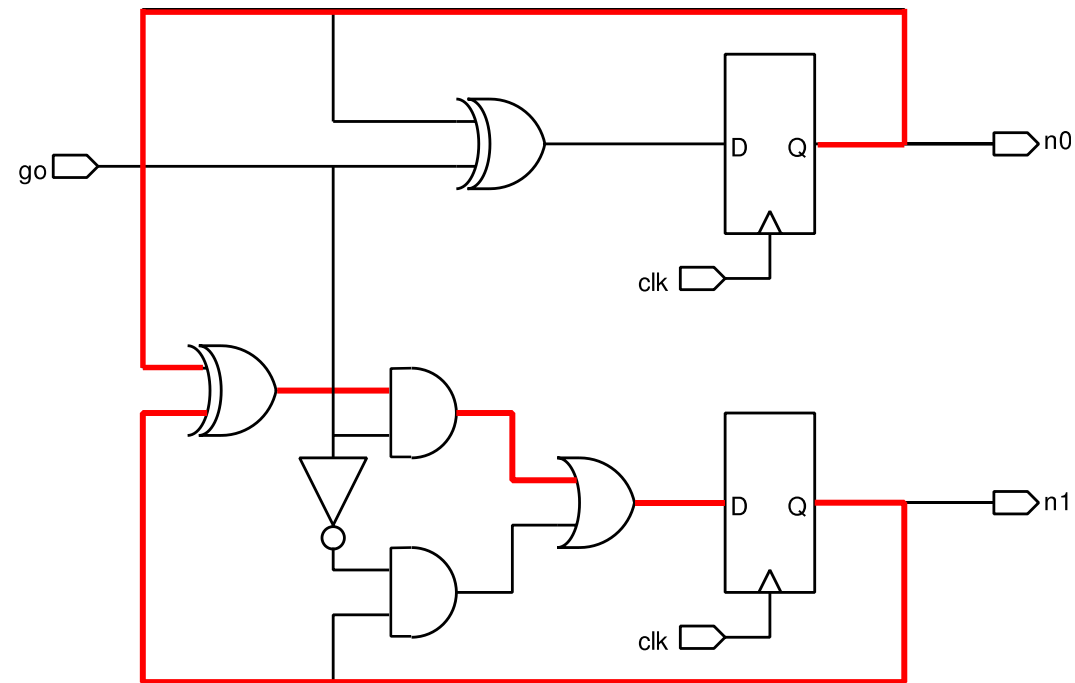
Place & Route (sometimes called *layout*)

- Starting point
 - Netlist (directed graph) of components linked by wires
- Place & route is the process of:
 - Placing components onto physical locations on a chip or FPGA
 - Routing wires between these components to complete a physical netlist
- Constraints applied:
 - Area constraints
 - Location constraints, e.g. a wire connecting to a physical pin going on/off chip
 - Timing (performance) constraints
- Automated approaches are highly effective but are time consuming for large designs with constraints that are difficult to meet
- Constraints are critical: the automated process stops once constraints are met

Timing analysis

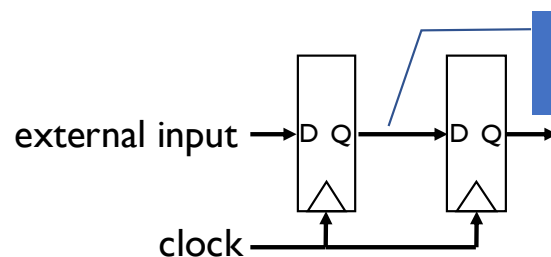
- Given the physical netlist and a detailed physical model, determine the longest combinatorial paths in a circuit
 - Using the same constraints information as place & route including clock frequencies
 - Physical model includes characteristics of circuits as temperature changes and statistical variability in performance
- Use to determine whether the maximum clock frequency ($f_{\max}$) is safe
 - Not useful to determine what the maximum $f_{\max}$ could be; need to change the constraints and repeat place & route to see if the circuit could go faster
- Ensures that our digital time abstraction is still valid, including setup and hold times on D flip-flops (DFFs) and managing clock skew
- Timing analysis sign-off is critical for commercial use
 - Want to ensure that circuits will not fail in the field

Example of a critical path (red/thick line)



Synchronisation and metastability

- External inputs (e.g. a communication input or a push-button) will not be synchronised to the clock
 - This breaks out clocked digital abstraction
- A changing input that violates setup or hold times can cause metastability, where the DFF outputs neither 0 or 1
- Two flip-flop synchroniser is often used to synchronise asynchronous inputs. t is design dependent; can result in an MTBF of once a minute to once in the lifetime of the universe



$$MTBF = \frac{e^{t/\tau}}{f_d f_c T_o}$$

t = time for first DFF to resolve
 τ and T_o = DFF parameters
 f_d and f_c = frequency of data & clock

Simulation

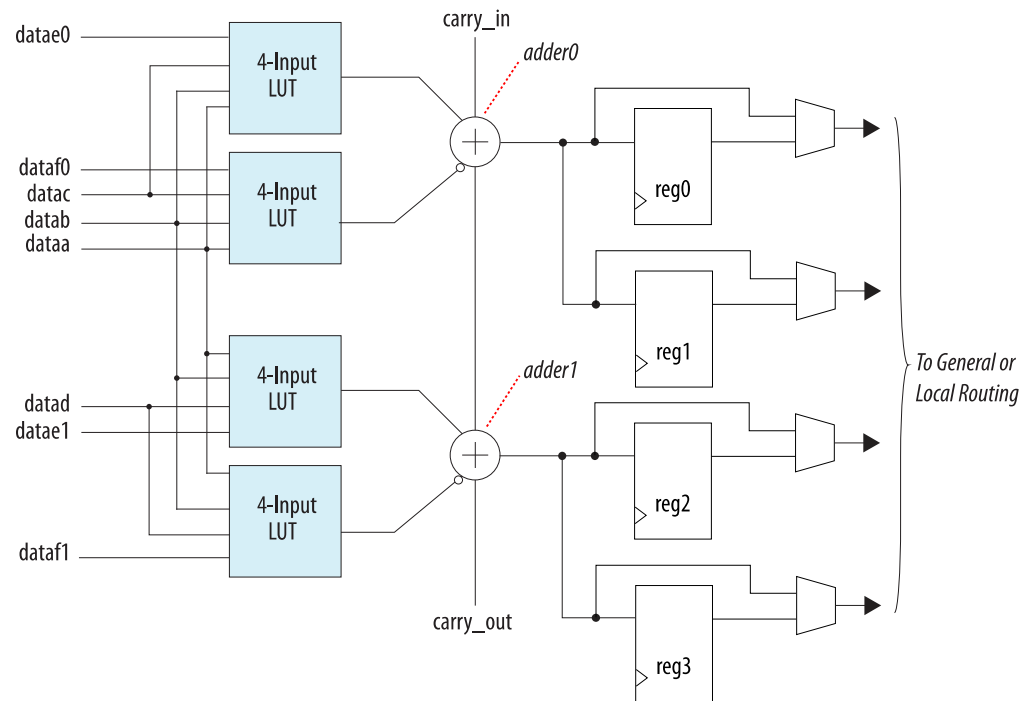
- Circuits are inherently parallel
 - Need to model parallel changes
- Often only a small proportion of signals change in any clock cycle
 - Discrete event simulation used, with state updates based on signal changes
- High-level simulation of SystemVerilog
 - Rapidly load HDL code into the simulator with no place and route required
 - Detailed delays (gate, wire) are not available
 - The clock is modelled, so cycle accurate simulation is achieved
 - Problem: simulator has it's own semantic interpretation of SystemVerilog, which may differ from synthesis tool
- Gate-level simulation
 - Post synthesis and place+route, so more accurately reflects implementation
 - Waiting for synthesis + place + route slows development time

Structure of an FPGA – architectural components

- FPGAs are made up of several reconfigurable components
- Intel Cyclone-V FPGA (used in ECAD+Arch labs) names for these:
 - LUT – look-up tables for arbitrary combinational functions of n-inputs
 - ALM – adaptive logic modules = 4 x (LUT, DFF) + programmable wiring
 - LAB – logic array block = a set of ALMs (MLAB is a superset of LAB)
 - hierarchical programmable wiring
 - BRAM – block RAM (SRAM-based) to be used as memory or combinational logic
 - DSP – digital signal processing blocks
 - I/O – input/output block for slow speed signals or high-speed serial (Ethernet, SATA, PCIe, etc.) or DDR memory (to DRAM modules)

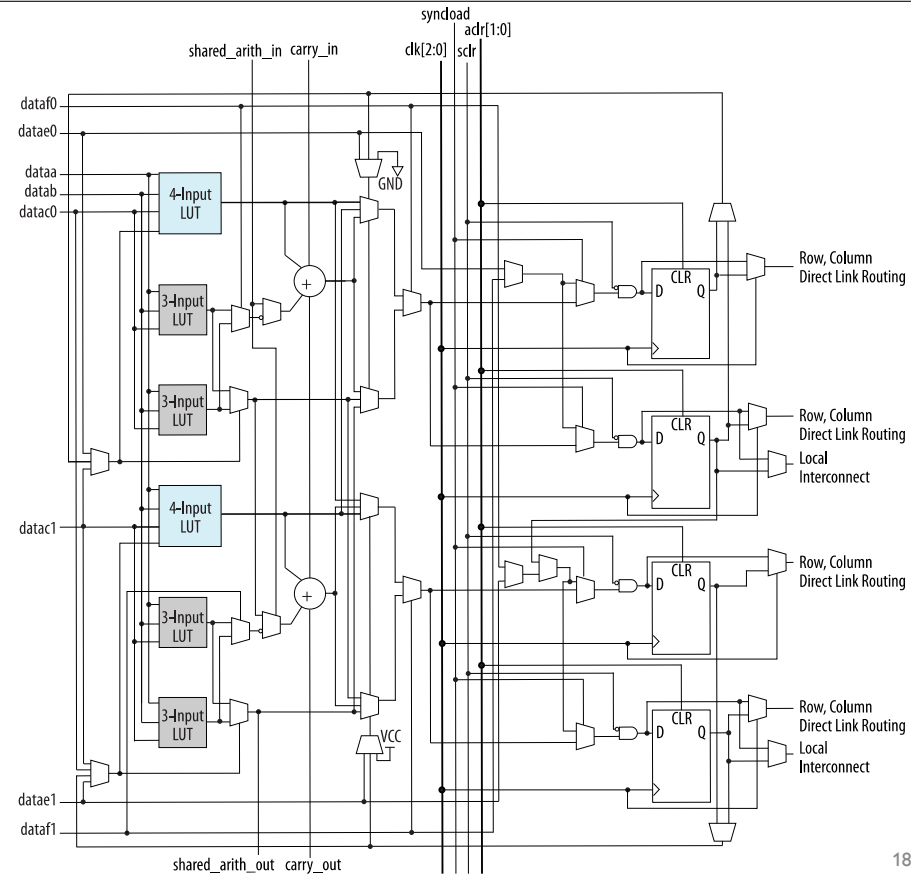
Structure of an FPGA – ALM on Cyclone V

- ALM abstracted for arithmetic

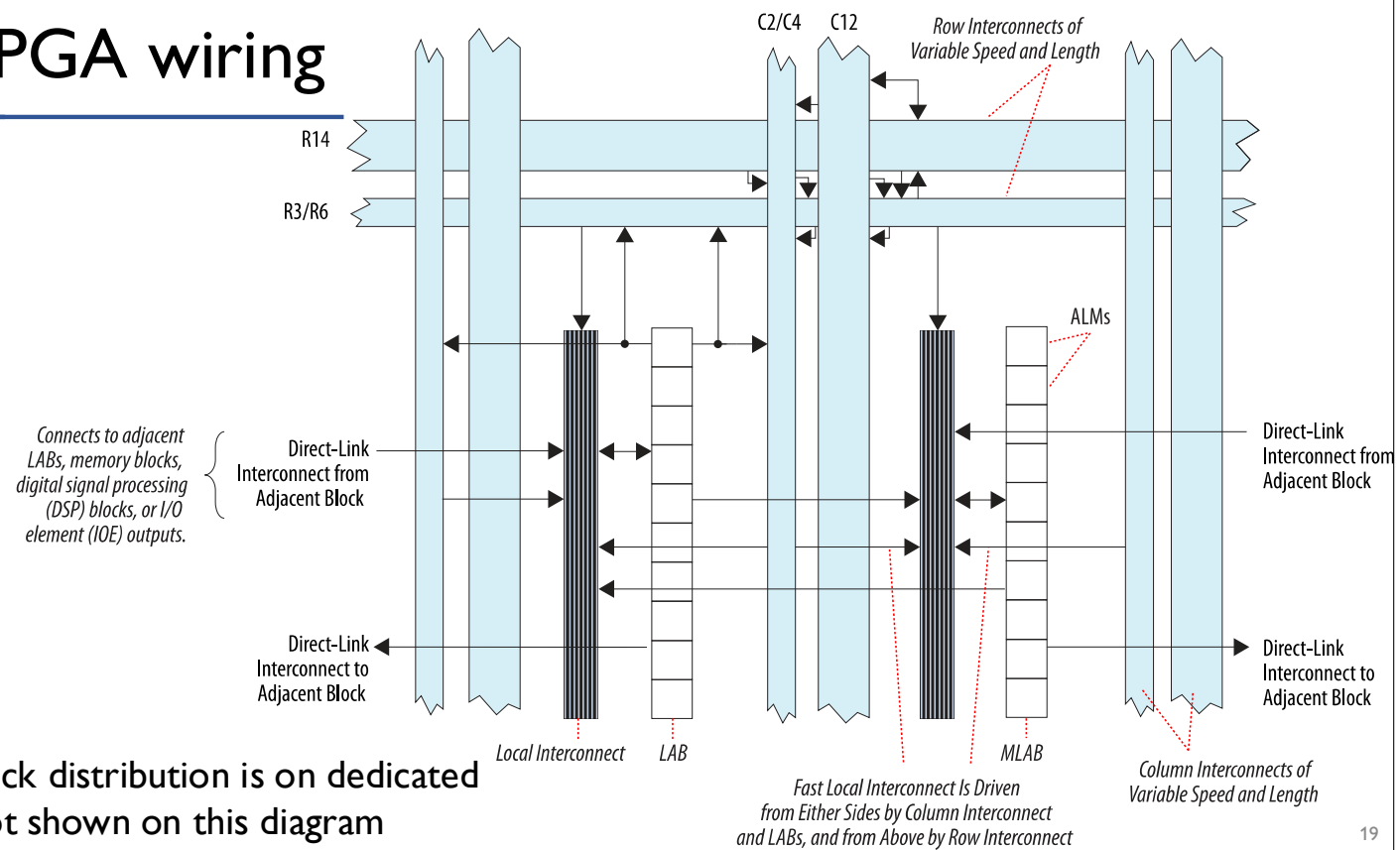


ALM on Cyclone V

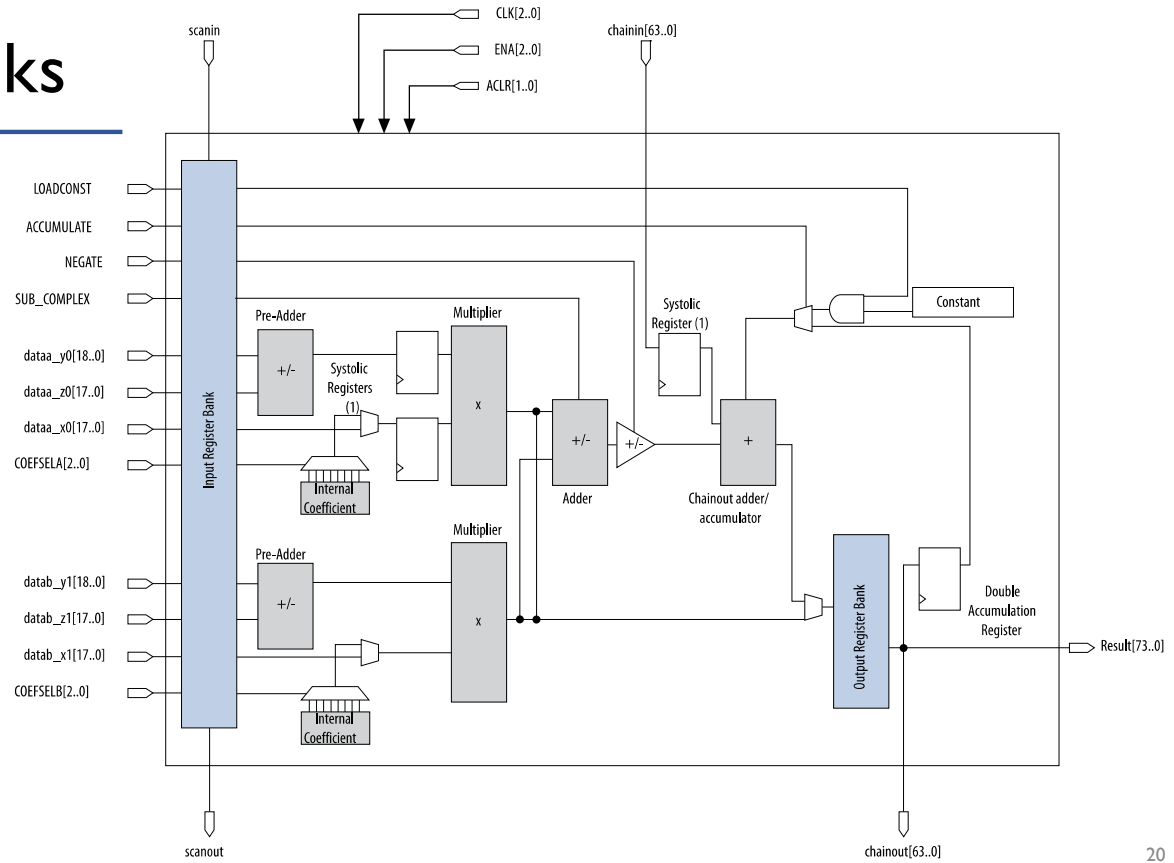
- Full ALM design (rather complex!)



FPGA wiring



DSP blocks

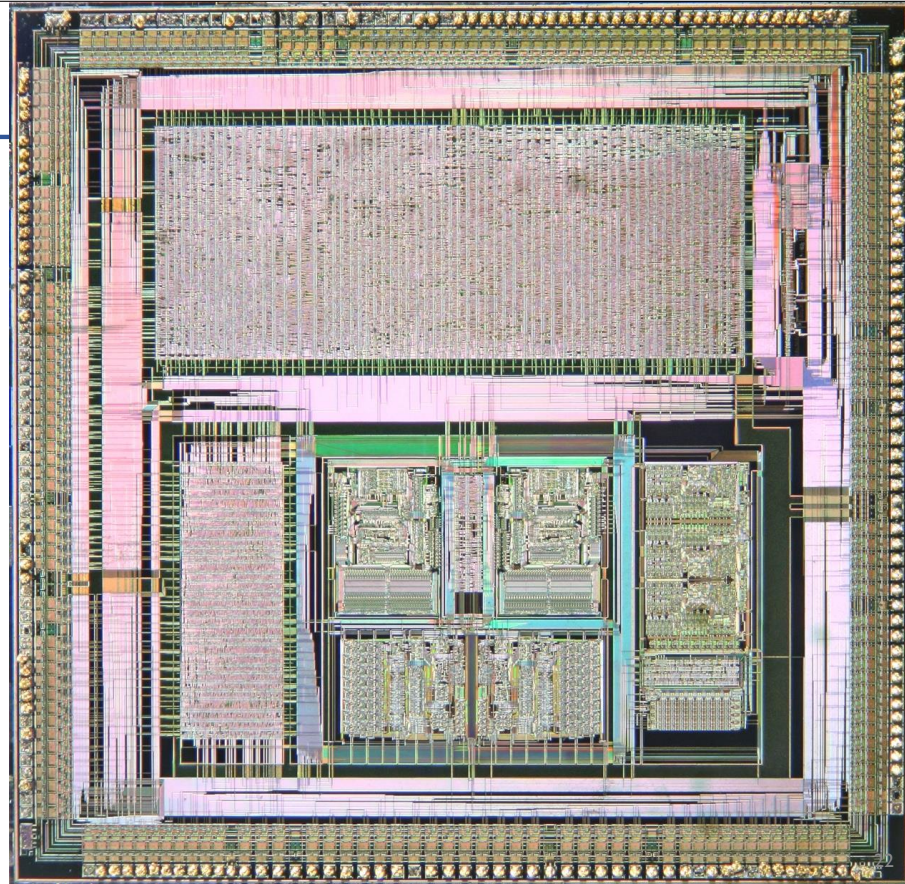


Structure of an ASIC

- ASIC = application specific integrated circuit
 - a full custom chip
- One-off costs (e.g. mask costs, design costs) limit ASICs to high volume product for economic reasons
 - Multi-project designs used for test chips (share mask costs, etc.)
- You could design every transistor yourself, but tend to use:
 - libraries of standard cells (gates, DFFs, etc)
 - memory generators
 - libraries of higher-level components including processor cores
- Lowest level design is geometric layout of every layer of the chip used to form transistors, wires, etc.

ASIC example

- Microscope photograph of custom ASIC (486 chipset) showing gate-based design on top and custom circuitry on bottom



By Antoinebercovi - Own work, CC BY-SA 3.0,
<https://commons.wikimedia.org/w/index.php?curid=29023372>

Summary

- Clocked digital design is in widespread commercial use today
- Preserving the clocked digital abstraction requires careful low-level design
 - Much low-level design is taken care of by FPGA manufacturers
 - Clock distribution networks
 - Reliably high-speed I/O
 - The designer still needs to ensure that their design
 - Meets timing requirements
 - Handles asynchronous inputs safely
- FPGAs are widely used for low to medium volume production
- ASICs for large-scale volumes

Introduction to Computer Architecture

Lecture 3

Eight great ideas in computer architecture

Prof. Simon W. Moore



Eight great ideas in computer architecture

- Design for **Moore's Law**
- Use **abstraction** to simplify design
- Make the **common case fast**
- Performance via **parallelism**
- Performance via **pipelining**
- Performance via **prediction**
- **Hierarchy** of memories
- **Dependability** via redundancy



Designing for Moore's law



- Traditionally this meant designing with the assumption that every new generation would improve performance
- e.g. Intel's tick tock model:
 - tick: new processor design but on existing well tested silicon node
 - performance gain through architectural improvements
 - tock: existing processor (with tweaks) but on the next generation silicon node
 - performance gain through silicon improvements
- Today there is greater focus on architectural improvements and how to use the larger number of transistors (Moore's law) without being reliant on silicon performance improvements

Abstraction: levels of program code



- High-level language
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- Assembly language
 - Textual representation of instructions
- Hardware representation
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for RISC-V)

```
swap:
  slli x6, x11, 3
  add x6, x10, x6
  ld x5, 0(x6)
  ld x7, 8(x6)
  sd x7, 0(x6)
  sd x5, 8(x6)
  jalr x0, 0(x1)
```

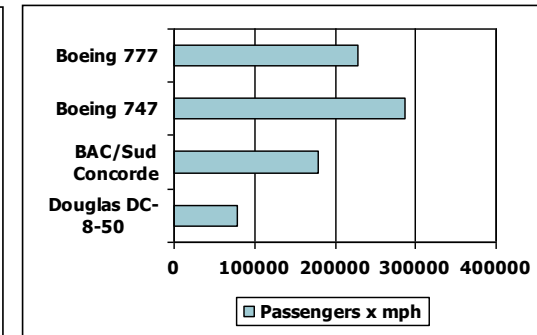
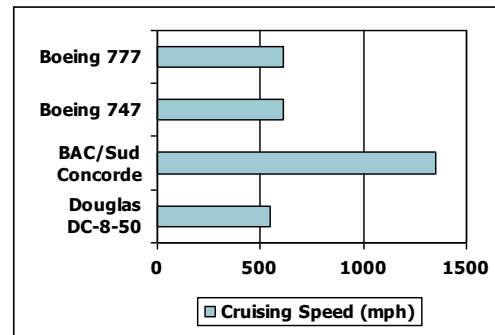
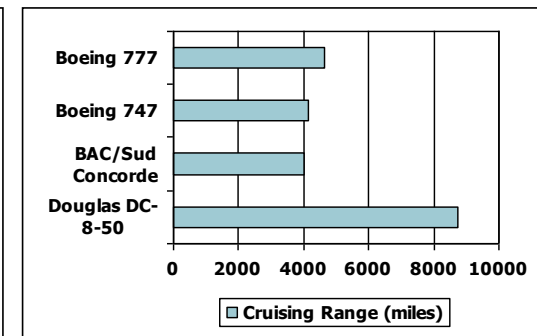
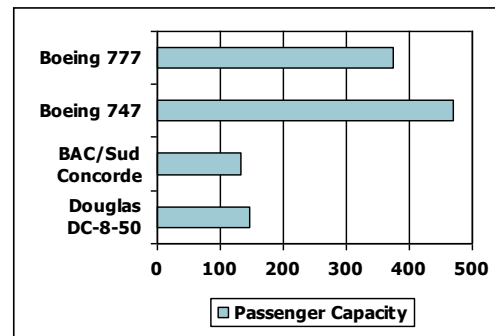
Assembler

Binary machine
language
program
(for RISC-V)

```
00000000001101011001001100010011
000000000011001010000001100110011
000000000000000110011001010000011
00000000100000110011001110000011
00000000011100110011000000100011
00000000010100110011010000100011
0000000000000000100000001100111
```

Defining performance

- Which airplane has the best performance?
- Processors are similar:
 - speed of execution (“performance”)
 - energy consumed (power)
 - silicon area
- Different applications have different requirements



Relative performance

- Define Performance = 1/Execution Time
- Relative performance: “X is n times faster than Y”
 - $\frac{\text{Performance}_x}{\text{Performance}_y} = \frac{\text{ExecutionTime}_y}{\text{ExecutionTime}_x} = n$
- Example: time taken to run a program
 - 10s on A, 15s on B
 - $\text{ExecutionTime}_B / \text{ExecutionTime}_A$
= 15s / 10s = 1.5
 - So A is 1.5 times faster than B

Measuring execution time

- Elapsed time
 - Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
 - Determines system performance
- CPU time (CPU = central processing unit = processor core)
 - Time spent processing a given job
 - Discounts I/O time, other applications scheduled
 - Comprises user CPU time and system CPU time
 - Different programs are affected differently by CPU and system performance

CPU time

$$\text{CPU Time} = \text{CPU Clock Cycles} \times \text{Clock Period}$$

$$= \frac{\text{CPU Clock Cycles}}{\text{Clock Frequency}}$$

- Performance improved by
 - Reducing number of clock cycles
 - Reducing the clock period (increasing clock frequency)
 - Hardware designer must often trade off clock frequency against cycle count

Instruction count and CPI (cycles per instruction)

Clock Cycles = Instruction Count $\times$ Cycles per Instruction

CPU Time = Instruction Count $\times$ CPI $\times$ Clock Cycle Time

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Frequency}}$$

- Instruction Count for a program
 - Determined by program, ISA (instruction set architecture) and compiler
- Average cycles per instruction
 - Determined by CPU hardware
 - If different instructions have different CPI
 - Average CPI affected by instruction mix

CPI example

- Computer A: Cycle Time = 250ps (4 GHz), CPI = 2.0
- Computer B: Cycle Time = 500ps (2 GHz), CPI = 1.2
- Same ISA
- Which is faster, and by how much?

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \end{aligned}$$

A is faster...

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps} \end{aligned}$$

$$\frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2$$

...by this much

CPI in more detail

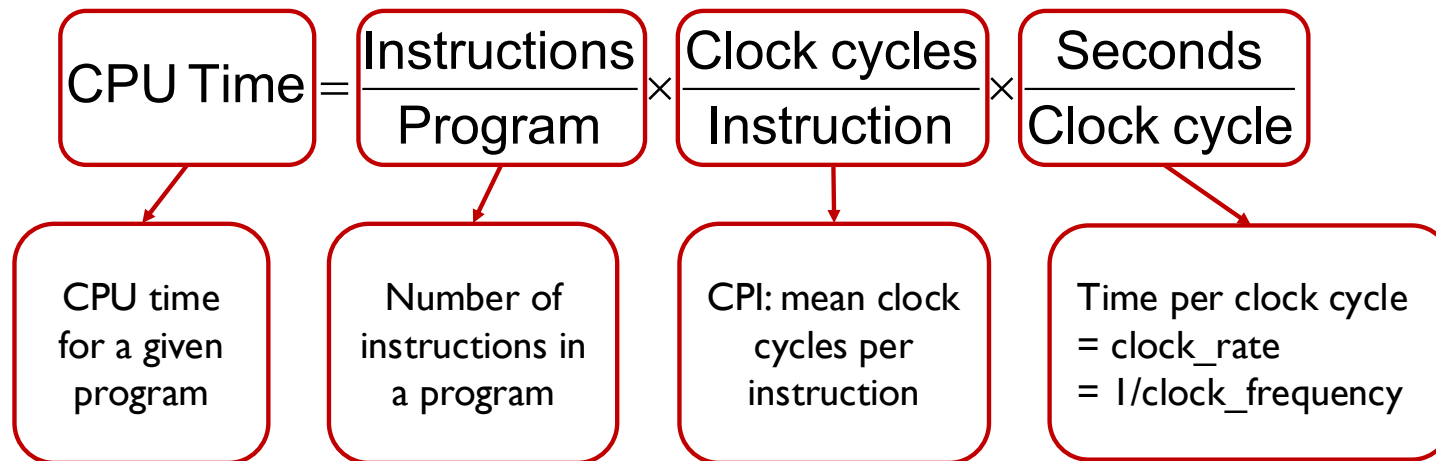
- If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

- Weighted average CPI

$$\text{CPI} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \underbrace{\frac{\text{Instruction Count}_i}{\text{Instruction Count}}}_{\text{Relative frequency of each instruction used}} \right)$$

Performance summary



What factors impact performance?

Factor:	Instruction count	CPI	Clock rate
Program	X	X	
Compiler	X	X	
ISA	X	X	X
Processor design		X	X
Implementation technology			X

Make the common case fast



COMMON CASE FAST

- Amdahl's law for sequential programs
execution time after improvement

$$= \frac{\text{execution time affected by improvement}}{\text{amount of improvement}} + \text{execution time unaffected}$$

Key to the RISC philosophy
(see later lecture)

- e.g., if integer division used 1% of the time, making it 10x faster results in a total improvement of 0.9%
- Adding integer division can make the processor clock more than 1% slower, so many early processors did not have an integer divide instruction and used a library function instead

Parallelism



- Many forms of parallelism (details in later lectures, e.g. Flynn's classification)
 - Not helped by the control-flow (“von Neumann”) model
- Instruction-level parallelism
 - i.e. try to execute more than one instruction in the same clock cycle
 - The approach taken by *superscalar* processors
- Thread-level parallelism
 - Run N sequential threads of code on N processors, or an N-way multithreaded processor, or some combination of the two
- Data-level parallelism
 - An instruction can act on more than one data item
 - e.g. vector processing: vector instructions (like add) operate on a vector of values
 - Widely used on today's CPUs and particularly GPUs (graphics processing units)

Pipelining



- Key idea: a production line for executing instructions
- Break the execution of each instruction into steps, e.g.:
 - instruction fetch (read from memory)
 - decode the instruction and fetch registers
 - execute the instruction (perform the arithmetic operation(s))
 - memory access (load or store to/from memory)
 - writeback the result to the register file
- So while one instruction is being executed, another is being decoded, etc.
- There are two lectures on this topic

Prediction



- Predict what might happen next to make the pipeline and memory subsystem more efficient
- Control-flow prediction
 - Predict what instructions will be executed next, including predicting the outcome of conditional branches
- Data-value prediction
 - Easier predictions include predicting that a value will not change, or that two different pointers do not alias to the same address
- Data-access pattern prediction
 - Ensure that data needed soon is close by for low-latency access
 - e.g. exploiting temporal and spatial statistical properties of data access or spotting stride patterns when reading data structures

Hierarchy of memories



- Motivation
 - The speed of light is too slow, and propagation of electrical signals on a chip is even slower
 - For a 5GHz processor, in one clock period light in a vacuum only travels 60mm and electrical signals on chip <20mm
- Accessing external DRAM chips can take 300+ clock cycles
- We are forced to have a hierarchy of memories, with the aim of having the one closest to the processor holding the working set
- Much more in a later lecture on memory hierarchy and caching

Dependability via redundancy



- Common forms of redundancy:
 - Extra memory bits and error correcting codes (ECC) to correct bit errors
 - Bit error detection and correction is even more critical on magnetic hard disks (~9% of disk used for correction codes) and flash memories/SSD
 - RAID: redundant array of inexpensive disks
 - RAID also applied DRAM in high reliable servers with multiple DRAM channels
 - Sizing of transistors to reduce impact of device variability
 - So larger transistors for arithmetic
 - Minimum sized transistors for memory
- For highly reliable systems, more extreme measures are taken
 - e.g. triple modular redundancy: three processors execute the same code (preferably three different codes written to the same spec. by three different teams) with key results being voted on
- Software techniques also used, e.g. MapReduce in Cloud applications

Summary

- Processor design is a multi-factor optimisation problem
- The operating environment (power, etc.) and the characteristics of the software to be run are critical design parameters
 - There is no “ideal” or “perfect” processor architecture, only compromises and domain specific optimisations
- The control-flow (“von Neumann”) processor model is a serious impediment to improvements
 - And yet the majority of software is written for such machines
 - The “Turing tax” of a general purpose machine is high: vast numbers of transistors used to store and move data with very few used for computation
- The available implementation technologies bound our ambitions and challenges our ingenuity

Introduction to Computer Architecture

Lecture 4

Reduced Instruction Set Computers (RISC)

Prof. Simon W. Moore

Instruction Set Architectures (ISAs)

- An ISA is the repertoire of instructions (primitive operations) that the processor can perform
- There are many ISAs
 - Common commercial ISAs:
 - Intel/AMD x86-64 in the Complex Instruction Set Computer (CISC) style
 - ARM (HQ in Cambridge) RISC ISAs for 32-bit and 64-bit processors (e.g. used in phones, Raspberry Pi, etc.)
 - Open source ISAs:
 - RISC-V from Berkeley is gaining traction
 - Now managed by the RISC-V foundation (riscv.org)
 - Used in Cambridge for various projects (e.g. CHERI-RISC-V for improved security: chericpu.org) and spin-outs (e.g. lowRISC for open source silicon: lowrisc.org)
 - Other open ISAs include old industry designs: OpenSPARC, MIPS Open; and community driven designs including OpenRISC

RISC philosophy

- Make the common case fast
 - Born out of a push for quantitative computer architecture
 - And greater use of compiled languages (back in the day when high performance programs were still in hand written assembler)
 - In contrast to CISC that focused on giving the assembler writer a rich instruction set to program against (minimising the semantic gap)
- Orthogonal instruction set design
 - E.g., all arithmetic instructions use the same operands
 - Registers are general purpose rather than being specialised
- Simple instruction decode
 - Often instructions are fixed length, e.g. 32-bits long perhaps with the option of a compressed 16-bit format

Reproduced under permission for Computer Design



RISC-V Introduction and Recent Developments

Krste Asanovic

krste@eecs.berkeley.edu

www.riscv.org





Instruction Set Architectures don't matter

Most of performance and energy running software on a processor is due to:

- Algorithms
- Application code
- Compiler
- OS/Runtimes
- ISA (Instruction Set Architecture)
- Microarchitecture (core + memory hierarchy)
- Circuit design
- Physical design
- Fabrication process
- In a *system*, there's also displays, radios, DC/DC converters, sensors, actuators, ...



ISAs do matter

- Most important interface in computer system
 - Large cost to port and tune all ISA-dependent parts of a modern software stack
 - Large cost to recompile/port/QA all supposedly ISA-independent parts of stack
 - If using proprietary closed-source, don't have code
 - If catch bit rot, no longer compile own source code
 - Lost your own source code
-
- Most of the cost of developing a new chip is developing software for it
 - Most current large chips have multiple ISAs



So...

If choice of ISA doesn't have much impact on system energy/performance,
and it costs a lot to use different ones

Why isn't there a free, open standard ISA that everyone can use for everything?



Open Software/Standards Work!

<i>Field</i>	<i>Standard</i>	<i>Free, Open Impl.</i>	<i>Proprietary Impl.</i>
Networking	Ethernet, TCP/IP	Many	Many
OS	Posix	Linux, FreeBSD	M/S Windows
Compilers	C	gcc, LLVM	Intel icc, ARMcc
Databases	SQL	MySQL, PostgreSQL	Oracle 12C, M/S DB2
Graphics	OpenGL	Mesa3D	M/S DirectX
Architecture	--	--	x86, ARM

- Why not successful free & open standards and free & open implementations, like other fields?



ISAs Should Be Free and Open

- ISAs proprietary for historical business reasons, no good reason for the lack of free, open ISAs:
- Not an error of omission by ISA owners
- Nor because owners do most software development
- Nor are most popular ISAs, wonderful ISAs
- Nor are companies great stewards of an ISA
- Nor can only owners verify ISA compatibility
- Not as if buying ISA protects you from patent lawsuits
- Finally, proprietary ISAs are not guaranteed to last, and many actually disappear



RISC-V Background

- In 2010, after many years and many projects using MIPS, SPARC, and x86 as basis of research, time to look at ISA for next set of projects
- Obvious choices: x86 and ARM
- x86 impossible – too complex, IP issues
 - 1300 instructions, x86 ISA manual 2900 pages, instruction length 1 to 15 bytes, ...
- ARM mostly impossible - baroque, IP issues
 - 400 instructions, ARMv7 ISA manual 2700 pages
- So we started “3-month project” in summer 2010 to develop our own clean-slate ISA
- Four years later, we released frozen base user spec.
 - But also many tape outs and several research publications



What is RISC-V?

- A new free and open ISA developed at UC Berkeley starting in 2010 (ParLab and ASPIRE)
 - Free as in “beer”, and free as in “speech”
- Designed for
 - research
 - education
 - commercial use
- Not just a free ISA, we also think it’s a good ISA



What's Different about RISC-V?

- *Simple*
 - Far smaller than other commercial ISAs
- *Clean-slate design*
 - Clear separation between user and privileged ISA
 - Avoids μ architecture or technology-dependent features
- A *modular* ISA
 - Small standard base ISA
 - Multiple standard extensions
- Designed for *extensibility/specialization*
 - Variable-length instruction encoding
 - Vast opcode space available for instruction-set extensions
- *Stable*
 - Base and standard extensions are frozen
 - Additions via optional extensions, not new versions



RISC-V is NOT an Open-Source Processor

- RISC-V is an ISA *specification*
- Want to encourage both open-source and proprietary implementations of the RISC-V ISA specification
- Most of cost of hardware design is software, so make sure software can be reused across many chip designs
- Expand to have open specifications for whole platforms, including I/O and accelerators

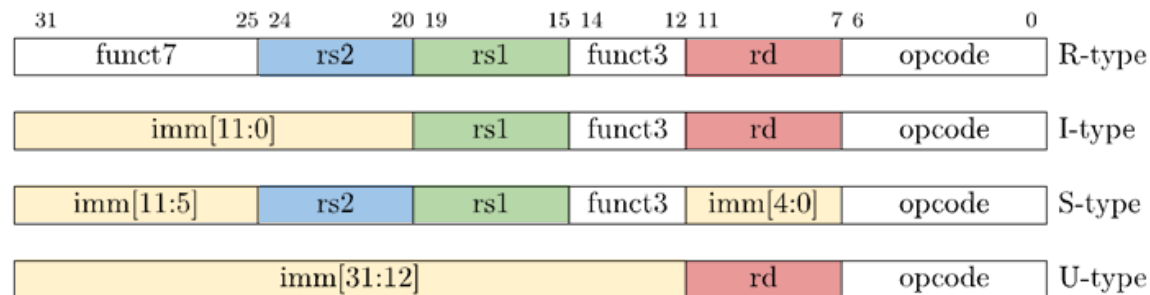


RISC-V Base Plus Standard Extensions

- Four base integer ISAs
 - RV32E, RV32I, RV64I, RV128I
 - RV32E is 16-register subset of RV32I
 - Only <50 hardware instructions needed for base
- Standard extensions
 - M: Integer multiply/divide
 - A: Atomic memory operations (AMOs + LR/SC)
 - F: Single-precision floating-point
 - D: Double-precision floating-point
 - G = IMAFD, “General-purpose” ISA
 - Q: Quad-precision floating-point
- All the above are a fairly standard RISC encoding in a fixed 32-bit instruction format
- Above user-level ISA components frozen in 2014
 - Supported forever after



RISC-V Standard Base ISA Details



- 32-bit fixed-width, naturally aligned instructions
- 31 integer registers x1-x31, plus x0 zero register
- **rd/rs1/rs2** in fixed location, no implicit registers
- Immediate field (instr[31]) always sign-extended
- Floating-point adds f0-f31 registers plus FP CSR, also fused mul-add four-register format
- Designed to support PIC (position independent code) and dynamic linking

RISC-V registers & memory

- RISC-V has 32 integer registers called $x0 \dots x31$ where $x0=0$ always
 - The register file provides very low-latency access and is multiported, allowing multiple simultaneous read (minimum 2) and write (minimum 1) operations
- Memory usage is conventional:
 - a linear address space for both instructions and data
 - optional page-based virtualisation for protection and to simplify memory allocation (see later lecture for details)
- Addresses are 32b for RV32I base instruction set
 - RV64I = 64b addresses
 - RV128I = 128b addresses
 - All still based around a 32b instruction format (with 16b compressed instructions and extended instructions possible)

RISC-V example instructions

- Add two registers together (e.g. $x3 = x2 + x1$):
add x3, x2, x1
- Add 1 to register x3 using add immediate:
addi x3, x3, 1
- Put constant -5 into register x5:
addi x5, x0, -5
- Load a word of data from memory location in register x3 and put data into x4 (address calculation is $x3 + 0$):
lw x4, 0(x3)
- Branch back by four instructions (16 bytes) if $x4 = x5$:
beq x4, x5, -16
N.B. the “-16” is usually specified in terms of a label

Assembler vs. Machine Code

- Example assembler instruction:
addi x5, x0, -5
- This human readable form is converted by the assembler into binary machine code as follows
- It is an I-type instruction:

12-bits	5-bits	3-bits	5-bits	7-bits
imm[11:0]	rs1	funct3	rd	opcode

imm[11:0] = -5 rs1 = x0 = 0 rd = x5 = 5
funct3 = 3'b111 opcode = 7'b0010011

- In machine binary (‘_’ characters are just spacers):
32'b111111111011_00000_111_00101_0010011

Calling Subroutines

- Jump-and-link instruction:
jal rd, immediate
- Performs:
 - $rd = pc + 4$ (i.e. instruction after jal)
 - $pc = pc + \text{signExtend}(\text{immediate})$
- Pseudo instructions:
 - call a function: call immediate
 - replaced by: jal ra, immediate
 - return from a function: ret
 - replaced by: jalr zero, ra, 0
 - i.e. $pc = ra$
 - where “jalr” – jump-and-link-register (i.e. adds a register offset which is added to the immediate)

RISC-V calling convention

Register	ABI name	Saver	Description
x0	zero	-	always zero
x1	ra	caller	return address
x2	sp	callee	stack pointer
x3	gp	-	global pointer
x4	tp	-	thread pointer
x5-x7	t0-t2	caller	temporaries
x8	s0/fp	callee	saved register/frame pointer
x9	s1	callee	saved register
x10-x11	a0-a1	caller	function arguments/return values
x12-x17	a2-a7	caller	function arguments
x18-x27	s2-s11	callee	saved registers
x28-x31	t3-t6	caller	temporaries

Example fib() in assembler

```
.text
.global fib
.type main, @function
# fib function start
# N.B. only temporary registers clobbered, so none saved

fib:
    mv    t0, a0          # t0=a0=n (since n is the first argument it is passed in via register a0)
    mv    a0, zero        # a0=0
    addi  a1, zero, 1     # a1=1
    beq   t0, zero, fibend # if t0==0 goto fibend

fibloop:
    addi  t0, t0, -1      # t0--
    add   a0, a0, a1      # a0=a0+a1
    sub   a1, a0, a1      # a1=a0-a1
    bne   t0, zero, fibloop # if t0!=0 goto fibloop

fibend:
    # end point for fib
    ret                # return result in a0
```

```
int fib(int n)
{
    int a0=0;
    int a1=1;
    int j;
    for(j=n; j>0; j--) {
        a0=a0+a1;
        a1=a0-a1;
    }
    return a0;
}
```

Summary

- Despite RISC processors often being simpler and lower power while offering similar performance, CISC continues to dominate the PC space due to requirements for backward compatibility
- RISC dominates the mobile and microcontroller space
- RISC-V is a clean simple complete open-source ISA
- RISC-V is great for teaching and research
 - Extensively used for the rest of these lectures
 - You will be using it in ECAD+Arch labs
 - We're using and extending RISC-V for research in the Computer Architecture Group

Introduction to Computer Architecture

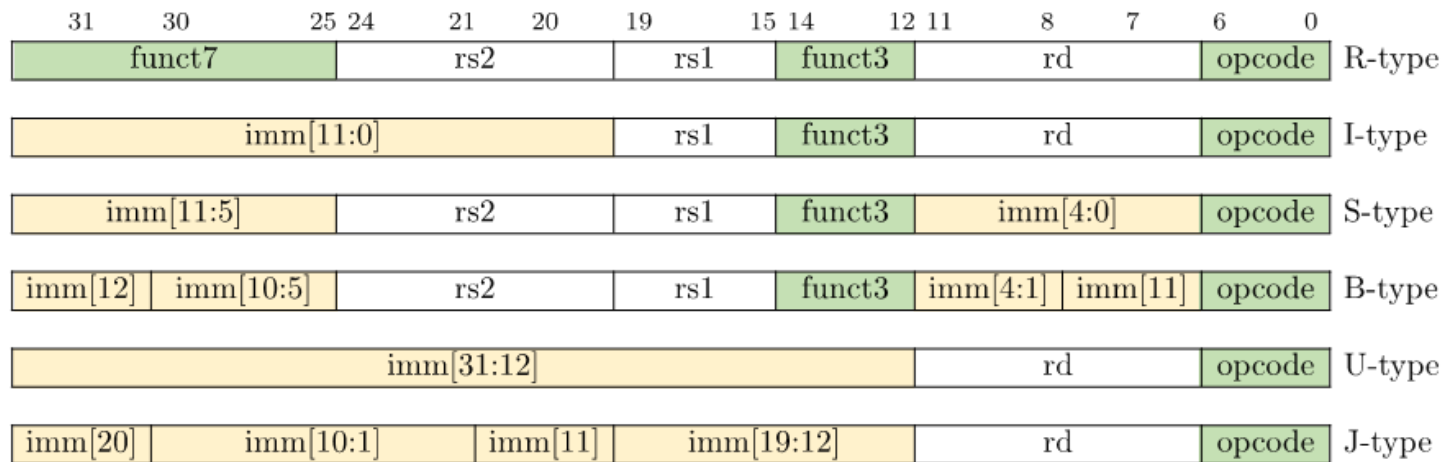
Lecture 5

Executable processor models

Prof. Simon W. Moore



RISC-V instruction types



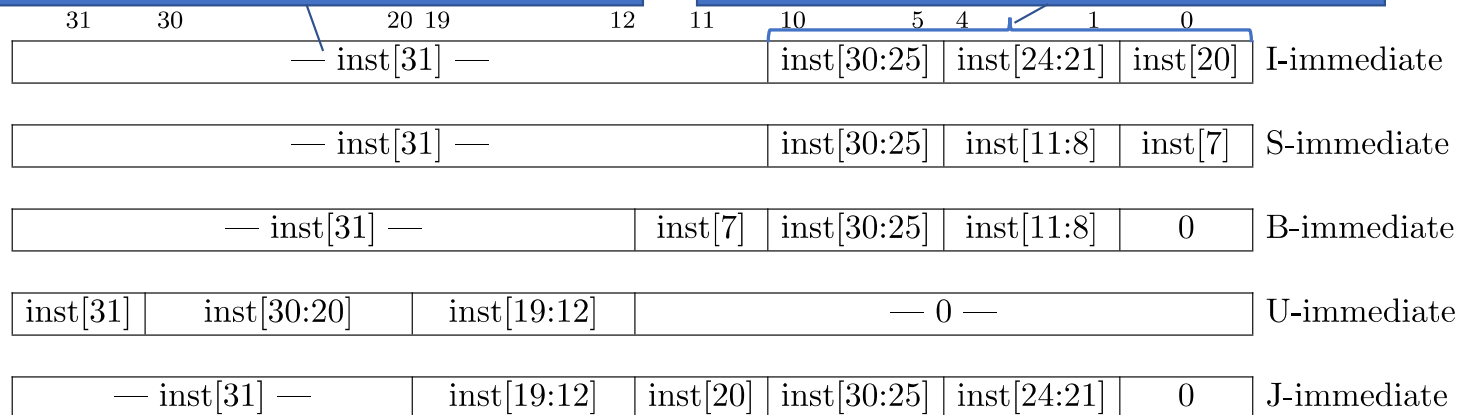
- Instruction function encoded in the opcode, funct3 and funct7 fields (where present)
 - So there can be a large number of R-type instructions but few U-type, etc.
- Source (rs1, rs2) and destination (rd) registers are always in the same place
- Immediates formed in various ways

Extracting immediates

- Encoding of immediates is designed to keep decoding hardware simple
- e.g. B-type for branch where $\text{imm}[0]=0$ is similar to S-type but looks complicated because it avoids needing to shift lots of bits

sign extend 12-bit immediate to 32-bits

i.e. $\text{inst}[30:20]$



RV32I – R-type instructions

- RV32I = RISC-V 32-bit integer base instructions
- R-type register-to-register operations
 - many instructions under the same opcode, with funct3 and funct7 specifying the function

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
0000000				rs2		rs1		000		rd		0110011		ADD
0100000				rs2		rs1		000		rd		0110011		SUB
0000000				rs2		rs1		001		rd		0110011		SLL
0000000				rs2		rs1		010		rd		0110011		SLT
0000000				rs2		rs1		011		rd		0110011		SLTU
0000000				rs2		rs1		100		rd		0110011		XOR
0000000				rs2		rs1		101		rd		0110011		SRL
0100000				rs2		rs1		101		rd		0110011		SRA
0000000				rs2		rs1		110		rd		0110011		OR
0000000				rs2		rs1		111		rd		0110011		AND

RV32I – R-type instructions cont...

- arithmetic operations: ADD, SUB
- logical operations: XOR, OR, AND
- shifts:
 - SLL: shift left logical (zeros shifted in) $\text{rf}[\text{rd}] = \text{rf}[\text{rs1}] \ll \text{rf}[\text{rs2}]$
 - SRL: shift right logical (zeros shifted in) $\text{rf}[\text{rd}] = \text{unsigned}(\text{rf}[\text{rs1}]) \gg \text{rf}[\text{rs2}]$
 - SRA: shift right arithmetic (sign extended) $\text{rf}[\text{rd}] = \text{signed}(\text{rf}[\text{rs1}]) \gg \text{rf}[\text{rs2}]$
- comparisons
 - SLT: $\text{rf}[\text{rd}] = (\text{rf}[\text{rs1}] < \text{rf}[\text{rs2}]) ? 1 : 0$
 - SLTU: unsigned version of SLT

where $\text{rf}[]$ = register file



RV32I – I-type shift immediate instructions

- I-type shift operations where shamt = shift amount
 - SLLI = logical shift left (zeros shifted into the bottom bits)
 - SRLI = logical shift right (zeros shifted into the top bits)
 - SRAI = arithmetic shift right (the original sign bit shifted into the vacated bits)
- Note that is defined as a “specialised I-type” encoding that shares features with the R-type format but with the rs2 field being used for shamt

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
0000000				shamt		rs1		001		rd		0010011		SLLI
0000000				shamt		rs1		101		rd		0010011		SRLI
0100000				shamt		rs1		101		rd		0010011		SRAI



RV32I - I-type arithmetic/shift instructions

- Arithmetic operations where one operand is a constant provided by a 12-bit sign extended immediate
- Note that there is no funct7 field, but we still have the funct3 field, so there have to be far fewer I-type instructions than R-type
- SLTI: $rf[rd] = (rf[rs1] < imm) ? 1 : 0$
- SLTIU: unsigned version of SLTI

imm[11:0]	rs1	funct3	rd	opcode	I-type
imm[11:0]	rs1	000	rd	0010011	ADDI
imm[11:0]	rs1	010	rd	0010011	SLTI
imm[11:0]	rs1	011	rd	0010011	SLTIU
imm[11:0]	rs1	100	rd	0010011	XORI
imm[11:0]	rs1	110	rd	0010011	ORI
imm[11:0]	rs1	111	rd	0010011	ANDI



RV32I - I-type load instructions

- $\text{address} = \text{rf}[\text{rs1}] + \text{imm}$
- Read from memory, e.g. logically $\text{memory}[\text{address}]$
- LW = load 32-bit word
- LB & LH = byte or half word (two-byte) read sign extended into $\text{rf}[\text{rd}]$
- LBU & LHU = byte or half word (two-byte) read unsigned into $\text{rf}[\text{rd}]$

$\text{imm}[11:0]$	rs1	funct3	rd	opcode	I-type
$\text{imm}[11:0]$	rs1	000	rd	0000011	LB
$\text{imm}[11:0]$	rs1	001	rd	0000011	LH
$\text{imm}[11:0]$	rs1	010	rd	0000011	LW
$\text{imm}[11:0]$	rs1	100	rd	0000011	LBU
$\text{imm}[11:0]$	rs1	101	rd	0000011	LHU



RV32I - S-type store instructions

- $\text{address} = \text{rf}[\text{rs1}] + \text{imm}$
- Write $\text{rf}[\text{rs2}]$ to memory, logically $\text{memory}[\text{address}] = \text{rf}[\text{rs2}]$
 - but taking care of bit width of the write
- SB = store byte
- SH = store half-word (two-bytes)
- SW = store word

no rd so use bits
for immediate

imm[11:0]		rs1	funct3	rd	opcode	I-type
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW



RV32I - B-type branch instructions

- if(condition) pc = pc + imm
- Note: imm is signed and imm bit zero=0
- BEQ: condition = rf[rs1] == rf[rs2]
- BNE: condition = rf[rs1] != rf[rs2]
- BLT: condition = rf[rs1] < rf[rs2]
- BGE: condition = rf[rs1] >= rf[rs2]
- BLTU & BGEU: unsigned versions of BLT and BGE

no rd so use bits
for immediate

imm[12 10:5]	rs2	rs1	funct3	imm[4:1 11]	opcode	B-type
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU



U-type: Forming large constants

- LUI – load upper immediate
 - U-type instruction where constant is formed from the upper 20 bits are from the immediate and the bottom 12 bits are zero
 - i.e. $rf[rd] = U_immediate$
 - Use an ADDI instruction to add in the bottom 12 bits to form a 32-bit constant
- AUIPC – add upper immediate to PC for far jumps
 - $rf[rd] = pc + U_immediate$
 - Combine with JALR (jump and link with register offset) instruction to provide the bottom 12 bits and perform the jump

imm[31:12]	rd	opcode	U-type
imm[31:12]	rd	0110111	LUI
imm[31:12]	rd	0010111	AUIPC



J-type and I-type jump to subroutine instructions

- JAL = jump and link
 - $rf[rd] = pc + 4$ (i.e. the next instruction after the jump)
 - $pc = pc + imm$
- JALR = jump and link register
 - $rf[rd] = pc + 4$
 - $pc = rf[rs1] + imm$
- $rd=ra$ for subroutines, $rd=zero$ for a plain jump

imm[20 10:1 11 19:12]			rd	opcode	J-type
imm[11:0]	rs1	funct3	rd	opcode	I-type
imm[20 10:1 11 19:12]			rd	1101111	JAL
imm[11:0]	rs1	000	rd	1100111	JALR



Pseudo assembler instructions

- There are a number of pseudo assembler instructions that aim to make code more human readable
- Examples

Pseudo instruction	Real instruction	Description
JR rs	JALR zero, rs, 0	Jump to a register
CALL label	JAL ra, pc_offset	Jump and link at an immediate offset to PC
RET	JALR zero, ra, 0	Return from subroutine
NOP	ADD zero, zero, zero	No operation
MV rd, rs	ADDI rd, rs, 0	Copy register
BEQZ rs, offset	BEQ rs, zero, offset	Branch if rs==0
BGT rs, rt, offset	BLT rt, rs, offset	Branch if rs>rt
LI rd, immediate	Sequence of instructions depending on the size of the immediate	Load immediate into a register

Steps in executing an instruction

- Instruction fetch
 - `instruction_register = memory[pc]`
 - `pc = pc+4`
- Decode
 - unpack the machine code instruction
 - fetch operands from the register file
- Execute
 - perform an arithmetic or logical operation
- Branch
 - update PC for branch/jump instructions
- Memory access
 - read or write data for load and store instructions
- Writeback
 - write the result to the destination register

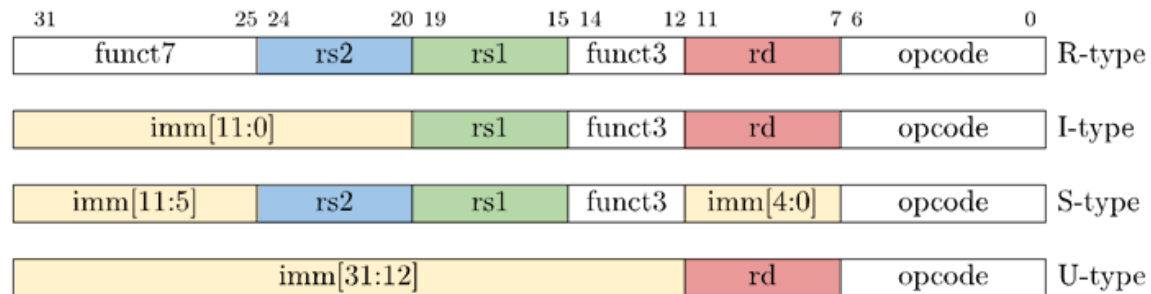
Decoding instructions

Overview:

- Decode is quite easy on RISC-V
- The opcode field determines either:
 - the instruction to be performed (e.g. for U-type instructions)
 - or the class of instruction to be performed (e.g. integer R-type instructions)
 - funct fields then determine the exact instruction
- The source (rs1, rs2) and destination (rd) registers are always encoded in the same bit fields
- The immediate bits (constant value) needs to be extracted

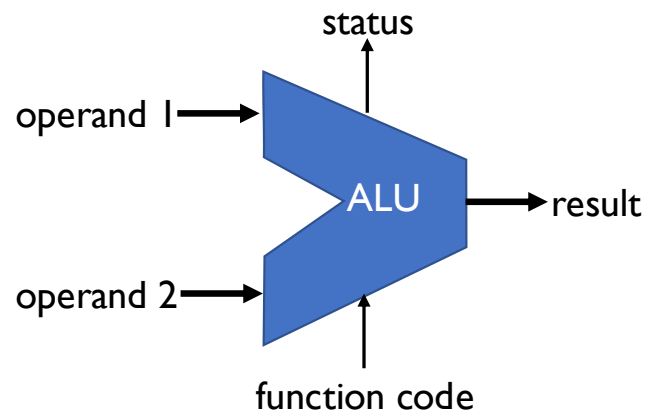
Register fetch

- Register fields never move
 - destination register is rd
 - source registers are rs1 and rs2
- rs1 and rs2 can be used as addresses to read register values from the register file as soon as the instruction arrives
 - The values read can be ignored if they are not needed (an efficient approach in hardware)



Execute

- Operands (register values/immediate) are fed into an arithmetic logic unit (ALU)
- The function code (or ALU *opcode*) selects the function, e.g. add, subtract, or, and, shift...
- The result is produced together with status, e.g. zero, overflow, carry
- Surprisingly small given that it does most of the work



Branch

- Branch (or *jump*) instructions change the flow of control by modifying the program counter (PC)
- Conditional and unconditional branches are often handled in parallel with decode or execute
- Conditional branches (B-type) have a 12-bit immediate that is shifted left one bit and sign extended before being added to the PC
 - Jump PC $\pm$ 4KiB ($\pm$ 1K 32-bit instructions or $\pm$ 2K 16-bit compressed instructions)
- Unconditional jumps (J-type) have a 20-bit immediate that is shifted left one bit and sign extended before being added to the PC
 - Jump PC $\pm$ 1MiB ($\pm$ 256K 32-bit instructions or $\pm$ 512K compressed instructions)
 - Jump-and-link-register (JALR) jump to a register specified value, so can jump anywhere in the address space



Memory access

- An address will have been computed during execute
- Send the address to the memory (e.g. the level-1 cache)
- Receive the data returned from memory
- Sounds simple, but can be time consuming due to latency in the memory hierarchy
 - One cycle of latency for a cache hit on a low-end core
 - Three cycles of latency for a cache hit on a high-end core
 - It can be hundreds of cycles of latency if there is a cache miss and we need to fetch data from DRAM



Writeback

- A result from execute or memory-access is written back to the destination register if required



ISA-level simulation

- A program that interprets machine code to emulate a processor
 - Contains a mechanism to load code into memory
 - Then performs the decode, execute, branch, memory access, writeback step for each instruction
- May also contain one or more models of I/O
 - e.g. a simple UART for console output
- Often has support for remote debug
- Used to bring up software before the hardware is available
- For RISC-V, Spike (<https://github.com/riscv/riscv-isa-sim>) is the primary ISA simulator



Example program to run

```
int fib(const int a)
{
    if(a<2)
        return a;
    else
        return fib(a-1) + fib(a-2);
}

int main(void)
{
    int *outputDevice = (int*) 0xf0000000; // magic output device
    int result;
    result = fib(8);
    outputDevice[0] = result;
    return result;
}
```



Compile and link for simulation

- To run this code “bare metal” (without an OS) we need a simple bit of assembler to initialise the stack pointer and jump to main()

```
# RISC-V baremetal init.s
```

```
# This code is executed first.
```

```
.section .text.init
```

```
entry:
```

```
    la    sp, __sp-32  # set up the stack pointer, using a constant defined in the linker script
```

```
    call  main         # call the main function (call is a pseudo instruction for jal)
```

```
end:
```

```
    j     end          # loop when finished if there is no environment to return to.
```

Simple linker script (link.ld)



See ECAD+Arch labs
for further details

```
/*
 * The stack starts at the top of memory, defined by __sp, and grows down.
 * Unfortunately this means we must hardcode the size of memory.
 * Here we choose 64K. Change this value to change the working memory size.
 */
__sp = 0x10000;

SECTIONS
{
    .text.init : {
        *(.text.init)
    }
```

```
.text : {
    *(.text)
}
.data : {
    *(.sbss)
    *(COMMON)
    *(.bss)
    *(.sdata)
    *(.rodata)
    *(.data*)
    *(.got*)
}
}
```

fib.S part 1

```
.file "fib.c"
.option nopic
.text
.align 2
.globl fib
.type fib, @function
```

} Assembler directives

fib:

```
li    a5,1           # load immediate: a5=1
bgtu  a0,a5,.L8       # branch if a0 > a5 to label .L8
li    a0,1           # else part of if( ) statement: return value a0=1
ret                    # return from subroutine: pc=ra
# N.B. ret is a pseudo for jalr zero, ra
```

fib.S part 2

.L8:

addi	sp,sp,-16	}	Stack operations: Grow stack down: $sp = sp - 16$ Store the return address (ra) and two saved registers (s0,s1)
sw	ra,12(sp)		
sw	s0,8(sp)		
sw	s1,4(sp)		
mv	s0,a0		# s0=a0, i.e. save operand in s0
addi	a0,a0,-1		# a0=a0-1
call	fib		# call fib function, i.e. fib(a-1)

fib.S part 3

```
mv    s1,a0      # save result of fib(a-1) in register s1
addi  a0,s0,-2    # a0=s0-2
call  fib         # call fib, i.e. fib(a-2)
add   a0,s1,a0    # a0=s1+a0 sums results from two function calls
lw    ra,12(sp)   # restore values from the stack
lw    s0,8(sp)
lw    s1,4(sp)
addi  sp,sp,16    # deallocate stack
ret                   # return from subroutine: pc=ra
```

fib.S part 4: the main() function

main:

```
addi  sp,sp,-16    # allocate stack space
sw    ra,12(sp)    # save the return address on the stack
li    a0, 8        # a0=8
call  fib          # call fib, i.e. fib(8)
lw    ra,12(sp)    # restore the return address
addi  sp,sp,16     # deallocate stack space
ret                   # return from main
```

Java simulator for RISC-V

- A simple ISA simulator that is just enough to simulate fib() can be found here:
 - <https://github.com/swm11/riscv-lite/tree/master/jsim>
- Start of Execute function:

```
ExecuteState executeStep(Memory mem) {  
    // Move onto the next pc  
    this.pc = this.nextpc;  
    // Ensure register zero is always 0  
    this.rf[0] = 0;  
    // Fetch and decode instruction  
    DecodedInst d =  
        DecodedInst.decode(mem.load(this.pc));  
    // By default the nextpc is the next instruction  
    this.nextpc = this.pc + 4;  
}
```

```
switch (d.inst) {  
    case ADD: // add two registers  
        this.rf[d.rd] = this.rf[d.rs1] + this.rf[d.rs2];  
        break;  
    case ADDI: // add a register and an immediate  
        this.rf[d.rd] = this.rf[d.rs1] + d.imm;  
        break;  
    case AUIPC: // PCrelative immediate  
        this.rf[d.rd] = this.pc + d.imm;  
        break;  
}
```


Java simulator for RISC-V: execute cont...

```
case LUI: // load upper immediate
    this.rf[d.rd] = d.imm;
    break;
case BLT: // branch less than
    if (this.rf[d.rs1] < this.rf[d.rs2])
        this.nextpc = this.pc + d.imm;
    break;
case JAL: // jump and link (pc + immediate)
    this.nextpc = this.pc + d.imm;
    this.rf[d.rd] = this.pc + 4; // x1 = ra (return address)
    break;
case JALR: // jump and link (register + immediate)
    this.nextpc = this.rf[d.rs1] + d.imm;
    this.rf[d.rd] = this.pc + 4; // x1 = ra (return address)
    break;
```

```
case LW: // load word
    this.rf[d.rd] = mem.load(this.rf[d.rs1] + d.imm);
    break;
case SW: // store word
    mem.store(this.rf[d.rs1] + d.imm, this.rf[d.rs2]);
    break;
default:
    System.out.format("ERROR: Undefined instruction at
pc=0x%08x%n", this.pc);
    this.nextpc = this.pc; // trigger stop condition
}
```

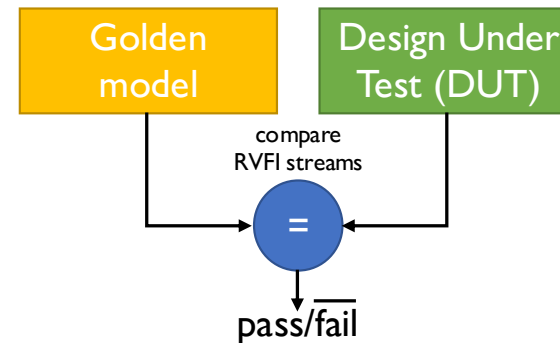


Formal modelling of ISAs

- Traditionally processors are described in English in an Instruction Set Reference Manual, e.g. for RISC-V: <https://riscv.org/specifications/>
- Written descriptions can easily be ambiguous
- A formal model is preferred
- For RISC-V there is a formal model in the Sail language developed in Cambridge: <https://github.com/rem-s-project/sail-riscv>
- This formal model is executable, so an ISA simulator can be generated
- The Sail code can also be transformed into descriptions for theorem provers like HOL and Isabelle (from Cambridge) or Coq (from INRIA) to allow properties to be verified

Tandem verification

- RVFI: RISC-V Formal Interface
 - Standardised format to report completed instructions
- Tandem verification
 - Run two models or implementations of processors and compare them side-by-side
 - Can end up with divergent behaviour due to interrupts or subtle differences in I/O models; fixed by sending the control-flow information to the golden model and using it as a checker



Summary

- The Instruction Set Architecture (ISA) provides the interface between hardware and software
- An ISA reference manual is often provided as English prose
 - but needs a formal model to be rigorous
- A processor can be simulated at an ISA level
 - Encapsulates the algorithmic behaviour of the processor
 - but without the hardware implementation details to make a high performance design
 - ISA simulators are useful to bring up software before hardware is available and to compare how software runs on a “golden” executable specification vs. an implementation
- Processor models can be compared using techniques like tandem verification (arguably “tandem test”)

Introduction to Computer Architecture

Lecture 6

Processor Pipelines

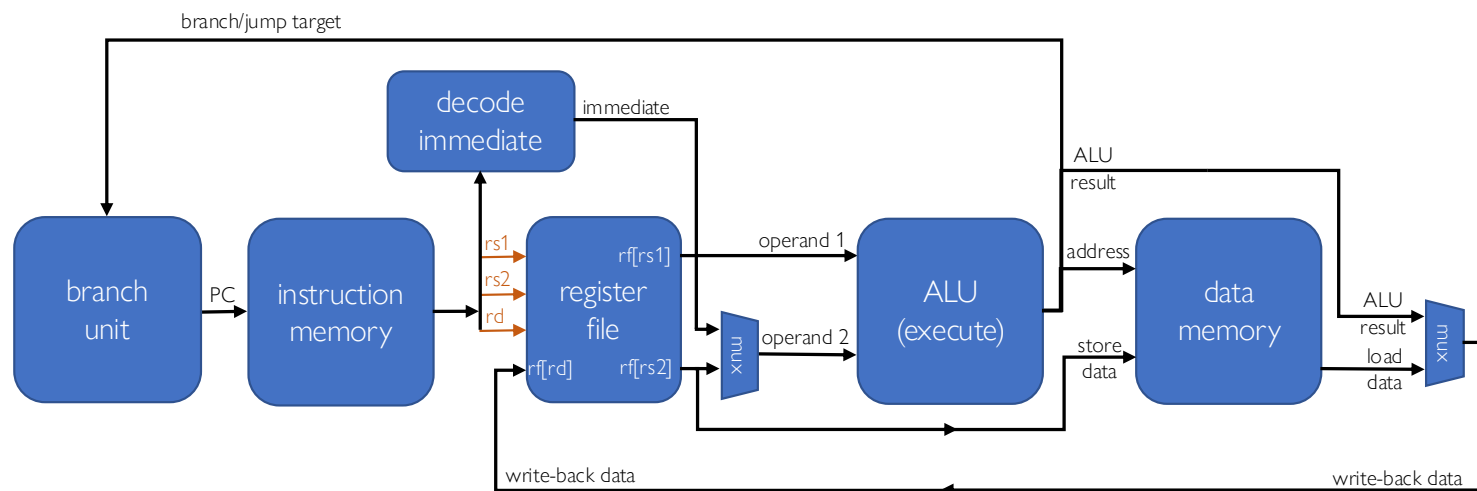
Prof. Simon W. Moore



Reminder: instruction execution steps

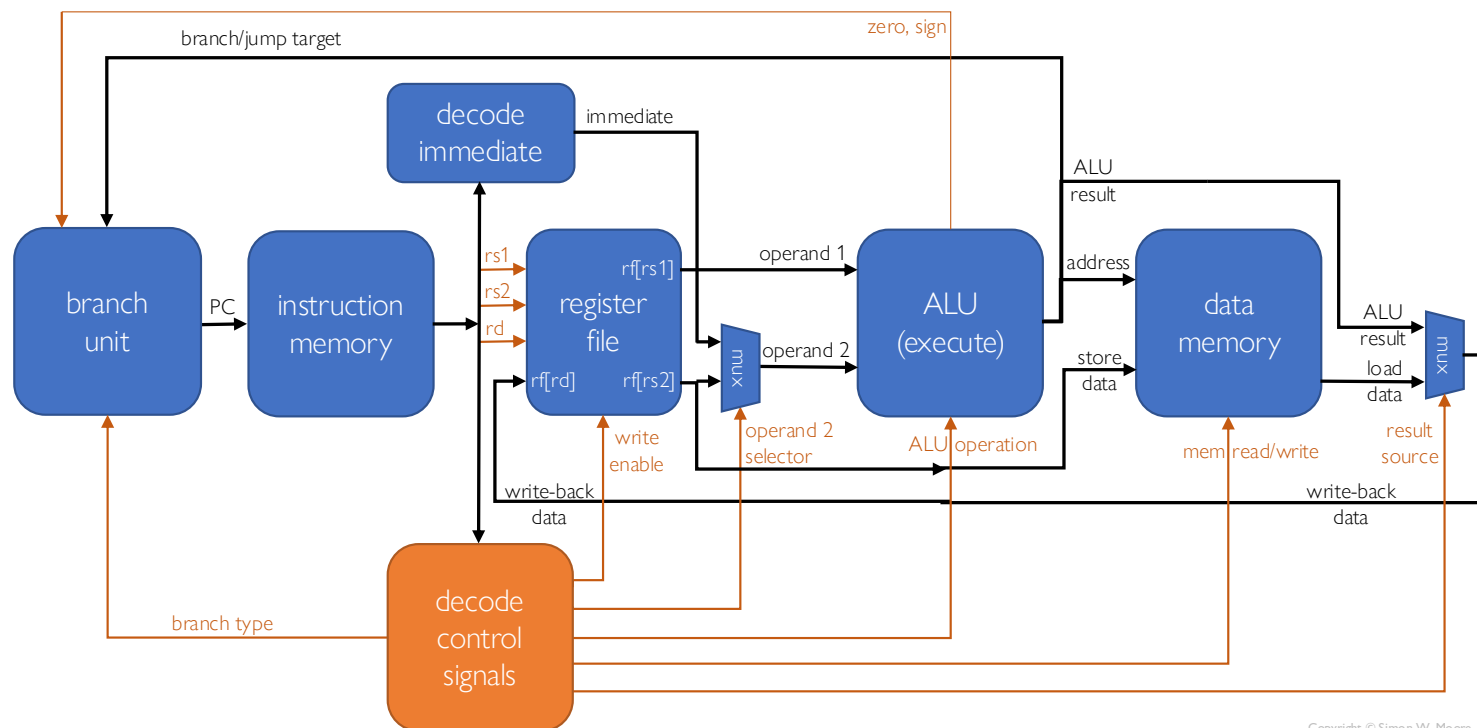
- PC → instruction memory, fetch instruction
- Register numbers → register file, read registers
- Depending on instruction class
 - Use ALU to calculate
 - Arithmetic result
 - Memory address for load/store
 - Branch comparison
 - Branch or next instruction: $PC \leftarrow \text{target address or } PC + 4$
 - Access data memory for load/store
 - Write any result back to the register file

CPU overview focusing on data path



- Remember that hardware is parallel, so it is often quicker to do things and later decide if we need the result
 - E.g. read **rf[rs2]** even though it might be replaced by an immediate
- We're starting to see some control logic added: **rs1**, **rs2**, **rd** fields extracted from the instruction

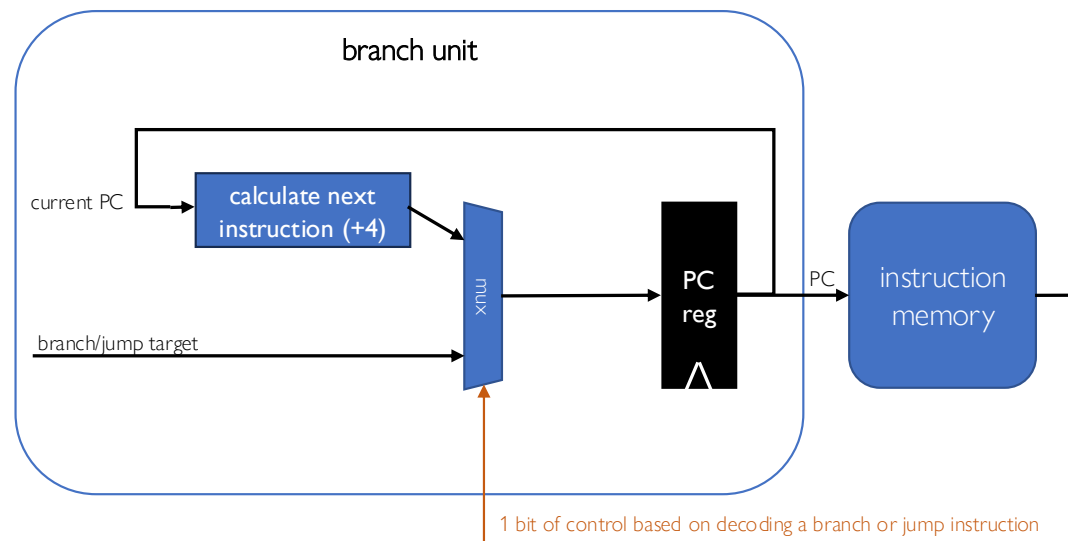
Control logic – quick overview



Building a datapath

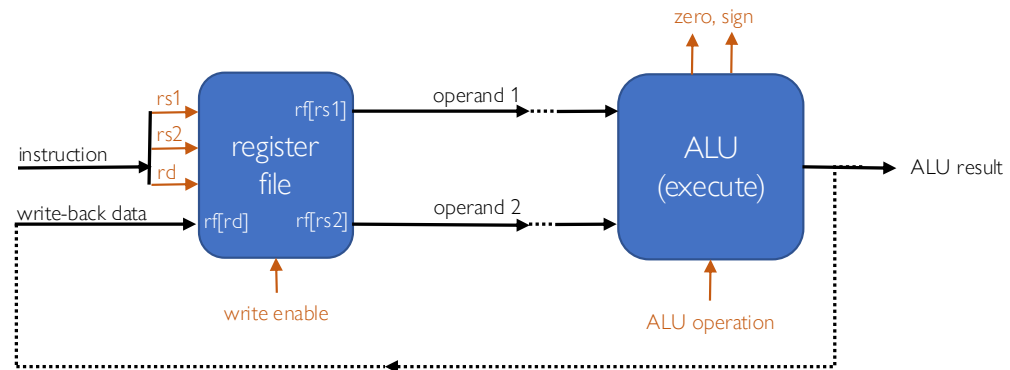
- Datapath
 - Elements that process data and addresses in the CPU
 - Registers, ALUs, mux's, memories, ...
- We will build a RISC-V datapath incrementally
 - Refining the overview design

Instruction fetch



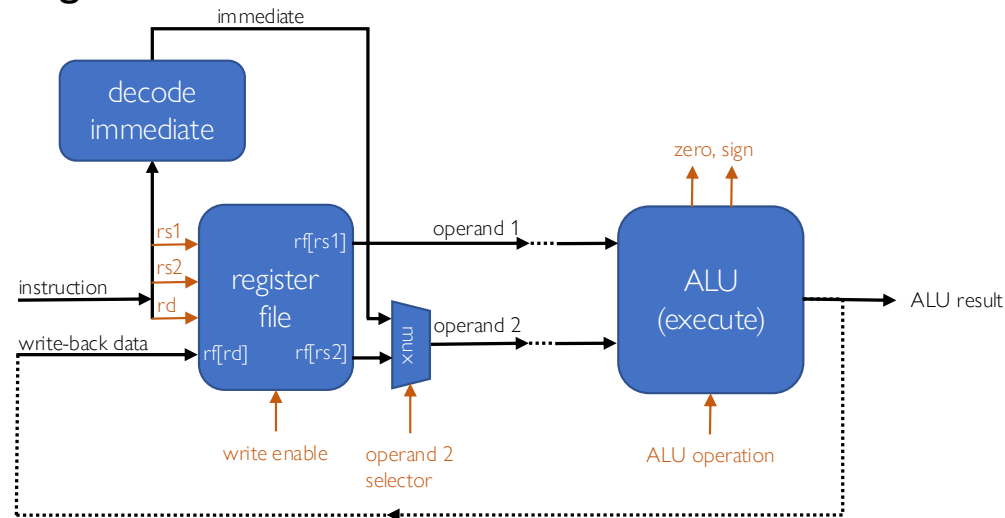
R-type instructions: $rf[rd] = alu(rf[rs1], rf[rs2])$

- Read two register operands
- Perform arithmetic/logical operation
- Write register result



I-type instructions: $rf[rd] = alu(rf[rs1], imm)$

- Read two register operands (ignore op. 2) & decode the immediate
- Perform arithmetic/logical operation
- Write register result

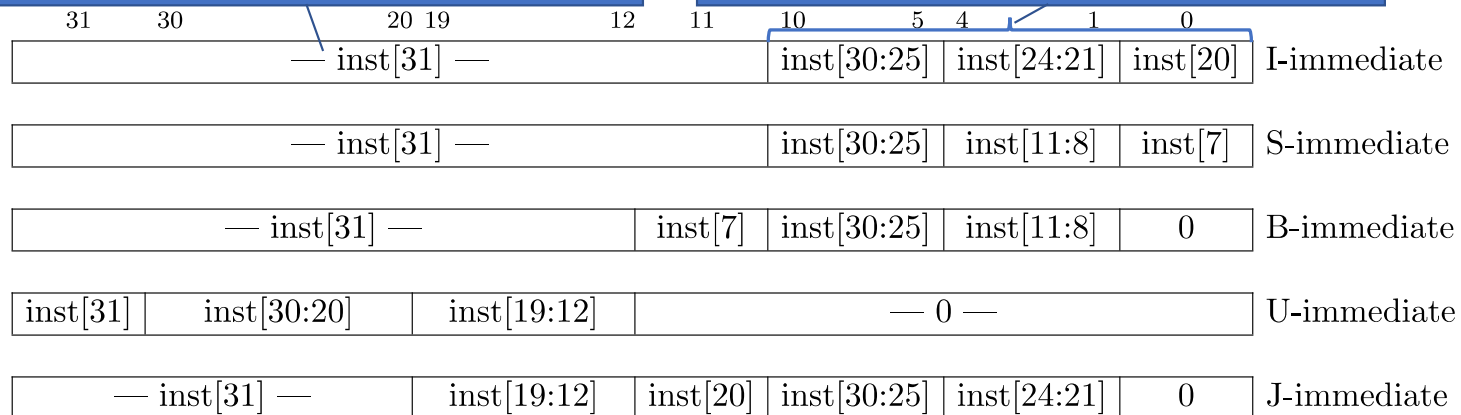


Reminder from lecture 5: Extracting immediates

- Encoding of immediates is designed to keep decoding hardware simple
- e.g. B-type for branch where $\text{imm}[0]=0$ is similar to S-type but looks complicated because it avoids needing to shift lots of bits

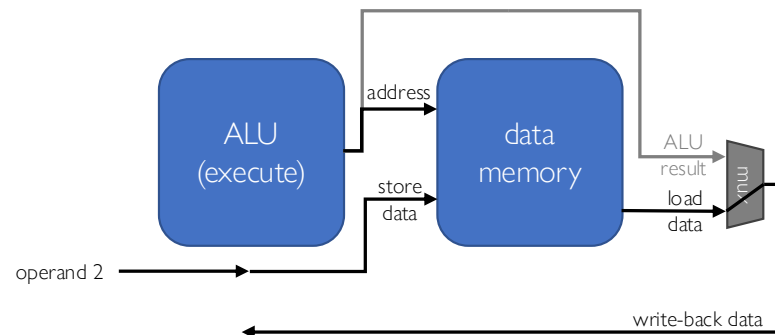
sign extend 12-bit immediate to 32-bits

i.e. $\text{inst}[30:20]$



Load/Store instructions: $\text{addr} = \text{rf}[\text{rs1}] + \text{imm}$

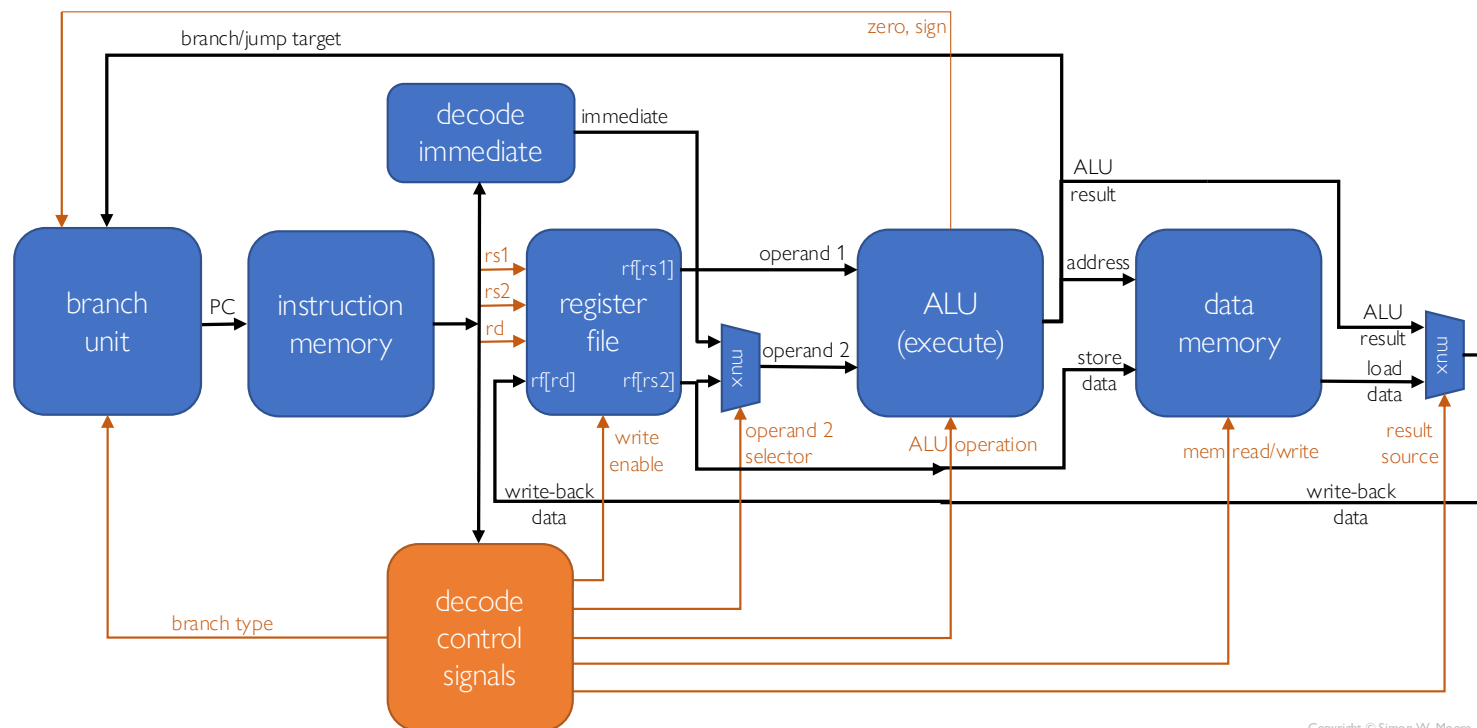
- Starts out like an I-type instruction to do the address calculation:
 - Read register operands
 - Calculate address $\text{rf}[\text{rs1}] + \text{sign_extended_immediate}$
- Store: Write register $\text{rf}[\text{rs2}]$ value to memory
- Load: Read memory and update destination register: $\text{rf}[\text{rd}]$



Branch instructions

- Read register operands
- Compare operands
 - Use ALU, subtract and check (zero,sign) outputs
- Calculate target address
 - Sign-extend displacement
 - Shift left 1 place (halfword displacement since the smallest instruction is compressed to 16-bits, so we only need 2-byte access, not 1-byte)
 - Add to PC value

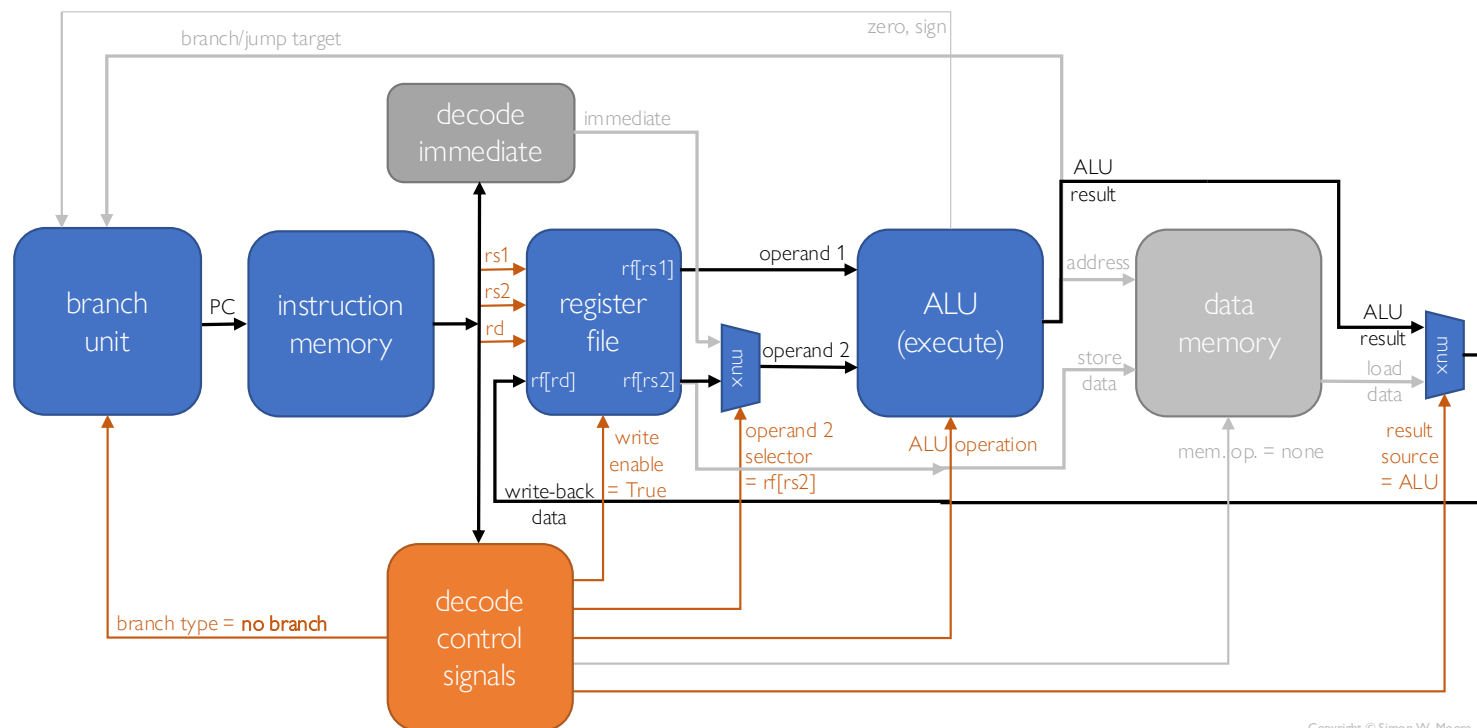
Composing the elements



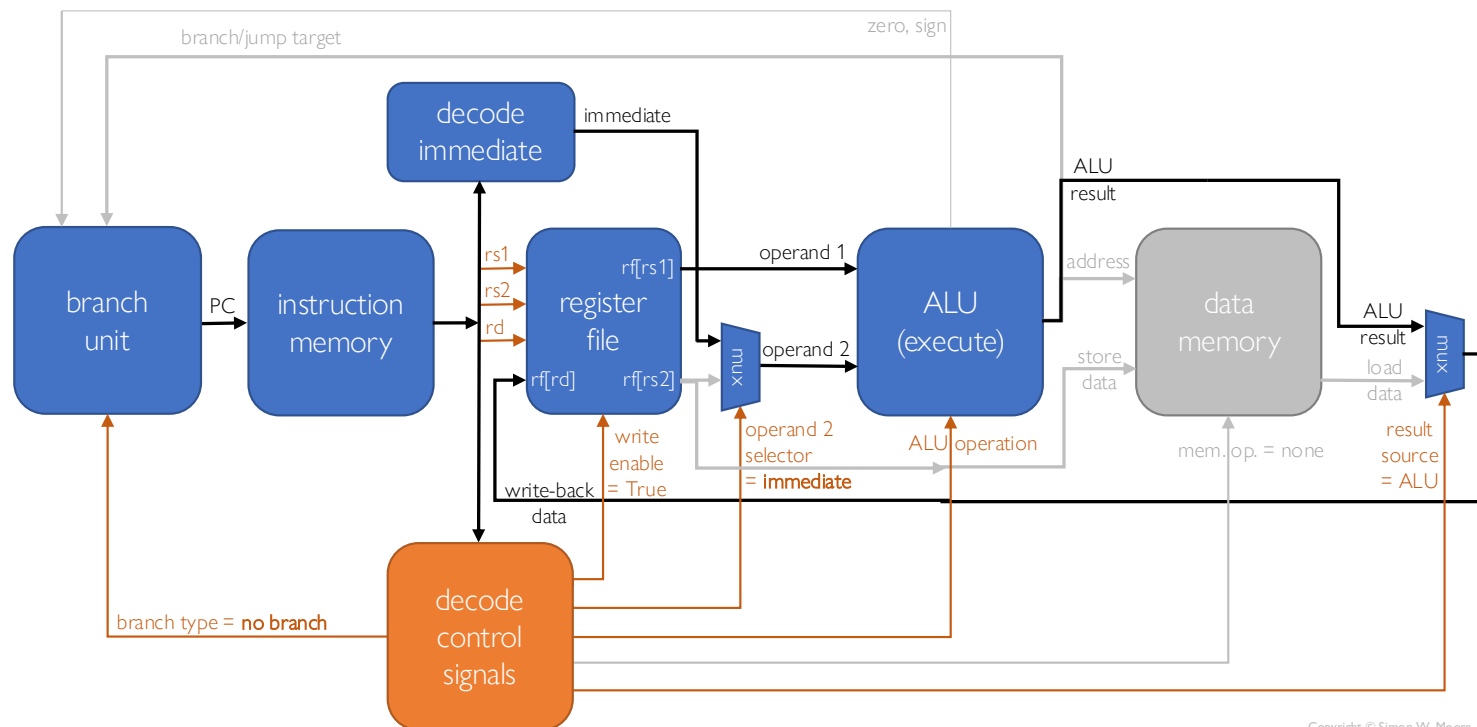
Data-path vs. control-path

- The data-path is the blue boxes & black lines in the previous figure
 - Data flows through the data path, with some of the data coming from the instruction (immediates, etc)
- The control-path is the orange boxes & lines in the figure
 - The control-path signals are determined by the instruction and other microarchitectural state (e.g. result of ALU operation is zero or negative)
- Remember that hardware is physical!
 - The data-path is fixed and control determines how each part of the data-path is used to perform each instruction
 - Examples:
 - Two registers (rs1 and rs2) are read every time even if one or both are not needed since that is the fastest option
 - ADD, ADDI, LW and SW all cause the ALU to do addition, but of these only LW and SW access data memory

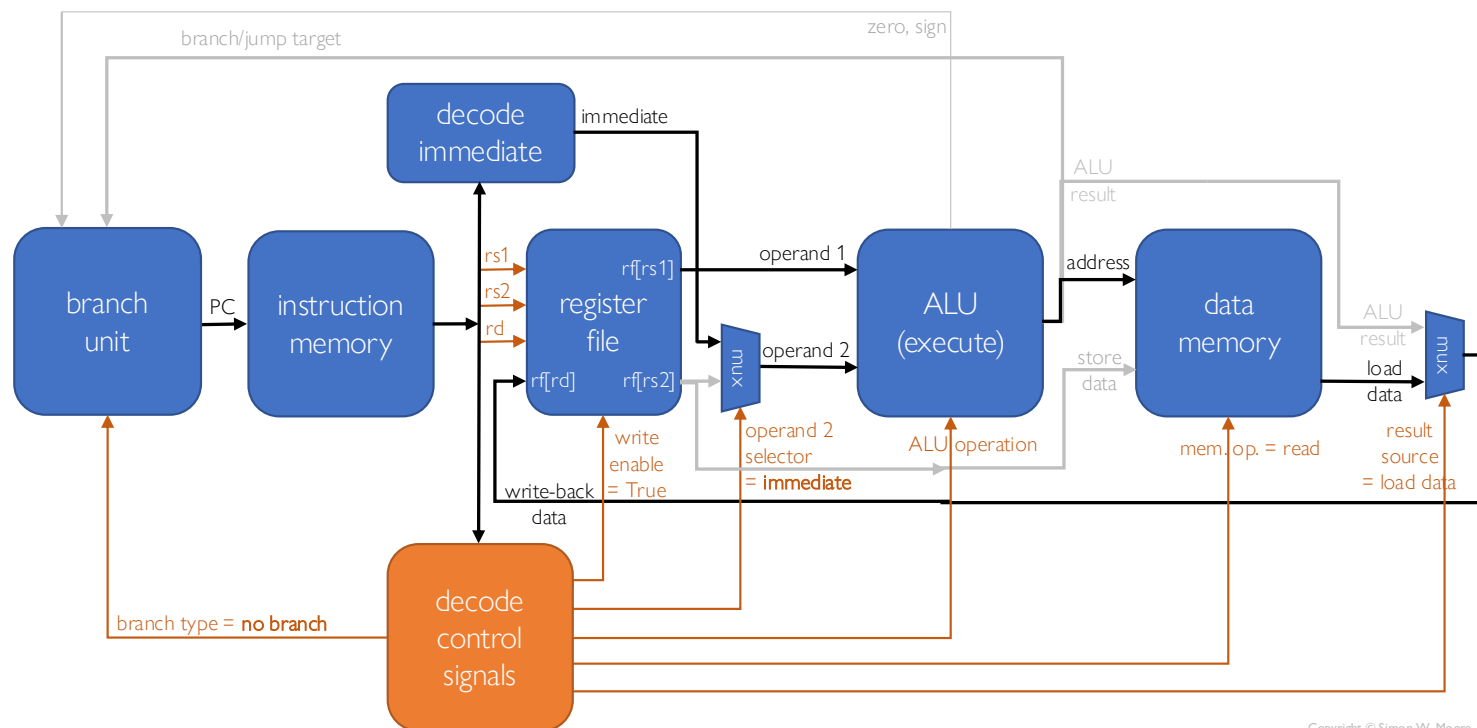
Influence of control for R-type instructions



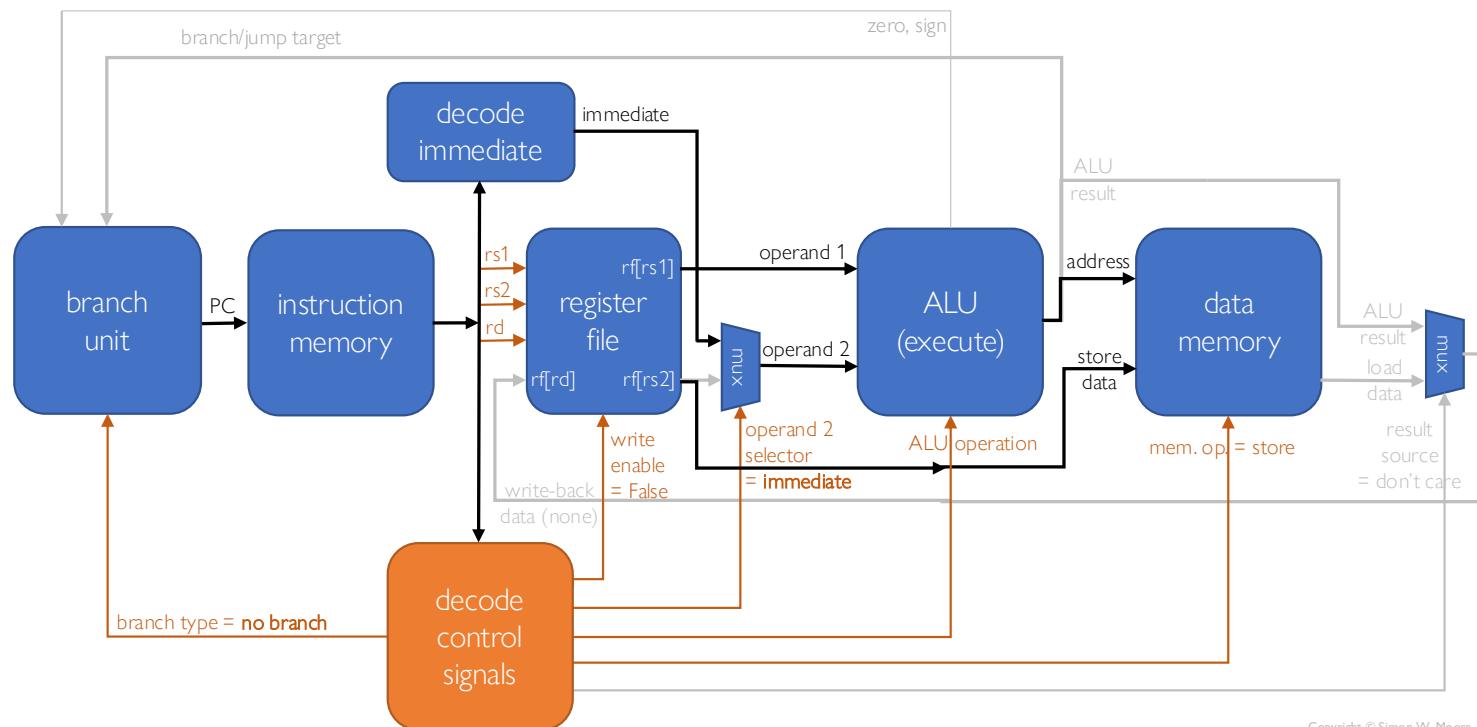
Influence of control for I-type arith. instructions



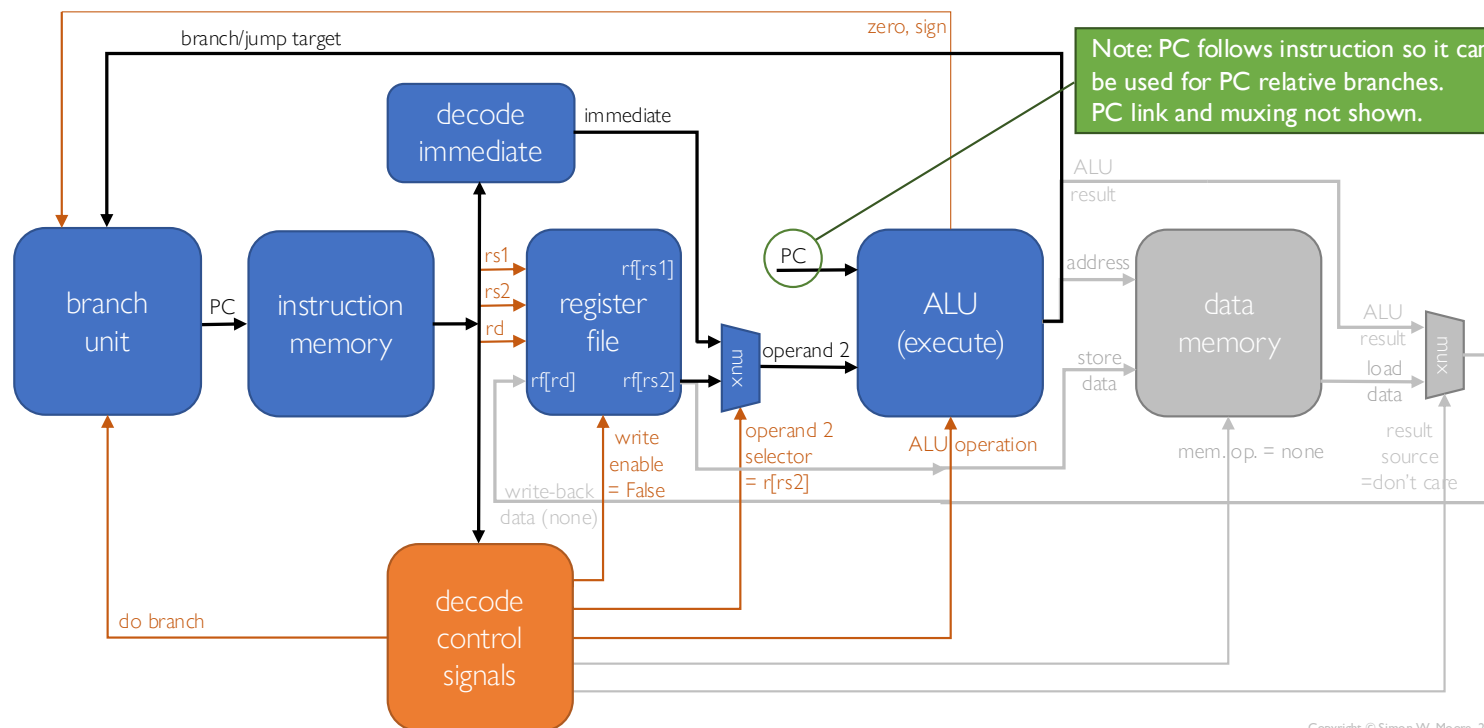
Influence of control for I-type load instructions



Influence of control for S-type store instructions



Influence of control for B-type instructions





Performance issues

- Longest delay determines clock period
 - Critical path: load instruction
 - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle
 - Making the common case fast
- We will improve performance by pipelining

Pipelining analogy: the assembly line

Key points:

- Assembling one device may not be much faster, but you can start assembling a new device far more frequently
- Each stage of assembly is specialised and optimised
- The assembly line can only go as fast as the slowest operation



https://en.wikipedia.org/wiki/Assembly_line#/media/File:Ford_assembly_line_1913.jpg Copyright © Andrew W. Moore, 2025



Example RISC-V pipeline

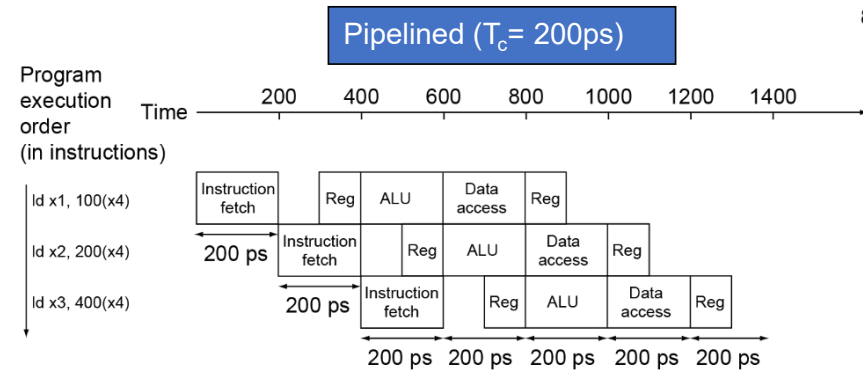
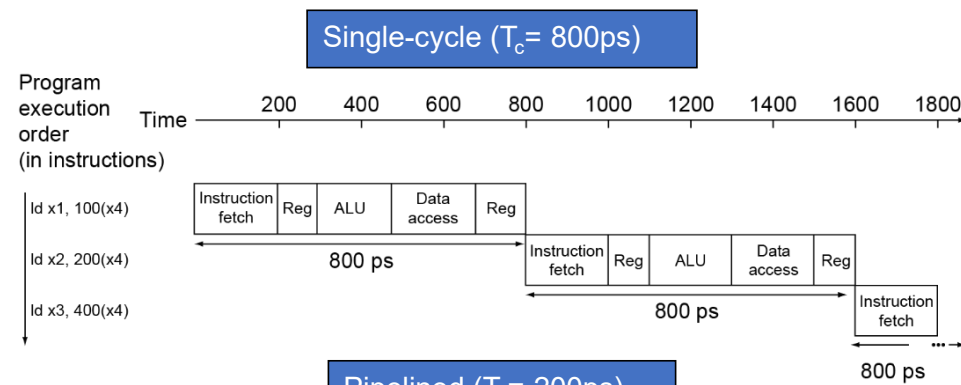
- Five stages, one step per stage
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register fetch
 3. EX: Execute operation or calculate address
 4. MEM: Memory access
 5. WB: Write back result to register
- This is referred to as a **scalar pipeline**, i.e. it is a simple linear pipeline that fetches one instruction at a time and sends it down the pipeline to be executed

Pipeline Performance

- Assume time for stages is:
 - 100ps for register read or write
 - 200ps for other stages
- Compare pipelined data path with single-cycle data path

Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100ps	200ps	200ps	100ps	800ps
sw	200ps	100ps	200ps	200ps		700ps
R-type	200ps	100ps	200ps		100ps	600ps
beq	200ps	100ps	200ps			500ps

Pipeline performance



Pipeline speedup

- If all stages are balanced
 - i.e., all take the same time
 - Time between instructions pipelined =
$$\frac{\text{Time between instructions nonpipelined}}{\text{Number of stages}}$$
- If not balanced, speedup is less
- Speedup due to increased throughput
 - Latency (time for each instruction) does not decrease

Pipelining and ISA design

- RISC-V ISA designed for pipelining
 - All instructions are 32-bits (or 16-bits if the compressed instructions are supported)
 - Easier to fetch and decode in one cycle
 - c.f. x86: 1- to 17-byte instructions
 - Few and regular instruction formats
 - Can decode and read registers in one step
 - Load/store addressing
 - Can calculate address in 3rd stage, access memory in 4th stage

Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structural hazards
 - A required resource is busy
- Data hazard
 - Need to wait for previous instruction to complete its data read/write
- Control hazard
 - Deciding on control action depends on previous instruction

Structural hazards

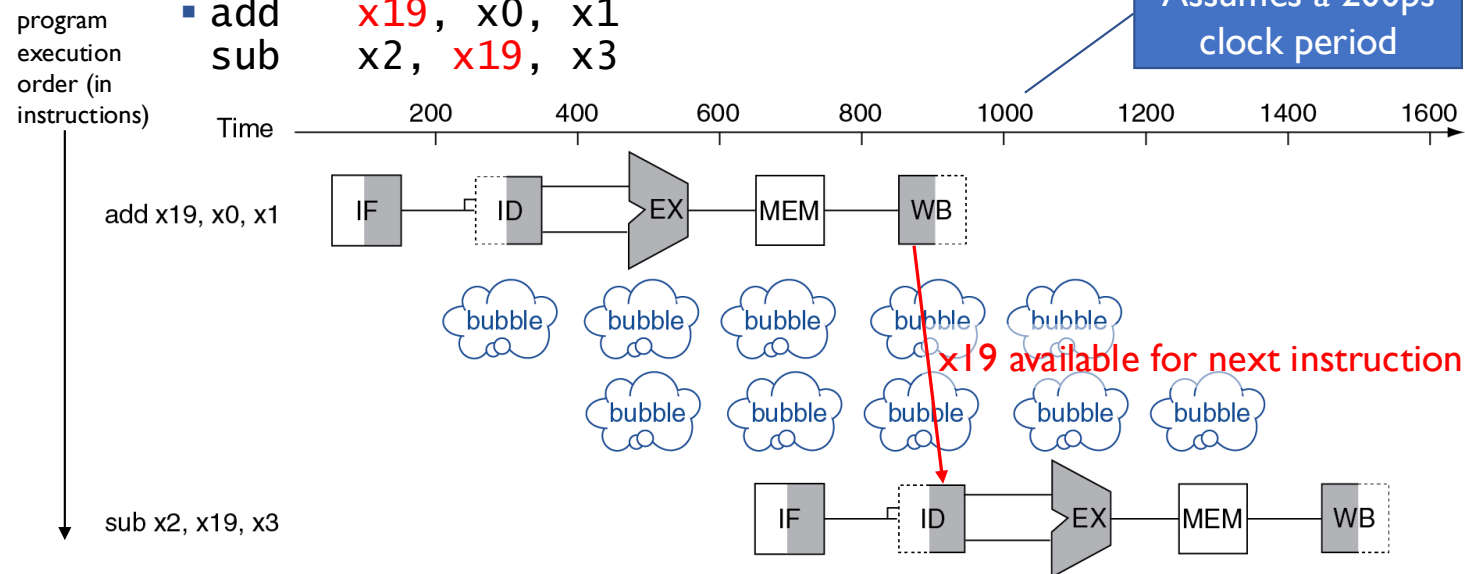
- Conflict for use of a resource
- In RISC-V pipeline with a single memory
 - Load/store requires data access
 - Instruction fetch would have to *stall* for that cycle
 - Would cause a pipeline “bubble”
- Hence, pipelined data paths require separate instruction/data memories
 - Typically separate instruction and data caches that get data from one memory hierarchy (see later lecture on caches)

Data hazards

- An instruction depends on completion of data access by a previous instruction

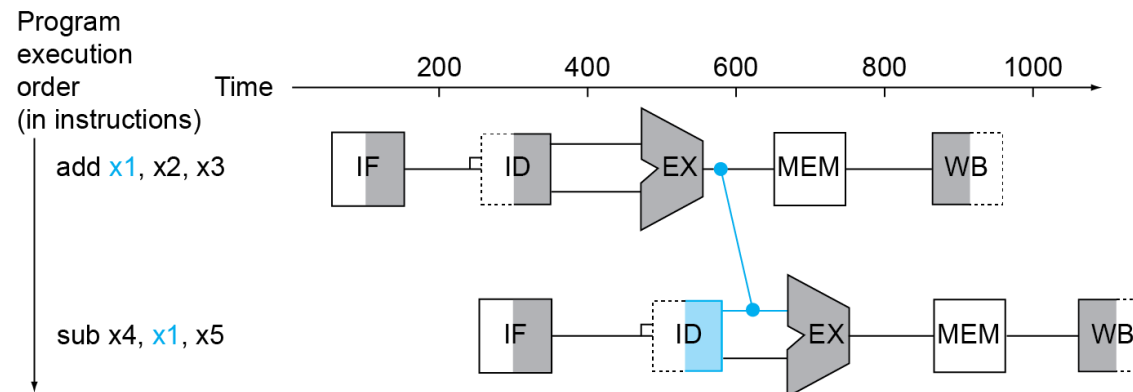
- add x19, x0, x1
- sub x2, x19, x3

Assumes a 200ps
clock period



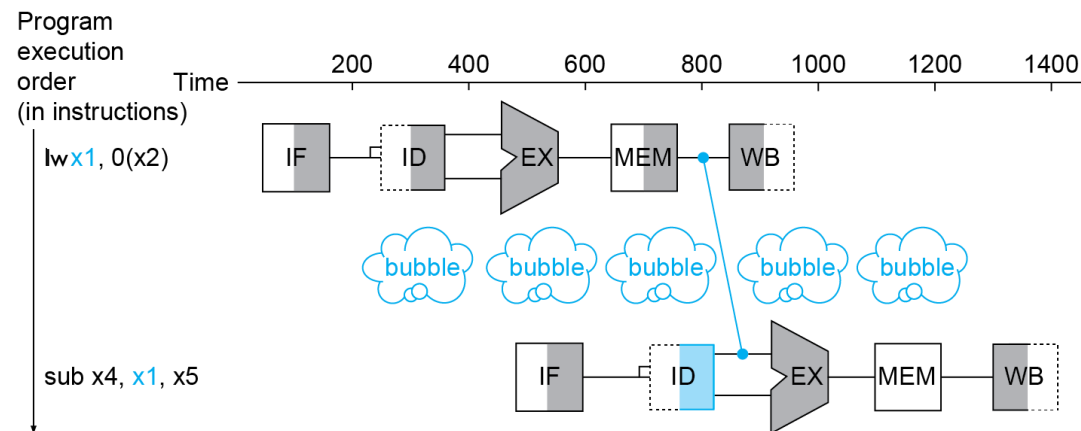
Forwarding (aka Bypassing)

- Use result when it is computed
 - Don't wait for it to be stored in a register
 - Requires extra connections in the datapath



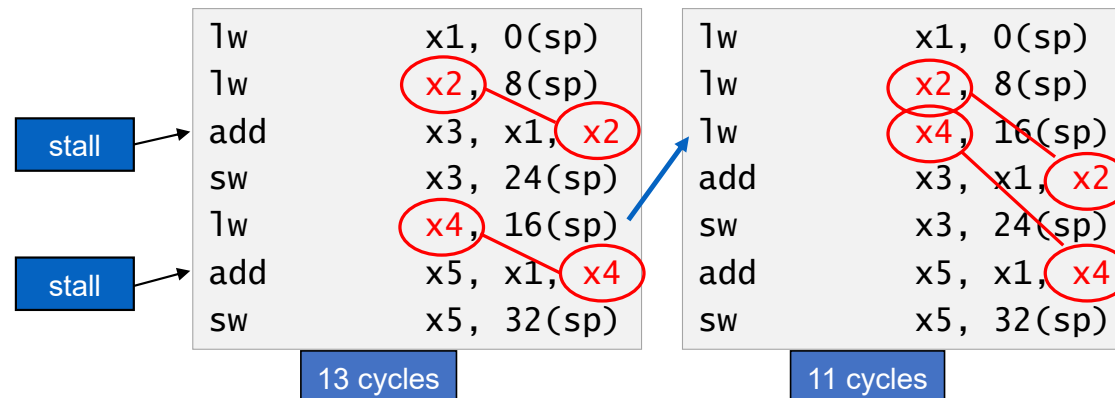
Load-use data hazard

- Can't always avoid stalls by forwarding
 - If value not computed when needed
 - Can't forward backward in time!



Code scheduling to avoid stalls

- Reorder code to avoid use of load result in the next instruction
- C code (variables stored on the stack)
a = b + e;
c = b + f;

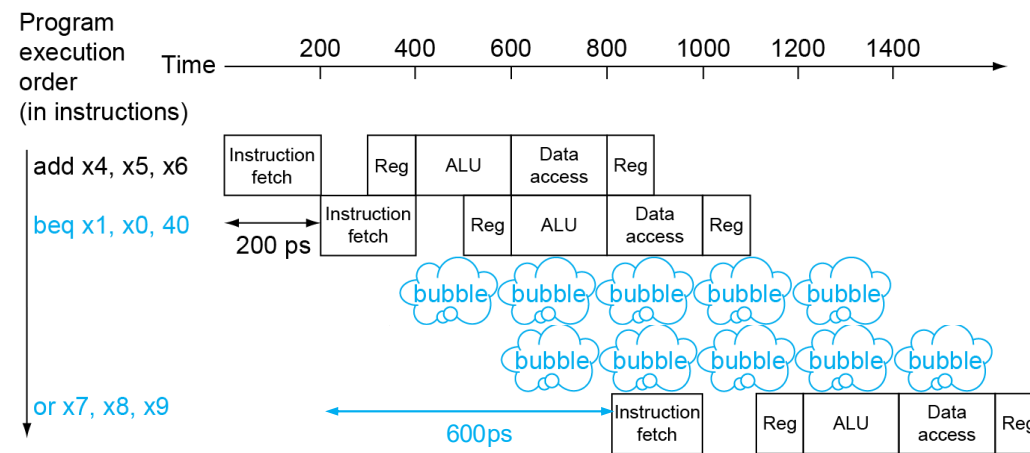


Control hazards

- Branch determines the flow of control
 - Fetching the next instruction depends on the branch outcome
 - The pipeline cannot always fetch correct instruction
 - Still working on ID stage of branch
- In a RISC-V pipeline
 - Unconditional jumps can be undertaken in the ID stage
 - Conditional branches are dependent upon register comparisons typically done in the EX stage
 - Indirect jumps (jump to an address in a register) can also be handled by the EX stage

Stall on Branch

- Wait until branch outcome determined before fetching next instruction



Branch Prediction

Much more in Part II
Comparative Architectures

- Longer pipelines can't readily determine branch outcome early
 - Stall penalty becomes unacceptable
- Predict outcome of branch
 - Only stall if prediction is wrong
 - If prediction is wrong, state must be cleaned up to preserve the sequential execution model
- Static branch prediction
 - Based on typical branch behavior
 - Example: loop and if-statement branches
 - Predict backward branches taken
 - Predict forward branches not taken
- Dynamic branch prediction
 - Hardware measures actual branch behavior
 - e.g., record recent history of each branch
 - Assume future behavior will continue the trend
 - When wrong, stall while re-fetching, and update history

Summary

- Pipelining improves performance by increasing instruction throughput
 - Executes multiple instructions in parallel
 - Each instruction has the same latency
- Subject to hazards
 - Structure, data, control
- Instruction set design affects the complexity of pipeline implementation

Introduction to Computer Architecture

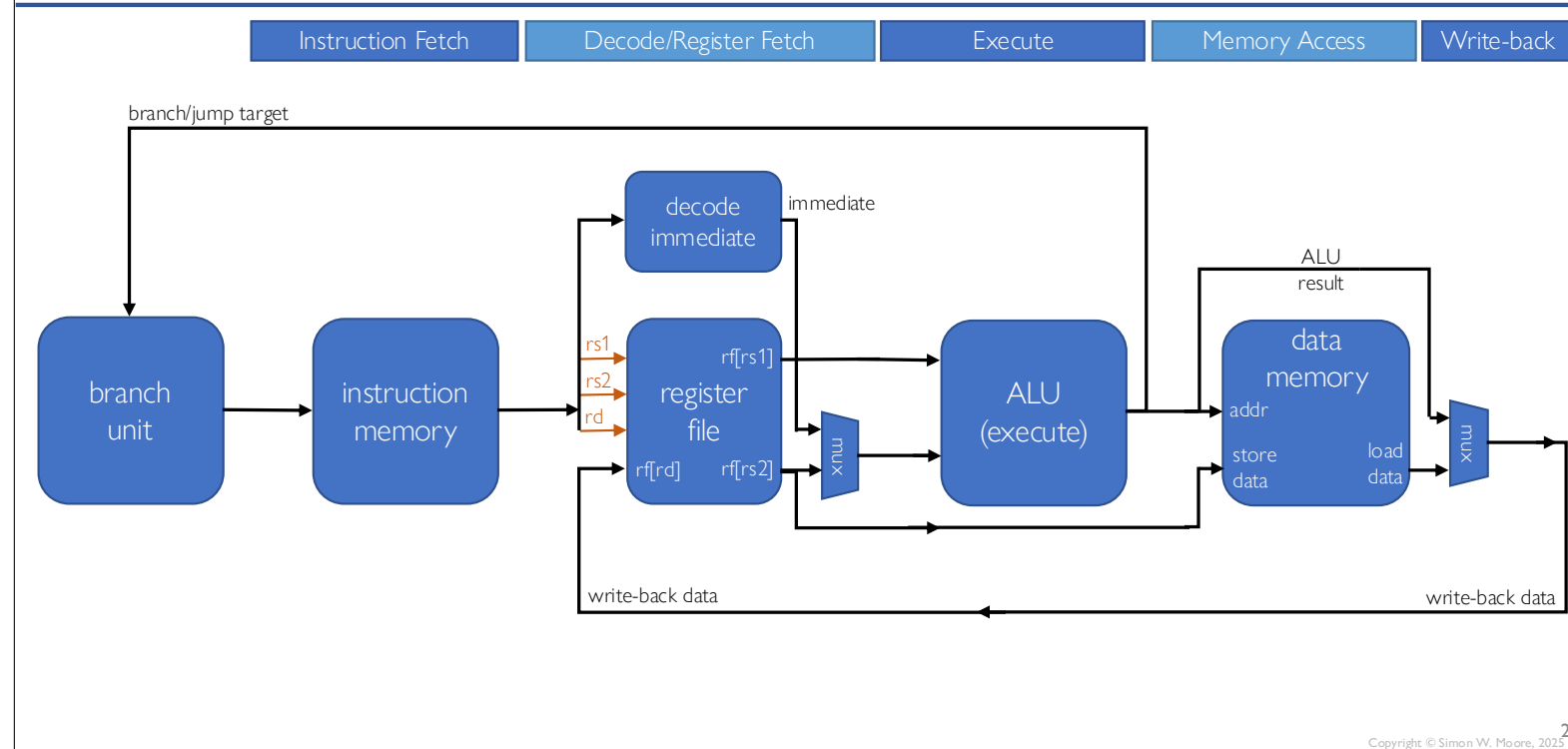
Lecture 7

More Processor Pipelines

Prof. Simon W. Moore

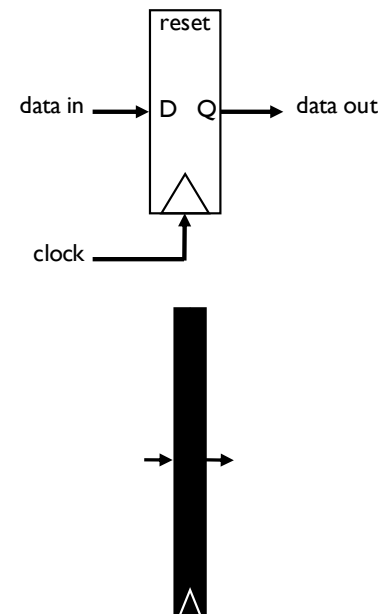


CPU overview from last lecture

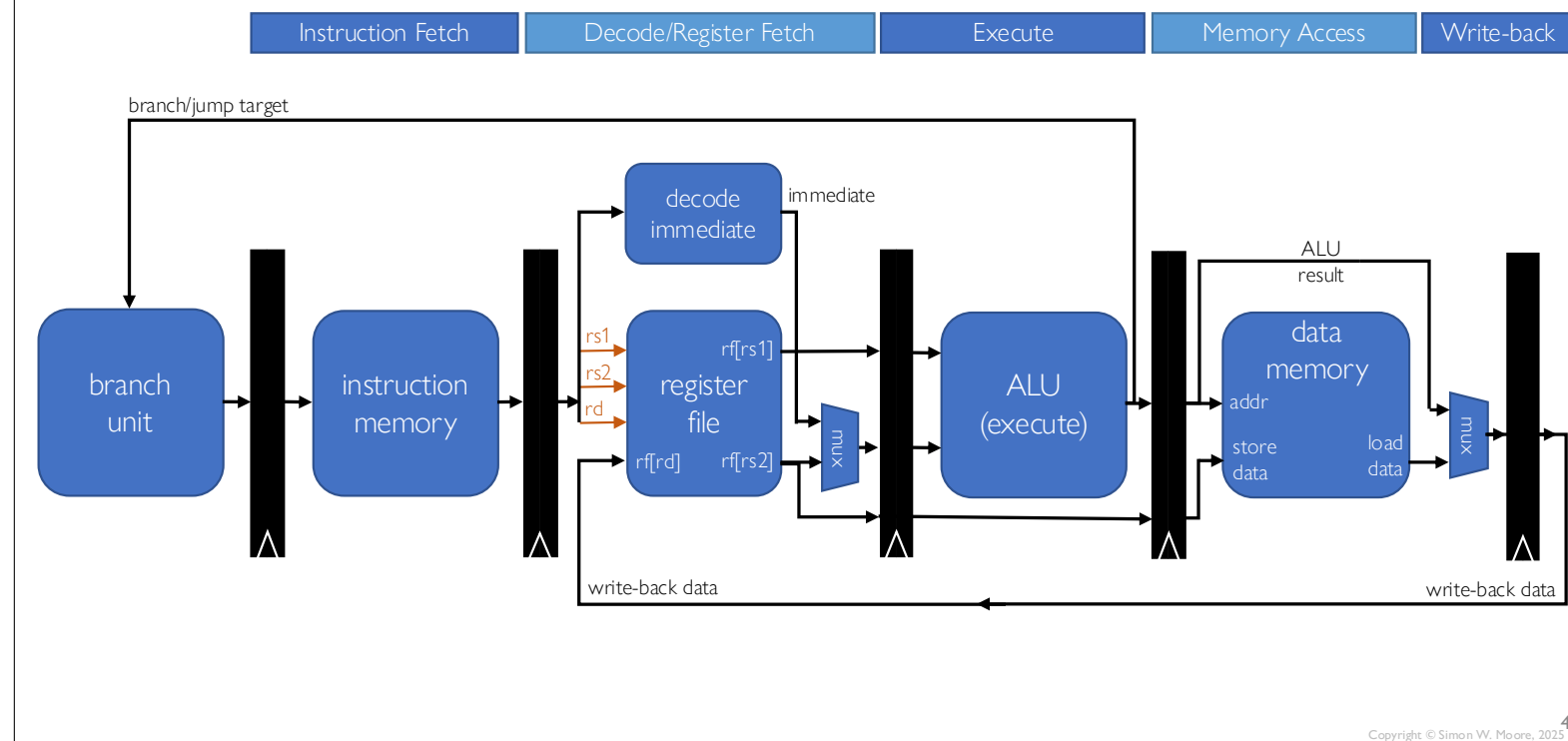


Reminder: D flip-flops

- We will use D flip-flops (DFFs) to control the flow of data in our pipeline
- DFF symbol from lecture 2
- I will abstract to the following where D is on the left, Q on the right
- The block represents a whole bank of DFFs, one for each bit of data
- Clock and reset need to be connected but I won't draw the wires



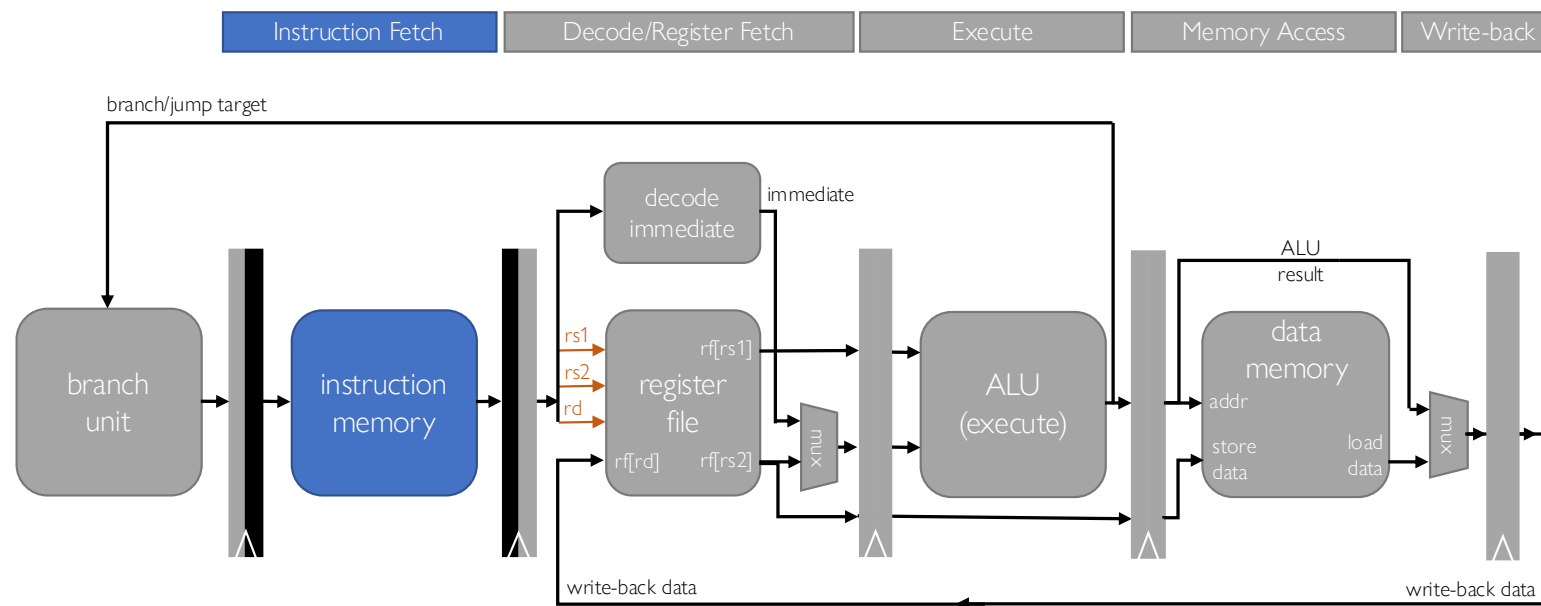
Adding pipeline latches: 5-stage pipeline



Pipeline operation

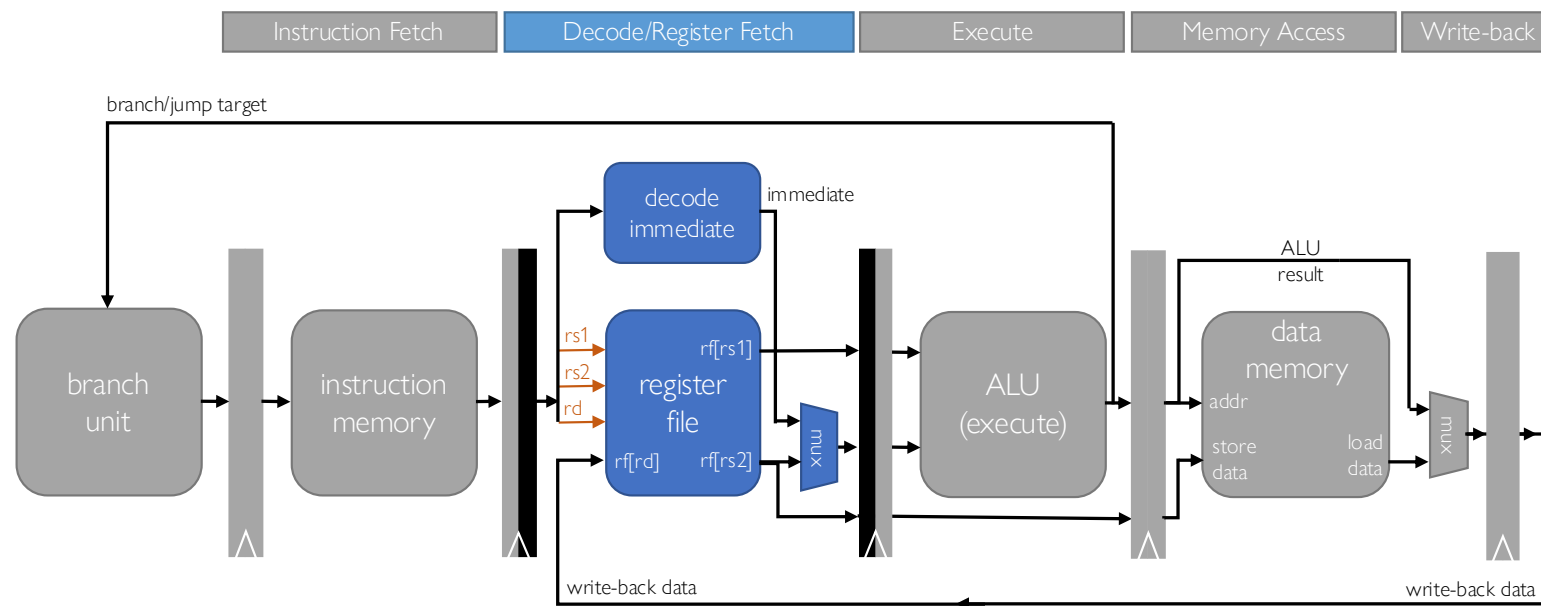
- First, we'll look at executing one instruction at a time
 - Like an assembly line where you complete manufacture of one item before starting on the next
- Then we will look at pipelined operation
 - Like an assembly line, we want to keep fetching instructions while in parallel we do decode, execute, memory access and write-back steps for other instructions
- Objectives:
 - Try to fetch a new instruction every clock cycle
 - Try to keep all stages of the pipeline busy
 - Preserve the programmer's model of sequential execution

Instruction fetch for Load/Store



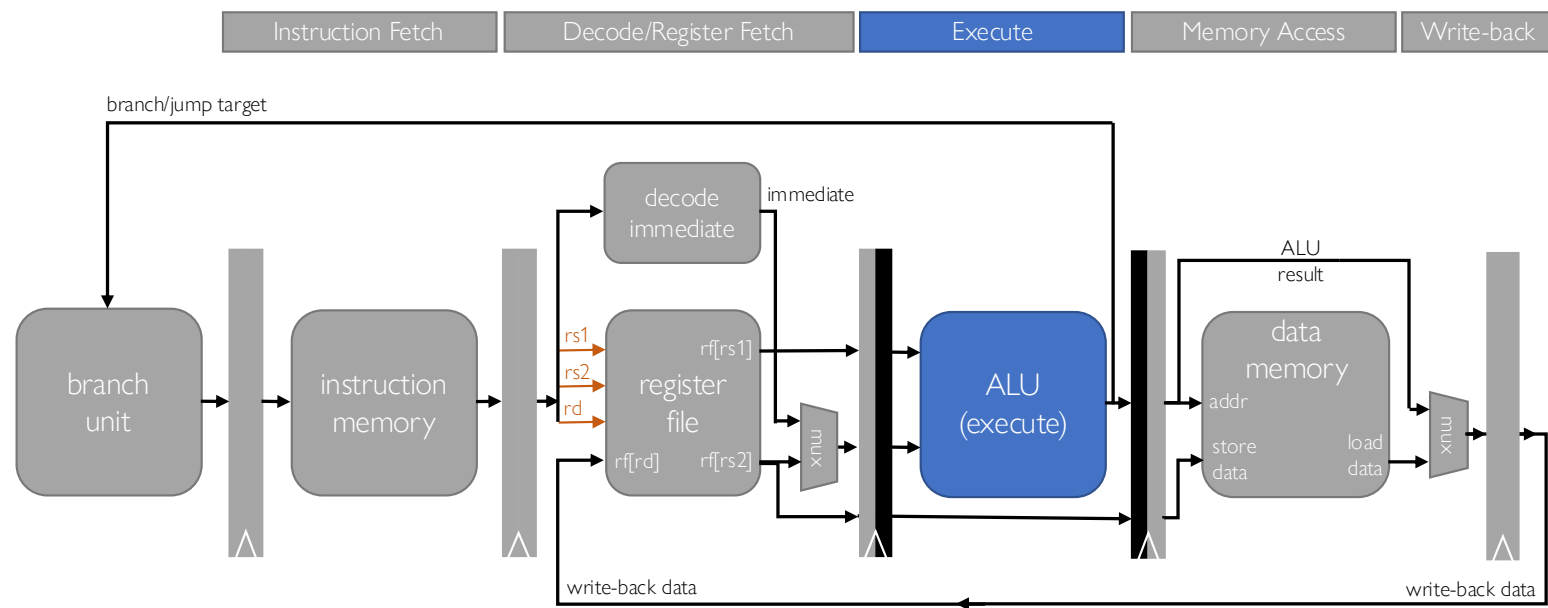
- Operation: fetch instruction from memory

Instruction decode for Load/Store



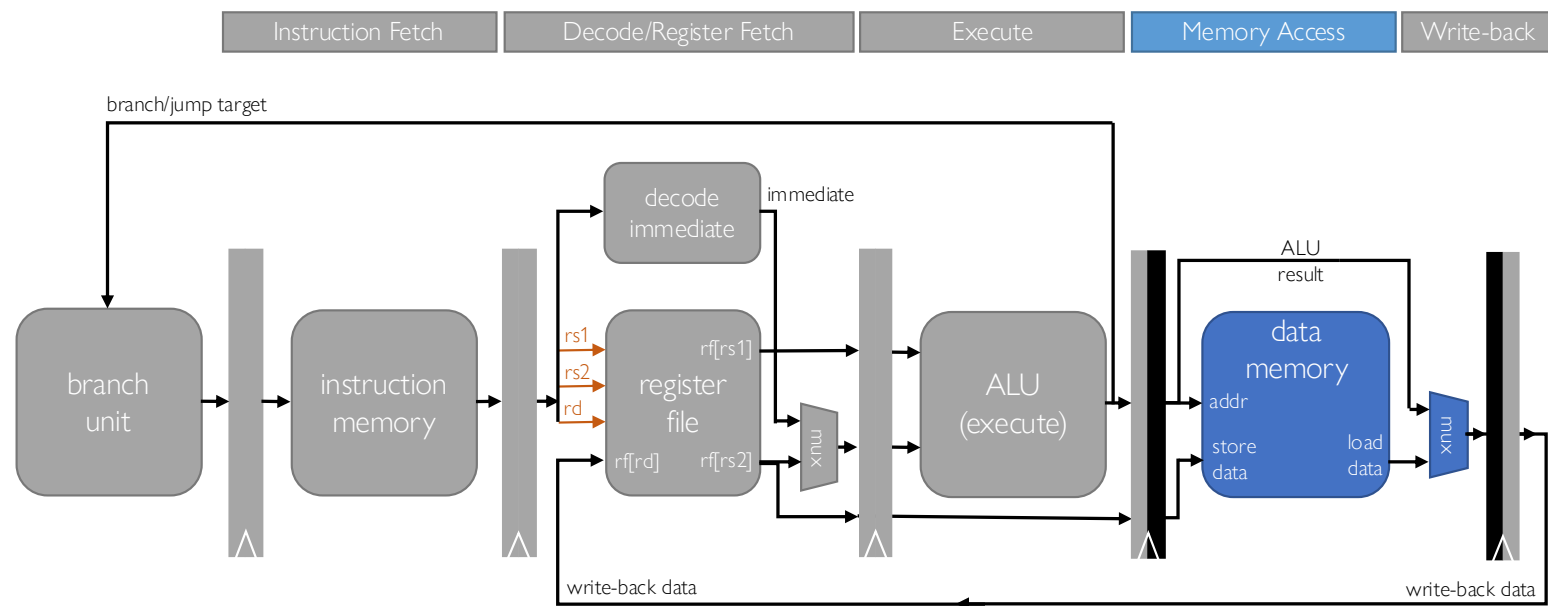
- Operation: fetch registers and decode the immediate

Execute for Load/Store



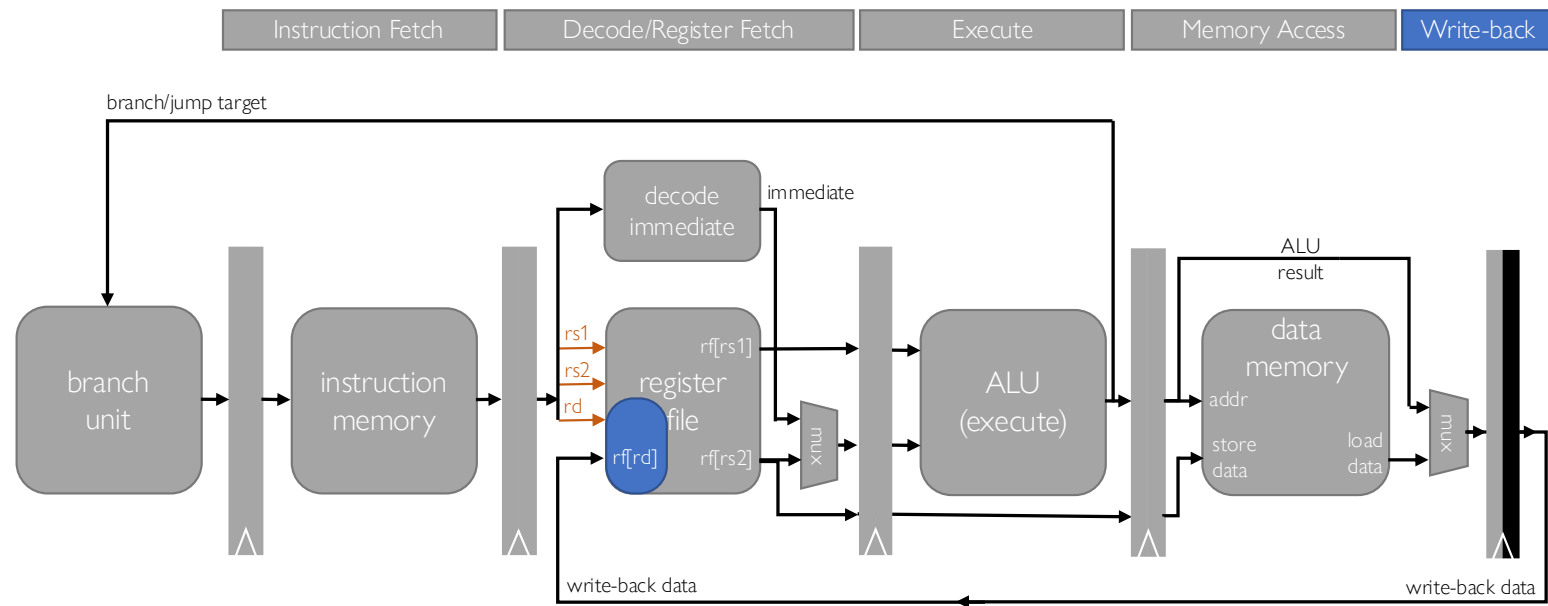
- Operation: address calculation: $\text{address} = rf[rs1] + \text{immediate}$

Memory access for Load/Store



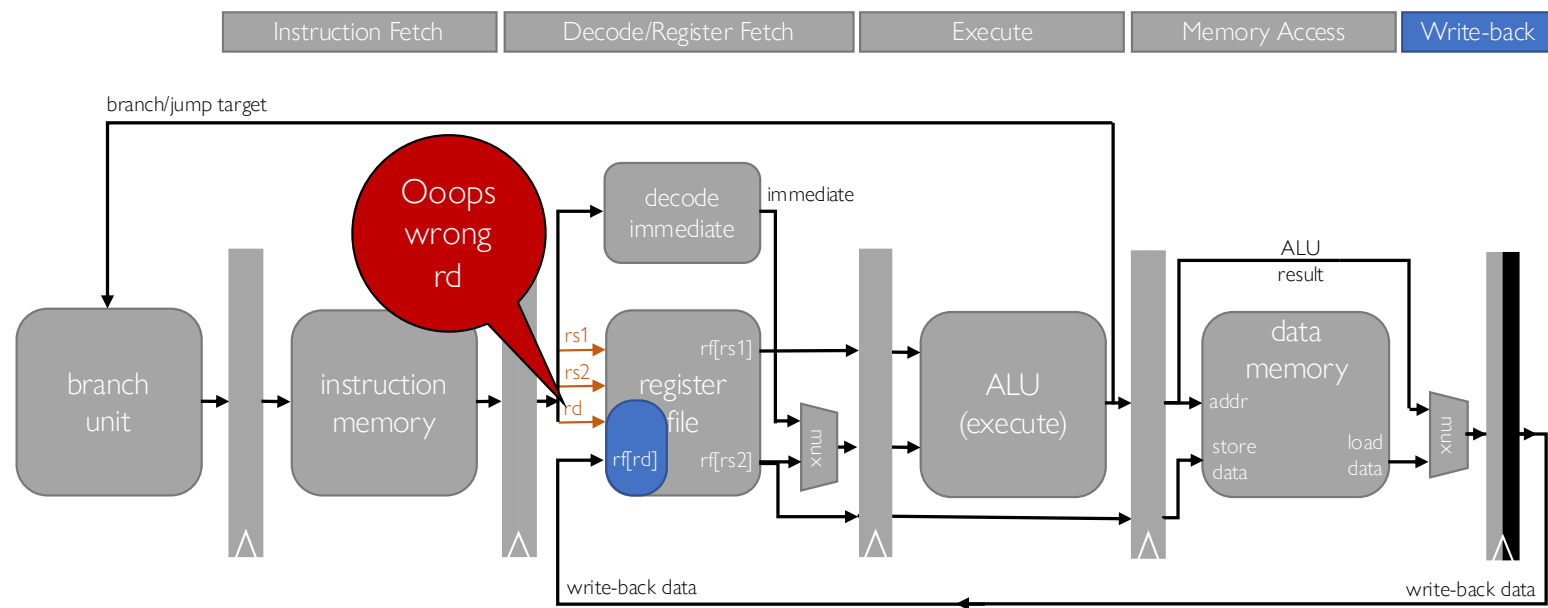
- Store writes $rf[rs2]$ to the data memory at computed address
- Load reads data from computed address

Write-back for Load (not Store)



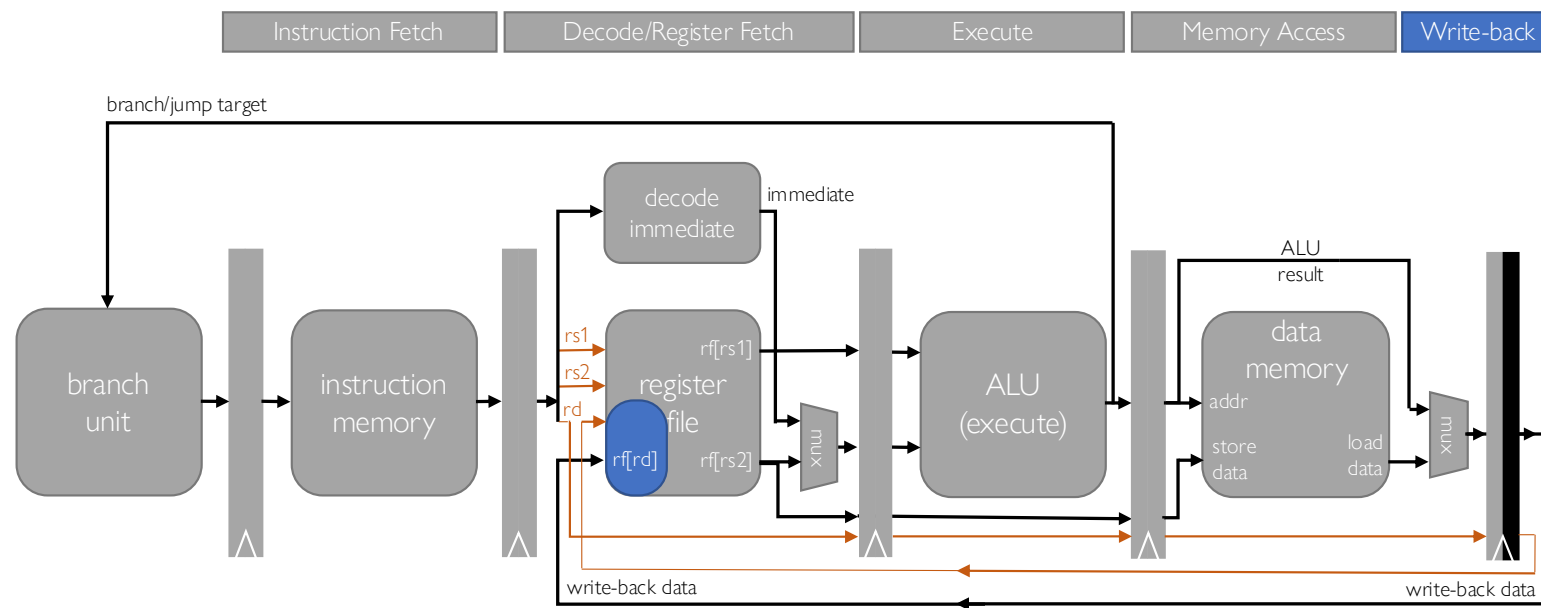
- Operation: write loaded data back to the register file

Pipelining issue from write-back for Load



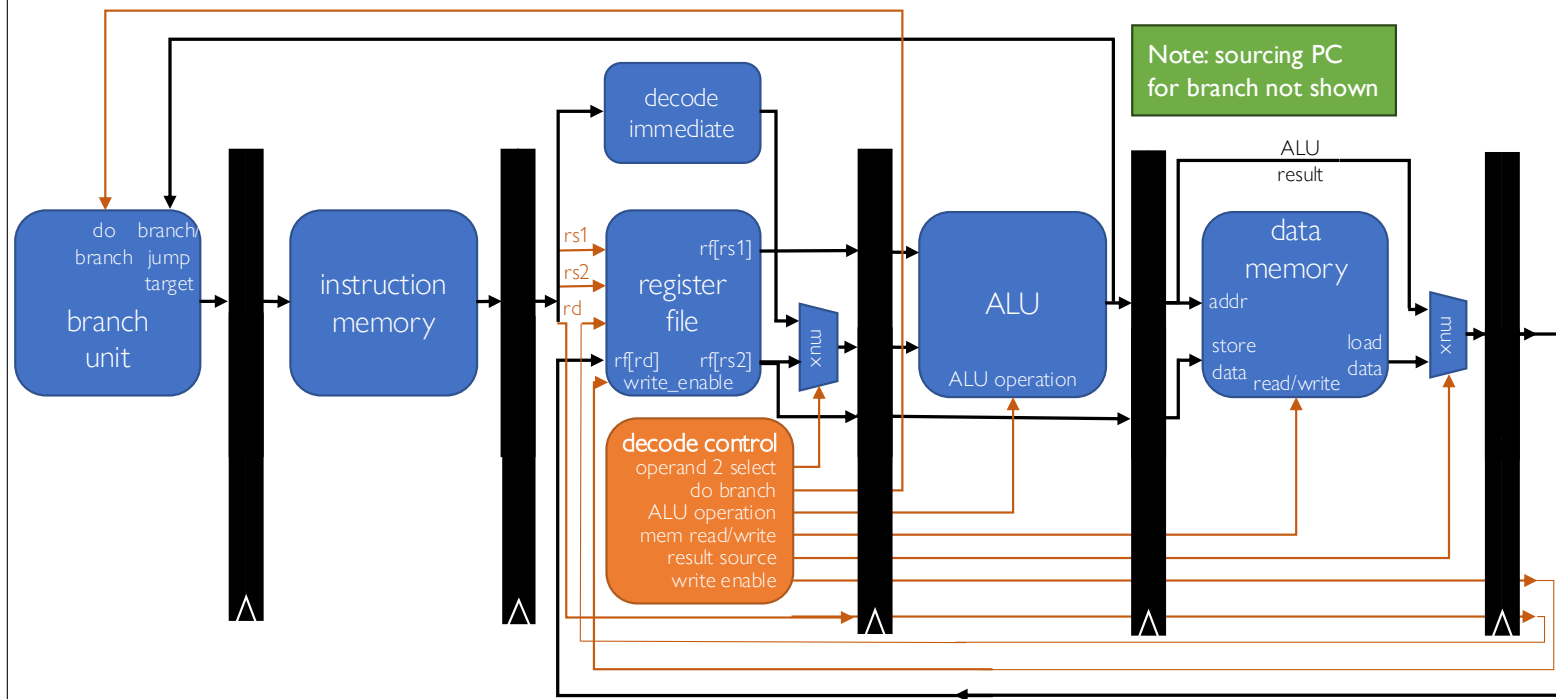
- Operation: write loaded data back to the register file

Fix pipelining issue from write-back for Load



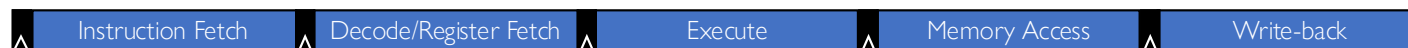
- Idea: propagate control information along the data-path

Pipelined control: control flows with data-path



Pipeline hazards

- The challenge:
 - We want to preserve the illusion of a sequential programming model
 - But we want to run the code fast
 - Hazards (problems) exhibited by our pipeline:
 - Data hazards: the risk of stale data being used
 - Control hazards: we fetch the wrong instructions and start executing them
- Examples follow
- To illustrate issues, we're going to abstract the pipeline to:

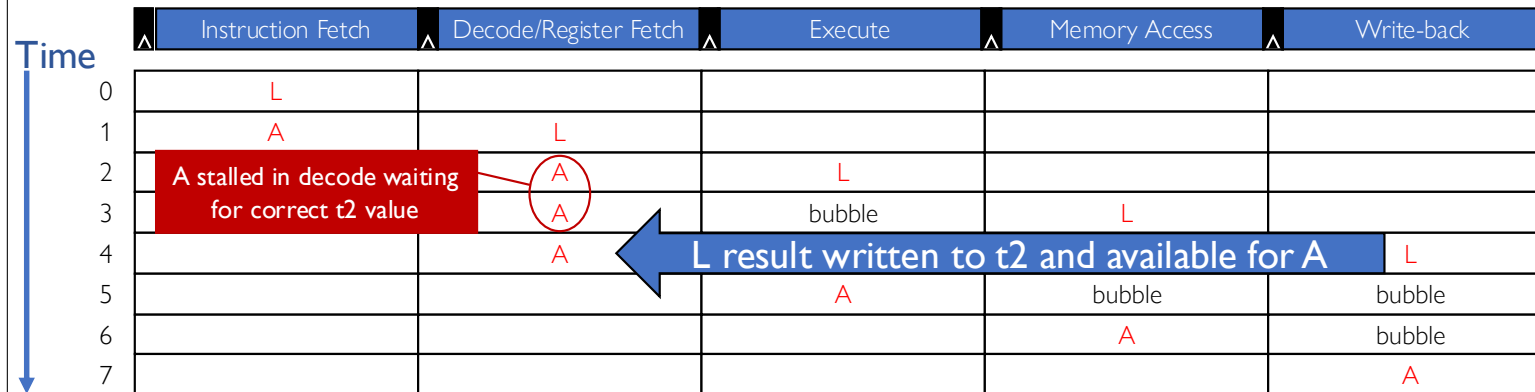


Load to use data hazard

- Example code:

lw t2, 0(t1) # L: t2=load(rf[t1]+0)
add t3, t3, t2 # A: t3=t3+t2

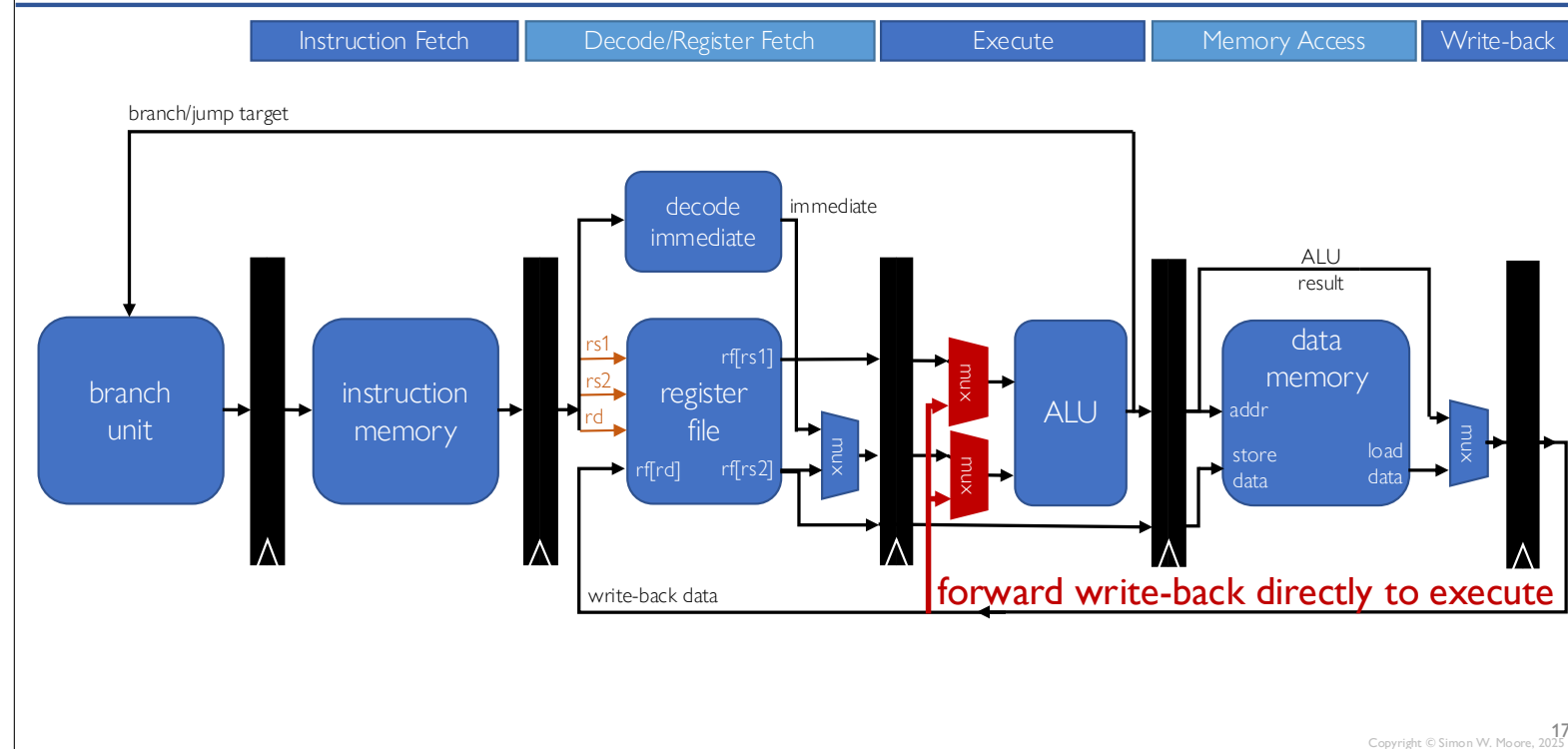
- This has a load-to-use dependency on register t2



Removing pipeline bubbles using forwarding

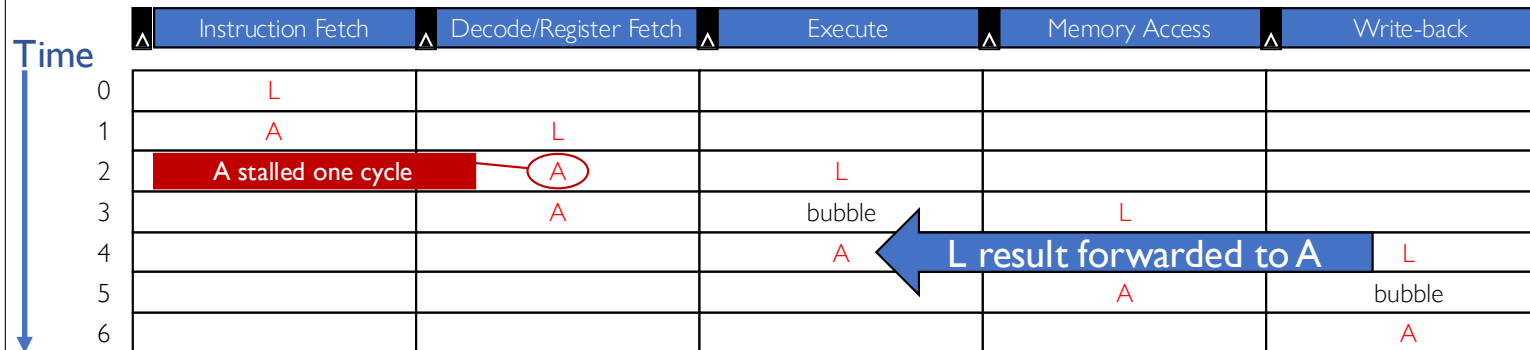
- Objective:
 - Remove as many pipeline bubbles as possible to speed things up
- Approach:
 - Forward results from the producer pipeline stage to the consumer stage
 - This is called **forwarding**

Forwarding result from Load



Load to use data hazard after forwarding

- Example code (as before):
`lw t2, 0(t1) # L: t2=load(rf[t1]+0)`
`add t3, t3, t2 # A: t3=t3+t2`
- This has a load-to-use dependency on register `t2`



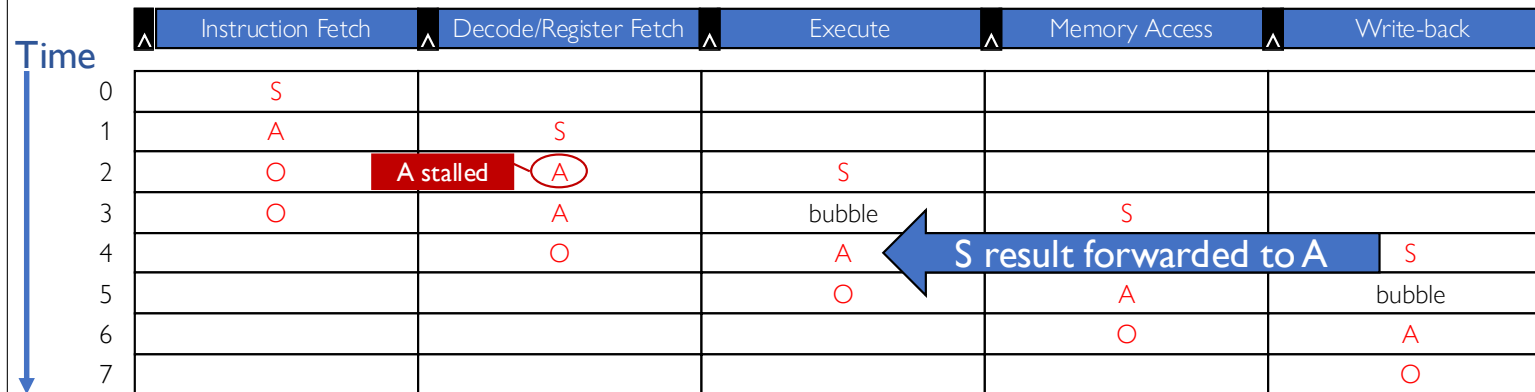
- Forwarding removes one of the two bubbles, saving one clock cycle

Generalising Load-to-use hazard avoidance

- There is always a load-to-use hazard
 - Often presented to programmers/compiler writers as a **load delay slot**
- More complex high performance processors typically have more complex pipelining, e.g. data memory may take at least three clock cycles to access
 - So the Load-to-use hazard is greater: a bigger load delay slot
- General compiler optimisation: **Load hoisting**
 - Aim to issue Load instructions as early as possible
 - Example given in previous lecture

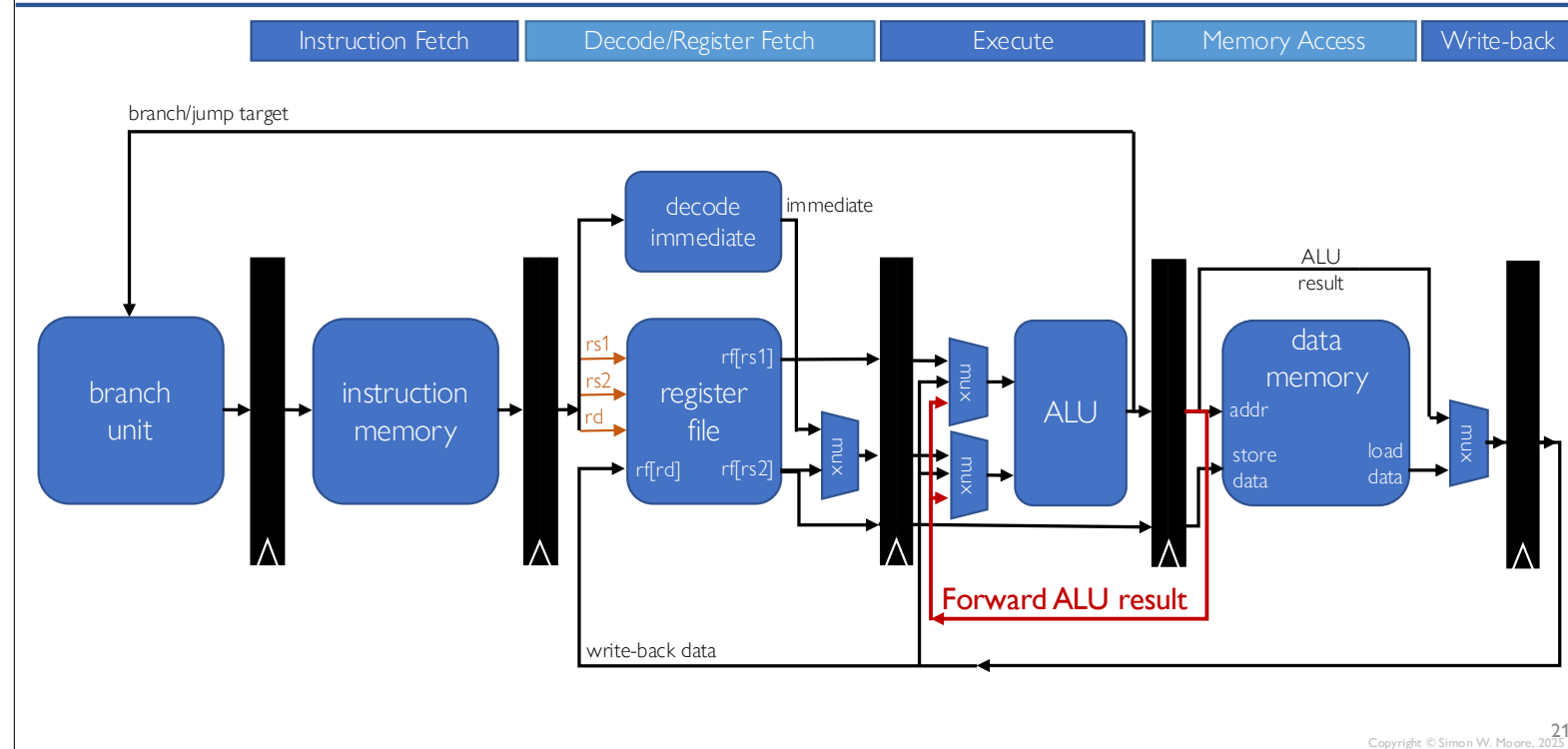
Execute-to-execute data hazard

- Example code:
 sub t1, t2, t3 # S: t1 = t2 - t3
 and t4, t1, t3 # A: t4 = t1 & t3
 or t5, t3, t1 # O: t5 = t3 | t1
- This has a dependencies on register t1



- How can we remove the bubble due to the dependency S→A ?

Forwarding result from Execute

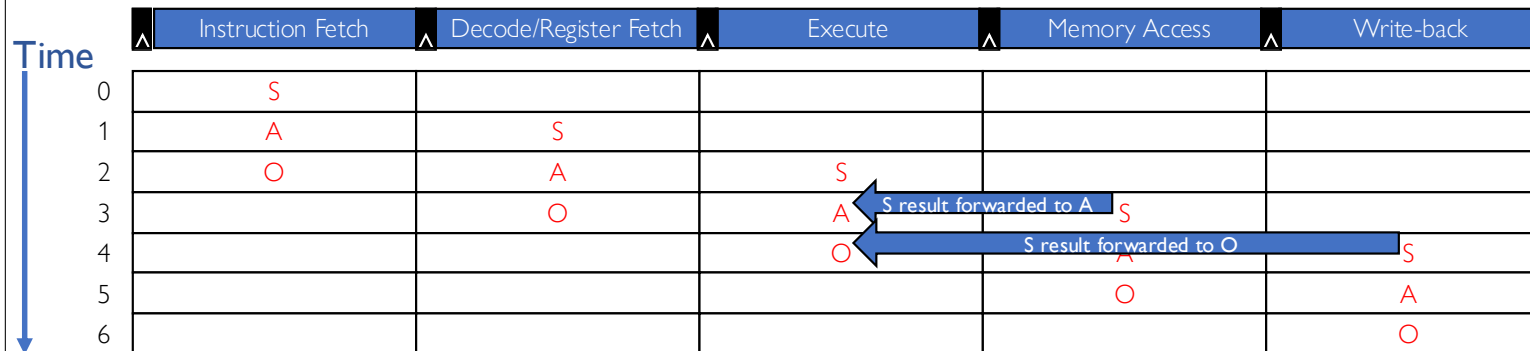


Using both forwarding paths

- Example code (as before):

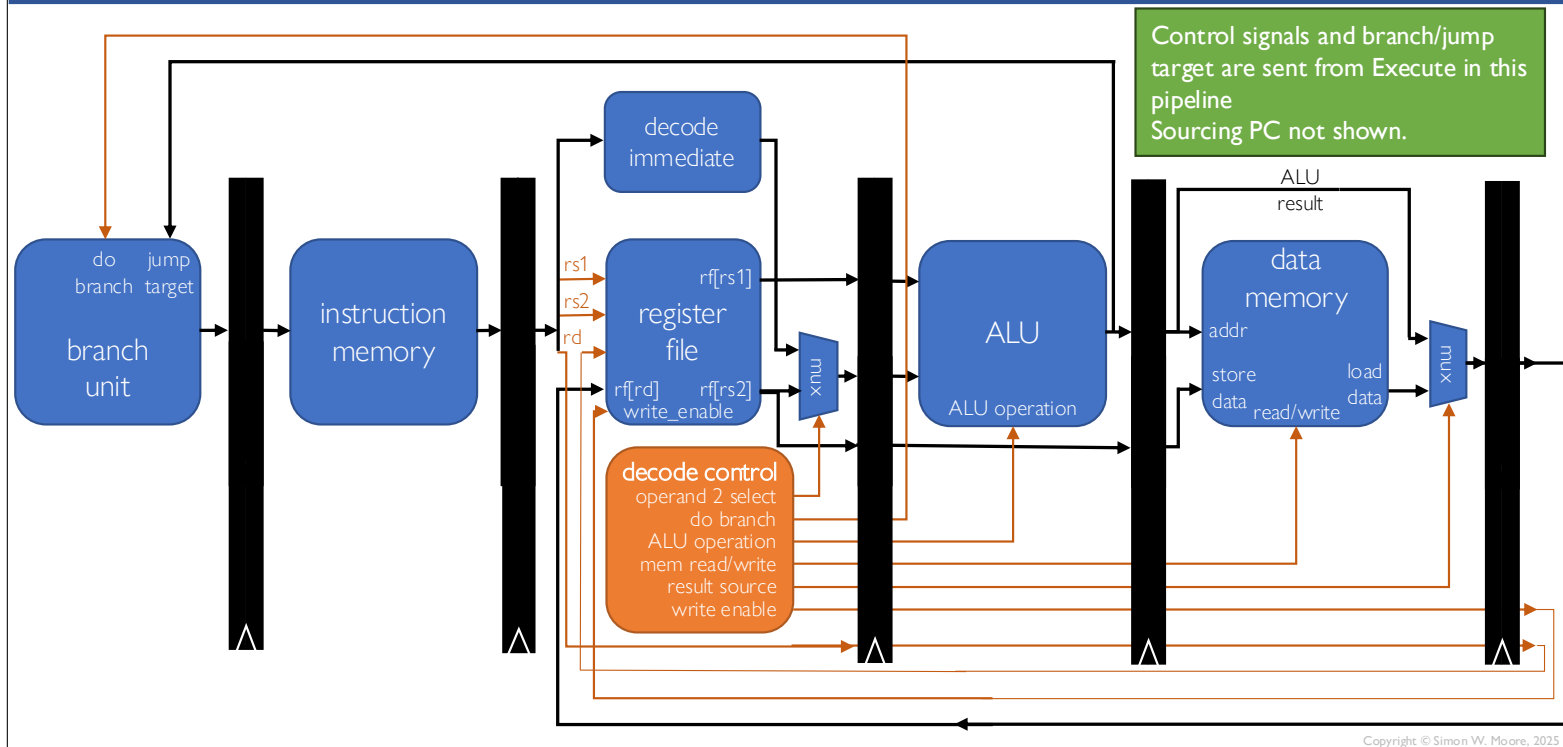
```
sub  t1, t2, t3    # S:  t1 = t2 - t3
and  t4, t1, t3    # A:  t4 = t1 & t3
or   t5, t3, t1    # O:  t5 = t3 | t1
```

- This has a dependencies on register t1



- For arithmetic operations, two forwarding paths eliminates all data hazards in this pipeline

Where are branches handled?



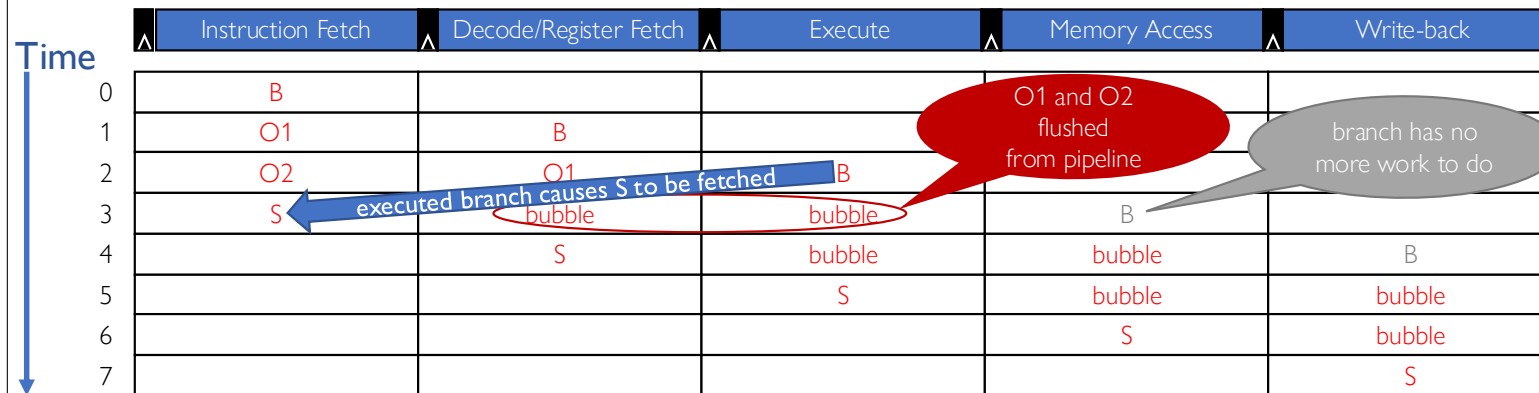
Branches cause control-flow hazards

Example code:

```

blt    t2, t1, skip    # B: if(t2<t1) goto skip
or     t3, zero, t2    # O1: t3 = t2
or     t2, zero, t1    # O2: t2 = t1
or     t1, zero, t3    # O3: t1 = t3
skip: sw    t1, 0(t4)   # S: store t1 at address 0+t4
    
```

branch taken in
this example



Notes on branches

- In the general case, several instructions may have been fetched in error before the pipeline realises that a branch should have happened
 - For this pipeline, two instructions were fetched in error
 - In a modern high performance processor, tens of instructions might have been fetched in error
- To avoid breaking the sequential execution model, we prevent the erroneously fetched instructions from completing
- Some approaches:
 - Flush the instructions from the pipeline, e.g. mark them as “invalid” or a bubble
 - Add an epoch number (a colour) to fetched instructions, and change the colour on a branch, causing instructions of the wrong colour to be ignored

Dynamic branch prediction

Much more in Part II
Comparative Architectures

- As mentioned last lecture, for more complex pipelines the branch penalty is more significant
 - E.g. in superscalar pipelines that aim to fetch and execute more than one instruction per cycle
- Use dynamic prediction
 - Branch prediction buffer (aka branch history table)
 - Indexed by recent branch instruction addresses
 - Stores outcome (taken/not taken)
 - To execute a branch
 - Check table, expect the same outcome
 - Start fetching from fall-through or target
 - If wrong, flush pipeline and flip prediction
- Branch target buffer
 - Cache of target addresses so that branch doesn't need to reach ID or EX to do prediction
 - Indexed by PC when instruction fetched
 - If hit and instruction is branch predicted taken, can fetch target immediately

Exceptions and interrupts

- “Unexpected” events requiring change in flow of control
 - Different ISAs use the terms differently
- Exception
 - Arises within the CPU
 - e.g., undefined opcode, syscall, ...
- Interrupt
 - From an external I/O controller
- Dealing with them without sacrificing performance is hard

Handling exceptions

- Save PC of offending (or interrupted) instruction
 - In RISC-V: Supervisor Exception Program Counter (SEPC)
- Save indication of the problem
 - In RISC-V: Supervisor Exception Cause Register (SCAUSE)
 - 32 bits, but most bits unused
 - Exception code field: 2 for undefined opcode, 12 for hardware malfunction, ...
- Jump to handler pointed to by STVEC register
- Precise/restartable exceptions (harder to implement)
 - Pipeline can flush the instruction and any earlier instructions in the pipeline
 - Handler executes, then returns to the instruction, which is refetched and executed
- Imprecise exceptions (easier to implement)
 - Pipeline flushes instructions but you cannot recover from the exception
 - e.g. illegal instruction exception (though you may want these to be precise if instruction emulation is to be performed)

Instruction-level parallelism (ILP)

Much more in Part II
Comparative Architectures

- Pipelining: executing multiple instructions in parallel
- To increase ILP
 - Deeper pipeline
 - Less work per stage $\Rightarrow$ shorter clock cycle
 - But many more forwarding paths and risk of bubbles due to hazards
 - Multiple issue
 - Replicate pipeline stages $\Rightarrow$ multiple pipelines
 - Start multiple instructions per clock cycle
 - $CPI < 1$, so use Instructions Per Cycle (IPC)
 - E.g., 4GHz 4-way multiple-issue
 - 16,000 MIPS, peak $CPI = 0.25$, peak $IPC = 4$
 - But dependencies reduce this in practice

Multiple issue

Much more in Part II
Comparative Architectures

- Static multiple issue
 - Compiler groups instructions to be issued together
 - Packages them into “issue slots”
 - Compiler detects and avoids hazards
- Dynamic multiple issue
 - CPU examines instruction stream and chooses instructions to issue each cycle
 - Compiler can help by reordering instructions
 - CPU resolves hazards using advanced techniques at runtime

Speculation

Much more in Part II
Comparative Architectures

- “Guess” what to do with an instruction
 - Start operation as soon as possible
 - Check whether guess was right
 - If so, complete the operation
 - If not, roll-back and do the right thing
- Common to static and dynamic multiple issue
- Examples
 - Speculate on branch outcome (branch prediction)
 - Roll back if path taken is different
 - Speculate on load
 - Roll back if location is updated

Side note:
Over the last couple
of years speculation
has been identified as
a security risk. See
Spectre, Meltdown,
Foreshadow, etc.

Fallacies

- Pipelining is easy (!)
 - The basic idea is easy
 - The devil is in the details
 - e.g., detecting and working around data and control hazards
- Pipelining is independent of technology
 - So why haven't we always done pipelining?
 - More transistors make more advanced techniques feasible
 - Pipeline-related ISA design needs to take account of technology trends
 - e.g., predicated instructions

Pitfalls

- Poor ISA design can make pipelining harder
 - e.g., complex instruction sets (VAX, x86-64)
 - Significant overhead to make pipelining work
 - x86-64 micro-op approach
 - e.g., complex addressing modes
 - Register update side effects, memory indirection
 - e.g., delayed branches (MIPS)
 - Advanced pipelines have long delay slots

Concluding remarks

- ISA influences design of data path and control
- Data path and control influence design of ISA
- Pipelining improves instruction throughput using parallelism
 - More instructions completed per second
 - Latency for each instruction not reduced
- Pipeline hazards covered: data, control
- Multiple issue and dynamic scheduling (ILP)
 - Dependencies limit achievable parallelism
 - Complexity leads to the power wall

Introduction to Computer Architecture

Lecture 8

Memory Hierarchy

Prof. Simon W. Moore

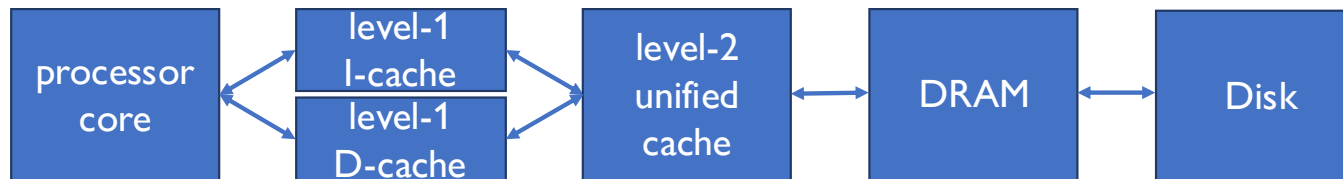
Motivation (from Lecture 1)

- *“Ideally one would desire an indefinitely large memory capacity such that any particular. . . word would be immediately available. . . We are. . . forced to recognize the possibility of constructing a hierarchy of memories, each of which has greater capacity than the preceding but which is less quickly accessible.”*

A. W. Burks, H. H. Goldstine and J. von Neumann

Preliminary Discussion of the Logical Design of an Electronic Computing Instrument (1946)

- Simplified memory hierarchy of a modern machine:



Memory technology characteristics

- SRAM on processor chip for on-chip caches
 - Low latency (proportional to size): 1 to 9 clock cycles
 - High bandwidth (depending on data width, e.g. 128-bits at 4GHz = 64GB/s)
 - Size limited by chip area (a few MiB) & can have many SRAM blocks
 - Read/write size can easily be byte-level
- DRAM off-chip
 - Latency: ~100 clock cycles
 - High bandwidth (e.g. ~25GiB/s per DDR4 channel (64-bit bus))
 - Reads/writes are in small blocks, transferred as bursts (rapid sequence of data)
 - DDR4: max 8 bursts of 64-bits = 64 bytes per transfer
- Magnetic hard disk or flash (SSD) for page-based virtual memory
 - High latency (~3ms magnetic disk, ~20μs for NVMe flash)
 - Lower bandwidth (<0.75GB/s for SATA, <16GB/s NVMe)
 - Reads/writes in larger blocks (512B) needed for extensive error detection and correction codes

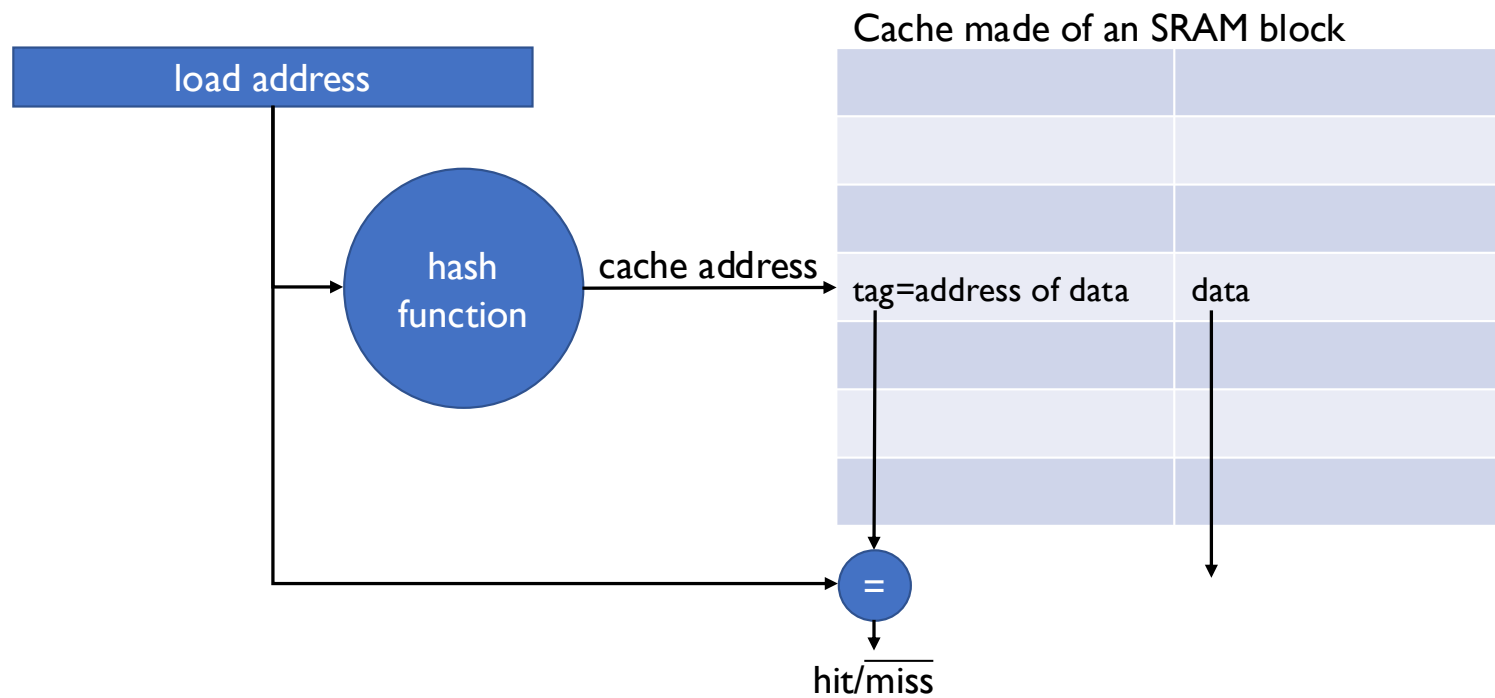
Principle of locality

- Programs access a small proportion of their address space at any time, giving rise to the following statistical properties
- Temporal locality
 - Items accessed recently are likely to be accessed again soon
 - e.g., instructions in a loop, loop counters and other repeatedly used variables
- Spatial locality
 - Items near those accessed recently are likely to be accessed soon
 - e.g., sequential instruction access, array data
 - Plays nicely with DRAM bursts: fetch data requested and also its neighbours

Cache terminology

- Cache = memory that holds recently used data so that it can be more rapidly accessed (low-latency) next time
 - to alleviate the von Neumann bottleneck
- Cache hit
 - data found in cache, so read
- Cache miss
 - data not in cache, so fetch from next level in the hierarchy
- Miss rate = $\frac{\text{num. cache misses}}{\text{num. memory accesses}}$

Crude direct-mapped cache

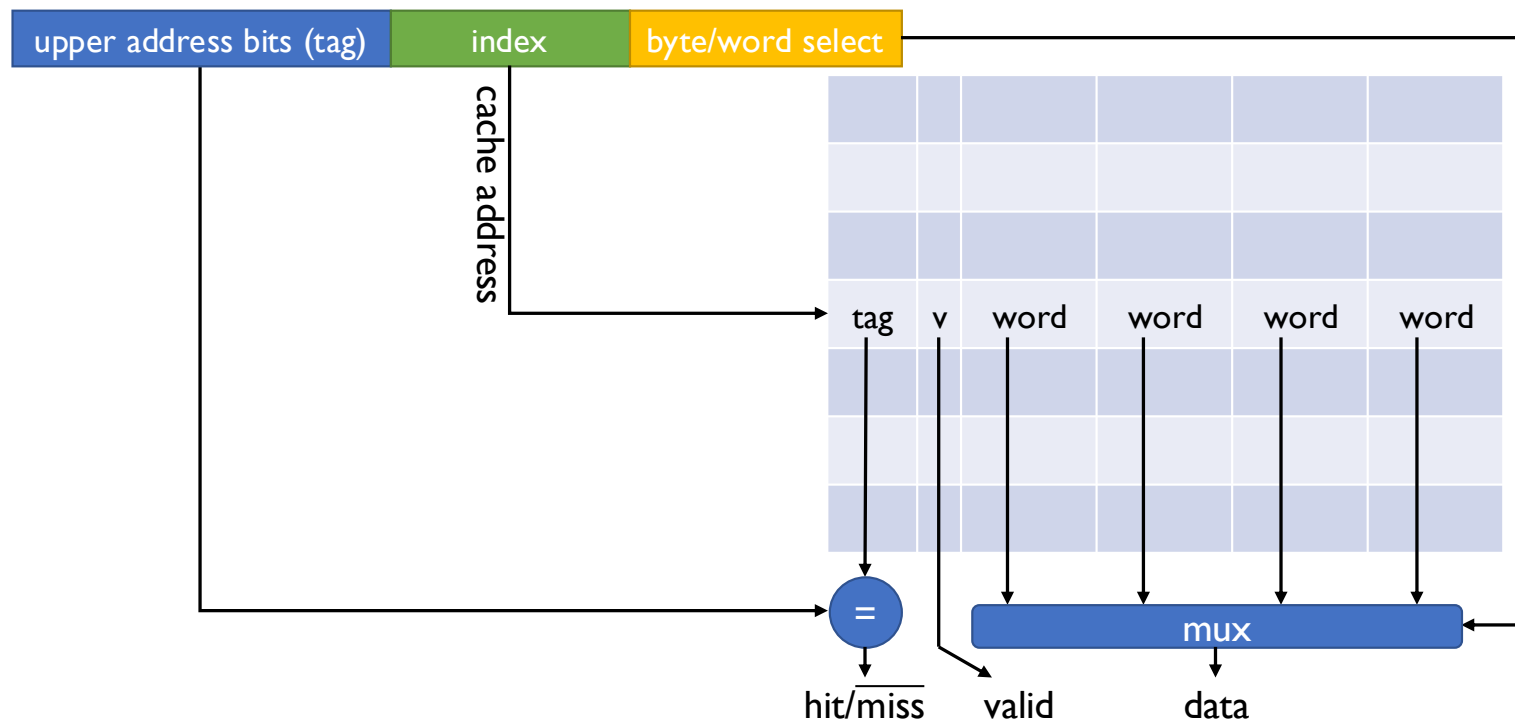


Simple direct-mapped cache

- Storing the address as a tag for every data item is a big overhead
- Amortise cost by storing several data items per tag
 - **Cache line** = some number (e.g. 4) of contiguous words in memory
 - Links with spatial locality
 - Links with DRAM bursts when filling cache lines from main memory
- Hash function is typically some bits of the address: the index bits



Simple direct-mapped cache



Cache line size considerations

- Larger cache lines should reduce miss rate
 - Due to spatial locality
- But in a fixed-sized cache
 - Larger cache lines $\Rightarrow$ fewer of them
 - More competition $\Rightarrow$ increased miss rate
 - Larger cache lines $\Rightarrow$ pollution
 - More data read into the cache that may never be read by the processor core
- Larger miss penalty
 - Can override benefit of reduced miss rate
 - Early restart and critical-word-first can help

Cache Misses

- On cache hit, the processor core proceeds normally
- On cache miss
 - Stall the processor pipeline
 - Fetch block from next level of hierarchy
 - Instruction cache miss
 - Restart instruction fetch
 - Data cache miss
 - Complete data access

Write-Through

- On data-write hit, could just update the block in cache
 - But then cache and memory would be inconsistent
- Write-through, also update memory
- But makes writes take longer
 - e.g., if base CPI = 1, 10% of instructions are stores, write to memory takes 100 cycles
 - Effective CPI = $1 + 0.1 \times 100 = 11$
- Solution: **write buffer**
 - Holds data waiting to be written to memory
 - Processor core continues immediately
 - Only stalls on write if write buffer is already full

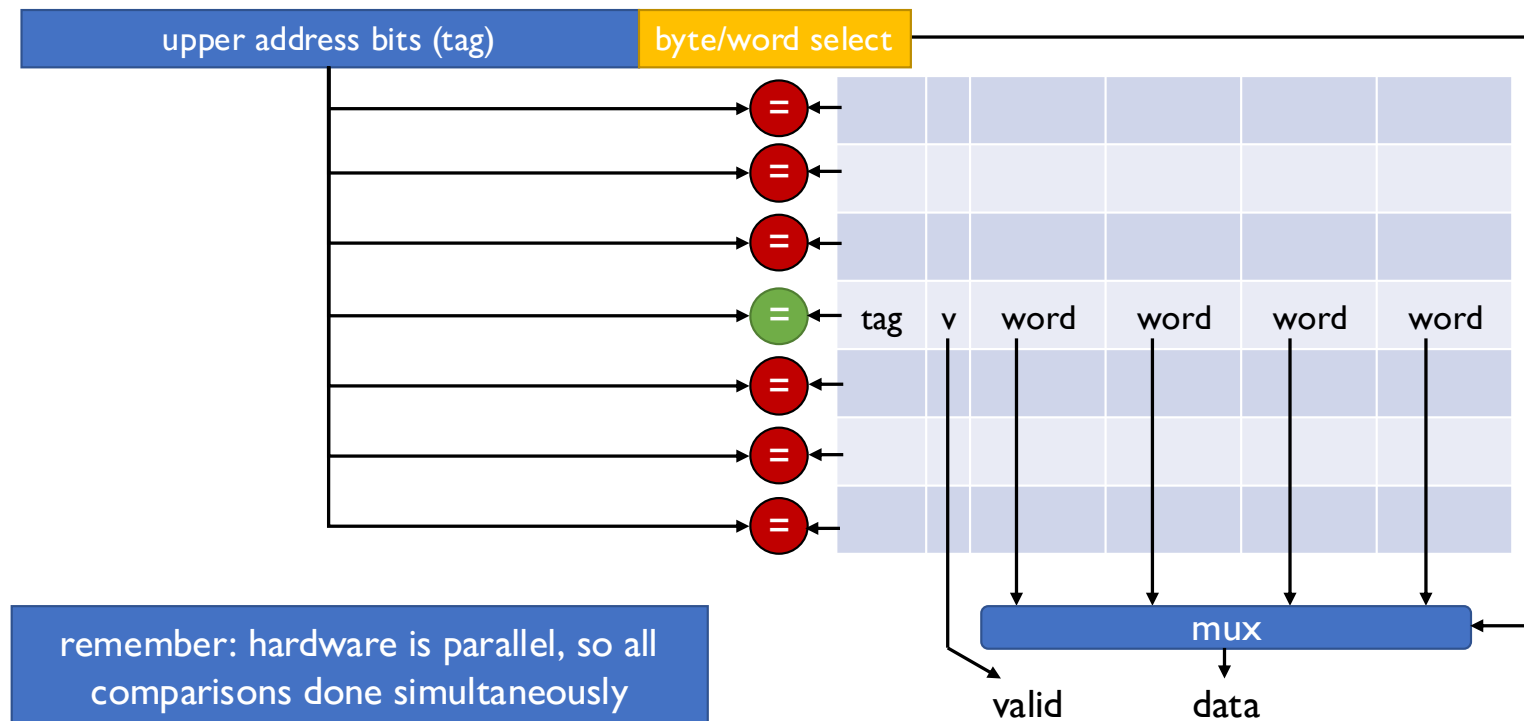
Write-Back

- Alternative: on data-write hit, just update the cache-line
 - Keep track of whether each cache-line is “dirty”, i.e. if it holds new data
- When a dirty block is replaced
 - Write it back to memory
 - Can use a write buffer to allow replacement cache-line to be read first

Write Allocation

- What should happen on a write miss?
- Alternatives for write-through
 - Allocate on miss: fetch the cache-line
 - Write around: don't fetch the cache-line but instead write the data to the next level in the hierarchy
- For write-back
 - Usually allocate on miss: fetch the cache-line to avoid having a cache-line where some words or bytes are invalid

Fully associative cache



Direct-mapped vs. fully-associative caches

- Circuit area
 - ASIC: direct-mapped caches use less circuit area than fully-associative caches
 - FPGA: direct-mapped caches can be efficiently implemented using embedded block RAM (BRAM) whereas fully-associative caches have to be made out of registers + logic, so are huge!
- Efficiency
 - Fully-associative caches tend to have a higher hit rate than a direct-mapped cache that stores the same amount of data
- Alternative approach combining the efficiency of direct-mapped caches with the higher hit rate of a more associative cache:
 - the set-associative cache...

Set associative caches

- Set associative cache idea:
 - have N direct-mapped caches
 - reads look in all N caches for data
 - thus the cache has N-way associativity
- In use:
 - Set-associative caches are widely used
 - E.g., Intel Core i9-9990XE – 14 cores
 - 14 x L1 instruction and data caches are each: 32KiB 8-way set associative
 - 14 x L2 unified cache (instructions and data): 1MiB 16-way set associative
 - 1 x L3 last-level cache: 19.25MiB 11-way set associative
 - E.g., ARM A72 used in the Raspberry Pi 4 – 4 cores
 - 4 x L1 instruction cache 48KiB 3-way associative & L1 data cache 32KiB 2-way associative
 - 1 x L2 unified cache: 1MiB 16-way set associative

Replacement Policy

- Direct mapped: no choice
- Set associative
 - Prefer non-valid entry, if there is one
 - Otherwise, choose among entries in the set
 - **Least-recently used (LRU)**
 - Choose the one unused for the longest time
 - Simple for 2-way, manageable for 4-way, too hard beyond that
 - **Not last used**
 - Approximates LRU and is simpler to implement for 8+ ways
 - **Random**
 - Gives approximately the same performance as LRU for high associativity
 - Simple to implement and avoids pathological misses

Victim cache/buffer

- Augment a direct mapped cache with a tiny full-associative cache/buffer
 - Could be as small as one cache line in size
- Evicted cache lines get a “second chance” in the victim cache
- Eliminates some of the pathological cases of a direct-mapped cache
- Tends not to be as effective as a set-associative cache

Sources of Misses

- Compulsory misses (aka cold start misses)
 - First access to a block
- Capacity misses
 - Due to finite cache size
 - A replaced block is later accessed again
- Conflict misses (aka collision misses)
 - In a non-fully associative cache
 - Due to competition for entries in a set
 - Would not occur in a fully associative cache of the same total size

Cache Design Trade-offs

Design change	Effect on miss rate	Negative performance effect
Increase cache size	Decrease capacity misses	May increase access time
Increase associativity	Decrease conflict misses	May increase access time
Increase cache-line size	Decrease compulsory misses	Increases miss penalty. For very large cache-line size, may increase miss rate due to pollution.

Multilevel Caches

- Level-1 or “primary” cache attached to the processor core
 - Small, but low latency
- Level-2 cache services misses from primary cache
 - Larger, slower, but still faster than main memory
- Main memory services L2 cache misses
- Some high-end systems include a Level 3 (L3) cache
- The cache closest to DRAM is called the *last level cache*

Multilevel Cache Example

- Given
 - Core base CPI = 1, clock rate = 4GHz
 - Miss rate/instruction = 2%
 - Main memory access time = 100ns
- With just an L1 cache
 - Miss penalty = $100\text{ns}/0.25\text{ns}$
= 400 cycles
 - Effective CPI = $1 + 0.02 \times 400 = 9$
- Now add an L2 cache
 - Access time = 5ns
 - Global miss rate to main mem. = 0.5%
- L1 cache miss with L2 hit
 - Penalty = $5\text{ns}/0.25\text{ns} = 20$ cycles
- L1 cache miss with L2 miss
 - Extra penalty = 400 cycles
- CPI = $1 + 0.02 \times 20 + 0.005 \times 400$
= 3.4
- Performance ratio = $9/3.4 = 2.6$

Cache topics not covered here, but of note

- **Prefetching** (not examinable)
 - Identify memory access patterns to further improve cache performance
 - E.g., spot sequential reads and ensure that caches prefetch data before the processor requests it
- **Context switch penalty** (not examinable)
 - Every context switch has a major impact on temporal and spatial locality
 - Expect much of the L1 cache to be clobbered, and a great deal of the L2
- In later lecture: Coherent caches for multicore systems
 - Multicore systems typically share an L3 cache and DRAM
 - Coherency protocol used to maintain consistency between the caches
 - More in the System-on-Chip part of this course

Summary & final thoughts

- Physical constraints prevent construction of a large low-latency memory
 - So a hierarchy of memories has to be used to approximate a large low-latency memory, critical for high performance of a control-flow processor
- Caches exploit temporal and spatial statistical properties of memory access to store data likely to be used close to the processor
 - Not a panacea, e.g. last level cache often has >80% dead data (i.e. data that will never be accessed again before it is evicted)
- Set associative caches are widely used
 - Good balance between the simplicity of a direct-mapped cache while giving some of the associativity of a full-associative cache
- It is ironic that we devote so many transistors to moving and storing data and so few on computation (more “Turing tax”)

Introduction to Computer Architecture

Lecture 9

OS Support

Prof. Simon W. Moore



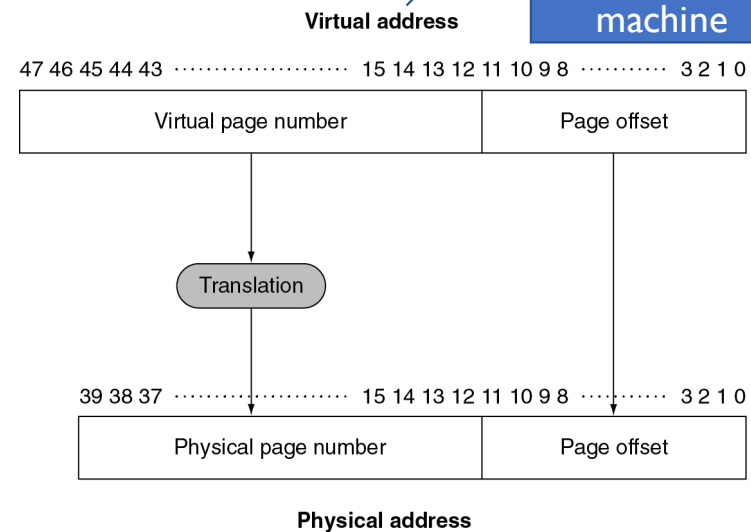
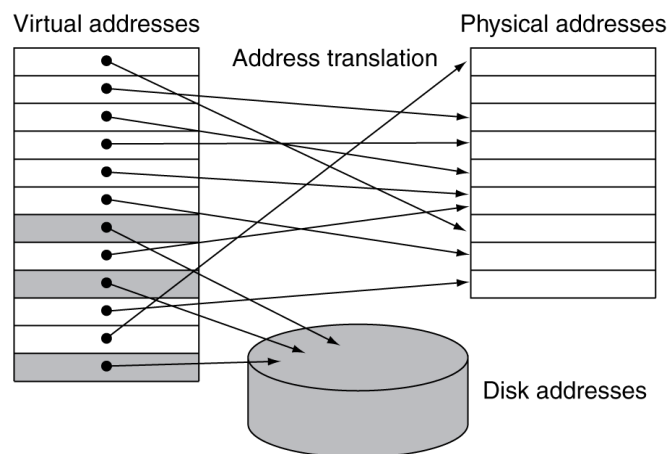
Introducing virtual memory

- Give each application the illusion that it has its own address space
- Processor + OS translates virtual to physical addresses
 - Memory broken into blocks or **pages** (e.g. 4KiB) or **superpages** (e.g. 4MiB)
 - Each translation fault is called a **page fault**
- Aids memory allocation
- Also provides memory protection
 - Permission bits per page determine what is shared and what is not
- Also allows rarely used pages to be migrated to disk: **paging**
 - Gives the illusion of a bigger memory
 - Can be useful to tolerate long running applications with memory leaks
 - But slow if paging to magnetic disk and risks wearing out flash disk (SSD)
- Can have virtual memory without paging (e.g. most smart phones)

Concept of address translation

- Fixed-size pages: e.g., 4KiB page or 4MiB superpage

example for
64-bit
machine



Page Tables

- Stores virtual to physical translation information
 - Array of page table entries (PTEs), indexed by virtual page number
 - Page table register in CPU points to page table in physical memory
- If page is present in memory
 - PTE stores the physical page number
 - Plus other status bits (referenced, dirty, ...)
- If page is not present
 - PTE can refer to a location in swap space on disk
 - Software **Page Fault Handler** invoked

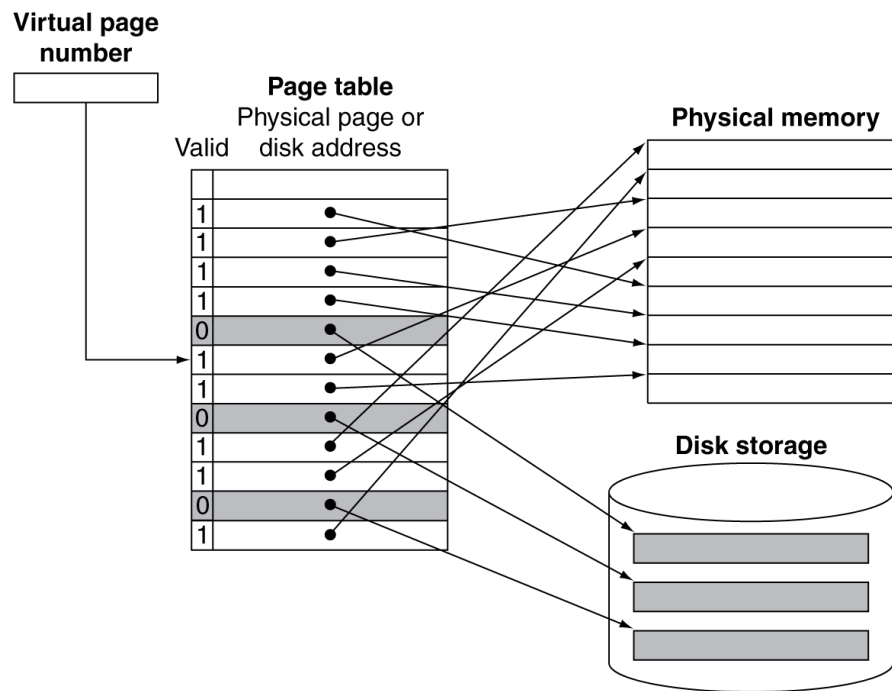
Page Fault Handler

- Page fault taken when PTE indicates that page is on disk
 - Software (the OS) has to get involved at this point
- Use faulting virtual address to find PTE
- Locate page on disk
- Choose page to replace
 - If dirty, write to disk first
- Read page into memory and update page table
- Make process runnable again
 - Restart from faulting instruction

Page Fault Penalty

- On page fault, the page must be fetched from disk
 - Handled by OS code
 - Can take millions of clock cycles
- Try to minimize page fault rate
 - Fully associative placement (any page on disk can be mapped to any page in memory)
 - Smart replacement algorithms

Mapping Pages to Disk

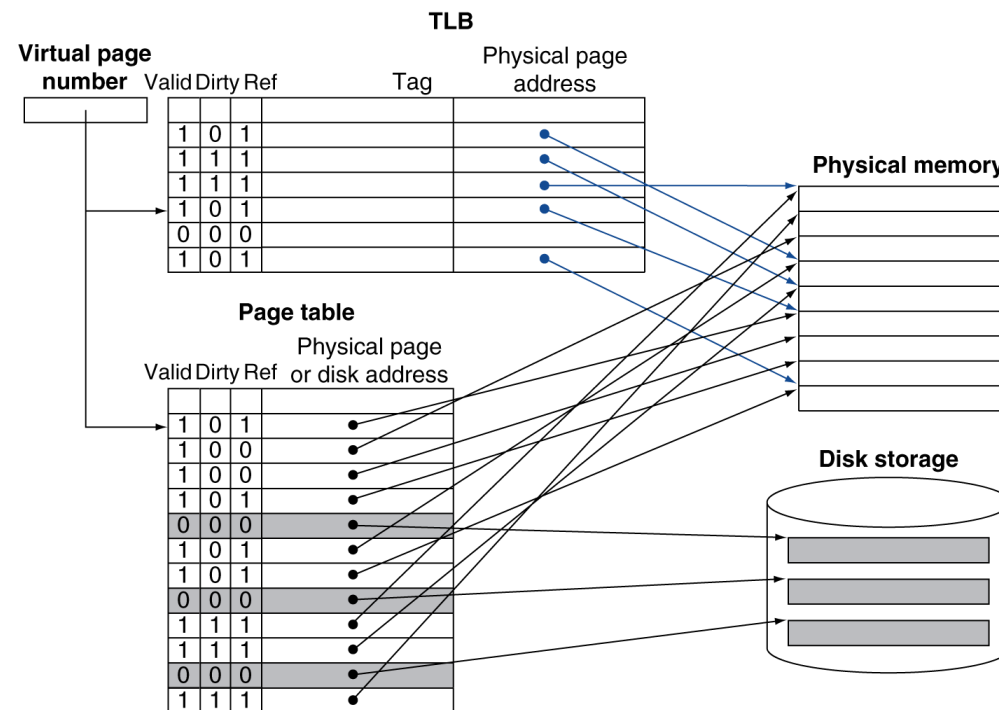


- DRAM becomes cache of the disk
- Disk accesses are slow, so careful management is critical, e.g. dirty bit – if not set then don't write the page back to disk
- Page table allows fully associative mapping
- LRU replacement policy

Translation Look-aside buffer (TLB) = cache of PTEs

- Address translation would appear to require extra memory references
 - One to access the PTE
 - Then the actual memory access
- But access to page tables has good locality
 - So use a fast cache of PTEs within the CPU
 - Called a Translation Look-aside Buffer (TLB) = cache of PTEs
 - Typical: 16–512 PTEs, 0.5–1 cycle for hit, 10–100 cycles for miss, 0.01%–1% miss rate
 - TLB lookup usually performed before L1 cache access
 - TLB misses could be handled by hardware or software
 - Older processors like MIPS handled in software
 - Modern processors all have hardware page table walkers

Fast Translation Using a TLB



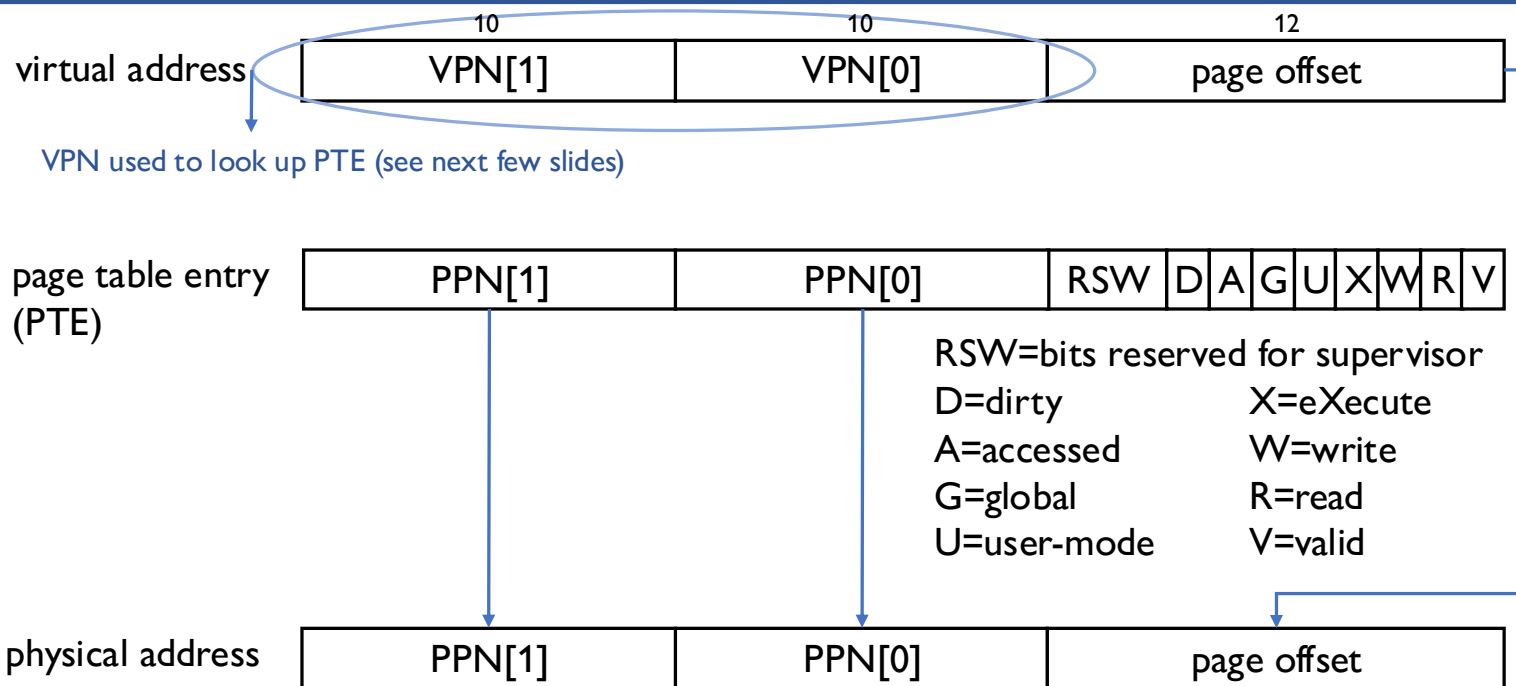
TLB Misses

- If page is in memory
 - Load the PTE from memory and retry
 - Could be handled in hardware
 - Can get complex for more complicated page table structures
 - Usually worth it for increased performance
 - Or in software
 - Raise a special exception, with optimized handler
- If page is not in memory (page fault)
 - OS handles fetching the page and updating the page table
 - Then restart the faulting instruction

Hardware page table walker

- TLB miss causes the hardware page table walker to search for the PTE, i.e. the TLB is a cache of the page table
 - PTE found gets put into the TLB
 - PTE not found results in an exception
- Must recognize TLB miss before destination register overwritten
 - Typically memory operations are replayed (re-run) after the TLB miss has been performed

Page table structure (RISC-V Sv32 32-bit addressing)



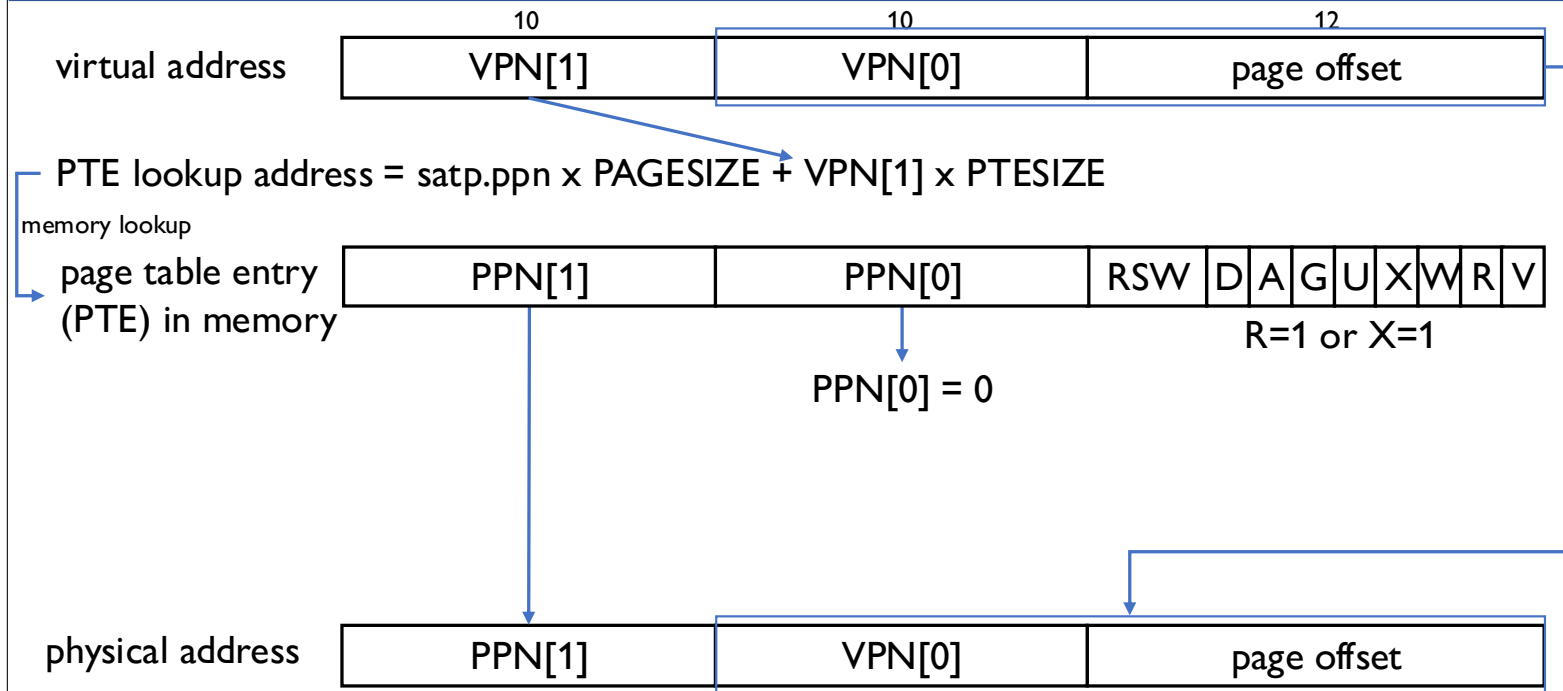
eXecute, Write, Read bits (RISC-V)

X	W	R	Meaning
0	0	0	Pointer to next level page table
0	0	1	Read-only page
0	1	0	Reserved for future use
0	1	1	Read-write page
1	0	0	Execute-only page
1	0	1	Read-execute page
1	1	0	Reserved for future use
1	1	1	Read-write-execute page

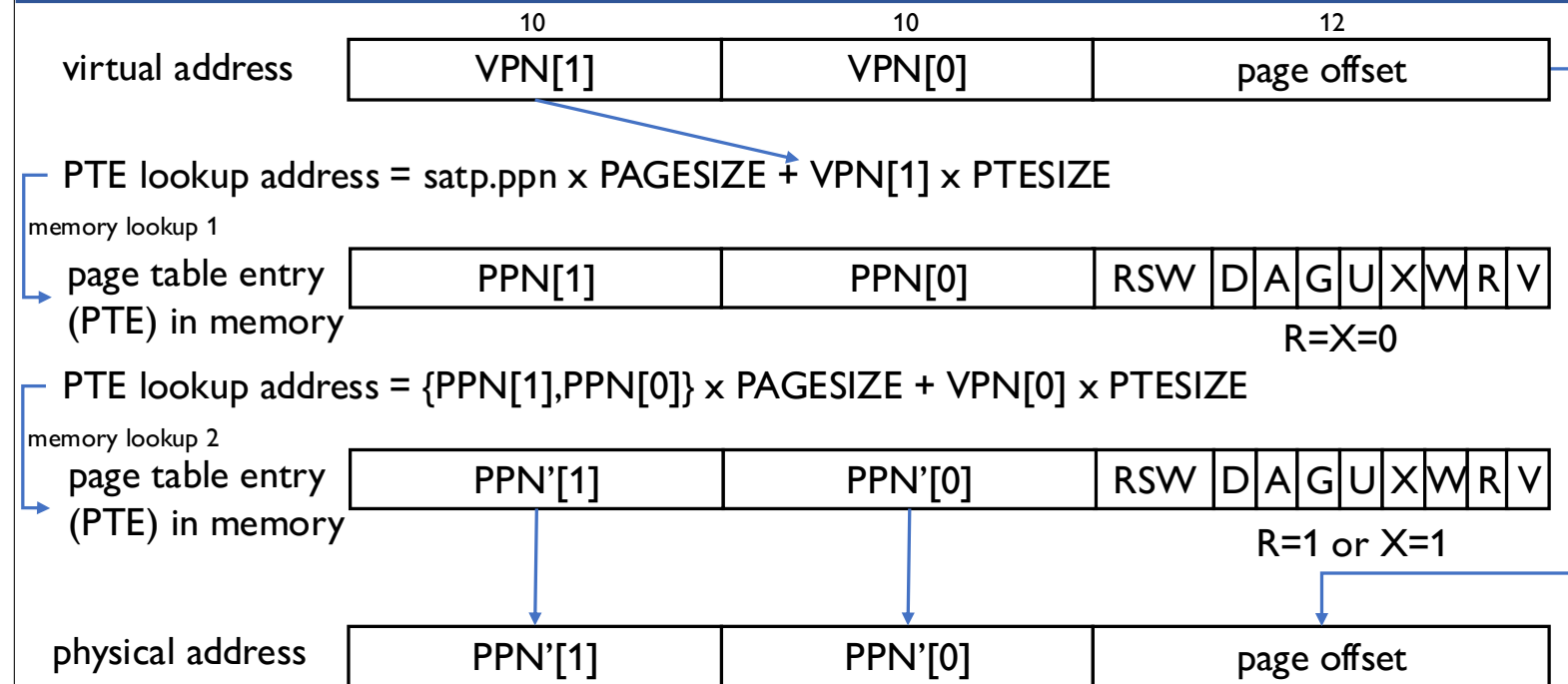
Multilevel page table (RISC-V)

- One-level page table would be $2^{20} \times 4 = 4\text{MiB}$, much of it unused
- Page table represented as a tree
 - compacts the structure
 - each section of page table fits into a page = $2^{12} = 4\text{KiB} = \text{PAGESIZE}$
 - PTESIZE = 4 bytes
 - supervisor address protection and translation (satp) register hold the base of the tree in the physical page number (ppn) field
- First PTE lookup address = $\text{satp.ppn} \times \text{PAGESIZE} + \text{VPN}[1] \times \text{PTESIZE}$
- If $X=1$ or $R=1$ it's a superpage (where $\text{PPN}[0]=0$)
else lookup address = $\{\text{PPN}[1], \text{PPN}[0]\} \times \text{PAGESIZE} + \text{VPN}[0] \times \text{PTESIZE}$
- More translation levels for a 64-bit machine

Superpage (=4MiB page) lookup (RISC-V)



Page (=4KiB page) lookup (RISC-V)



Memory Protection

- Different applications (processes or tasks) are allocated their own virtual address space
 - Pointer to page table changed during context switch (satp on RISC-V)
 - satp includes a 9-bit ASID (Address Space Identifier)
 - TLB entries often include the ASID so that the TLB can hold translations for different virtual address spaces
- Different applications can share parts of their virtual address space
 - But need to protect against errant access
 - Requires OS assistance
- Hardware support for OS protection
 - Privileged supervisor mode (aka kernel mode)
 - Privileged instructions
 - Page tables and other state information only accessible in supervisor mode
 - System call exception (e.g., ecall in RISC-V)

Operating modes on RISC-V

- RISC-V operating modes

Level	Encoding	Name	Abbreviation
0	00	user/application	U
1	01	supervisor	S
2	10	reserved	
3	11	machine	M

- Modes control

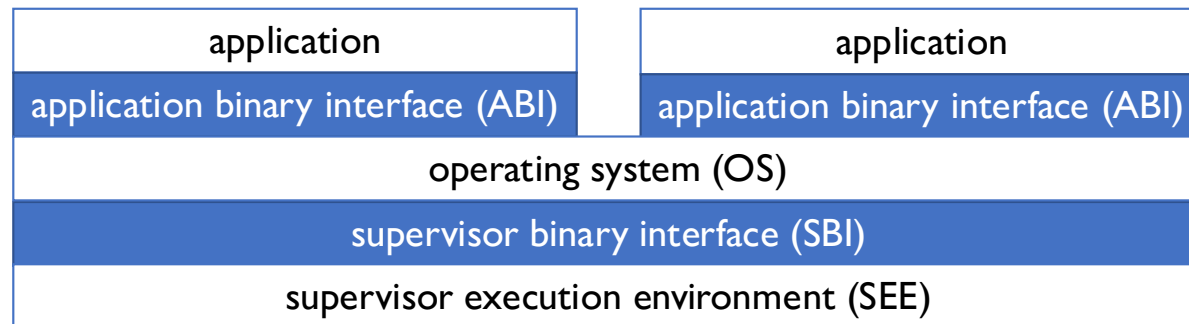
- How much of the physical address space is visible (e.g. access to peripherals or not): M-mode gives unrestricted access
- Limit access to privileged instructions
 - return from trap or interrupt
 - memory management
- Limit access to control-status registers (CSRs)

Operating modes – implementation choices

- Choice of operating modes implemented for different scenarios

Number of levels	Supported modes	Intended usage
1	M	Simple embedded systems
2	M, U	Secure embedded systems
3	M, S, U	Systems running UNIX-like OSs

Example software implementation stack



- U-mode: applications, ABI acts as the interface
- S-mode: OS-layer
- M-mode: SEE which could be a simple BIOS-style IO system

Environment call (or *software interrupt*) for RISC-V

▪ ECALL

- used to call into a more privileged mode
- e.g. used by the ABI to transition from U-mode to S-mode
- pc of ECALL is saved in the **epc** register (not the address after the ECALL)
- epc = exception PC register, hidden to U-mode
- mtvec = machine trap vector
 - if $\text{LSB}(\text{mtvec}) == 0$
pc = mtvec
 - else
pc = $\text{mem}[\text{mtvec} + \text{cause} * 4]$, where cause=0 for user-mode software interrupt

▪ SRET

- trap return from S-mode
- e.g. used by the OS to transition from S-mode back to U-mode
- PC=EPC and perform mode change

Interrupt/exception **cause** (RISC-V)

- **cause** register in each mode specifies the cause of a trap
- **cause=0** for user software interrupt

Interrupt	Exception Code	Description
1	0	User software interrupt
1	1	Supervisor software interrupt
1	2	<i>Reserved for future standard use</i>
1	3	Machine software interrupt
1	4	User timer interrupt
1	5	Supervisor timer interrupt
1	6	<i>Reserved for future standard use</i>
1	7	Machine timer interrupt
1	8	User external interrupt
1	9	Supervisor external interrupt
1	10	<i>Reserved for future standard use</i>
1	11	Machine external interrupt
1	12–15	<i>Reserved for future standard use</i>
1	≥ 16	<i>Reserved for platform use</i>

Interrupts and exceptions

- Interrupts, exceptions and software exceptions all use very similar hardware: jump to some privileged code and change privilege mode
- Interrupts also cause a trap but based on external factors rather than instruction execution
 - Timer interrupts used for scheduling, etc.
 - DMA interrupts used to indicate the end of a transfer
 - I/O device interrupts to trigger execution of a device driver
- Exceptions are caused by the instruction stream
 - Illegal instruction
 - Division by zero
 - Page fault
 - etc.
- Software exception: e.g. ECALL

Summary

- Hardware support for operating systems includes:
 - Execution modes (user, supervisor, machine, etc.)
 - Exceptions and interrupts, and software exceptions
 - Privileged instructions
 - Privileged registers
 - Virtual memory
 - Great for memory allocation
 - Also used for memory protection
- Different processors have different implementations
 - E.g. hardware vs. software page table walking on a TLB miss
 - But the same abstract model (virtual addressing, execution modes, etc.) works across all

Introduction to Computer Architecture

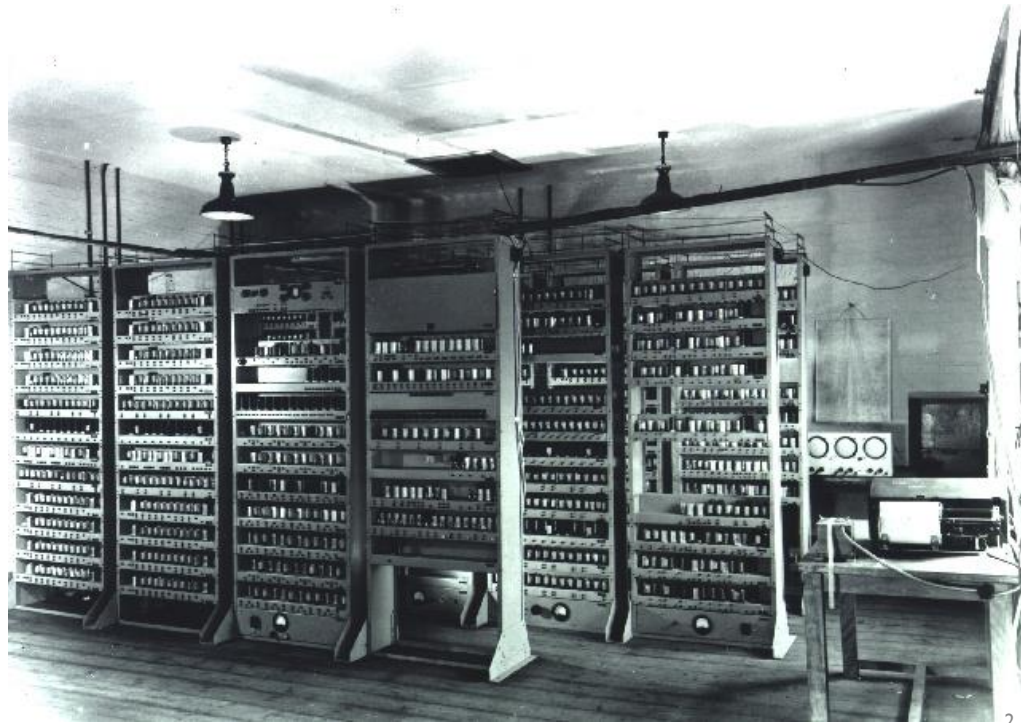
Lecture 10

Other Instruction Sets

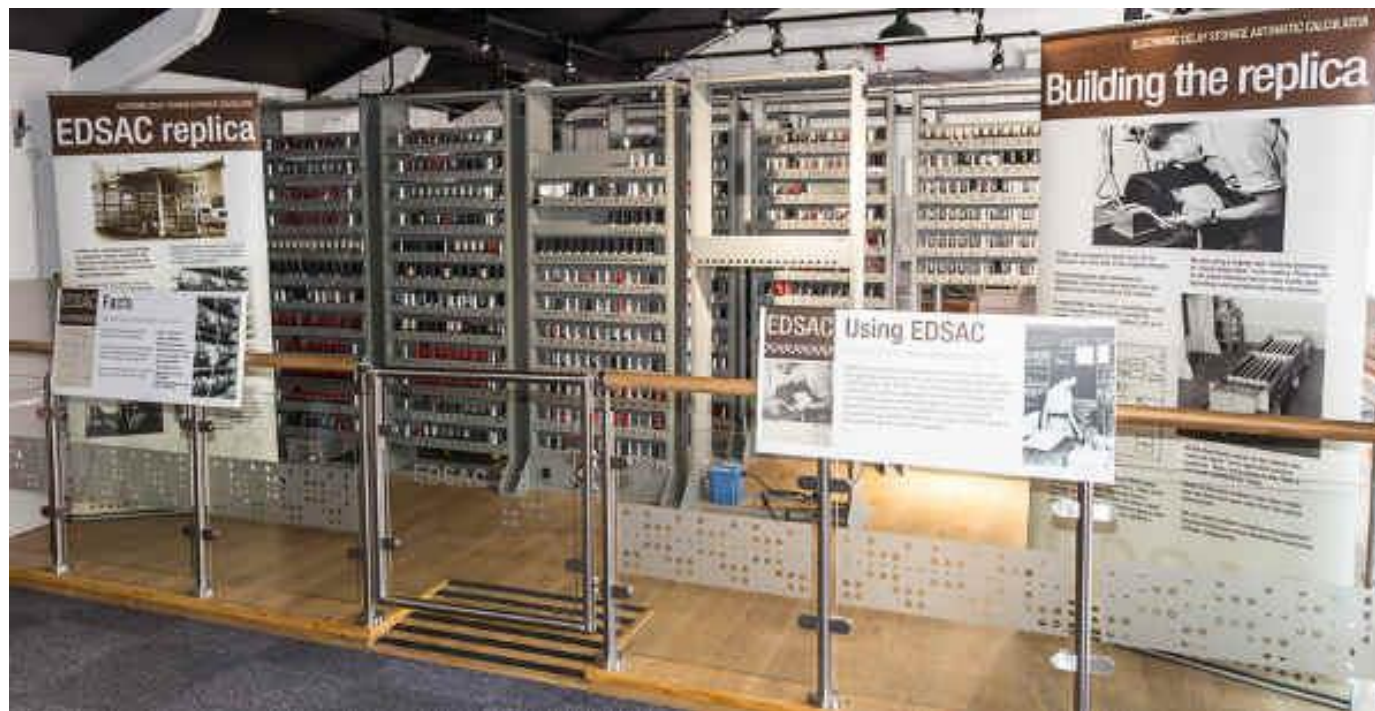
Prof. Simon W. Moore

1940's Cambridge history: EDSAC

- 650 instructions/s
- 1k x 17 bits memory
- paper tape input
- teletype output
- floor plan: 4m x 3m
- 3000 valves
- 12kW



EDSAC rebuild at TNMOC in Bletchley



1940's Cambridge history: EDSAC

- Project started in 1947 and ran its first program in 1949
- Project lead: Prof Sir Maurice Wilkes (who always liked to look forward...)
- Input/output: paper tape, teleprinter, switches
- Memory: mercury delay lines
 - 1024 x 17 bits
- Logic technology: valves (vacuum tubes)
- Speed: 714 operations/second
- Footprint: medium room
- Used widely in Cambridge and scientific computations are linked to two Nobel prizes

EDSAC instruction set (1 of 2)

Instruction	Description	
P n	pseudo code (for constants)	instruction is n
A n	add	$\text{acc} = \text{acc} + \text{mem}[n]$
S n	subtract	$\text{acc} = \text{acc} - \text{mem}[n]$
H n	initialise multiplier	$\text{mr} = \text{mem}[n]$
V n	multiply and add	$\text{acc} = \text{acc} + \text{mr} \times \text{mem}[n]$
N n	multiply and subtract	$\text{acc} = \text{acc} - \text{mr} \times \text{mem}[n]$
T n	store and clear acc	$\text{mem}[n] = \text{acc}, \text{acc} = 0$
U n	store	$\text{mem}[n] = \text{acc}$
C n	bitwise AND	$\text{mr AND mem}[n]$

EDSAC instruction set (2 of 2)

Instruction	Description	
R n	right shift	$\text{acc} = \text{acc} \gg n$
L n	left shift	$\text{acc} = \text{acc} \ll n$
E n	conditional branch positive	if($\text{acc} \geq 0$) $\text{pc} = n$
G n	conditional branch negative	if($\text{acc} < 0$) $\text{pc} = n$
I n	input 5-bit from punched tape	$\text{mem}[n] = \text{punch_tape_input}$
O n	output top 5 bits to teleprinter	$\text{output} = n$
F n	read next output character	$\text{mem}[n] = \text{output}$ (Ed: not used)
X	round accumulator to 16-bits	
Y	round accumulator to 34-bits	
Z	stop the machine and ring the warning bell	

Observations about EDSAC

- Accumulator machines like EDSAC have a huge memory churn as every instruction reads or writes data to/from memory
- What's missing from the instruction set?
 - Subroutines
 - invented in Cambridge by David Wheeler: the Wheeler jump!
 - involved self-modifying code on EDSAC due to lack of instruction set support
 - Interrupts
 - Virtual memory
 - Rich I/O (though it did have a bell (!) and a teleprinter)
- Memory made from mercury delay-lines
 - Using valves for memory would have been too expensive (and possibly less reliable)

Cray-1 supercomputer

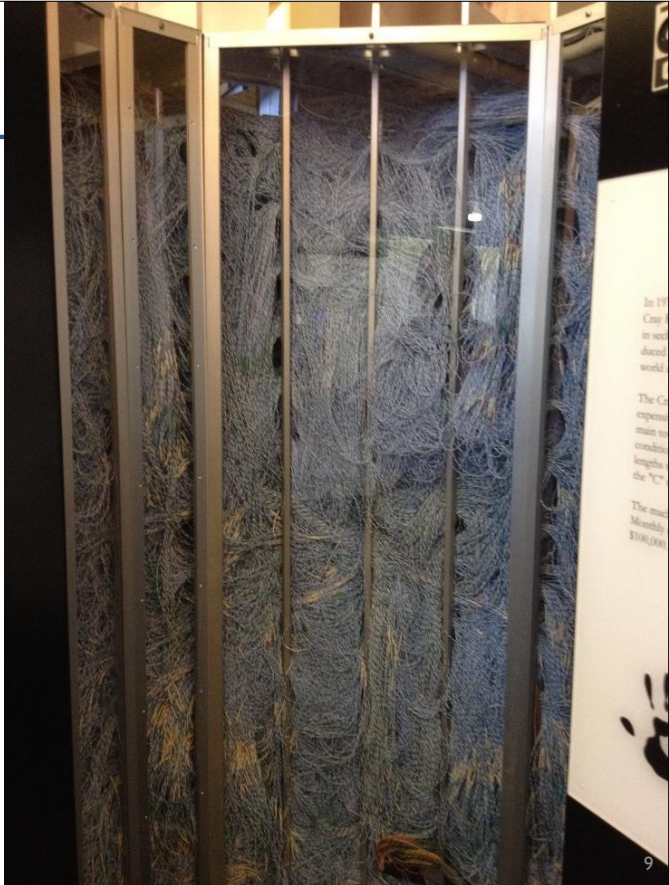
- \$7.9m in 1977
- 5.5 tons, 115kW
- 64-bit processor @ 80MHz
 - included powerful vector instructions
- 8.39MiB memory
- 303MiB storage
- Separate minicomputer (1970's notion of a “minicomputer”!) provides much of the management of the supercomputer, and I/O
 - Somewhat analogous to today's GPUs hosted by a lower performance x86



Cray-1

- Observation from Seymour Cray:
Performance was limited by heat removal and the thickness of the mat (i.e. the wiring)
- Still major design constraints today

- # Cray-1
- Observation from Seymour Cray:
Performance was limited by heat removal and the thickness of the mat (i.e. the wiring)
 - Still major design constraints today



Cray-1 ISA

- Instructions:
 - 16-bit instructions mainly for arithmetic operations
 - 32-bit instructions mainly for memory operations
 - instructions follow a RISC-style
- Registers:
 - 8 x vector registers
 - each 64 x 64-bit registers (integers or floating-point values)
 - lowest 64-bit register can also be used as a scalar register
 - 8 x 24-bit address registers
 - used for memory addresses
 - also loop counting

Xerox Alto

- \$32k in 1979
- First computer with:
 - Graphical user interface
 - Mouse
 - Ethernet
 - Laser printer
- Designed by Charles P. Thacker
- Software initially written in BCPL (from Cambridge)
 - Included a WYSIWYG word processor
- Major influence on Steve Jobs



Modern CISC: Intel x86 ISA

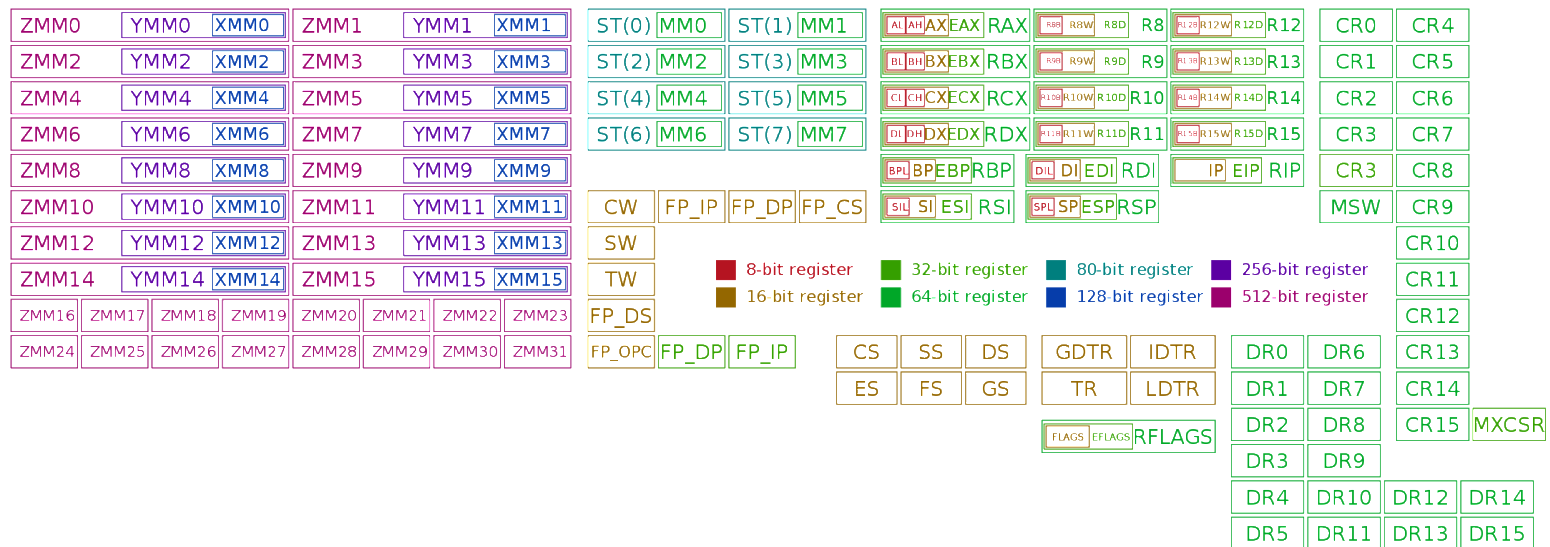
- Brief history
 - 1978: 16-bit ISA
 - 1985: extended to 32-bit ISA – originally the 80386 (i386 on Linux distros)
 - 2003: extended to 64-bit ISA – x86-64
 - Widely deployed for laptops, PCs and servers
- Complex Instruction Set Computer (CISC) design
 - Variable length instructions (1 to 15 bytes) that are highly complex to decode
 - Many registers are special purpose
 - Though more recent extensions are more RISC like (orthogonal register file usage)
 - x86 machines decode CISC instructions into micro-operations that are similar to RISC instructions
- Despite complexity, it runs legacy software
 - The rise of “app store” delivery model is slowly removing this dependency

x86-64 general purpose registers

- 64-bit mode general purpose registers: R0 to R15
- R0 to R7 have historical names with some specialised functionality
 - RAX — Accumulator for operands and results data (EAX=lower 32-bit...)
 - RBX — Pointer to data in the DS segment
 - RCX — Counter for string and loop operations
 - RDX — I/O pointer
 - RSI — Pointer to data in the segment pointed to by the DS register; source pointer for string operations
 - RDI — Pointer to data (or destination) in the segment pointed to by the ES register; destination pointer for string operations
 - RSP — Stack pointer
 - RBP — Pointer to data on the stack



All of the x86-64 registers



A few x86 instructions

- AAA – ASCII adjust AL (register) after addition (used with unpacked BCD)
[old 16-bit instruction]
- ADD – D += S, where D=destination, S=source; D & S can be a register, immediate (**constant**) or **memory** address
 - Syntax
 - add <reg>, <reg>
 - add <mem>, <reg>
 - add <reg>, <mem>
 - add <con>, <reg>
 - add <con>, <mem>
- MOV – move {reg, mem, con} to {reg, mem}
 - So no separate integer load/store instructions
- LOOPccD – loop while equal:
 - if (cc && --ECX) goto lbl
 - where cc = **Z**(ero), **E**(qual), **N**on**Z**ero, **N**(on)**E**(qual)

Modern RISC: ARM

- Widely deployed in phones, Raspberry Pi, etc.
- Original 32-bit ISA (1985-) was typically RISC (with 16 int. registers) except:
 - R15 = PC, so any integer instruction could cause a branch
 - All instructions are conditional (on a hidden condition codes register)
 - There are load- and store-multiple instructions (rather CISC)
- 64-bit ISA – AArch64 – (2011-)
 - Many similarities with RISC-V and MIPS: 32 integer registers, hidden PC
 - Conditional branches, but also conditional move
 - other instructions are not conditional, unlike 32-bit ISA
 - Richer addressing modes:
 - register, register+immediate, register+register, register+register<<shift
 - particularly useful when compiling object-oriented code

Java bytecode

- An instruction set focused on portability
 - Intermediate representation – usually interpreted or compiled to native code
 - A Java byte code processor has been attempted, but was unsuccessful
 - ARM Jazelle DBX (Direct Bytecode eXecute) directly executes 95% of instructions with the others performed by an exception handler
 - Seems to be deprecated since ARM v7 (compile/JIT instead)
- Stack-based machine
 - An operand stack is used instead of registers
 - Values are pushed and popped from the stack
 - Top of stack is a dependency for almost all instructions
 - Can easily limit instruction-level parallelism
 - There are hidden registers for PC, stack pointer, etc.

Java bytecode instructions

- Variable length instruction set
 - First byte is the instruction opcode
 - n bytes of operands follow, where n is easily determined from the opcode
- Some example instructions
 - iadd – add top two integer items on the operand stack, result pushed on stack
 - fadd – like iadd but for floating-point numbers
 - ifeq – if (top_of_stack==0) then PC=PC+index
 - iaload – load an int from an array (array ref and index popped from stack)
 - tableswitch – useful for case statements: a long table of jump targets is part of this instruction with a dynamic index determining which jump to perform
 - On RISC-V as two instructions: load jump target from table; jump to register
 - new – create new object of type identified by class reference in the constant pool index – complex instruction not well suited for simple pipelines
 - On RISC-V this would be many instructions: basically a form of malloc()

Research Highlight: CHERI-RISC-V

Team effort: Robert N. M. Watson, Simon W. Moore, Peter Sewell, Peter G. Neumann

Hesham Almatary, Ricardo de Oliveira Almeida, Jonathan Anderson, Alasdair Armstrong, Rosie Baish, Peter Blandford-Baker, John Baldwin, Hadrien Barrel, Thomas Bauereiss, Ruslan Bukin, Brian Campbell, David Chisnall, Jessica Clarke, Nirav Dave, Brooks Davis, Lawrence Esswood, Nathaniel W. Filardo, Franz Fuchs, Dapeng Gao, Ivan Gomes-Ribeiro, Khilan Gudka, Brett Gutstein, Angus Hammond, Graeme Jenkinson, Alexandre Joannou, Mark Johnston, Robert Kovacsics, Ben Laurie, A. Theo Marketos, J. Edward Maste, Alfredo Mazzinghi, Alan Mujumdar, Prashanth Mundkur, Steven J. Murdoch, Edward Napierala, George Neville-Neil, Kyndylan Nienhuis, Robert Norton-Wright, Philip Paeps, Lucian Paul-Trifu, Allison Randal, Ivan Ribeiro, Alex Richardson, Michael Roe, Colin Rothwell, Peter Rugg, Hassen Saidi, Thomas Sewell, Stacey Son, Ian Stark, Domagoj Stolfa, Andrew Turner, Munraj Vadera, Konrad Witaszczyk, Jonathan Woodruff, Hongyan Xia, Vadim Zaliva, and Bjoern A. Zeeb

University of Cambridge and SRI International

Approved for public release; distribution is unlimited.

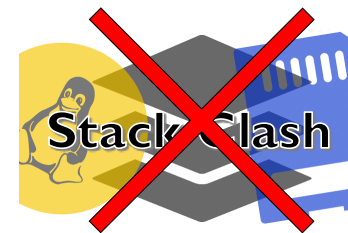
This work was supported by the Defense Advanced Research Projects Agency (DARPA) and the Air Force Research Laboratory (AFRL) under contract FA8750-10-C-0237 ("CTSRD"), with additional support from FA8750-11-C-0249 ("MRC2"), HR0011-18-C-0016 ("ECATS"), and FA8650-18-C-7809 ("CIFV") as part of the DARPA CRASH, MRC, and SSITH research programs.

The views, opinions, and/or findings contained in this report are those of the authors and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.

We also acknowledge the EPSRC REMS Programme Grant (EP/K008528/1), the ERC ELVER Advanced Grant (789108), the Isaac Newton Trust, the UK Higher Education Innovation Fund (HEIF), Thales E-Security, Microsoft Research Cambridge, Arm Limited, Google, Google DeepMind, HP Enterprise, and the Gates Cambridge Trust.

Motivation – The Eternal War in Memory*

- Many security vulnerabilities exploit memory safety violations

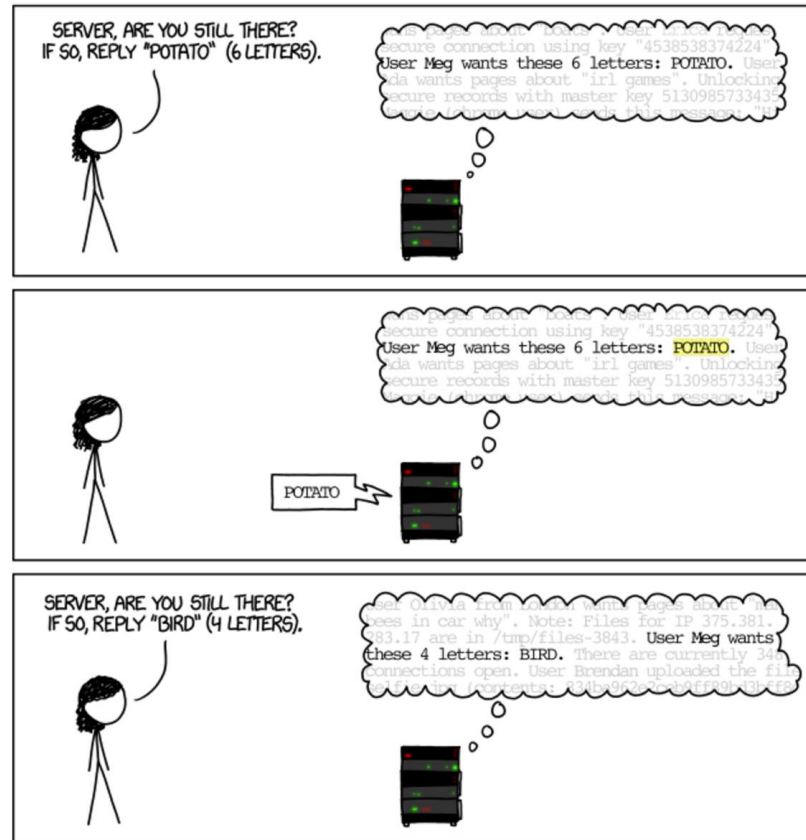


* Title based on Oakland 2013 paper: SoK: Eternal War in Memory, László Szekeres, Mathias Payer, Tao Wei, Dawn Song

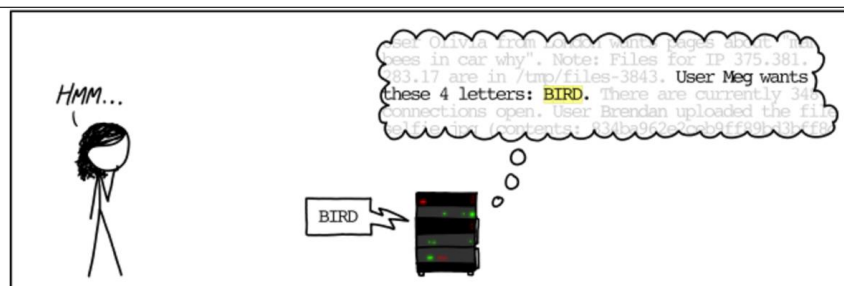
Example 1



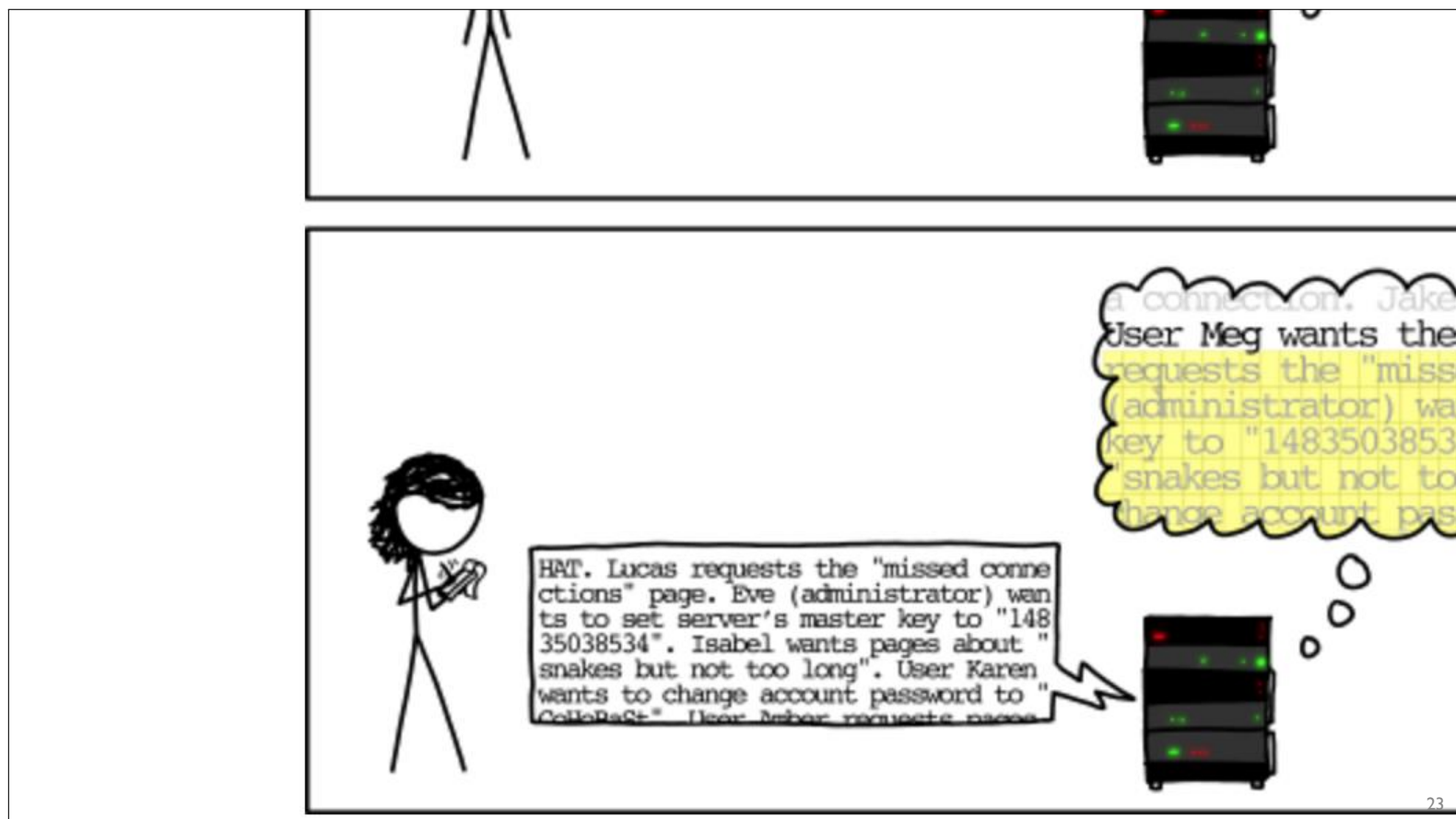
HOW THE HEARTBLEED BUG WORKS:



source: <http://xkcd.com/1354/>



source: <http://xkcd.com/1354/>



Went wrong? How do we do better?

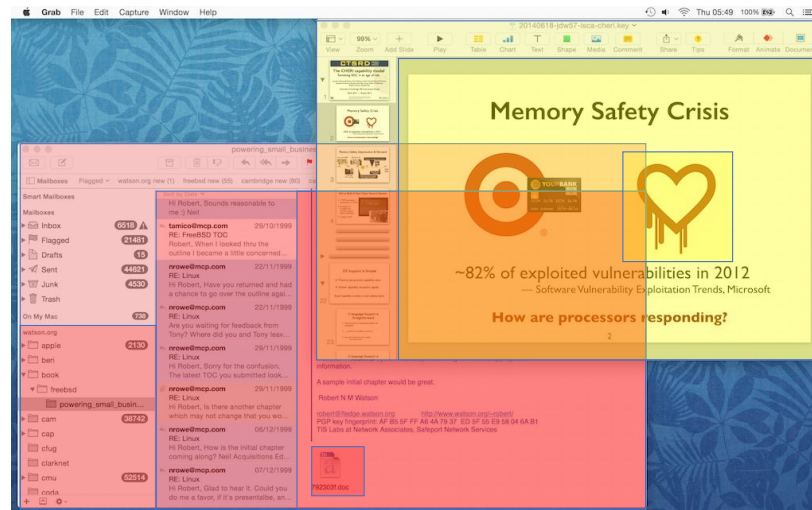
- Classical answer:
 - The programmer forgot to check the bounds of the data structure being read
 - Fix the vulnerability in hindsight – one line fix:
`if (1+2+payload+16 > s->s3->rrec.length) return 0;`
- Our answer:
 - Preserve bounds information during compilation
 - Use hardware (CHERI processor) to dynamically check bounds with little overhead and guarantee pointer integrity & provenance

Example 2: how to reduce the attack surface?

- The software attack surface keeps getting bigger
 - Applications just keep getting larger
 - Huge libraries of code aid rapid program development
 - Everything is network connected
- This aids the attacker: an expanding number of ways to break in

CHERI solution: application-level least privilege

Software compartmentalization decomposes software into **isolated compartments** that are delegated **limited rights**

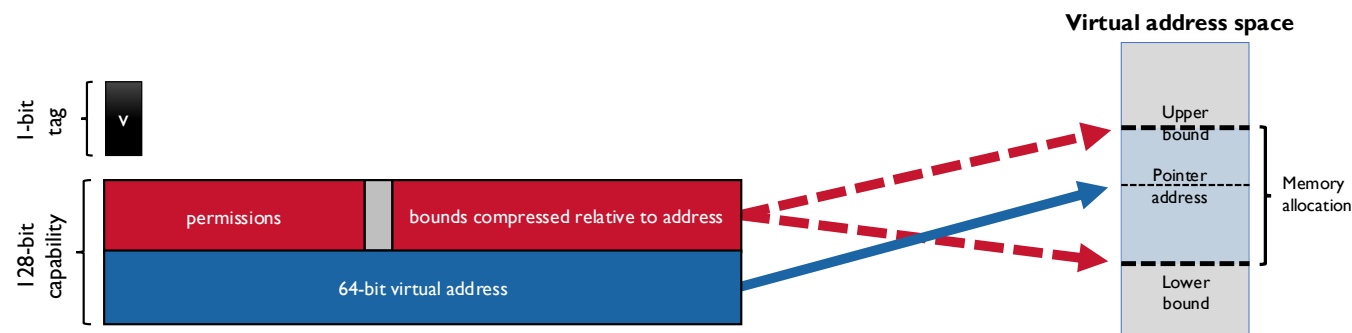


Able to mitigate not only **unknown vulnerabilities**, but also **as-yet undiscovered classes of vulnerabilities and exploits**

Principles CHERI helps to uphold

- The **principle of intentional use**
 - Ensure that software runs the way the programmer intended, not the way the attacker tricked it
 - Approach: guaranteed pointer integrity & provenance, with efficient dynamic bounds checking
- The **principle of least privilege**
 - Reduce the attack surface using software compartmentalization
 - Mitigates known and unknown exploits
 - Approach: highly scalable and efficient compartmentalization

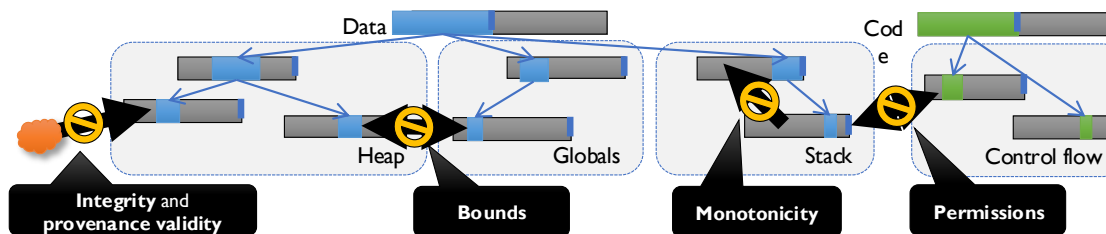
New CHERI data type: the **capability**



- **Capabilities** extend **integer memory addresses**
- **Metadata** (bounds, permissions, ...) control how they may be used
- **Guarded manipulation** controls how capabilities may be manipulated; e.g., **provenance validity** and **monotonicity**
- **Tags** protect capability integrity/derivation in registers + memory

Hardware
guarantees
correct usage

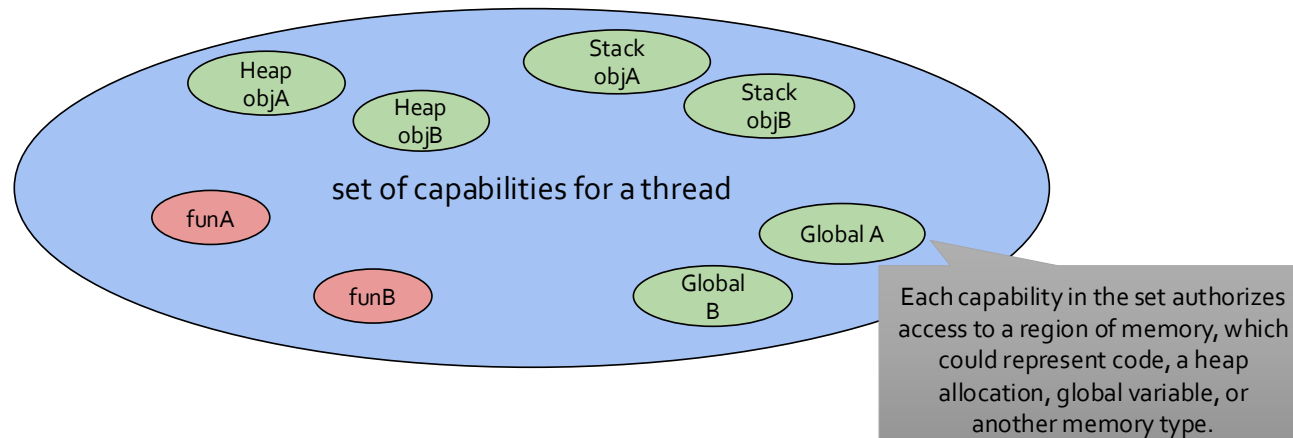
CHERI enforces protection semantics for pointers



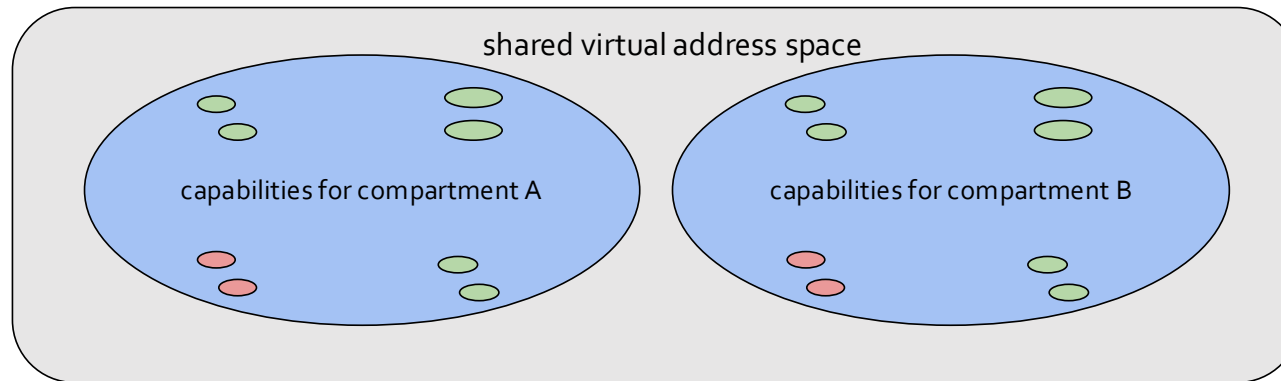
- **Integrity** and **provenance validity** ensure that valid pointers are derived from other valid pointers via valid transformations; **invalid pointers cannot be used**
- **Bounds** prevent pointers from being manipulated to access the wrong object
- **Monotonicity** prevents pointer privilege escalation – e.g., broadening bounds
- **Permissions** limit unintended use of pointers; e.g., W^X for pointers
- These primitives not only allow us to implement **strong spatial and temporal memory protection**, but also higher-level policies such as **scalable software compartmentalisation**

Reachable set of capabilities

- ✧ **Reachable capabilities** for a thread are those:
 - ✧ In its register file or program counter capability (PCC)
 - ✧ Transitively loadable via any capability in its register file
- ✧ All other capabilities are **unreachable** and unavailable to the thread
- ✧ **Memory safety** is a necessary first step toward CHERI compartmentalization

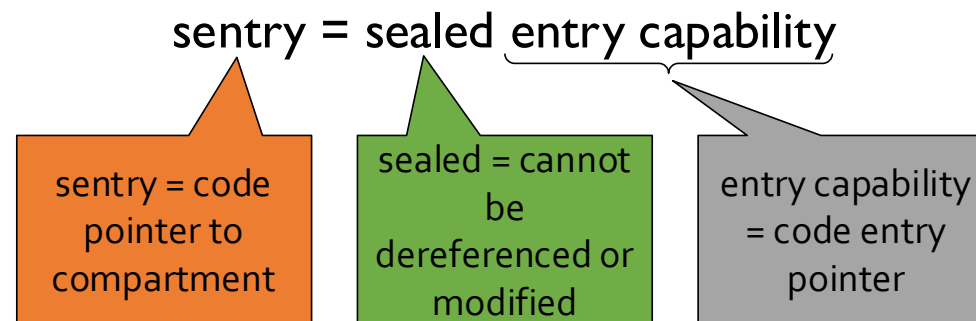


Two independent compartments



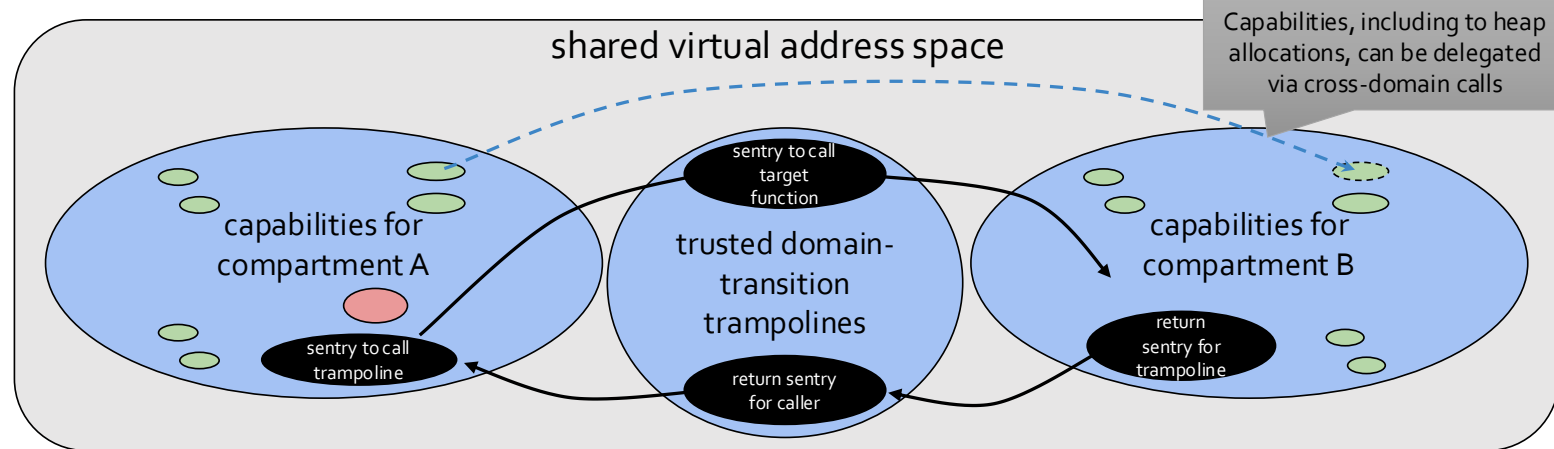
- ✎ **Capability provenance validity** and **monotonicity** prevent privilege escalation beyond reachable capabilities for each compartment
- ✎ These principles allow software to implement strong, architecturally supported **isolation** between compartments

Compartment transition with **sentries**



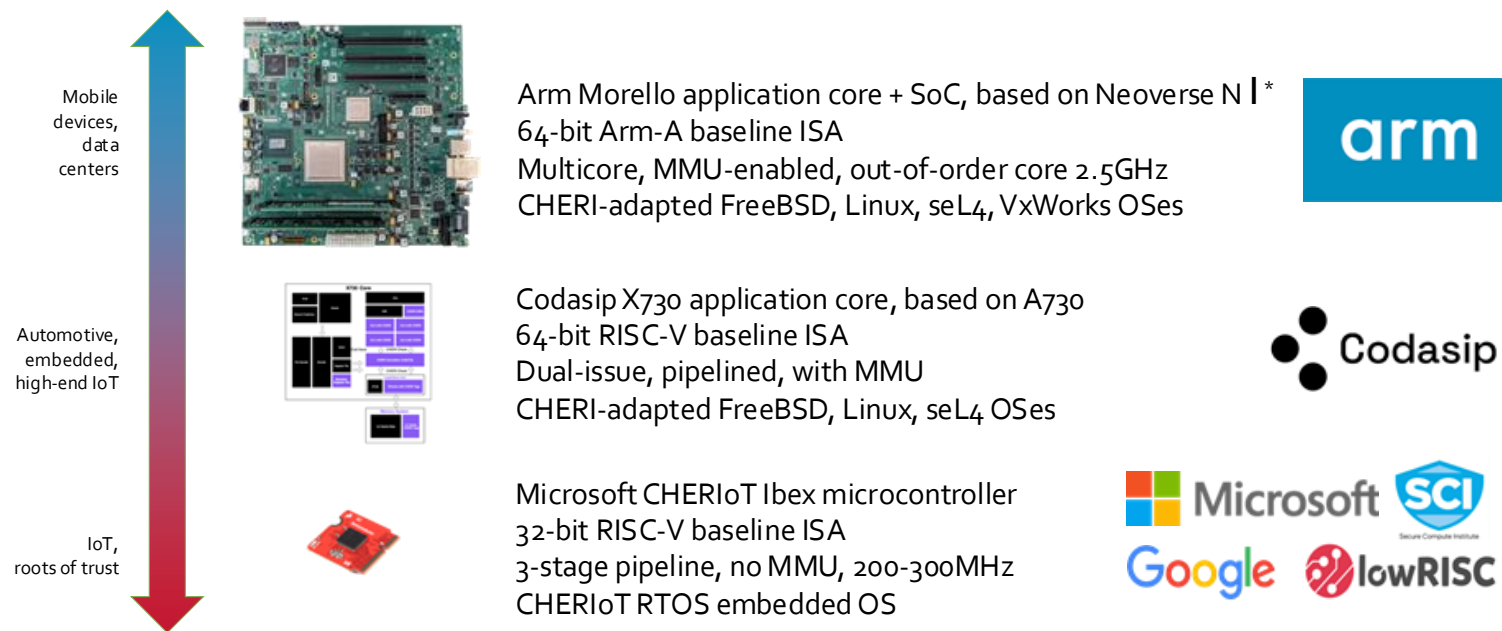
`jalr rd, o(sentry) # unseals the sentry and jumps to the address`

Compartment transitions with sentries



- ✧ Cross-compartment calls jump to **domain-transition trampolines** with additional capabilities
 - ✧ Trampolines implement software-defined properties such as **well-structured control flow, stack partitioning, and mutual distrust**
 - ✧ They can model various behaviours including **function calls** and **message passing**
- ✧ Register operands and return values can **pass capabilities** across domain transitions

CHERI demonstrated at a range of scales



* Supported by Innovate UK as part of Digital Security by Design (DSbD)

Summary

CHERI's memory-based capabilities:

- Guarantee pointer provenance and integrity
- Allows the compiler to encode bounds and permissions information
 - Automatically mitigating buffer-flow/over-run attacks
 - Toward the *principle of intentional use*
- Allow very efficient compartmentalisation
 - Toward the *principle of least privilege*
- Latest news:
 - CHERI for RISC-V is in the final stages of being standardised. ETA Q1 2026.

Finishing lectures 1-10

- Hope you've enjoyed the first part of this course exploring the architecture of single processors
- Lectures 11-16 will explore systems-on-chip (SoC) design including multiprocessors and graphics processing units (GPUs)

Introduction to Computer Architecture

Lecture 11

Overview of Systems on Chip (SoCs)

Prof. Simon W. Moore

Thanks to Dr Timothy Jones for the original version of these slides

Overview of Lectures

- We've got 6 lectures to talk about SoCs and their components
- This lecture: high-level SoC designs and memory
- Next 2 lectures on multicore
- Another 2 on GPUs
- Final lecture on the directions computer architecture is taking

What is a SoC?

- “An ensemble of processors, memories and interconnects tailored to an application domain”

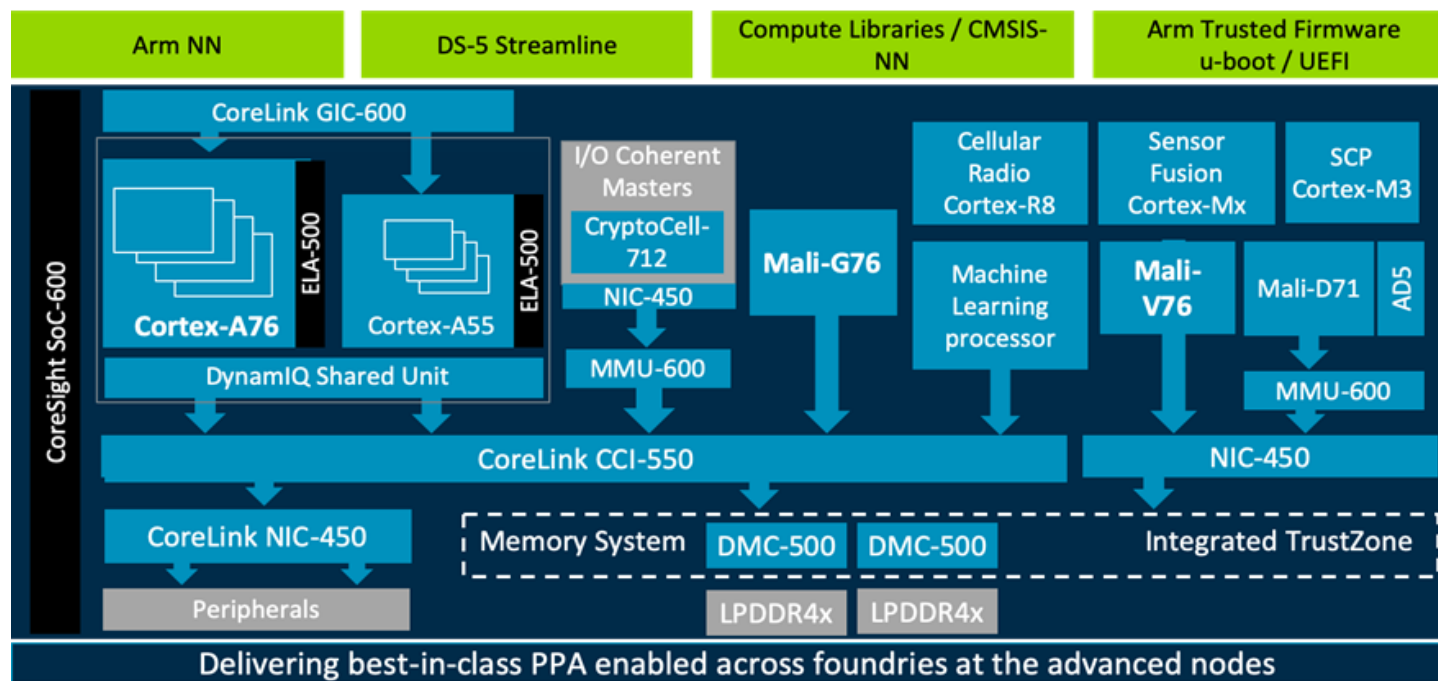
Flynn and Luk, Computer System Design: System-on-Chip

- An integrated circuit possibly containing:
 - Processors (CPU, GPU, DSP, etc)
 - Memory (ROM, RAM, etc)
 - Interconnects (a bus or NoC)
 - External interfaces (USB, ethernet)
 - Timing (phase-locked loops for clock generation; delay-locked loops for input signal deskewing)

Why use a SoC

- We design a SoC for several reasons
- SoCs have tight coupling between components
 - Beneficial for latency and power
- SoCs can be customised
 - Alter the components to target a specific domain
 - E.g. add cryptographic functionality when security required
- SoCs can be built from different parts
 - Buy IP from different vendors and integrate together
 - IP cores “glued” together by ASIC interconnect rendering tool
 - E.g. AXI interconnect for memory traffic

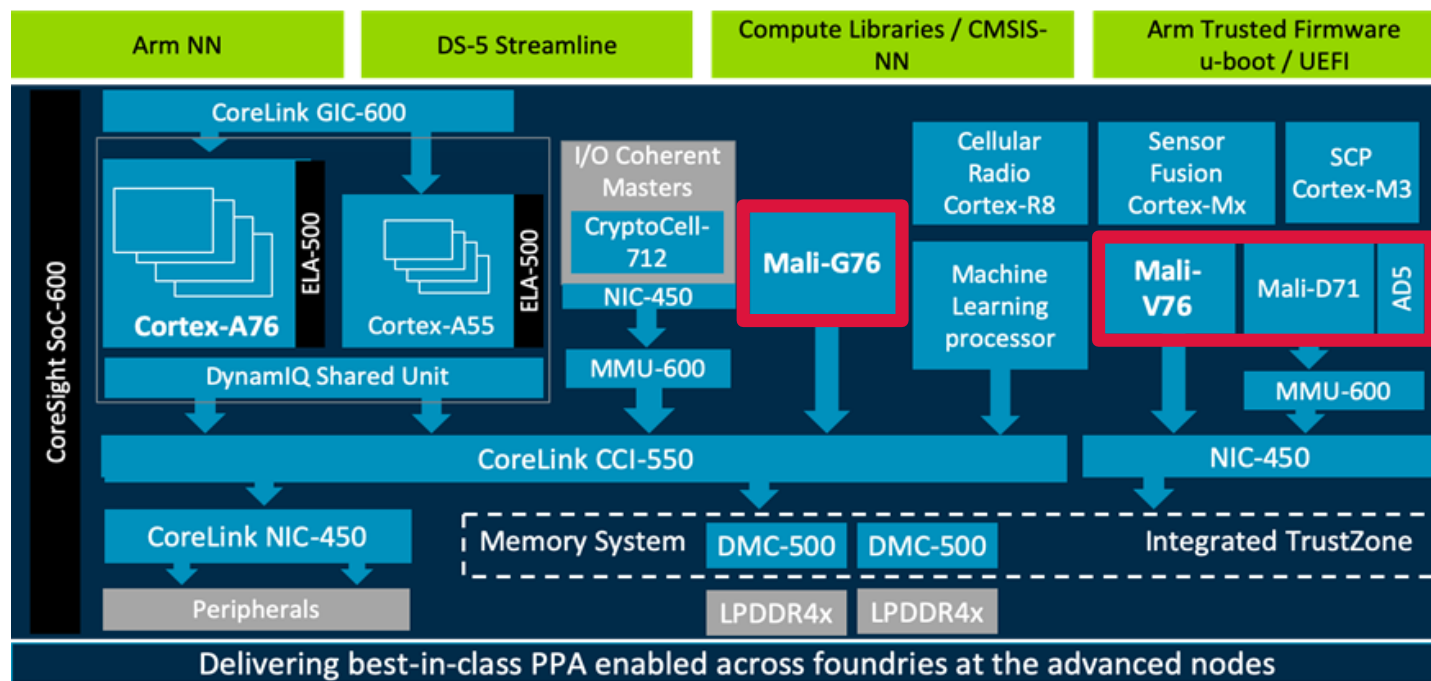
A Modern Arm SoC



Source: Arm

5
Copyright © Simon W. Moore, 2025

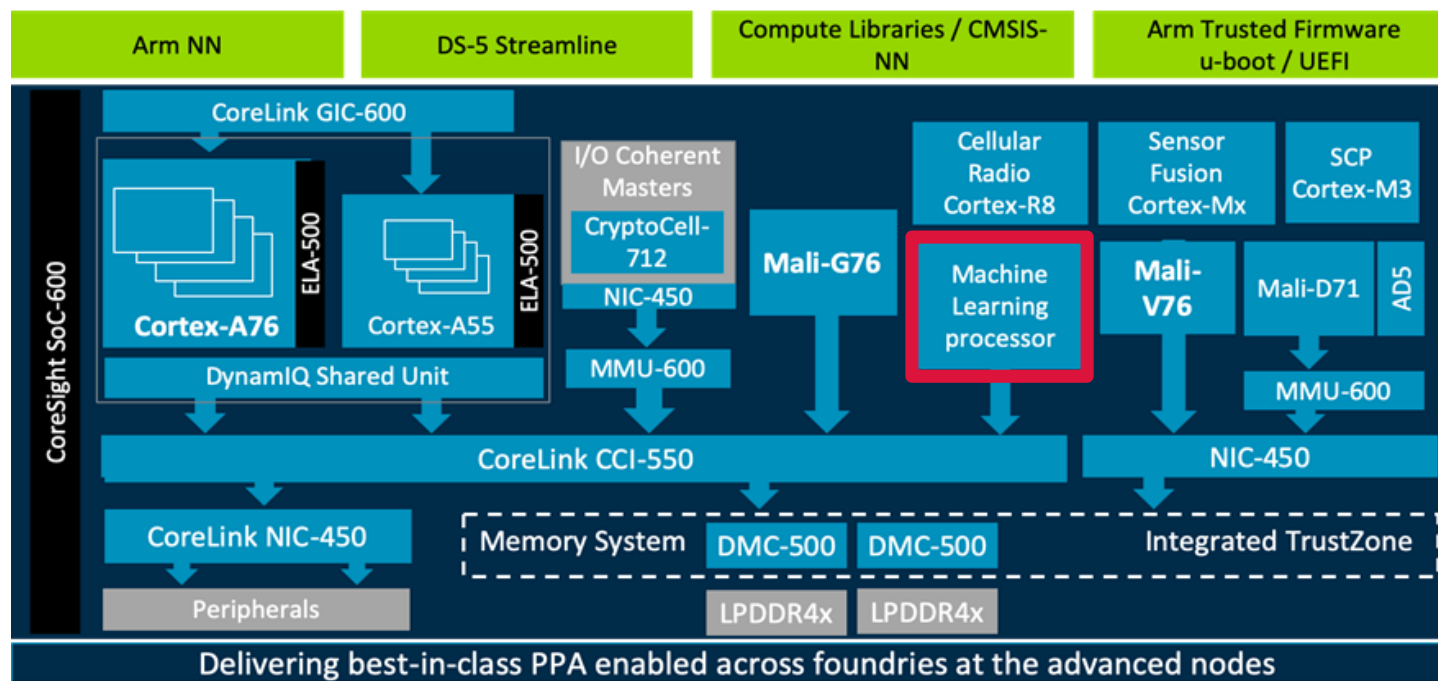
Graphics and Display IP



Source: Arm

6
Copyright © Simon W. Moore, 2025

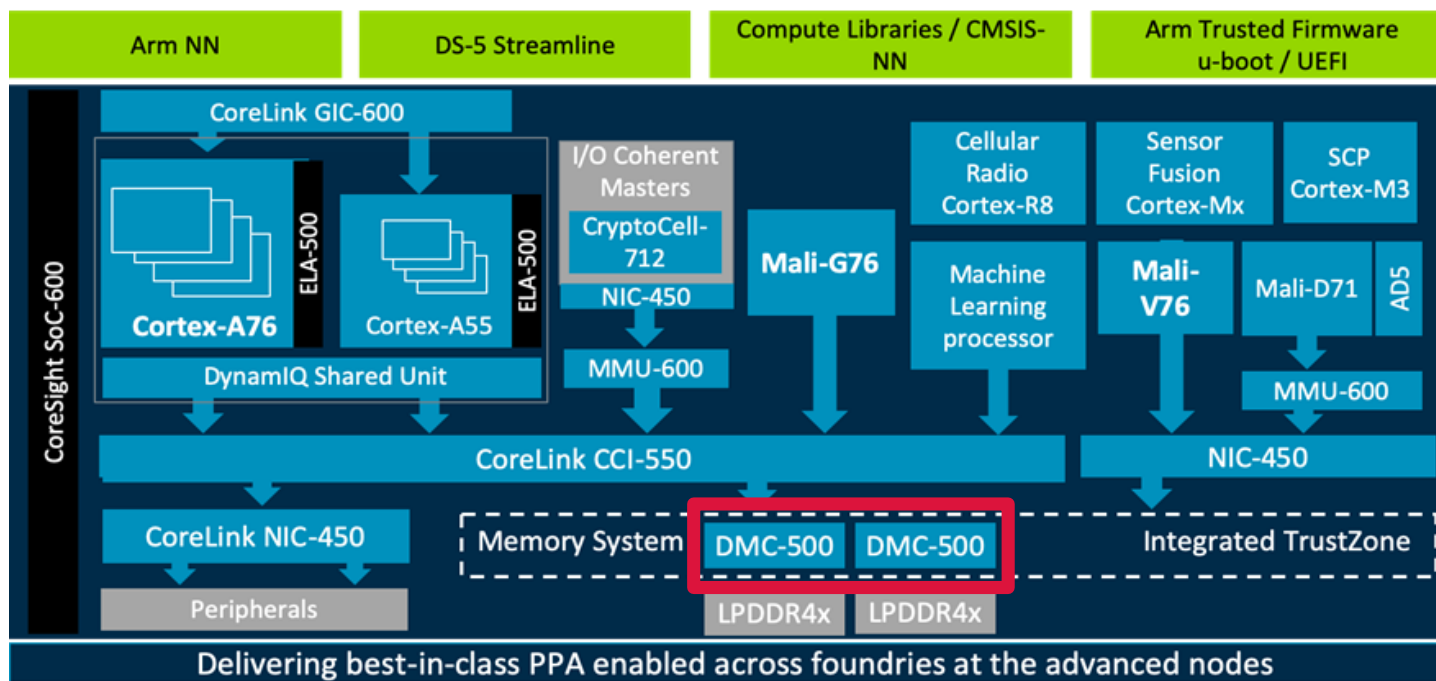
Specialisation



Source: Arm

7
Copyright © Simon W. Moore, 2025

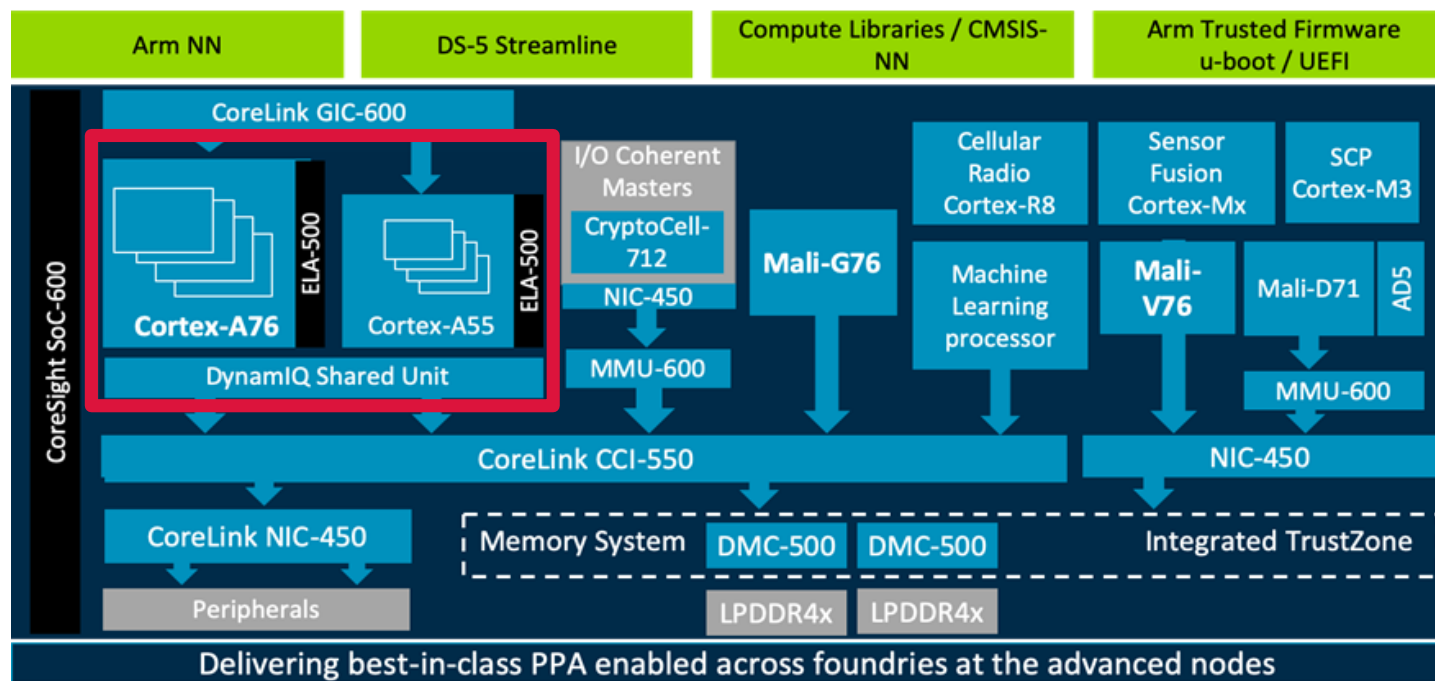
DRAM Interface



Source: Arm

8
Copyright © Simon W. Moore, 2025

Heterogeneous Cores



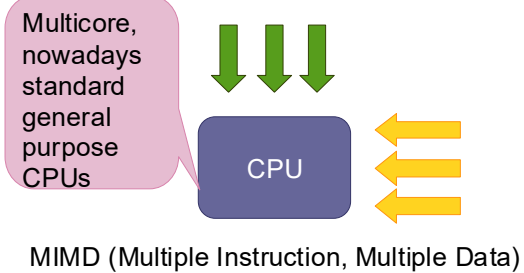
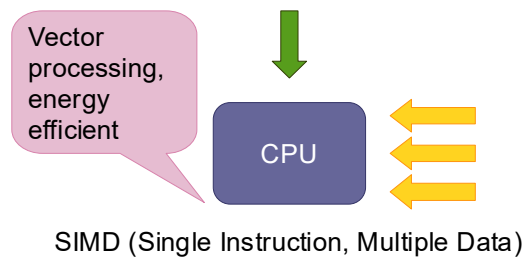
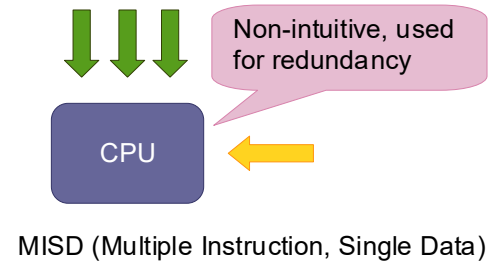
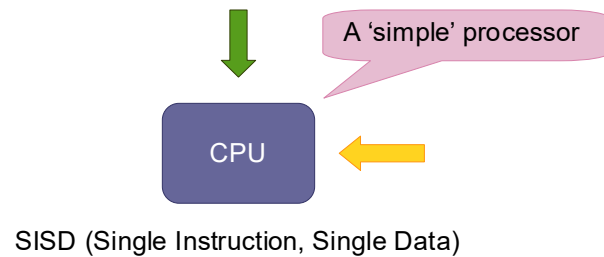
Source: Arm

9
Copyright © Simon W. Moore, 2025

Flynn's Taxonomy

- A classification of computing systems
- Shows parallelism in instruction and data streams
- Proposed by Michael Flynn
 - M. J. Flynn. Some Computer Organizations and Their Effectiveness, IEEE Trans. on Computers C-21 (9), September 1972

Flynn's Taxonomy



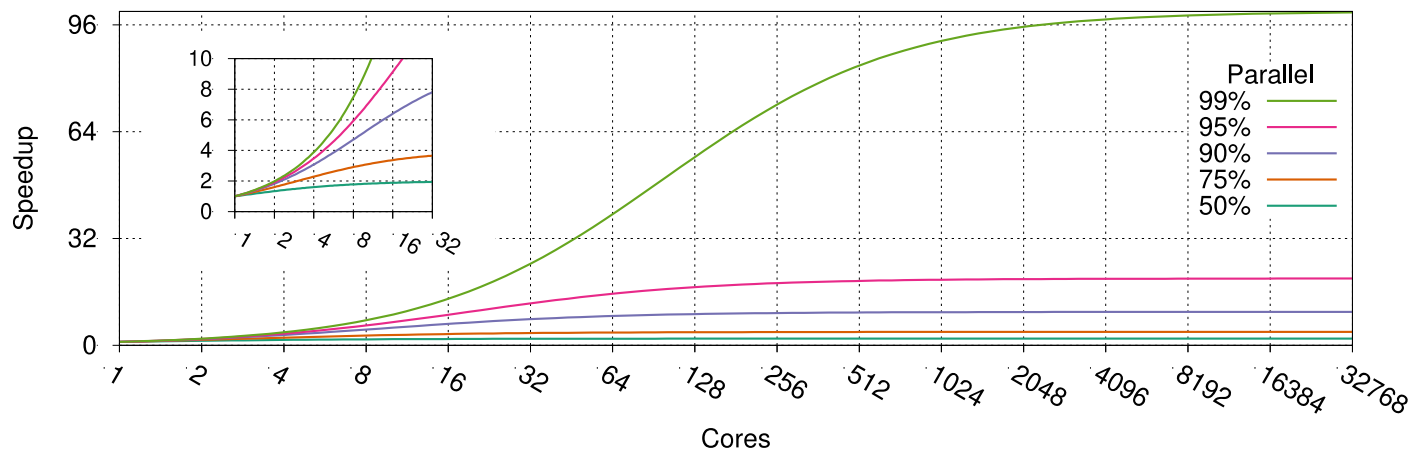
→ Instructions

→ Data

Limits of Parallelism

- Nowadays we can always add more cores
 - Especially in a GPU (see lectures 14 & 15)
- But performance will be limited by parallelism
 - Speedup depends on how much we use the cores
- Amdahl's law shows us how these are related
 - It is often used to determine speedup from parallelism
 - But can be used to find the maximum improvement for any system when only part is improved

Amdahl's Law



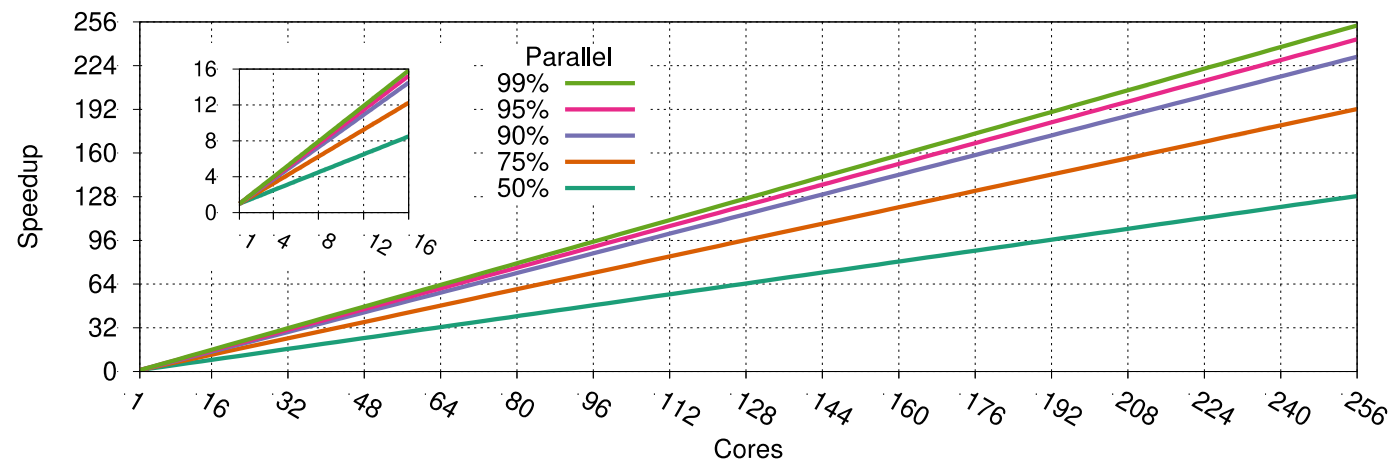
- Shows expected performance as cores increase
 - For a sequential fraction, B, and number of cores, n

$$Speedup(n) = \frac{1}{B + (1 - B)/n}$$

Amdahl's Law

- Amdahl's law can be used to estimate speedup for a workload when parallelising part of it
 - There are several assumptions built in
- It implies that we need to write very parallel programs to make best use of the cores
 - Basically speedup is limited by the sequential part
 - Therefore minimise this as much as possible
- Or thinking about generic system improvement
 - Maximum is limited by the fraction being improved
 - E.g. 75% of the system improved = 4x overall

Gustafson's Law



- Also shows speedup when cores increase
 - For a sequential fraction, B, and number of cores, n
$$Speedup(n) = n + (1 - n)B$$

Gustafson's Law

- Gustafson's law can be applied when a workload can be scaled up with increased compute
 - The parallel part scales, sequential part remains constant
- It implies that when given more computing power, programmers increase the problem size
 - In graphics, a better frame rate or resolution
 - In physics, the n-body problem, more particles
- Each law assumes one part is fixed
 - Amdahl's law assumes a fixed problem size
 - Gustafson's law assumes a fixed execution time

Memory Requirements

- What memory to provide depends on domain
- On-chip ROM and RAM may be sufficient
 - E.g. in a microcontroller
- May need to place memory off-chip
 - Because it won't all fit on the chip
 - E.g. A SoC for a mobile phone / tablet

Memory Placement

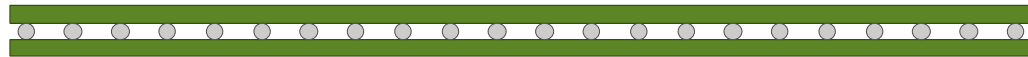
- Why place all memory on the chip?
 - Tighter integration is good for latency and bandwidth
 - Which in turn means less need for a cache
 - It may be cheaper since everything is self-contained
- Why leave memory on a separate chip
 - Space is a premium on-chip and may not be enough
 - DRAM process technologies differ from standard
 - Off-chip is more generic – greater applicability

Memory Considerations

- A number of other issues to consider
- Does there need to be virtual memory support?
 - E.g. to map disk space onto memory
 - See lecture 9
- Should memory be centralised or distributed?
 - Spread out across SoC modules
- How does the programmer see memory?
 - Shared memory (lecture 12) or message passing?

Chip Stacking

- One method is known as package on package
 - Here, a memory chip is put directly on top of the SoC
 - A ball grid array is used to connect the two

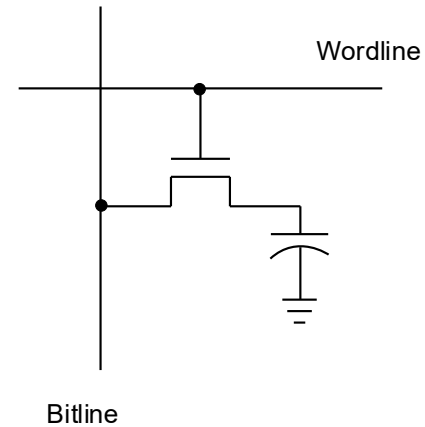


- An alternative is 3D die stacking
 - Here the silicon wafers are directly placed together
 - Through-silicon vias connect the dies



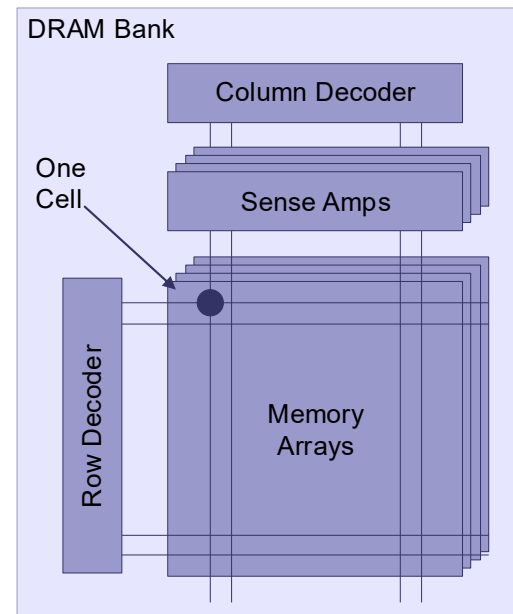
A DRAM Cell

- A 1T1C DRAM cell
 - 1 Transistor
 - 1 Capacitor
- Wordline selects the bit
- Bitline contains
 - Data read out
 - Data written in
- Capacitor not perfect
 - So needs refresh



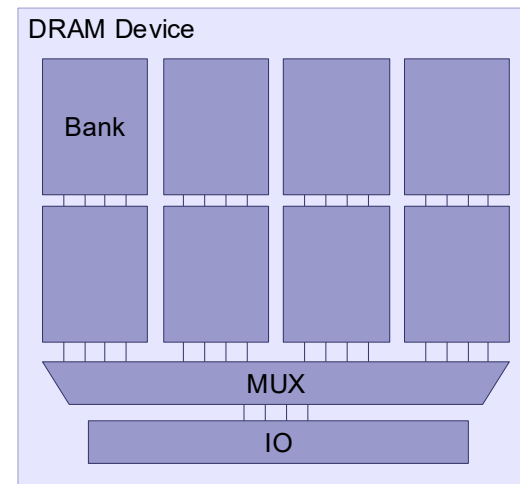
DRAM Banks

- Cells organised into arrays
 - Rows = wordlines
 - Columns = bitlines
- Each array provides a single bit
 - Multiple arrays needed to access more than one bit at a time
- A group of arrays is called a bank



DRAM Devices

- DRAM devices contain several banks
- Each bank operates independently
 - Each contains N arrays
 - Provides N bits of data
- Can pipeline access to different addresses
 - Contend with bank conflicts



DRAM Ranks

- DRAM devices may be arranged in ranks
 - All devices in a rank operate together
 - Each rank is independent
- Concurrency at ranks and banks gives bandwidth
- Synchronised operation at devices and arrays allows parallel accesses
- So, for example, a 64-bit read could come from
 - 1 rank of 4 devices, each containing 8 banks
 - Each bank has 16 arrays of 8192 rows, 512 columns
 - So 4 devices X 16 arrays = 64 bits

DRAM Commands

- DRAM is controlled through commands sent to it
- These identify different operations to perform
- They define low-level actions to take
 - Row access – move data to sense amps, then back
 - Column read – move data from sense amps to bus
 - Column write – move data from bus to sense amps
 - Precharge – reset sense amps and bitlines
 - Refresh – read out data then restore

Memory Controller

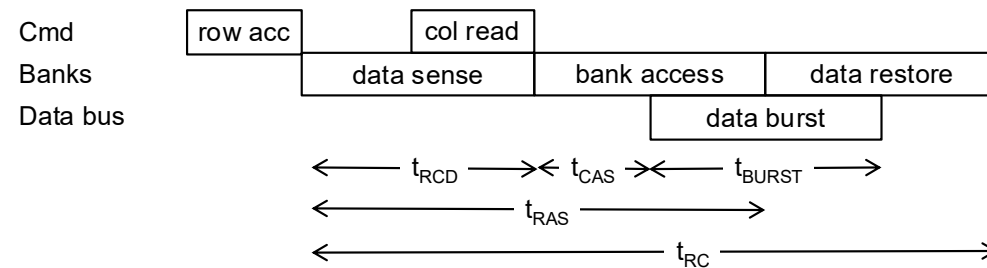
- The memory controller handles DRAM accesses
- It is responsible for converting requests from the processors into commands for the DRAM
- E.g. load address 0xffff0000 becomes:
 - Select bank x
 - Row access y
 - Column read z
 - Precharge (omitted with an open-page row-buffer policy)

Open-Page vs Close-Page

- The sense amps act like buffers
 - They store the most recent row that was accessed
 - To access another row, they must be reset
- An open-page policy doesn't reset
 - Reduced latency when reading from that row again
 - Great for sequential access to different columns
 - Requires explicit reset when accessing another row
- A close-page policy does reset and precharge
 - Which favours access patterns with little locality

DRAM Read

- Each command takes a certain amount of time
- The memory controller issues each command so in sequence to minimise this time



DRAM Timings

- DRAMs often denoted by key timing latencies
- For example DDR4-1600 11-11-11-28
 - That's DDR4 DRAM at 1600m transfers / second
 - With tCAS = 11 clock cycles (column address strobe)
 - And tRCD = 11 clock cycles (row to column delay)
 - And tRP = 11 clock cycles (row precharge)
 - And tRAS = 28 clock cycles (row access strobe)

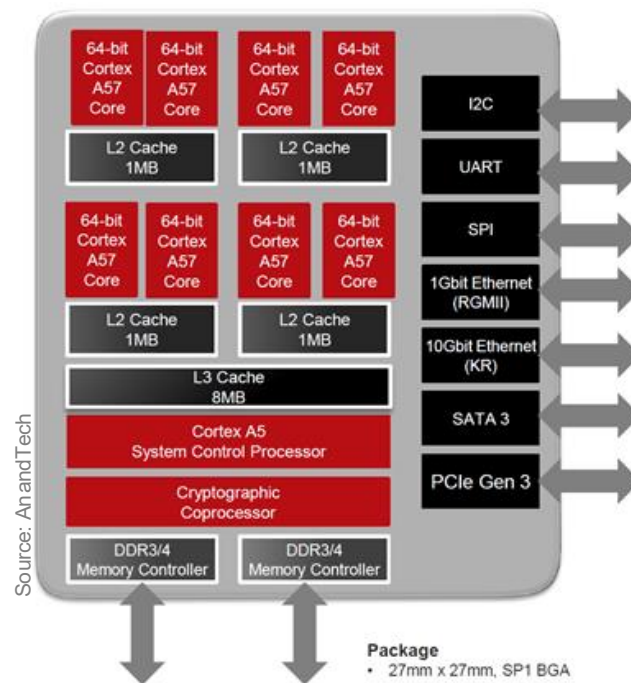
Key DRAM Timings

Timing	Description
t_{BURST}	Time that a data burst takes on the data bus
t_{CAS}	Column address / access strobe – time between column access command and the start of data from the DRAM device
t_{CMD}	Time that a command takes to go from the controller to devices
t_{RAS}	Row address / access strobe – time between row access command and data restoration in the DRAM array
t_{RC}	Row cycle – time interval between accesses to different rows in the same bank, so $t_{RC} = t_{RAS} + t_{RP}$
t_{RCD}	Row to column command delay – time between row access and data ready at the sense amplifiers
t_{RP}	Row precharge – time for an array to be precharged for another row access
t_{WR}	Write recovery – minimum time between a write burst and start of precharge

Putting It All Together

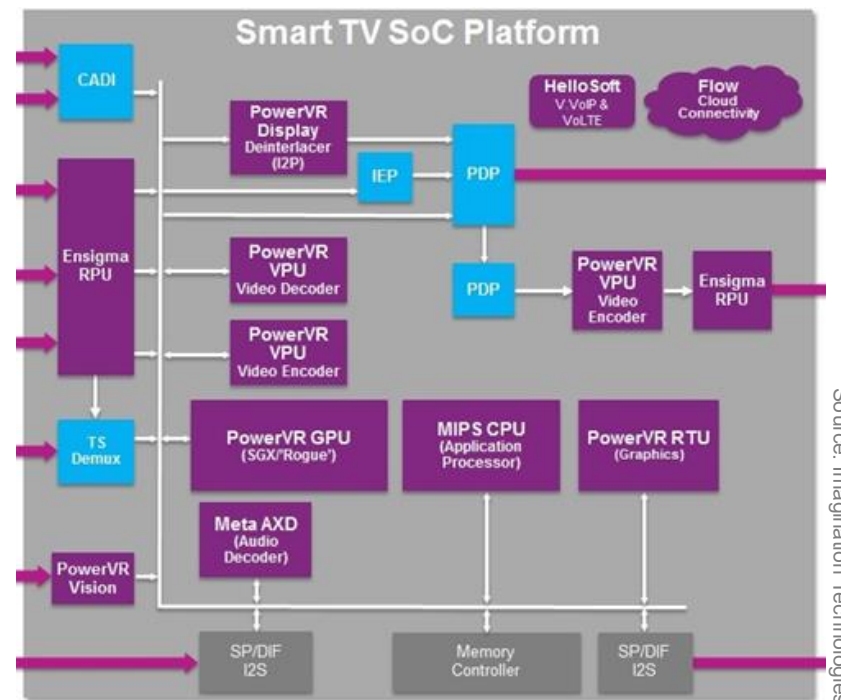
- We want to build SoCs that target a particular application domain
 - It meets certain requirements for that domain
 - But note that the domain needs to be large enough to warrant the large none-recurrent costs of fabricating chips, e.g. mask costs
- We must carefully consider all components used
 - How are they integrated with each other
 - Customisation if required
- Let's now see some examples of existing SoCs
 - Already looked at one Arm-based system

AMD Seattle



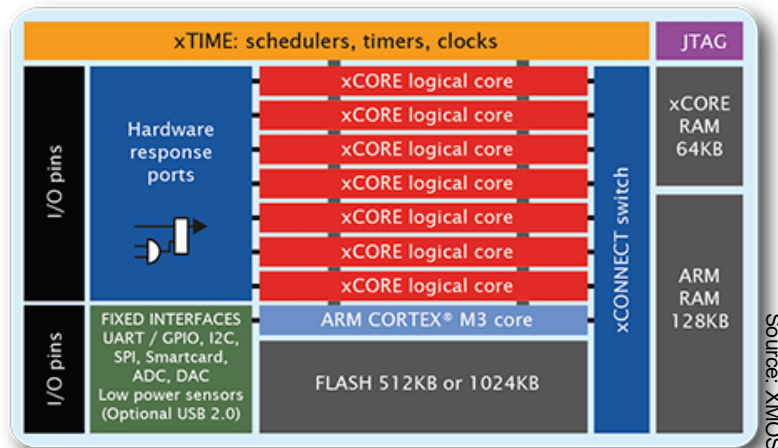
- This Opteron is designed for servers
- Cryptograph co-processor to offload functionality from A57s
- System control co-processor is a SoC itself used for services

Imagination Technologies



Source: Imagination Technologies

XMOS xCORE-XA



- Small ARM processor
- xCORE microcontrollers
- Benefits of both legacy ARM code & support and real-time xCORE

- XMOS xCORE-based processors used in
 - USB Audio 2.0 interfaces
 - Ethernet & industrial communications

Samsung Exynos

- Target market is high-end phones & tablets
- big.LITTLE clusters with a Mali GPU
- Jpeg (de)compression
- Memory compression

Source: AnandTech

35
Copyright © Simon W. Moore, 2025

- Source: AnandTech

Summary

- SoCs come in all shapes and sizes
- They are designed for a particular domain
- Lots of opportunities for customisation
 - Number and types of core
 - Memory layout
 - I/O, including wired and wireless links
 - Interconnect
- Next lecture looks in more depth at multicore

Introduction to Computer Architecture

Lecture 12

Multicore Processors

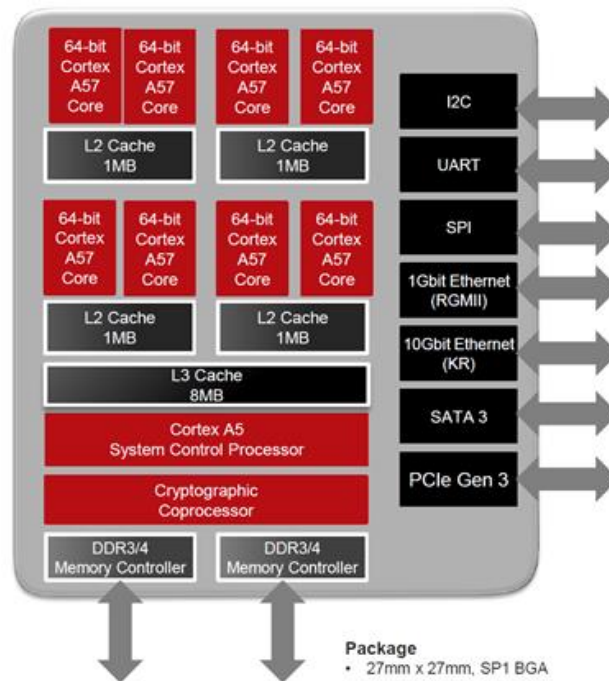
Prof. Simon W. Moore

Thanks to Dr Timothy Jones for the original version of these slides

Overview

- Last lecture
 - High-level view of SoCs
 - Different SoC designs
 - Flynn's taxonomy
- This lecture
 - Multicore organisations
 - Cache coherence

What Do We Mean By Multicore?

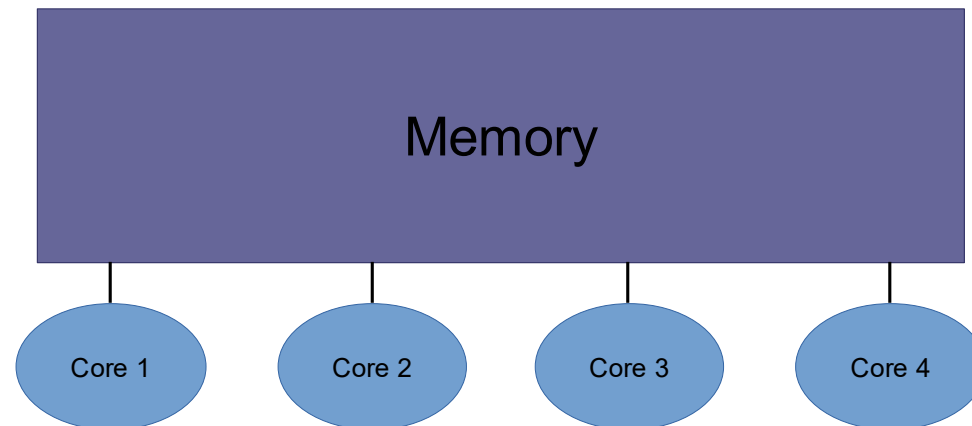


- Multiple cores together on the same die
- Each core capable of running a stream of instructions
- Cores are independent, but may work together
- MIMD parallelism

Multicore Organisation

- We must organise the cores somehow
- We need to enable communication between them
- Recall last lecture there were two forms
 - Shared memory
 - Message passing
- We're going to focus on shared memory systems

Shared Memory



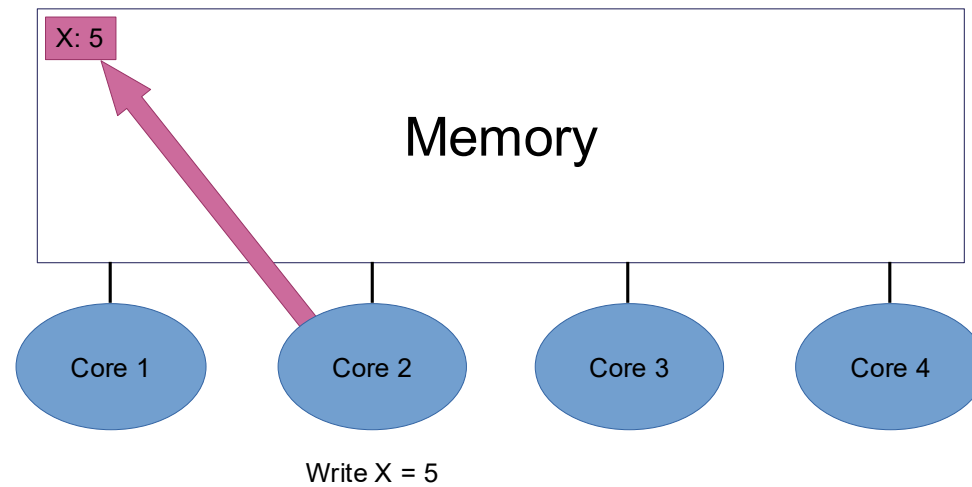
Shared Memory

- No matter how the cores are organised, they all see the same values in memory
- All cores see this as a single address space
- Writes by one core are eventually seen by others
 - This doesn't have to happen instantly
 - But it does mean that reads always get the right value, the one written by the latest write

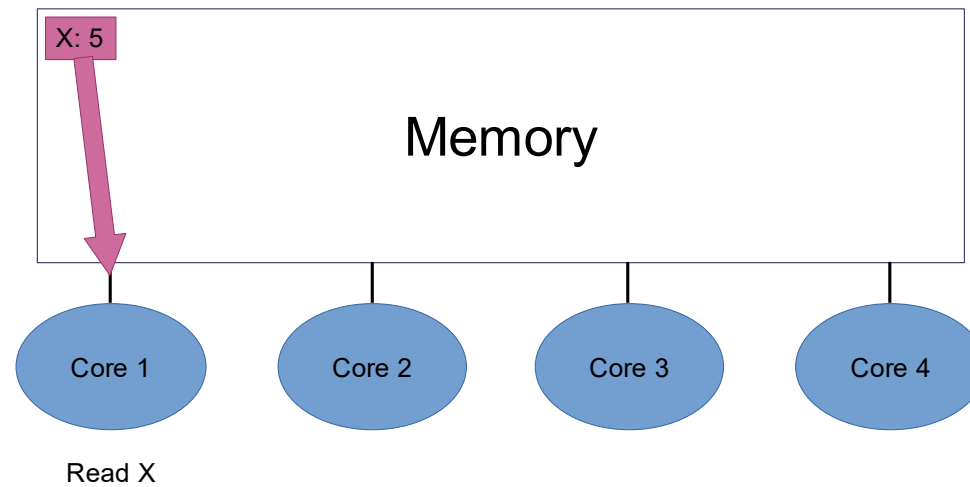
Shared Memory Benefits

- Shared memory is a useful concept
- Different cores can use memory to communicate
 - One core writes the value that the other reads
- This is the view programmers have
 - Therefore shared memory is easy to program
- Shared memory systems are common
 - The basis for desktop, server, mobile systems

Shared Memory



Shared Memory



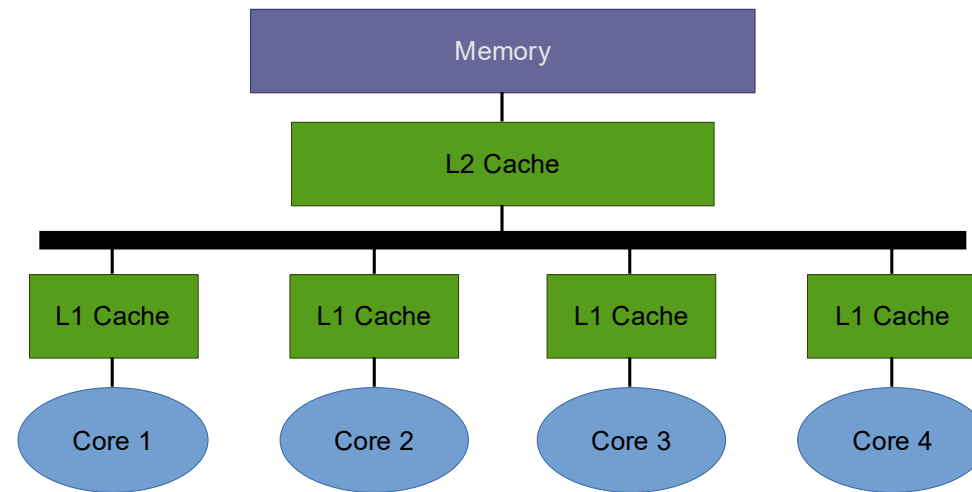
Multicore Caching

- DRAM memory is very slow
 - So we add a cache hierarchy
- With multiple cores, we have various organisation options
 - How many caches?
 - Which caches are shared?
 - Who shares them?
 - Is data duplicated?

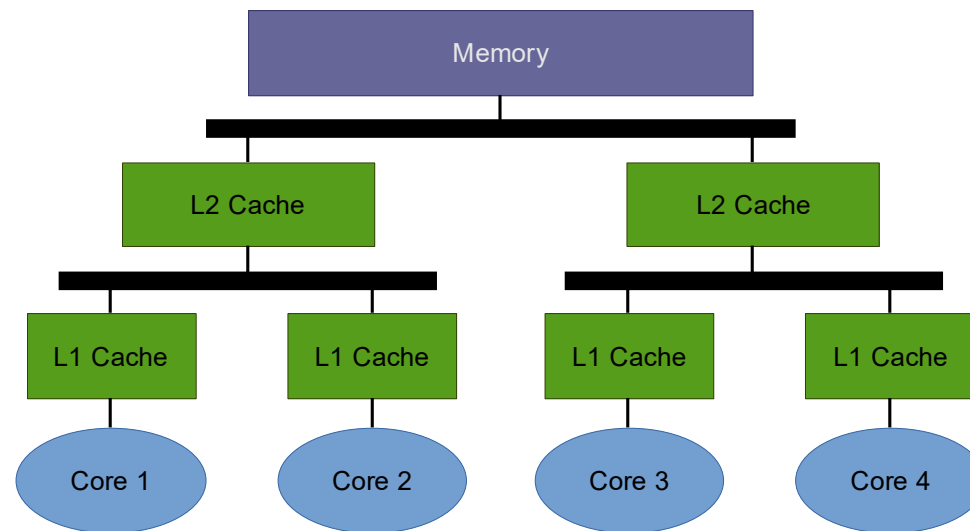
Cache Sharing

- Shared caches
 - Good use of space
 - Data shared between cores faster than memory
- Private caches
 - Provide guaranteed space for each core
 - May be faster to access
 - E.g. two 1MB private caches vs. one 2MB shared cache
- Choice may depend on applications
 - Are they multithreaded / sharing data?

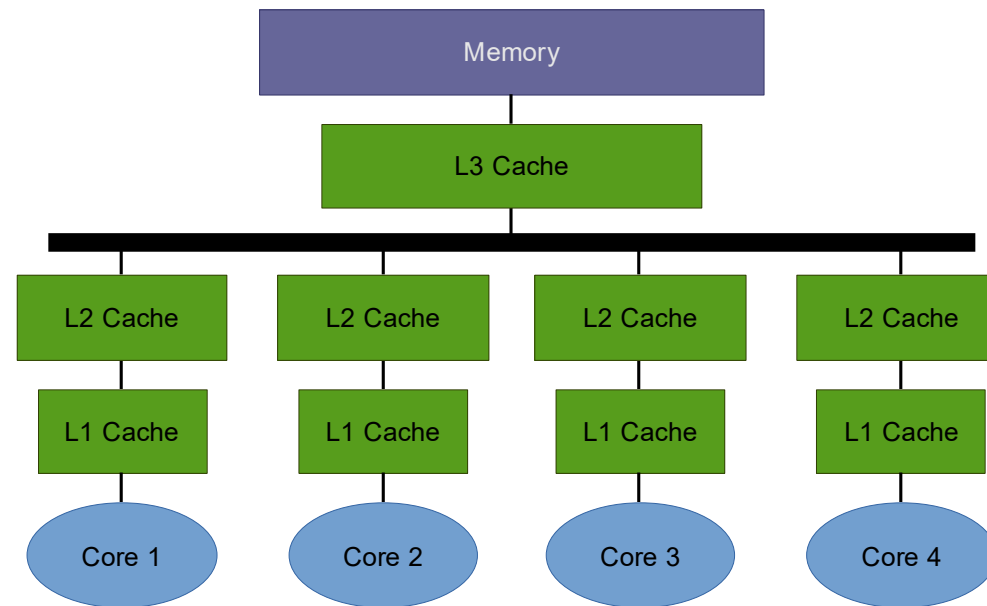
Single Level of Shared Cache



Multiple Shared Caches



Multiple Private Cache Levels

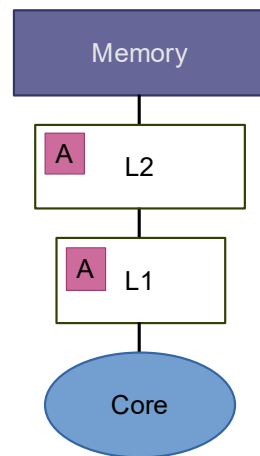


Cache Inclusion

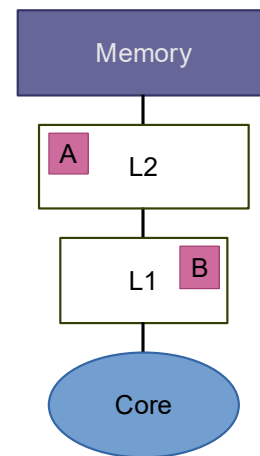
- Can data be duplicated at different cache levels?
- There are trade-offs
 - A miss may not have to go all the way to memory
 - But wasteful since L1 data is also in L2 (and L3)
- We have three schemes
 - Inclusive, Exclusive, Non-inclusive non-exclusive (NINE)
- Further reading
 - On the inclusion properties for multi-level cache hierarchies, Baer and Wang, ISCA 1998

Cache Inclusion

Inclusive

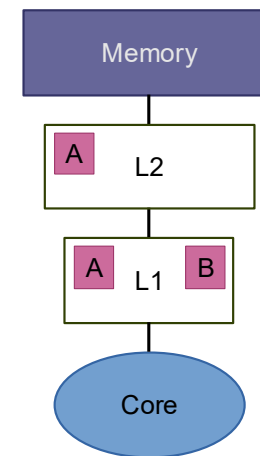


Exclusive

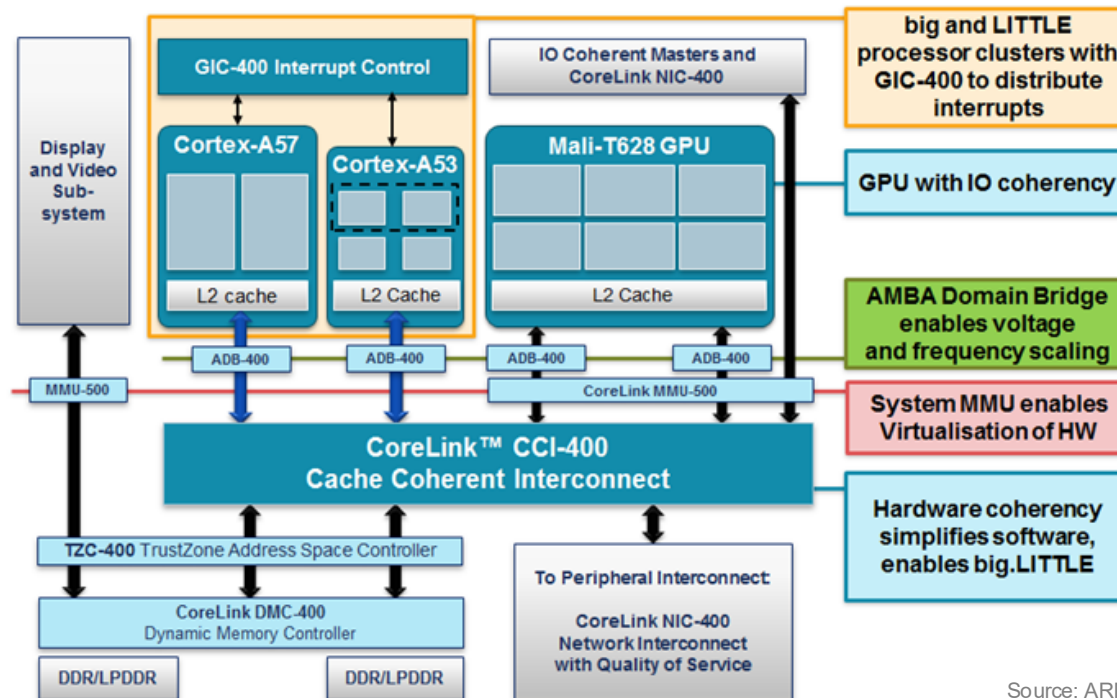


NINE

(Non-inclusive non-exclusive)



Example Cache Hierarchy

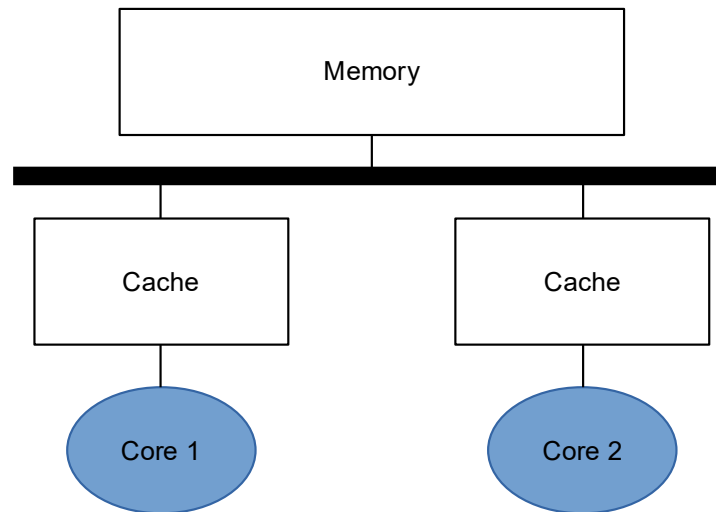


Source: ARM

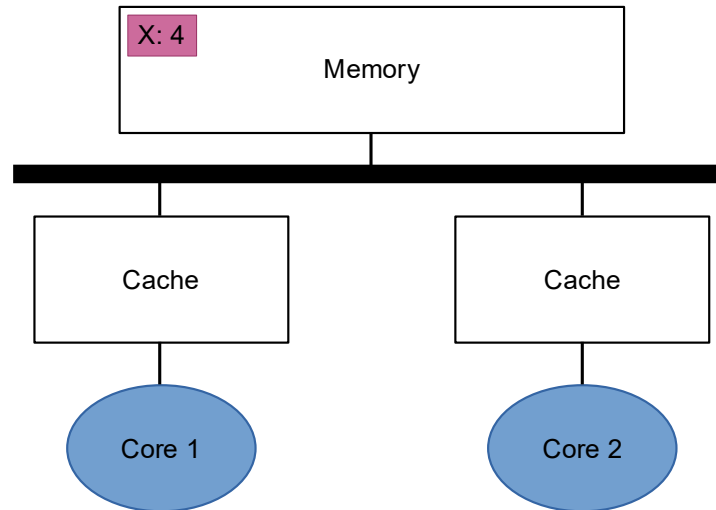
Cache Benefits

- Caches allow faster access to certain data
 - Faster than main memory
 - Data with temporal or spatial locality
- We prefer to use write-back caches
- But do these break our shared-memory model?

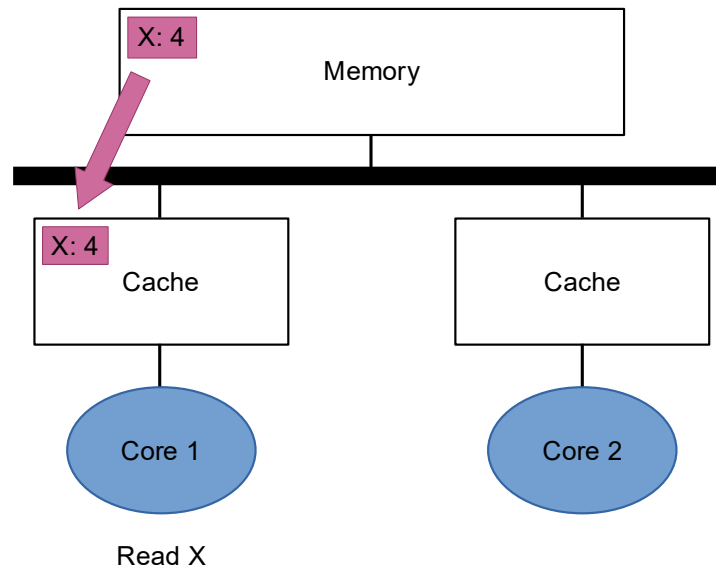
The Need For Coherence



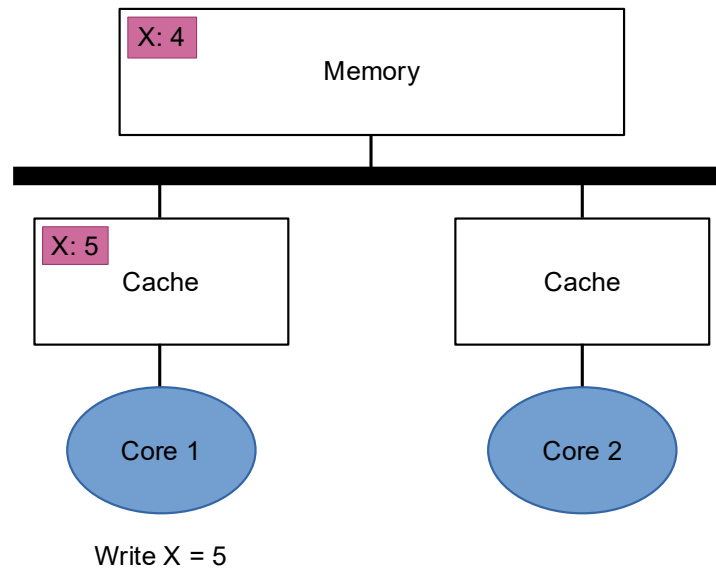
The Need For Coherence



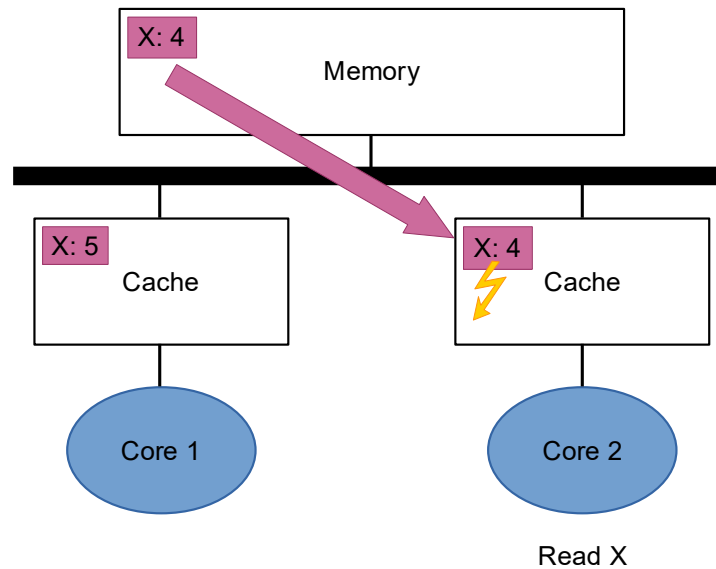
The Need For Coherence



The Need For Coherence



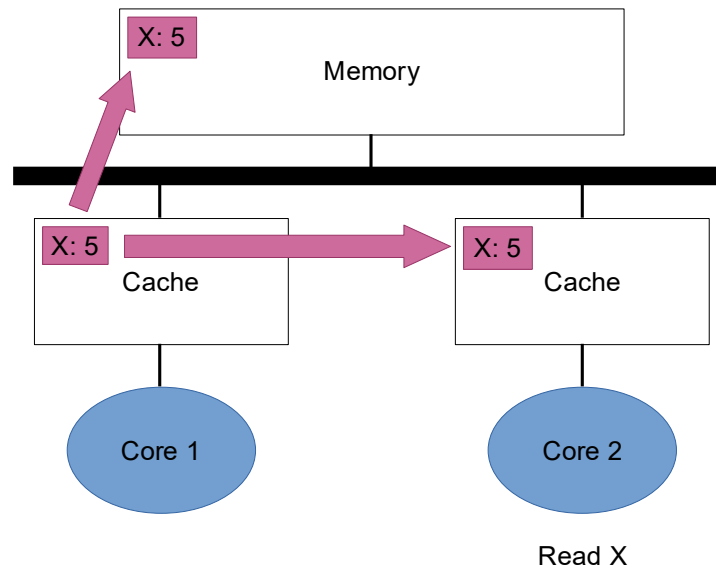
The Need For Coherence



The Need For Coherence

- Caches pose a problem for shared memory
- When some data has been updated in a cache, other caches can't see the new value
 - Not until it is flushed out or propagated down
- Write-through caches are one partial answer
 - But these create a large amount of traffic
 - We still need to propagate values to other cores
- Write-back caches are preferred
 - Ideally only propagate values when actually necessary (i.e. on reads by another core)

What We'd Like



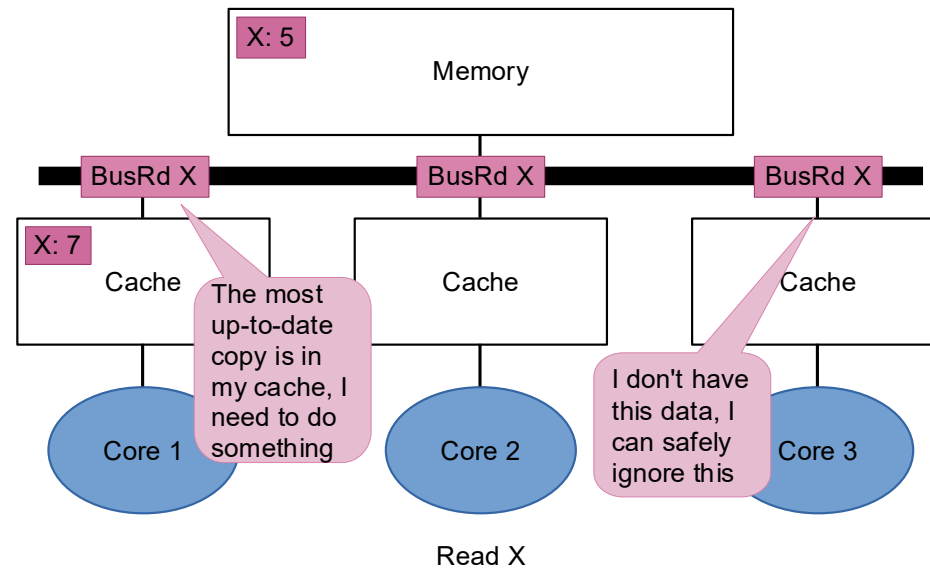
Cache Coherence Protocols

- We can implement hardware to propagate values
 - To keep caches and memory coherent
 - To ensure a consistent view of the system by all
- This hardware runs the cache coherence protocol
- The protocol ensures all cores read the most up-to-date value for each memory address
- We assume a snoopy bus
 - Caches can see (snoop) other cores' transactions
- No cache-to-cache sharing

Snoopy Bus

- A bus allows all caches to see what others do
 - We can 'snoop' on what they're doing
 - I.e. monitor the events happening around us
- Using this information we can make decisions about actions we might need to take
 - E.g. providing most up-to-date copy of data
- The following protocol relies on this knowledge

Snoopy Bus

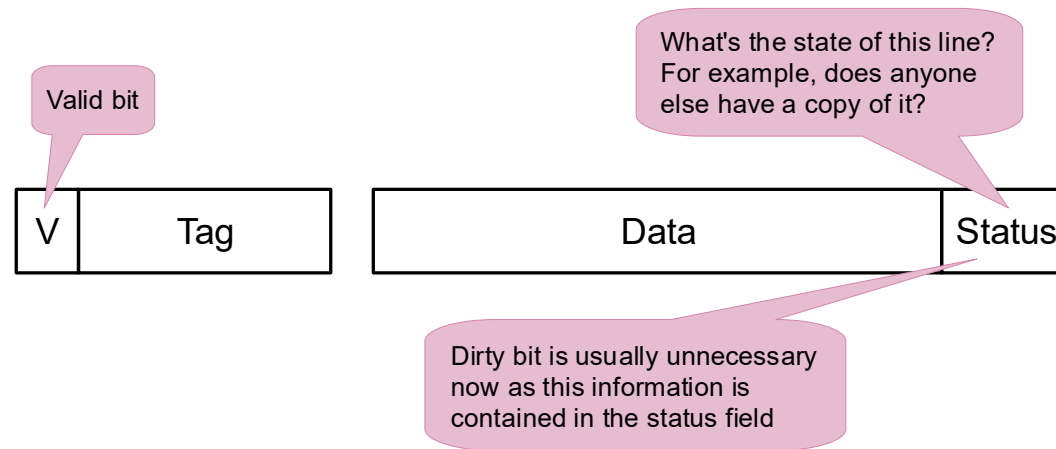


Cache Coherence Protocols

- But how do we know the most up-to-date value?
 - Associate status bits with each cache line
 - On access, the status tells us what actions to take
- The coherence protocol is run on each cache line
 - Therefore the status differs between lines
 - And the actions to take differ too
- We'll consider a protocol for one line

Example Cache Line

- Exactly what information the status bits convey is dependent on the cache coherence protocol that is being run

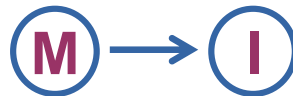


MSI Protocol

- **M**odified
 - Single up-to-date copy in this cache
 - Value in main memory is stale
- **S**hared
 - Copy in this cache matches memory
 - May be present in other caches
- **I**nvalid
 - Not valid (no copy) in this cache

MSI Protocol

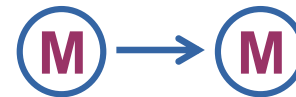
- We'll go through the protocol step by step
- At each stage, we'll see what happens to a cache upon seeing different events
- We'll see how the status of a line in this cache changes and the actions taken
- For example, transition from M state to I state



MSI Protocol: Local Read

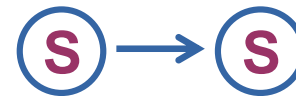
- **Status of line is M**

- We can simply return the data
- Read hit



- **Status of the line is S**

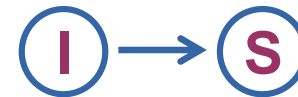
- We can simply return the data
- Read hit



MSI Protocol: Local Read

- **Status of line is I**

- We don't have a copy of the data
- Read miss
- Initiate a bus read to bring it into the cache

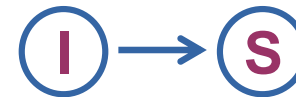


- **If the data is in another cache in M state**

- That cache snoops the bus and flushes the data back to memory
- It transitions to S state
- We get the data from memory
- We keep a copy in S state

MSI Protocol: Local Read

- **Status of the line is I**
- If the data is in other caches in S state
 - Those caches snoop the bus but do nothing
 - We get the data from memory
 - We keep a copy in S state
- If the data isn't in another cache
 - We get the data from memory
 - We keep a copy in S state



MSI Protocol: Local Write

- **Status of line is M**

- We can simply return the data
- Write hit



- **Status of the line is S**

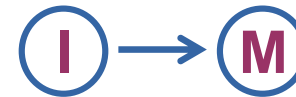
- Send an upgrade request to other caches
- Any caches that hold a copy (in S state) invalidate
- We transition to M
- Write hit



MSI Protocol: Local Write

- **Status of line is I**

- We don't have a copy of the value
- Write miss
- Initiate a bus read to bring it into the cache

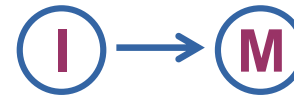


- **If the data is in another cache in M state**

- That cache snoops the bus and flushes the data back to memory
- It invalidates its own copy
- We get the data from memory
- We keep a copy in M state

MSI Protocol: Local Write

- **Status of the line is I**
- If the data is in other caches in S state
 - Those caches snoop the bus and invalidate their copies
 - We get the data from memory
 - We keep a copy in M state
- If the data isn't in another cache
 - We get the data from memory
 - We keep a copy in M state



Visualising The Protocol

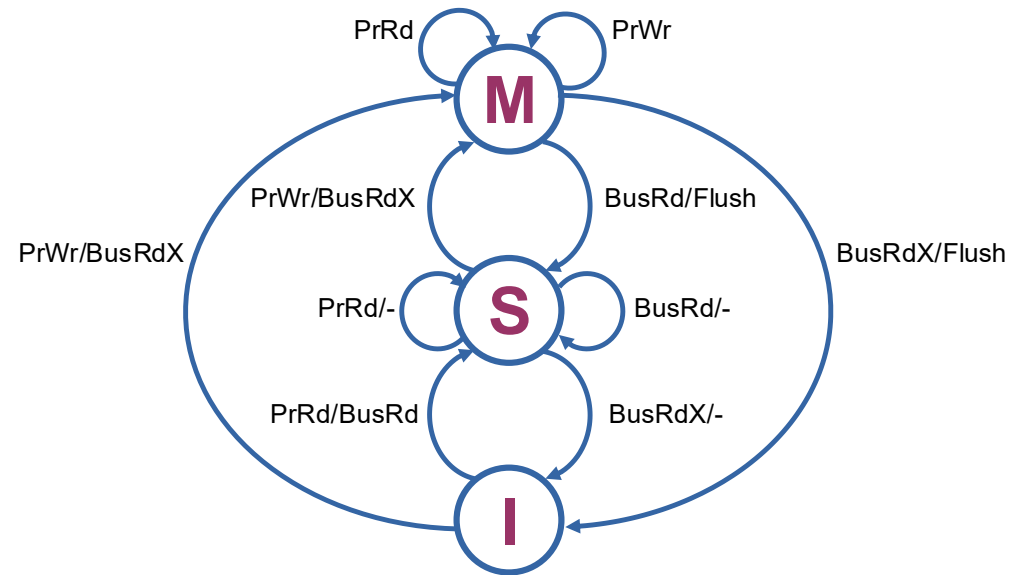
- State transition diagrams describe protocols
 - These show the events seen and actions
 - Helps simplify, visualise and understand the protocol
- State transitions are labelled
 - EventSeen / ActionTaken
 - E.g. PrRead / BusRd
 - Event: Local processor issues read request
 - Action: Initiate a read request on the shared bus

Events and Actions

- The protocol responds to events with actions
- This summarises those seen in the next slide

Short Name	Long Name	Description
PrRd	Processor Read	The processor attached to this cache wants to read a value
PrWr	Processor Write	The processor attached to this cache wants to write a value
BusRd	Bus Read	A read request transaction is placed on the bus
BusRdX	Bus Read Exclusive	An exclusive read request transaction is placed on the bus
Flush	Flush Data	Dirty data from the cache is flushed back to main memory

MSI Protocol

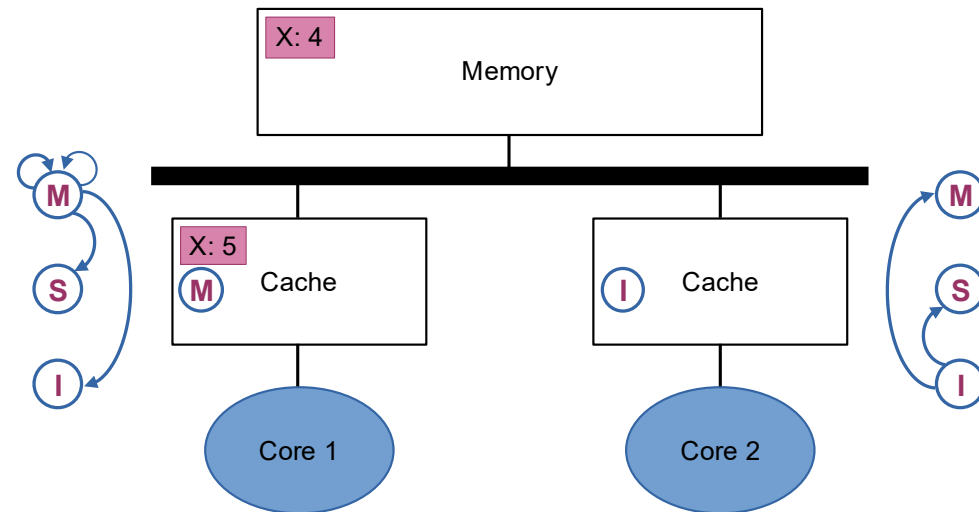


MSI Protocol

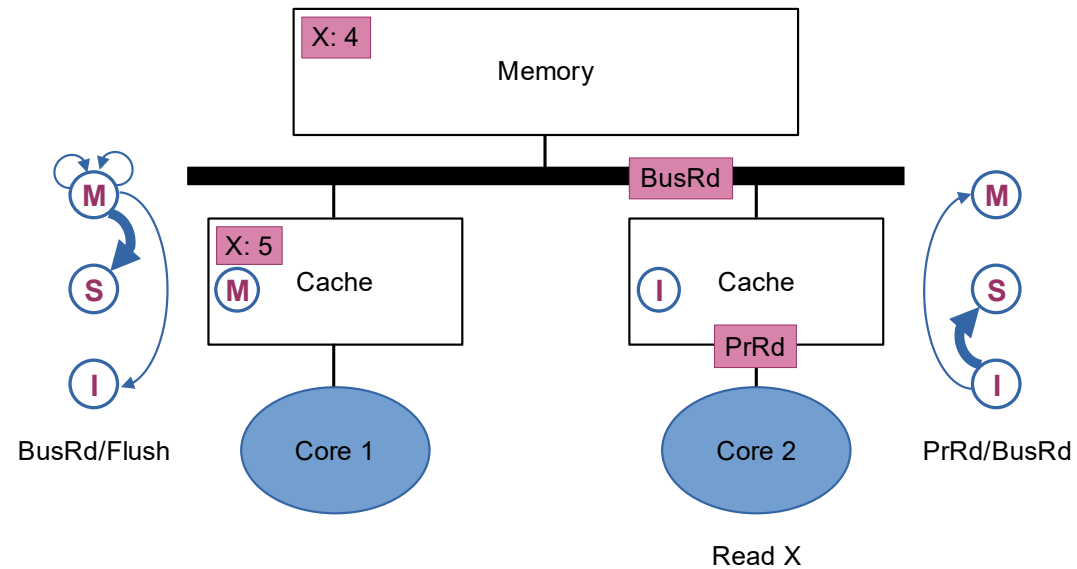
- The state of a line in one cache limits the states the same line in a different cache can have

	M	S	I
M	x	x	✓
S	x	✓	✓
I	✓	✓	✓

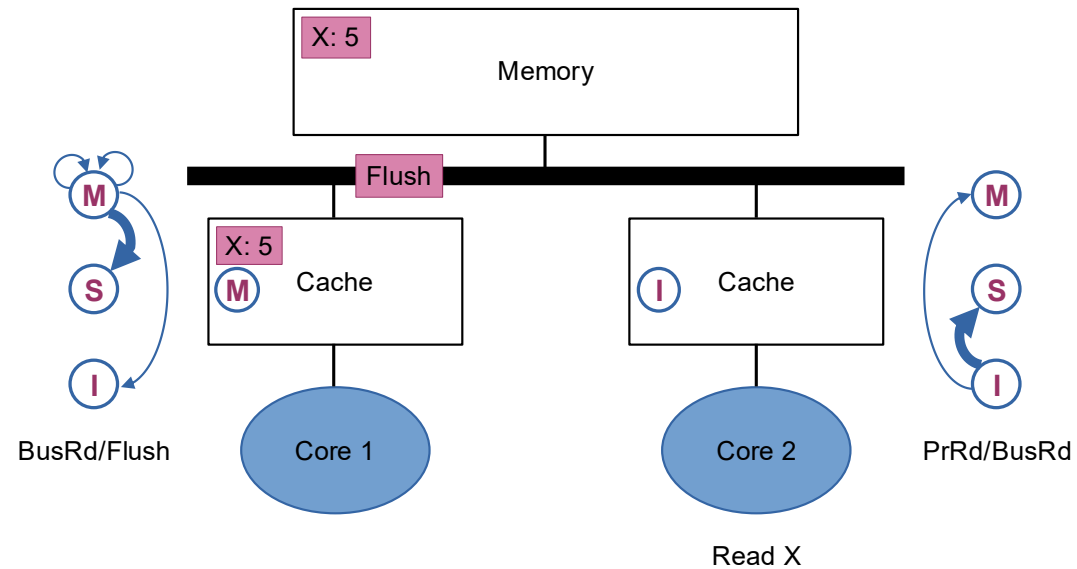
Coherency Example



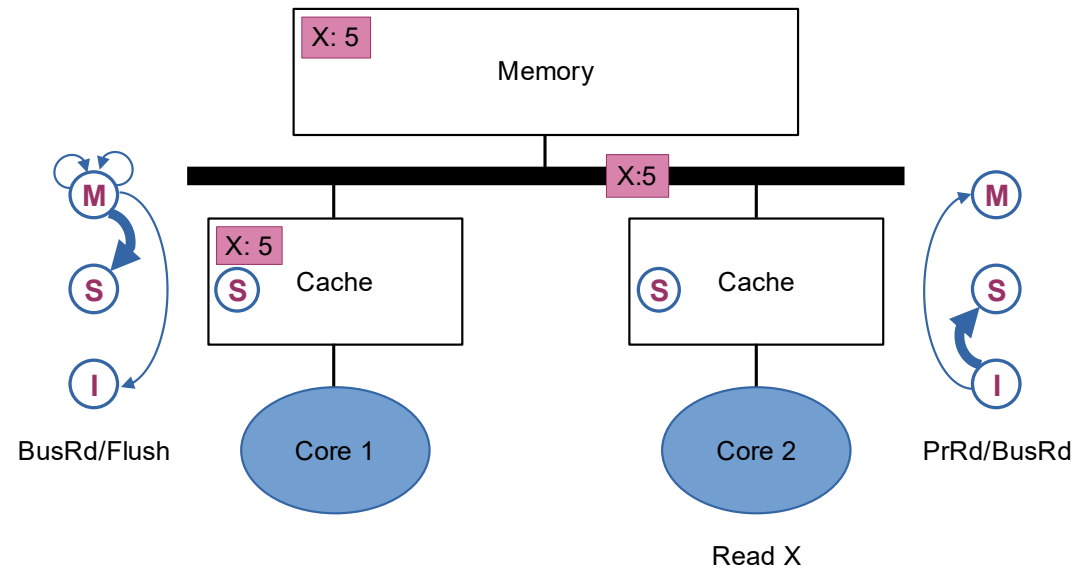
Coherency Example



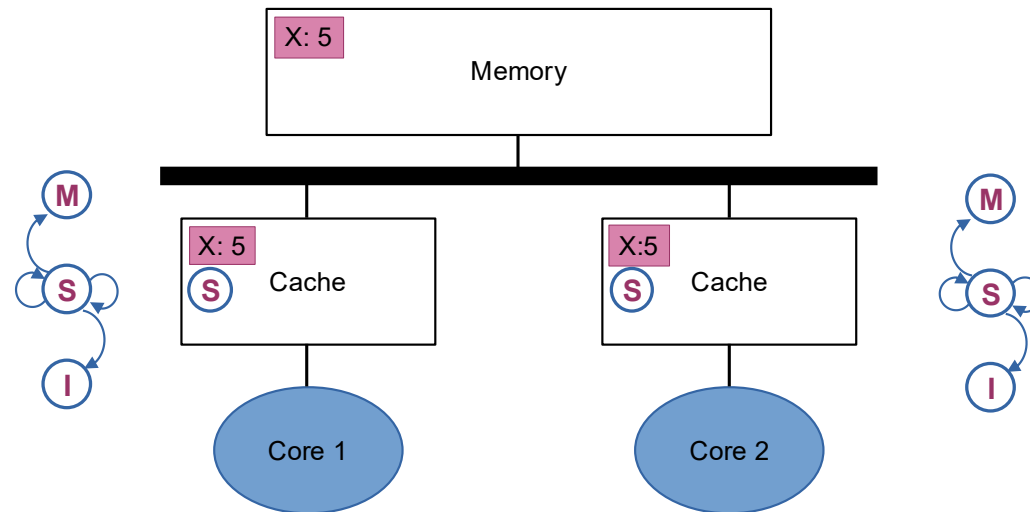
Coherency Example



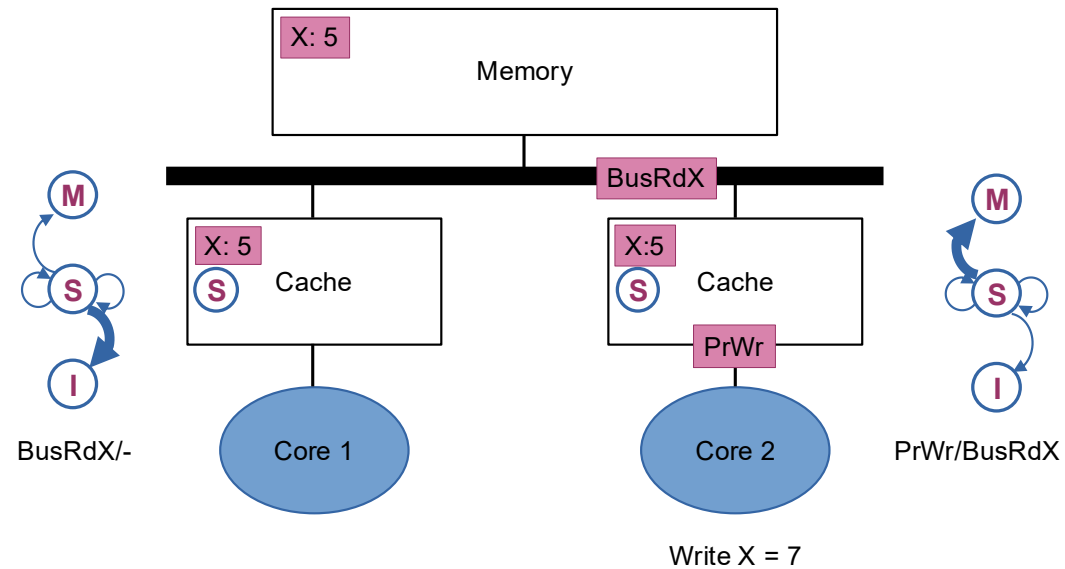
Coherency Example



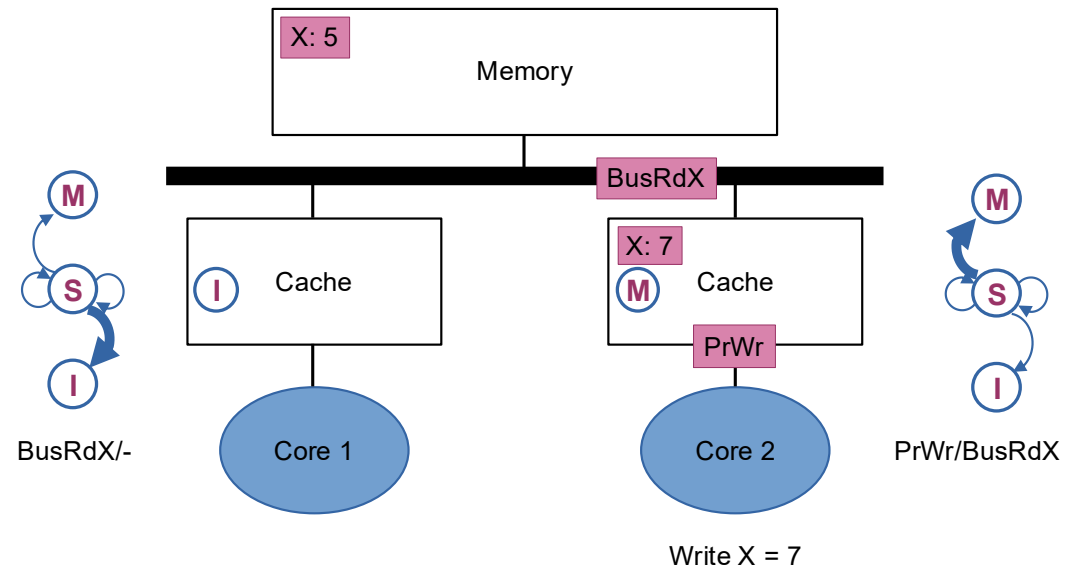
Coherency Example



Coherency Example



Coherency Example



MSI Extension

- The basic MSI protocol requires us to go to memory to get data
- If other caches have it, wouldn't it be better if they provided it (allow cache-to-cache sharing)?
- As an example
 - We are in I state but other caches are in S state
 - They snoop our bus read and respond with the data
 - This will generally be quicker than going to memory

Advanced Cache Coherence

- MSI protocol is simple
 - But sends a lot of invalidates for thread-private data
 - Solution is to add an extra 'Exclusive' state
- Snoopy protocols don't scale well
 - Directory-based schemes can be used instead
 - Then there's the issue of storing all the state...
- More cache coherence in Part II Comparative Architectures

Summary

- Multicore processors provide MIMD parallelism
- Using a shared memory paradigm is intuitive
 - Easy to program and understand
 - Simple communication
- Propagating writes to data causes problems
 - Cache coherence protocols tackle this in hardware
 - MSI is very simple; many complex schemes exist

Introduction to Computer Architecture

Lecture 13

Multicore Processors (2)

Prof. Simon W. Moore

Thanks to Dr Timothy Jones for the original version of these slides

Overview

- Last lecture
 - Multicore processors provide MIMD parallelism
 - Cache coherence takes care of propagating values
- This lecture
 - Memory consistency
 - Atomic operations
 - Synchronisation

Memory Consistency

- Cache coherence ensures data propagation
 - Values written into one cache will be seen by others
- But each location in memory is independent
 - What about ordering of writes to different addresses
 - Or reads and writes to different addresses
 - All writes will eventually be seen by all cores
 - But when will this occur?
 - What does being consistent mean?
- Essentially, what ordering do we see between reads and writes from another core?

Consistency Example

```
P1: A = 1
    if (B == 0) {
        X = 3
    }
```

```
P2: B = 1
    if (A == 0) {
        Y = 5
    }
```

- Assume initially $A = B = X = Y = 0$
- Possible outcomes

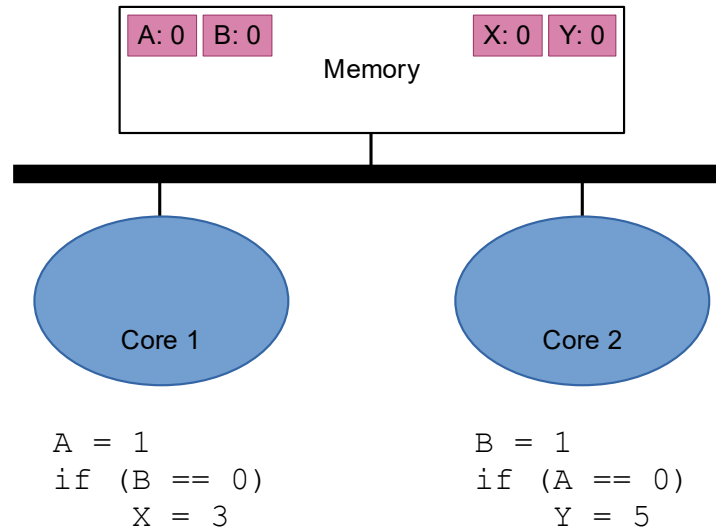
X:0, Y:0

X:3, Y:0

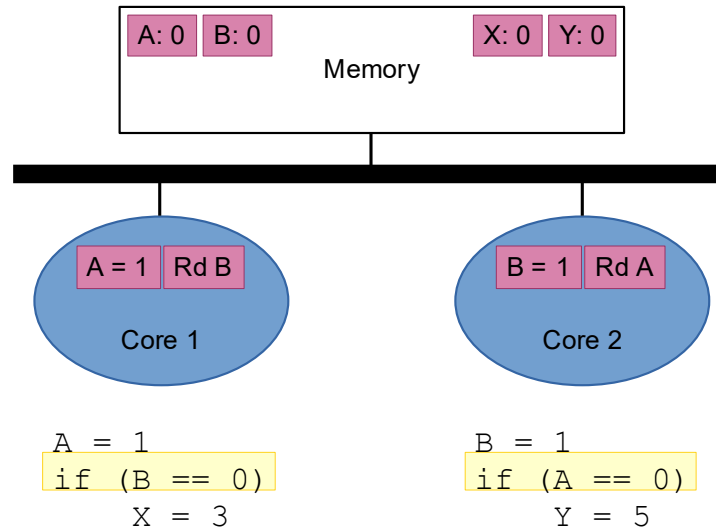
X:0, Y:5

⚡ X:3, Y:5 ⚡

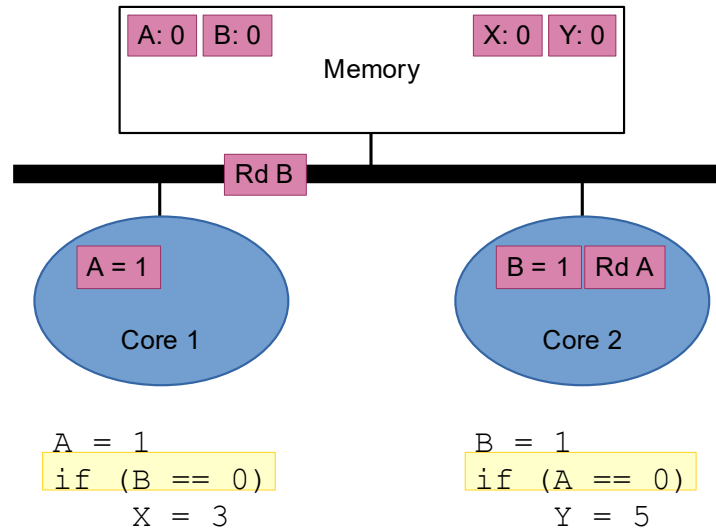
Reordering Memory Ops



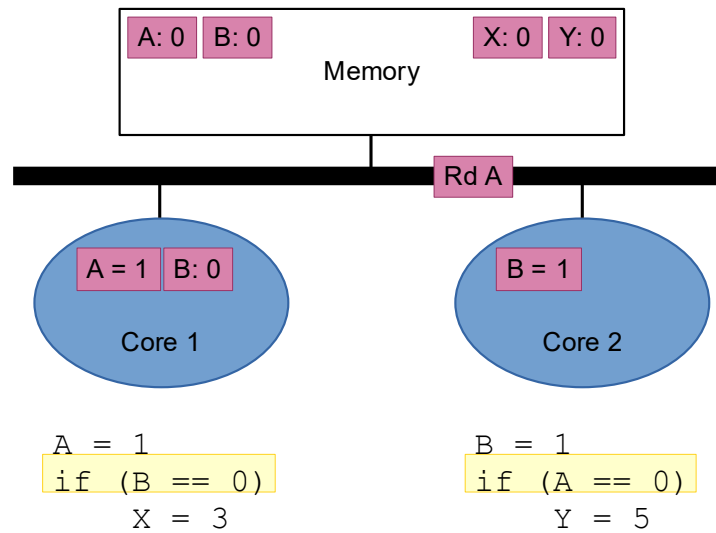
Reordering Memory Ops



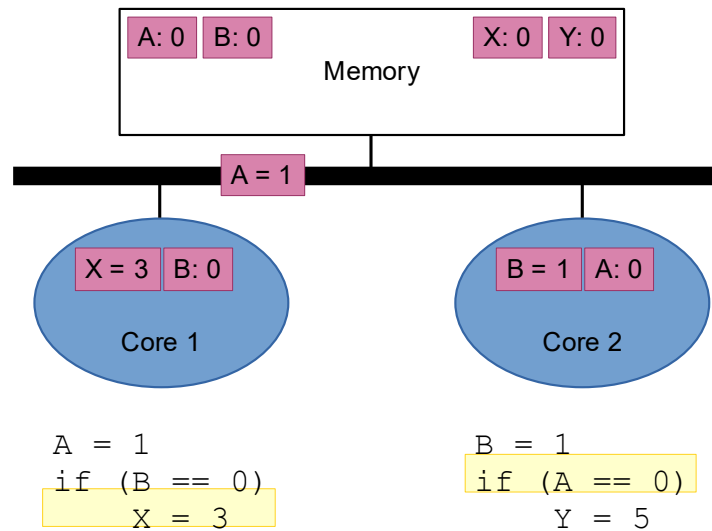
Reordering Memory Ops



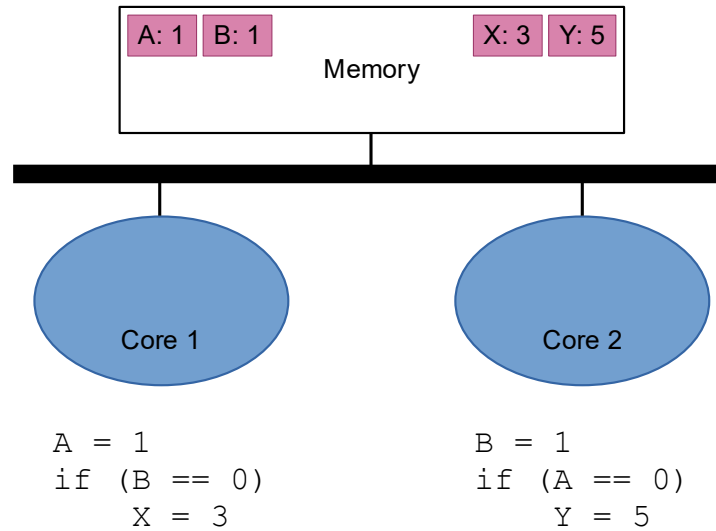
Reordering Memory Ops



Reordering Memory Ops



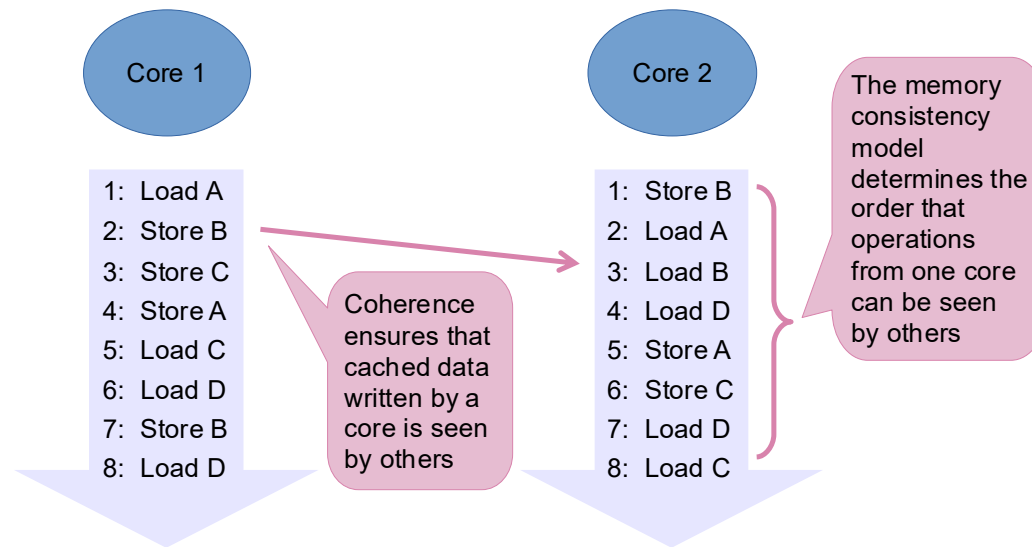
Reordering Memory Ops



Memory Consistency

- Sometimes loads and stores get reordered
 - This allows higher performance from the core
 - Could be loads or stores jumping over other stores
- However, this can lead to non-intuitive situations
 - As on the previous slide
- The memory consistency model describes the allowed behaviours

Coherence vs. Consistency

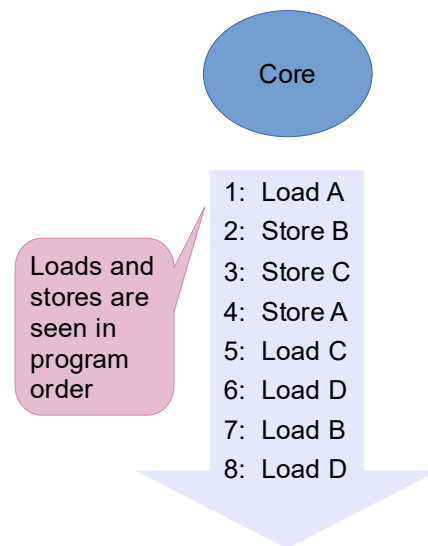


Memory Consistency

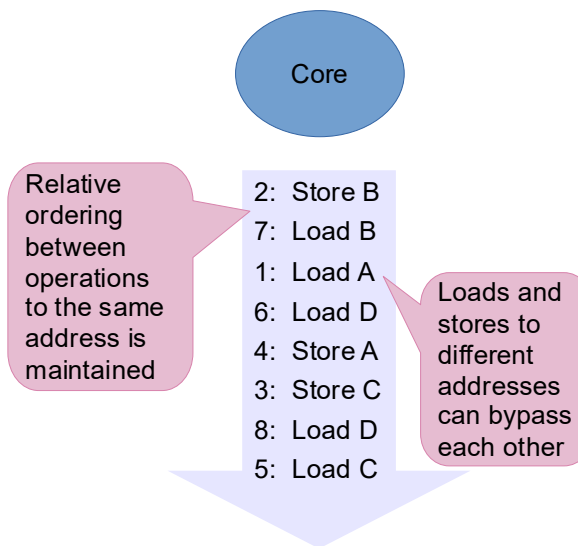
- The memory consistency model defines the ordering that reads and writes from one processor are seen by others
- The example showed relaxed consistency
 - Some loads and stores can bypass each other
- We want (the illusion of) sequential consistency
 - All reads and writes by a single processor are seen in the order they occur

Consistency Models

Sequential



Relaxed



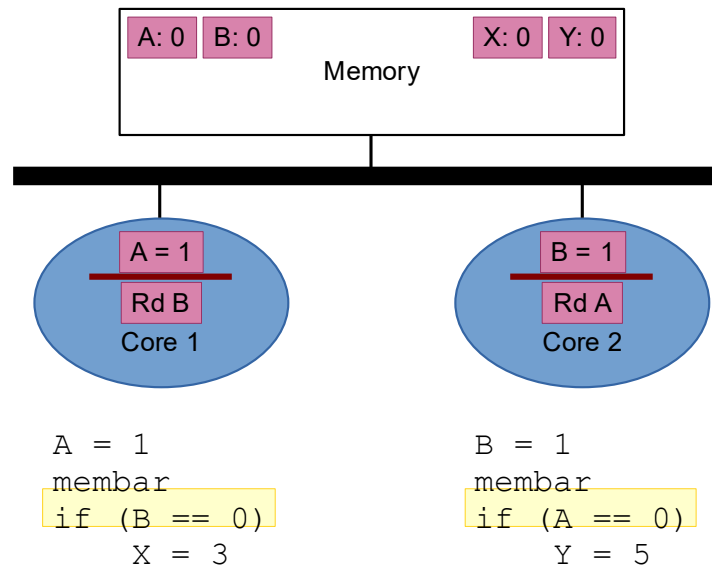
The Programmer's View

- Most of the time we don't care about ordering
 - Single-threaded applications
 - Accesses to thread-private data
- We'd like sequential consistency for shared data
 - But built on top of relaxed consistency processors
 - To do this, assume that programs are **synchronised**
 - I.e. locks are used around shared data
- We'll look at hardware operations that can be used to provide a consistency model we can program against

Memory Barriers

- Guarantee ordering of memory operations
 - Within a core, but not across cores
- All prior memory operations complete before the barrier finishes execution
- Store after a barrier can't overtake a load before it
- Sometimes called a **memory fence** instruction

Example of Memory Barrier



Original Example With Barriers

```
P1: A = 1
    membar
    if (B == 0) {
        X = 3
    }
```

```
P2: B = 1
    membar
    if (A == 0) {
        Y = 5
    }
```

- Assume initially $A = B = X = Y = 0$
- Possible outcomes

X:0, Y:0

X:3, Y:0

X:0, Y:5

~~X:3, Y:5~~

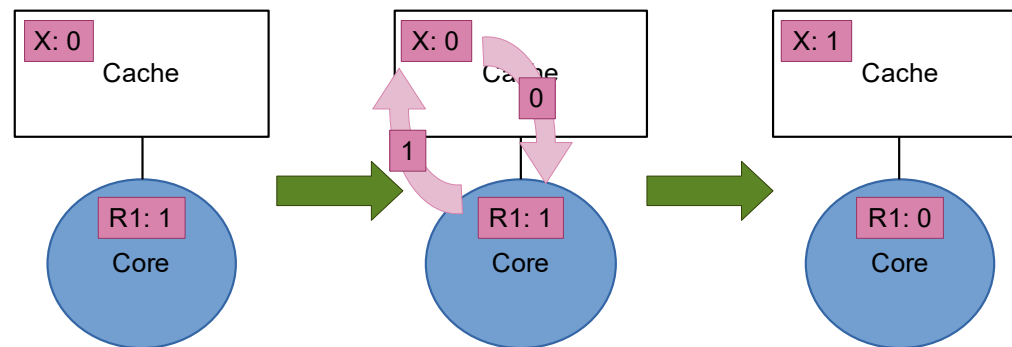
Atomic Operations

- Uninterruptible sequences of operations that appear to all occur as one
- Generally not seen by an application programmer
 - Hidden away in library routines
- Used to create software synchronisation primitives
 - Locks, thread barriers, etc
- We'll examine the instructions and uses

Read-Modify-Write (RMW)

- The most basic class of atomic operation
- These provide the ability to:
 - Read a memory location and
 - Simultaneously write a new value back
- A number of different examples
 - Atomic exchange
 - Test-and-set / compare-and-swap / fetch-and-add

Atomic Exchange



- Ideally, we want to be able to do this
 - Read and write in one uninterruptible instruction
 - Difficult in RISC machines (prefer simple load or store, not both at the same time)
- Coherence implications avoiding deadlock

Load Reserved / Store Conditional

- Instead of one instruction, provide 2 halves

```
atomic_exch X, r1
```

```
load_reserved r0, X  
store_conditional r1, X
```

- Instructions are linked together
 - Store only succeeds if X hasn't changed
 - Any write to X causes the store to fail
- Source register contains 1 on success, 0 on fail
 - In the example above this is r1
- N.B. “load_reserved” on RISC-V is sometimes called “load_linked” on other ISAs

Atomic Operations

- We can now start to build some atomic operations from instruction sequences, for example

- Atomic exchange

```
xchg: mov  r3, a0  
      lr   r4, 0(r1)  
      sc   r3, 0(r1)  
      beqz r3, xchg  
      mov  a0, r4
```

- Fetch-and-add

```
fadd: lr   r4, 0(r1)  
      add  r4, r4, 1  
      sc   r4, 0(r1)  
      beqz r4, fadd
```

Spin Locks

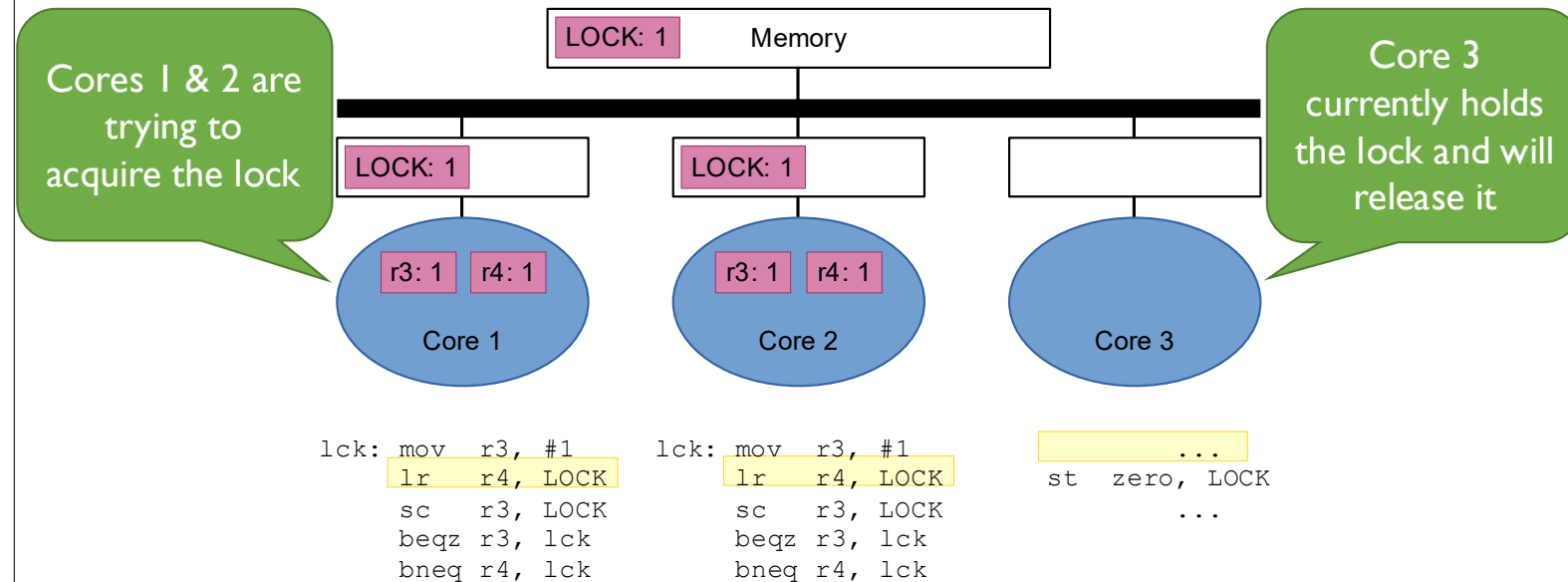
- And we can implement synchronisation primitives
- E.g. a simple lock and a more optimised version
- Right version spins on locally-cached copy of lock

```
lock: mov  r3, #1
      lr   r4, 0(r1)
      sc   r3, 0(r1)
      beqz r3, lock
      bneq r4, lock
```

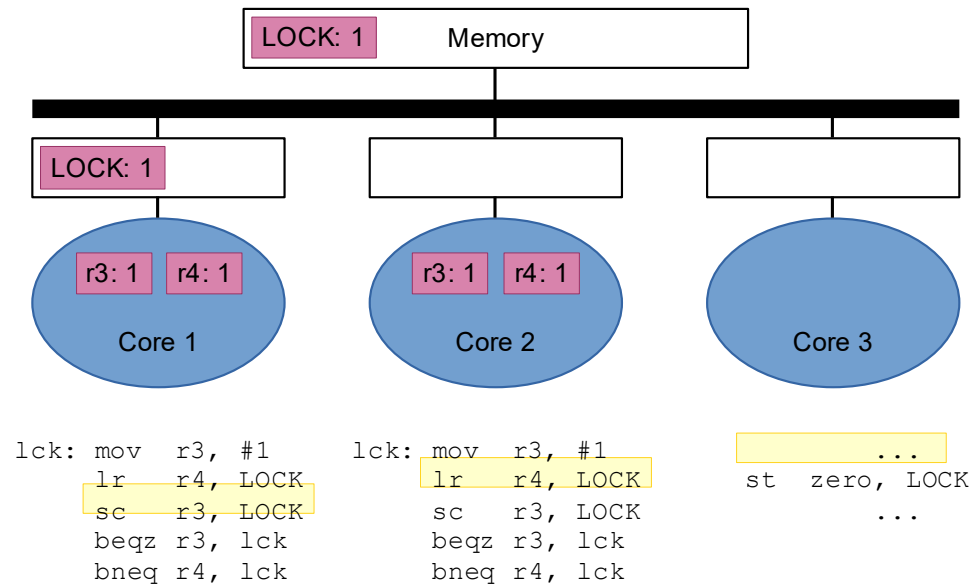
```
lock: lr   r4, 0(r1)
      bneq r4, lock
      mov  r3, #1
      sc   r3, 0(r1)
      beqz r3, lock
```

```
unlock: st  zero, 0(r1)
```

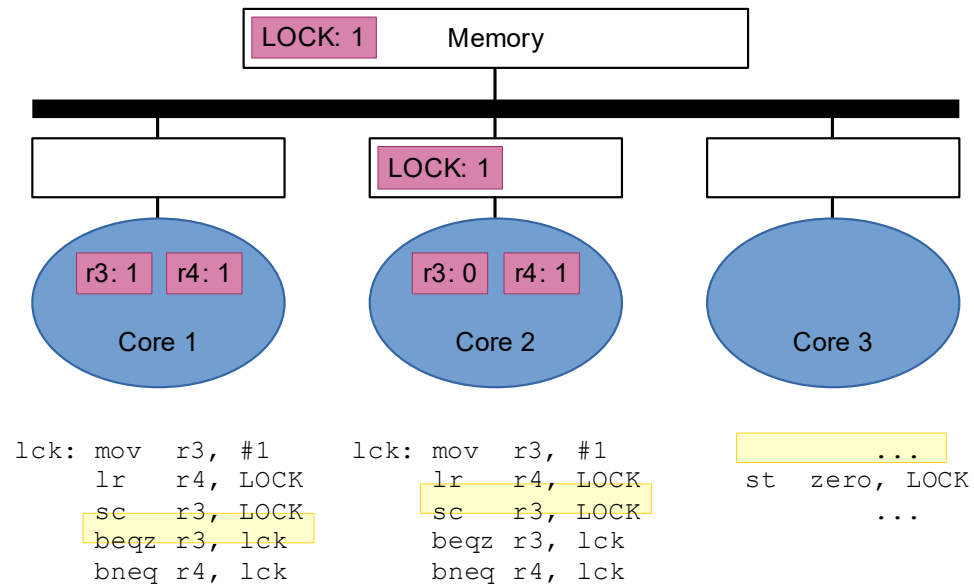
Naïve Spin Lock



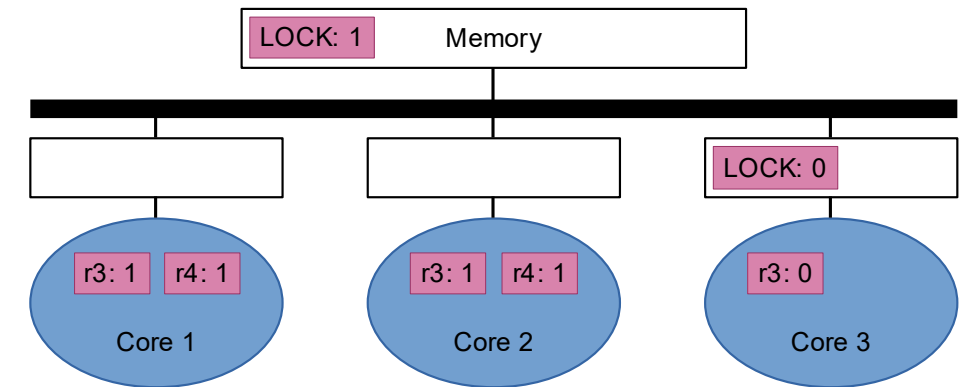
Naïve Spin Lock



Naïve Spin Lock



Naïve Spin Lock

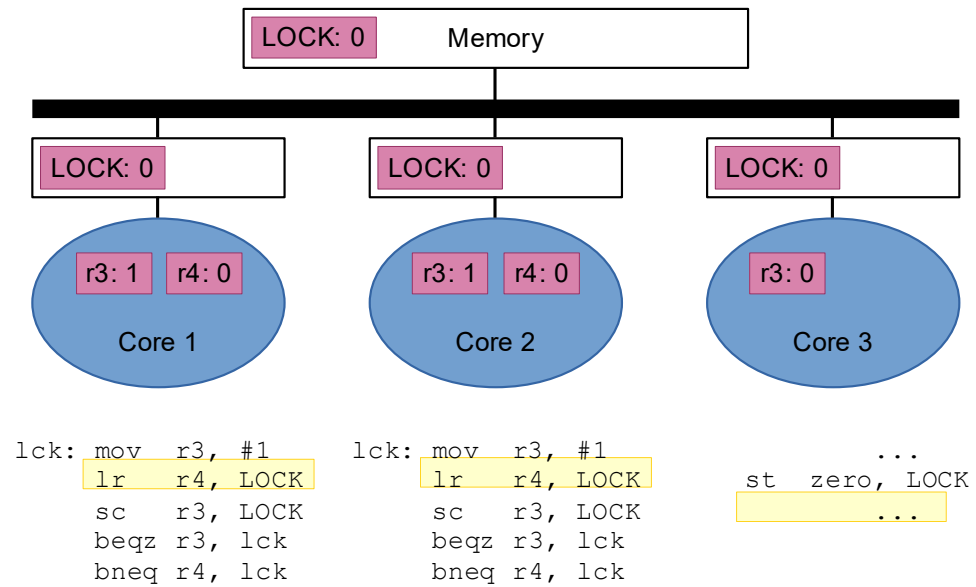


```
lck: mov  r3, #1
      lr   r4, LOCK
      sc   r3, LOCK
      beqz r3, lck
      bneq r4, lck
```

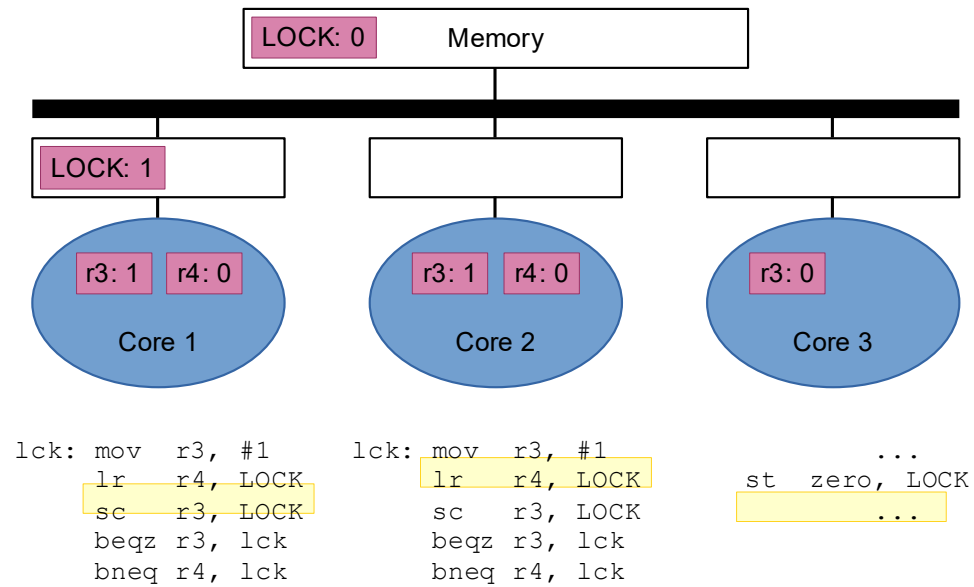
```
lck: mov  r3, #1
      lr   r4, LOCK
      sc   r3, LOCK
      beqz r3, lck
      bneq r4, lck
```

```
...
st  zero, LOCK
...
```

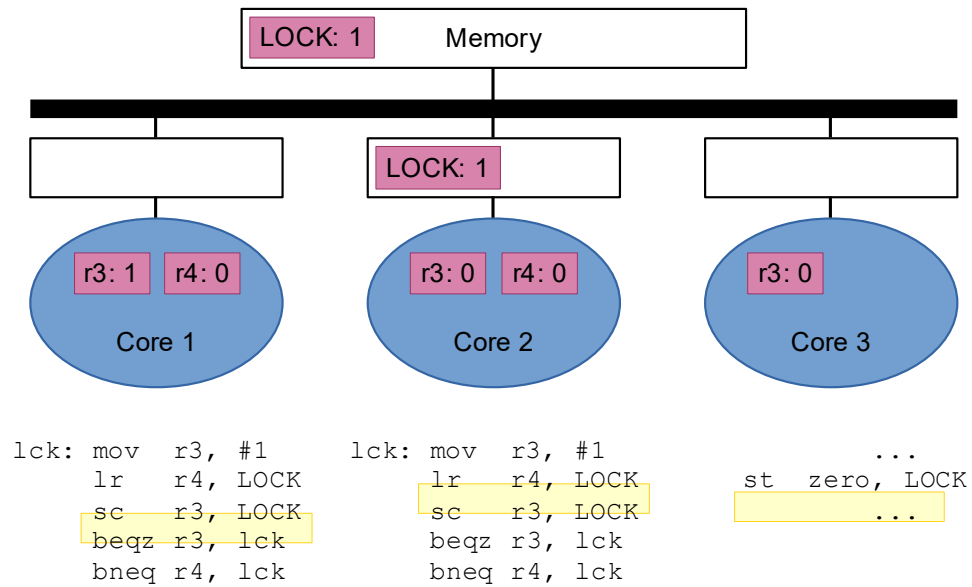
Naïve Spin Lock



Naïve Spin Lock



Naïve Spin Lock



Spin Lock Efficiency

- Naïve implementation causes lots of bus traffic
 - Cache coherence due to attempted writes
- Need to avoid writing unless likely to succeed
- So let's try the more efficient implementation...



```
lock: mov  r3, #1
      lr   r4, 0(r1)
      sc   r3, 0(r1)
      beqz r3, lock
      bneq r4, lock
```

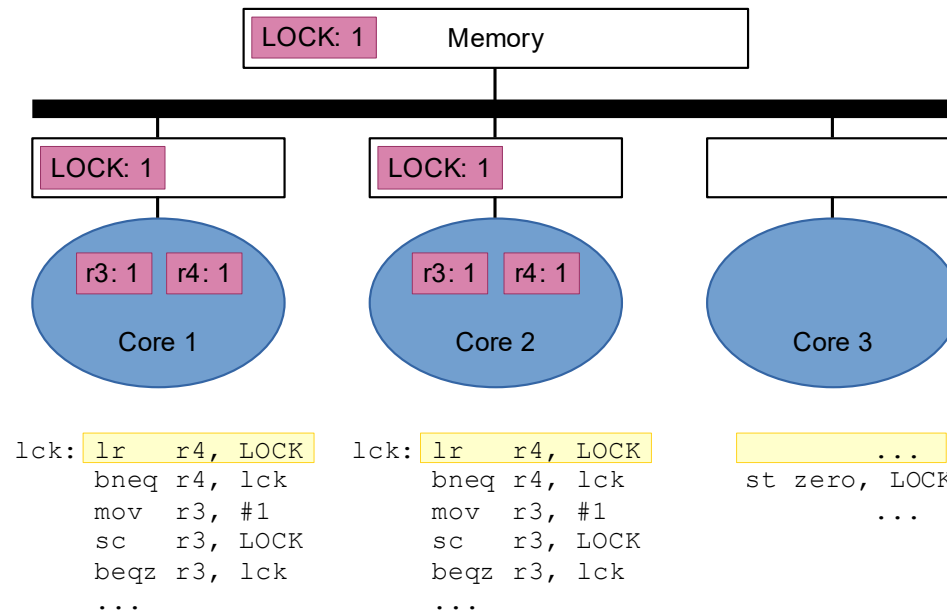


```
lock: lr   r4, 0(r1)
      bneq r4, lock
      mov  r3, #1
      sc   r3, 0(r1)
      beqz r3, lock
```

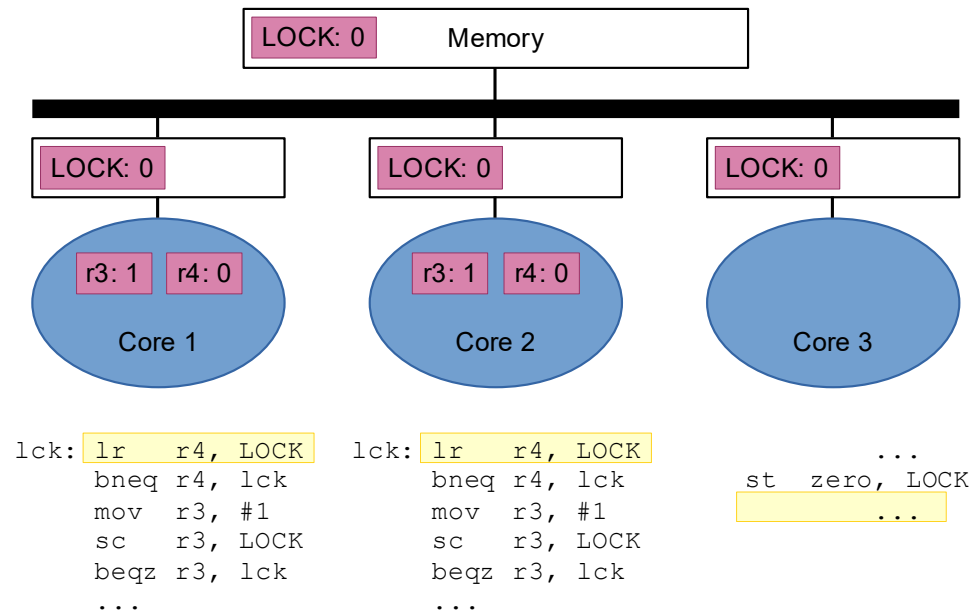
```
unlock: st  zero, 0(r1)
```

- Further improvement: `lr` is also expensive since it grabs the cache line in M (modify) state, so first read the lock using a regular load and if it looks like the lock can be taken then use `lr`

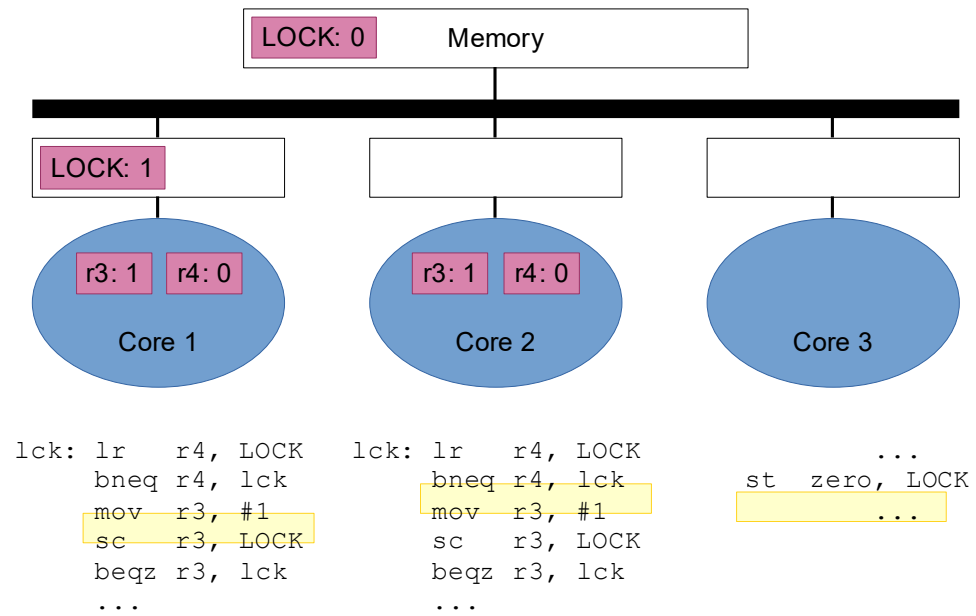
Spin Lock With Local Caching



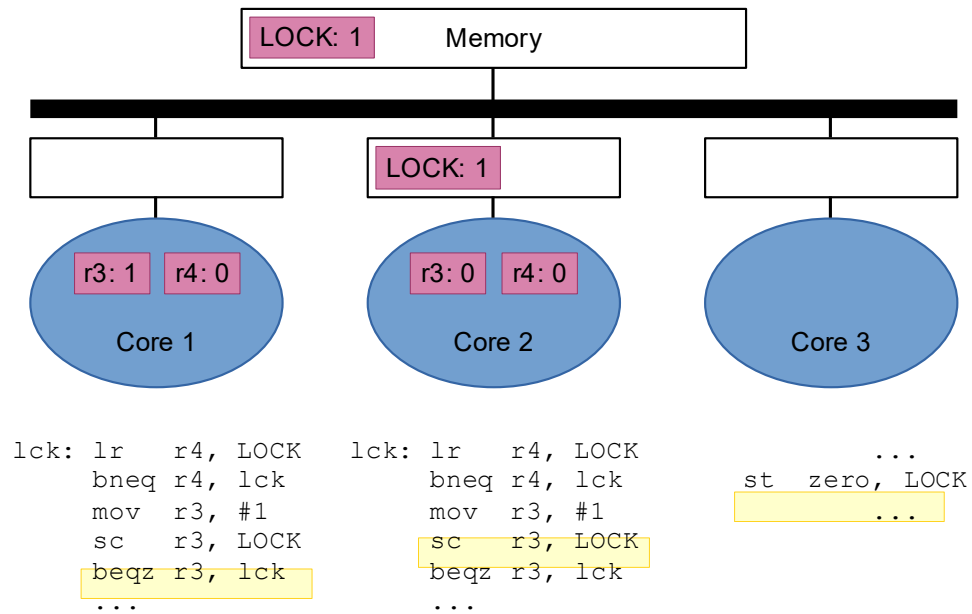
Spin Lock With Local Caching



Spin Lock With Local Caching



Spin Lock With Local Caching



Spin Lock With Barrier

- We add memory barriers to our locks too
 - So that nothing can be reordered against the lock
- All loads and stores are protected by the lock
 - Nothing performed inside can be seen outside

```
lock: lr    r4, 0(r1)
      bneq r4, lock
      mov  r3, #1
      sc   r3, 0(r1)
      beqz r3, lock
      membar
```

```
unlock: membar
       st zero, 0(r1)
```

Original Example With Locks

```
P1: lock
    A = 1
    if (B == 0) {
        X = 3
    }
    unlock
```

```
P2: lock
    B = 1
    if (A == 0) {
        Y = 5
    }
    unlock
```

- Assume initially $A = B = X = Y = 0$
- Possible outcomes

~~X:0, Y:0~~

X:3, Y:0

X:0, Y:5

~~X:3, Y:5~~

Original Example With Locks

```
P1: lock A
    A = 1
    unlock A
    lock B
    if (B == 0) {
        X = 3
    }
    unlock B
```

```
P2: lock B
    B = 1
    unlock B
    lock A
    if (A == 0) {
        Y = 5
    }
    unlock A
```

- Assume initially $A = B = X = Y = 0$
- Possible outcomes

X:0, Y:0

X:0, Y:5

X:3, Y:0

~~X:3, Y:5~~

TLB Coherence

- TLBs cache recent PTEs (page table entries) to make virtual memory efficient
- Updates (e.g. from the OS) to page tables can result in stale TLB entries
- Two approaches:
 1. TLB shoot-downs: the OS flushes TLB entries on every core
Relies on IPIs (inter-processor interrupts) to trigger these updates on every core
Expensive: every core gets involved and causes lots of page table lookups as the TLB repopulates entries
 2. Hardware keeps the TLBs coherent with page table entries
Adds complexity to the hardware

Summary

- The memory consistency model describes how loads and stores to different places are seen
- The architecture can provide low-level instructions
 - These might be atomic operations themselves
 - Or used to create atomic operations
 - Using load-linked store-conditional on RISC
 - Memory barriers prevent reordering of memory ops
 - All allow us to create synchronisation primitives
- Now we can write correct multithreaded code

Introduction to Computer Architecture

Lecture 14

Graphics Processing Units (GPUs)

Prof. Simon W. Moore

Thanks to Dr Timothy Jones for the original version of these slides

Overview

- High level GPU – what are they?
- Concepts behind GPUs
 - Motivating the use of parallelism
- SIMT execution & Multithreading
- Branching
- Scheduling
- Programming

What is a GPU?

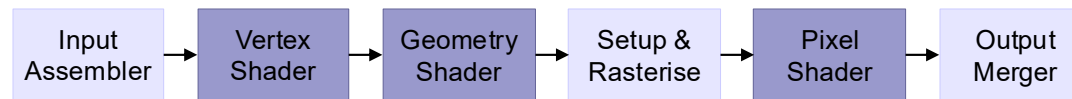
- Specialised hardware for graphics computation
 - Generating & manipulating 2D & 3D images & video

- Nvidia GeForce 256 (1999):

“The world's first 'GPU', or Graphics Processing Unit, a single-chip processor with integrated transform, lighting, triangle setup/clipping, and rendering engines that are capable of processing a minimum of 10 million polygons per second”

- Modern GPUs are highly parallel multiprocessors

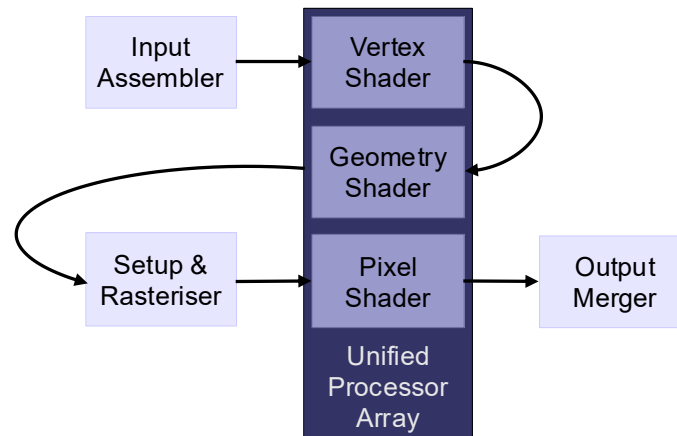
Graphics Pipeline



- Logical graphics pipeline
- Shaded stages are programmable
- In early GPUs each stage was separate
 - Largely because each was originally fixed
 - Programmability became desirable
- Increased programmability required more design
 - More complex processors; more verification

Unified GPU Architecture

- Instead map tasks to a unified array of processors
- Tasks pass through the array several times

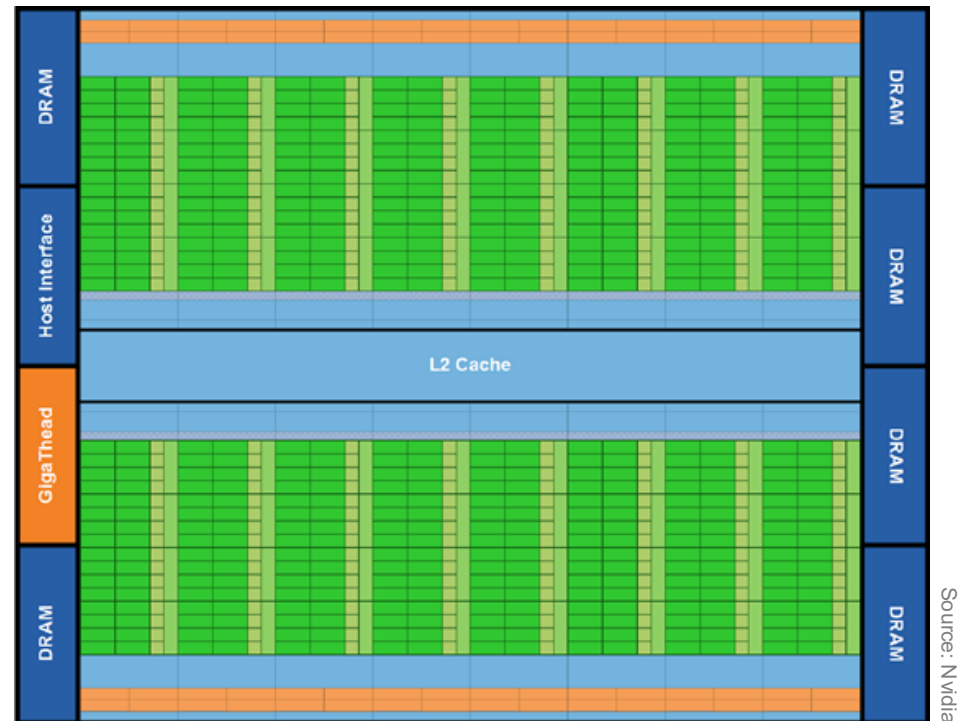


- Still contains fixed function
 - Compression
 - Anti-aliasing
 - Rasterisation
 - Video decoding
 - And others

Modern GPUs

- Modern GPUs are highly parallel multiprocessors
 - Exploiting SIMT (which is like SIMD) and MIMD
 - Multithreading
 - Multiprocessing
- GPUs take advantage of data level parallelism
- At their heart are arrays of many processing cores

Fermi GPU



Source: Nvidia

Concepts Behind GPUs

- We're going to now look at a GPU architecture
- To do this, we'll look at the techniques that GPUs use to get energy-efficient performance
 - Use of different forms of parallelism
- And issues that affect them

GPU Design Goals

- CPUs are designed for single-threaded performance
 - Logic is provided to reduce the likelihood of stalls
 - Large caches reduce accesses to memory
 - Aim to extract instruction-level parallelism (ILP)
- GPUs are designed for throughput
 - Individual thread performance is not critical
 - Little complex logic – not a focus on ILP extraction
 - Each core is relatively simple

Some Code To Run

- First, let's look at an example to use throughout
- Consider this code

```
for (i=0; i<1024; ++i) {  
    Z[i] = a*X[i] + Y[i]  
}
```

- Called a DAXPY loop
 - D = Double precision floating-point for A (scalar), X (vector) and Y (vector)
 - DAXPY is the benchmark name from: Double A X Plus Y
 - Operation: multiple vector of doubles (X) by constant (a) then add to vector of doubles (Y)

Sequential Execution

- Instructions execute one-by-one in the CPU
- During execution each instruction works on scalar data
- Only one value is read from or written to memory at a time

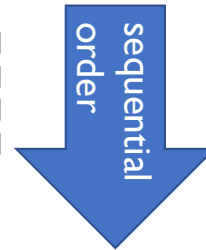
$Z[0] = a * X[0] + Y[0]$

$Z[1] = a * X[1] + Y[1]$

$Z[2] = a * X[2] + Y[2]$

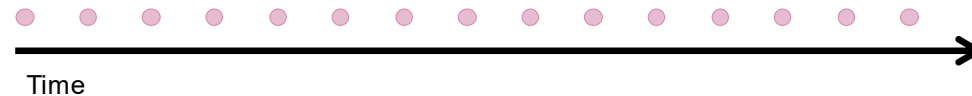
$Z[3] = a * X[3] + Y[3]$

...



Increasing Performance

- Execution proceeds sequentially through the loop



- But if we take another look at the code

```
for (i=0; i<1024; ++i) {  
    Z[i] = a*X[i] + Y[i]  
}
```

- We can see that all iterations are independent
 - No data dependences between them
 - We can parallelise this loop

Data Level Parallelism

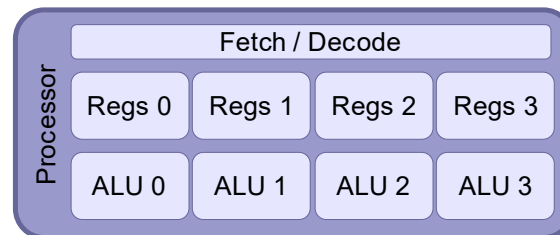
- Our loop exhibits huge data level parallelism
 - Independence between data items operated on

$Z[0]=a*X[0]+Y[0]$ $Z[1]=a*X[1]+Y[1]$ $Z[2]=a*X[2]+Y[2]$ $Z[3] = a*X[3] + Y[3] \dots$

- We can take advantage of this in the processor
 - Instructions are the same, data differs
 - Amortise cost of instruction fetch and decode

SIMT

- SIMT (vectorisation) is the way in which GPUs achieve this
- Each processor runs many threads concurrently
- Each thread has its own registers and memory
- All threads execute the same instructions
 - But on different data items



SIMT vs SIMD

- SIMT is Single Instruction, Multiple Threads
- SIMD is Single Instruction, Multiple Data
- Both take advantage of data-level parallelism
- SIMD is a form of vector processing
 - Vector registers each hold several data items
 - Vector operations work (e.g. vector add, vadd) work on vector registers
 - In modern cores, Intel SSE / AVX, ARM Neon / SVE
 - More in Part II Comparative Architectures
- In SIMT, each instruction operates on threads
 - Same instruction issued to each thread
 - But each thread has separate state (registers & memory)
 - E.g. the stack pointers may well be different for each thread

SIMT Parallel Loop

```
daxpy() {  
    int i = calcIdx();  
    Z[i] = a*X[i] + Y[i]  
}  
  
daxpy<<<256, 4>>>();
```

- Each thread executes the DAXPY kernel
- Kernel called 256 times with 4 threads each time
- So 4 data items processed each “iteration”

SIMT Execution with Four Threads

Thread 0

```
Z[ 0] = a*X[ 0]+Y[ 0]  
Z[ 4] = a*X[ 4]+Y[ 4]  
Z[ 8] = a*X[ 8]+Y[ 8]  
...
```

Thread 1

```
Z[ 1] = a*X[ 1]+Y[ 1]  
Z[ 5] = a*X[ 5]+Y[ 5]  
Z[ 9] = a*X[ 9]+Y[ 9]  
...
```

Thread 2

```
Z[ 2] = a*X[ 2]+Y[ 2]  
Z[ 6] = a*X[ 6]+Y[ 6]  
Z[10] = a*X[10]+Y[10]  
...
```

Thread 3

```
Z[ 3] = a*X[ 3]+Y[ 3]  
Z[ 7] = a*X[ 7]+Y[ 7]  
Z[11] = a*X[11]+Y[11]  
...
```

SIMT and GPUs

- We've looked at running just 4 threads at a time
 - This is called a warp, more next lecture
- In reality there are many more threads in a warp
 - E.g. in Fermi, there are 32 threads / warp
- All threads in a warp execute the same instruction
 - What if this shouldn't happen, e.g. for conditional execution?

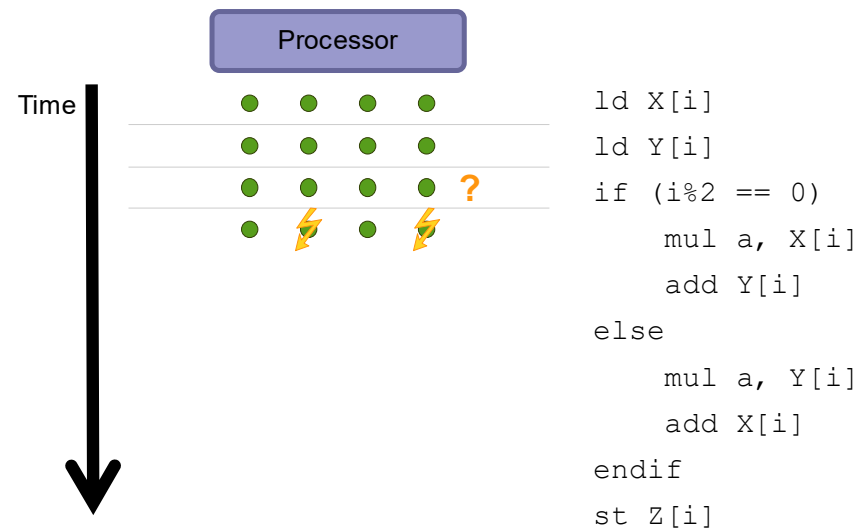
Branching in GPUs

- So far we've been considering a DAXPY loop
 - No control flow within this loop
- What about when we need to make decisions
 - How will our SIMT execution behave?

```
daxpy() {  
    int i = calcIdx();  
    if (i%2 == 0) {  
        Z[i] = a*X[i] + Y[i]  
    } else {  
        Z[i] = X[i] + a*Y[i]  
    }  
}
```

Branching in GPUs

- What do we do when we reach the condition given that we're going to issue the same instruction to every thread?

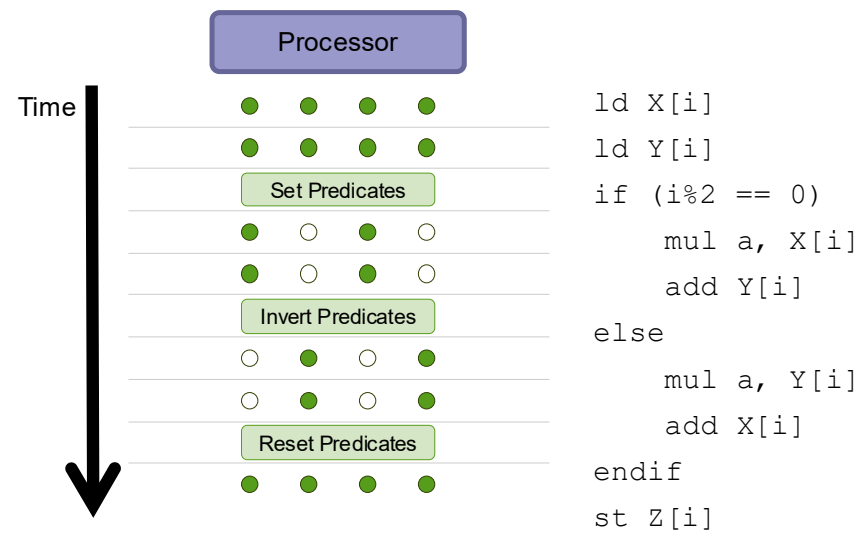


Conditional Execution

- We can add support for conditional execution
 - Only execute something if the condition holds
 - Often called predication or masking
 - Use a predicate register to hold conditions
 - Calculated in hardware, but may be special instrs too
- The thread sits idle if the predicate is false
 - No computation performed
 - Because all threads execute the same instructions
 - Although this leads to inefficiency

Branching in GPUs

- Turn simple branches into conditional execution

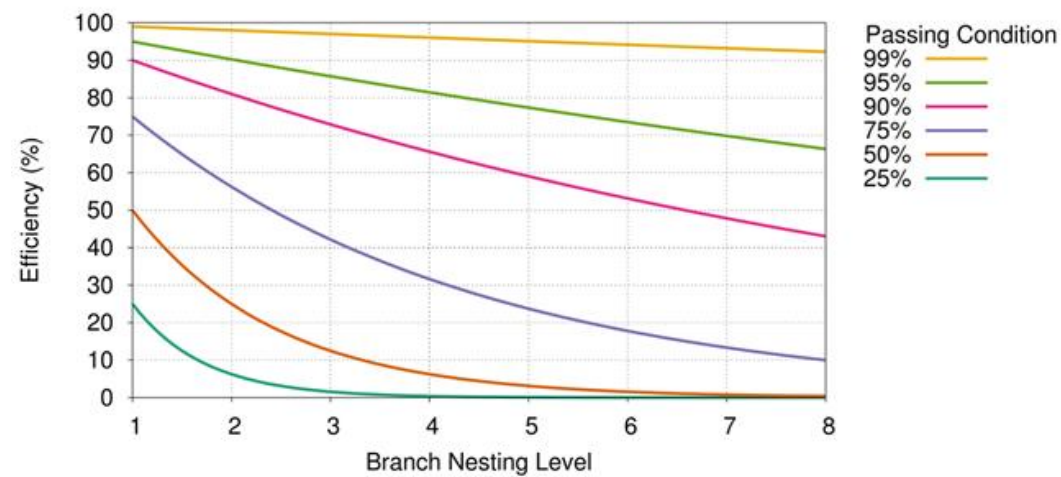


Complex Branching

- More complex branching is also possible
 - Procedure call and return
 - Nested if-then-else
 - Skip code entirely when no threads execute it
- Handled by the hardware with special instructions
- Each thread has its own stack
 - Predicates pushed to and popped from it
- Inefficient when some, but not all, threads branch
 - We call this branch divergence

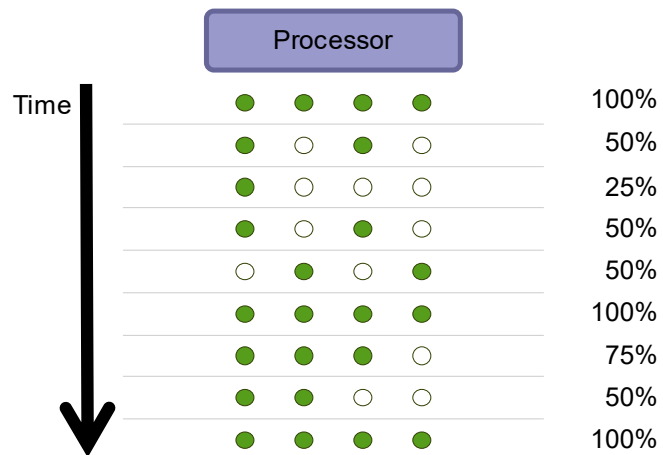
Branching & Efficiency

- Efficiency reduced when some threads branch
 - Fraction of threads doing useful work



Efficiency Example

- Total efficiency of this code block is 67%
- Note, predicate register manipulation ignored

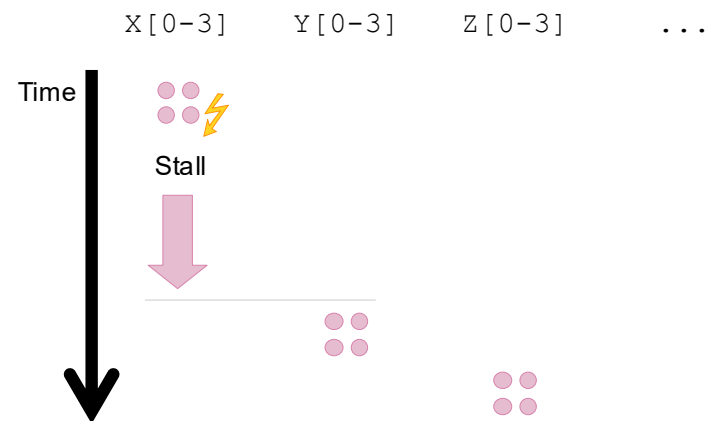


Stalls

- All these threads need access to their data
 - High memory bandwidth requirements
 - Memory accesses can be high latency
- Stalls are a big problem
 - These occur from long latency operations
 - Accessing main memory, for example
- Synchronisation between threads too
 - E.g. from a barrier (wait for all threads)

Stalls

- Usually we have to wait if we get a stall
 - Even if it's just in 1 thread



Overcoming Stalls

- No need to stall if we organise execution better

```
daxpy() {  
    int i = calcIdx();  
    Z[i] = a*X[i] + Y[i]  
}
```

- We already have a thread / call executed in warps
 - Exploit parallelism between warps

Warp 0:
Z[0-3] = a*X[0-3] + Y[0-3]
...

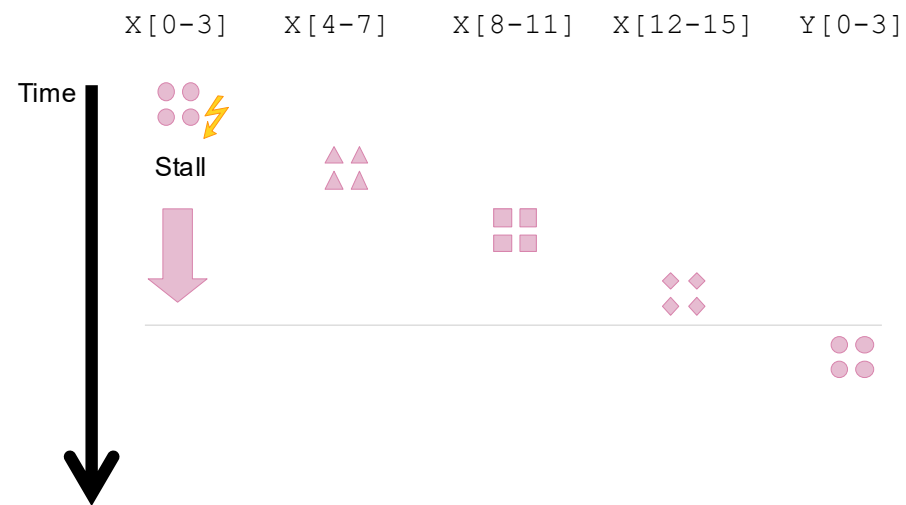


Warp 1:
Z[4-7] = a*X[4-7] + Y[4-7]
...



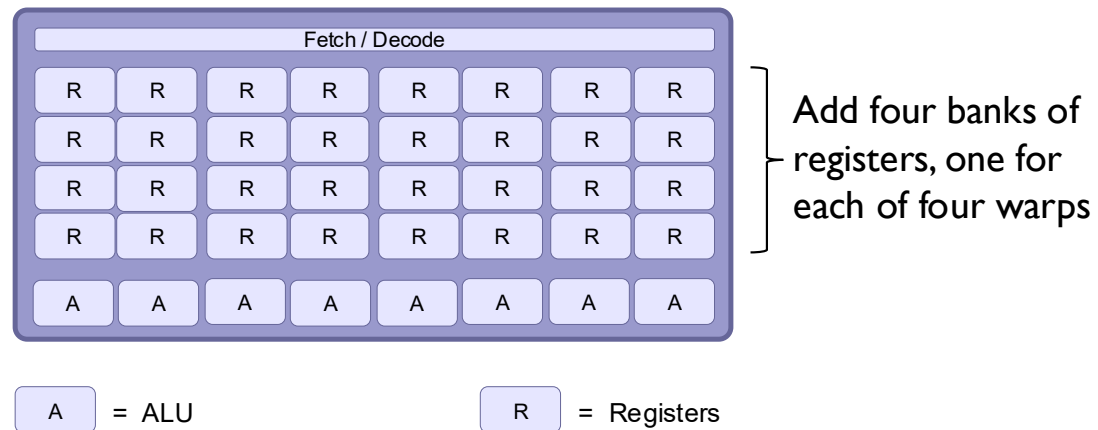
Overcoming Stalls

- Now we can hide the latency by doing useful work



Multithreading Support

- Much more thread state than processing resource



Multithreading Trade-Offs

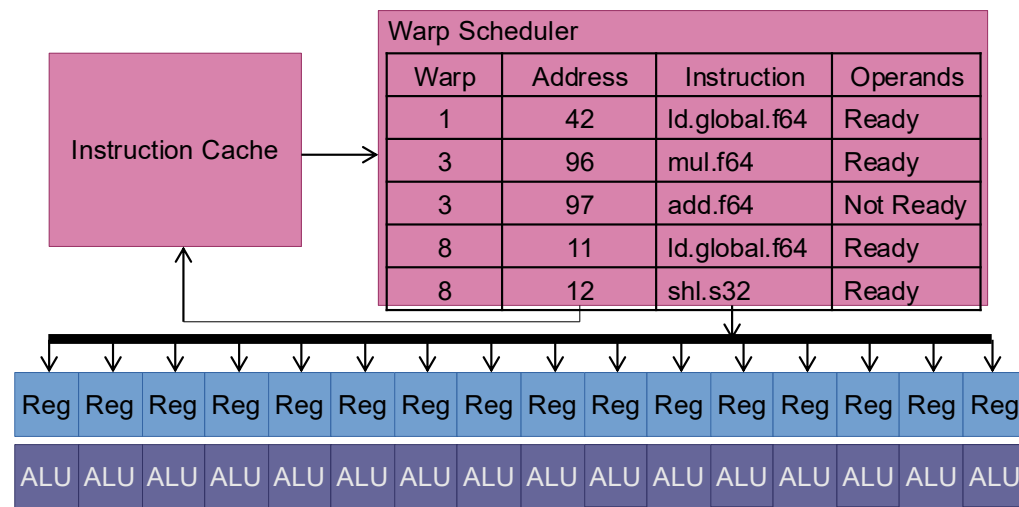
- Desire to support many threads
 - Enables high amount of latency-hiding
- Overhead in area from supporting lots of threads
 - Places to keep the thread state
- E.g. Nvidia Fermi architecture supports 48 warps
 - On each streaming multiprocessor
 - 1 warp = 32 threads, so 1,536 threads
 - And 32KiB for registers (minimum 21 per thread)

Scheduling

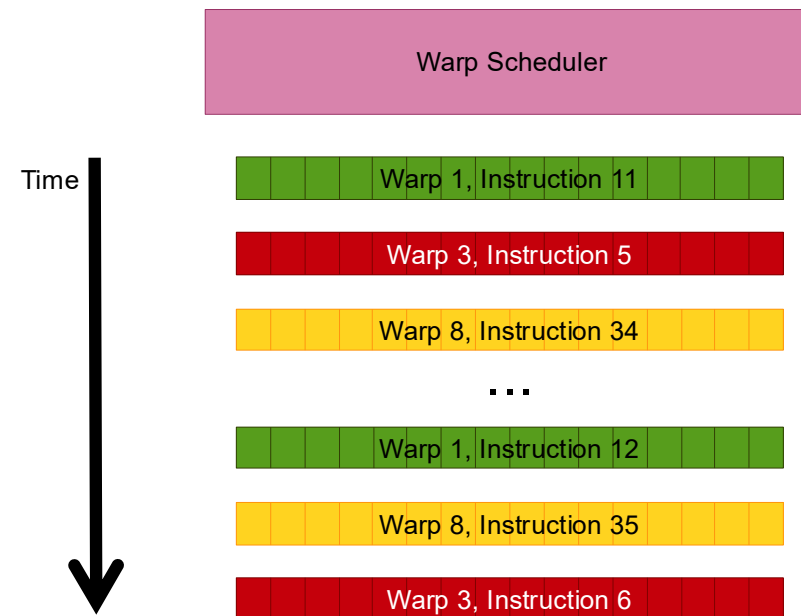
- Warp scheduler responsible for scheduling warps
 - I.e. choosing which ones to execute
- Each warp is independent of the others
 - Guaranteed by the compiler
- Selection based on when operands are ready
 - For latency hiding
 - No fixed ordering of warps

Scheduling

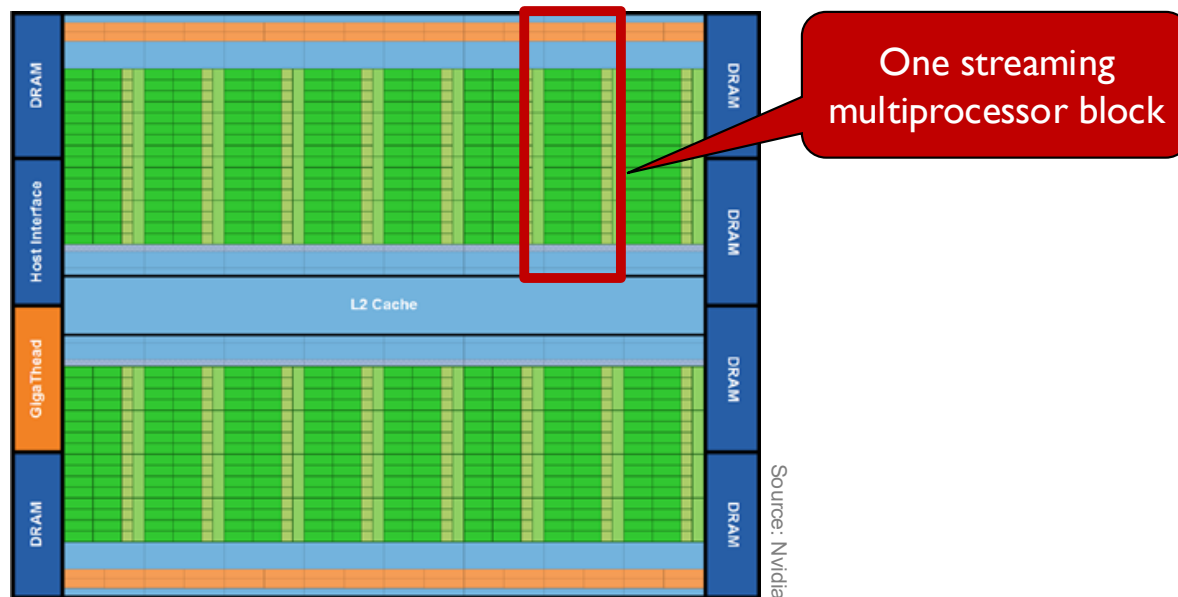
- Warp scheduler chooses ready instructions



Scheduling



Fermi GPU



Fermi Streaming Multiprocessor



- 32 CUDA core per Streaming Multiprocessor (SM)
 - Includes double precision FP
- 16 load / store units
- Special function units
 - E.g. sine, square root
- Dual warp scheduler
 - Executed concurrently

Summary

- GPUs are massively multithreaded processors
 - They support thousands of threads in parallel
- Threads are grouped together for efficiency
 - Only one instruction for a group of threads (warp)
 - Called SIMT execution
- Multithreading hides latency
 - Execute other (warps of) threads whilst waiting
- Next lecture looks at programming

Introduction to Computer Architecture

Lecture 15

Graphics Processing Units (GPUs) Part 2

Prof. Simon W. Moore

Thanks to Dr Timothy Jones for the original version of these slides

Overview

- GPU memory
- Programming
 - CUDA, for NVIDIA GPUs
 - OpenCL, for heterogeneous systems

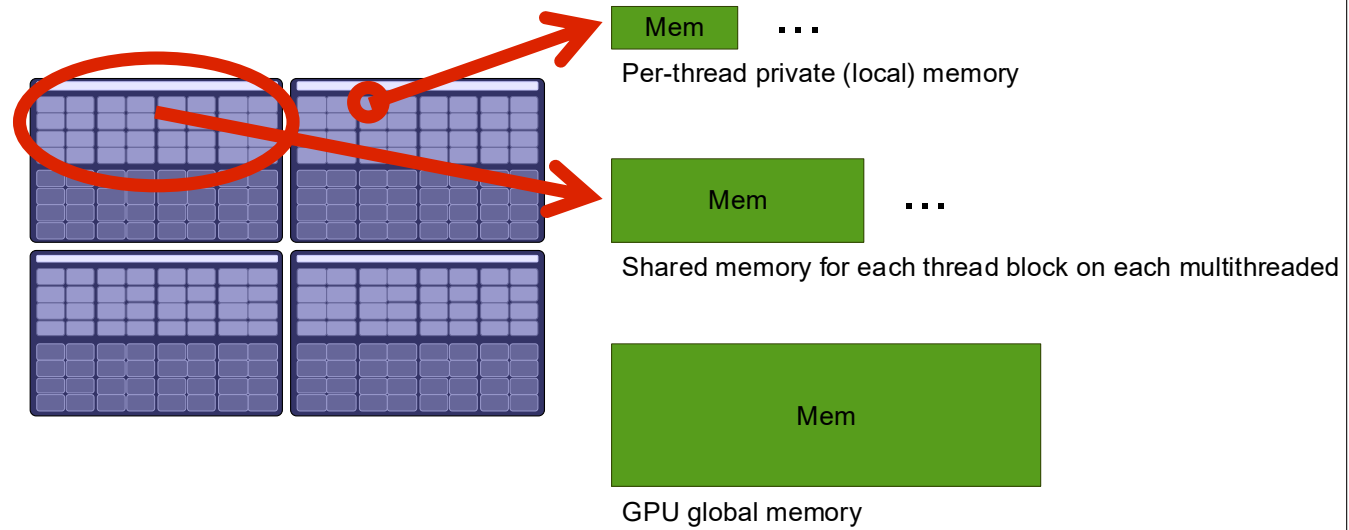
GPU Memory

- There are different hierarchies of memory within the GPU with various levels of sharing
- Balance the need for isolation vs. sharing
 - We want dedicated space for each operation
 - But to be able to easily share data too
- Modern GPUs also use caches
 - Even though multithreading hides DRAM latency
 - These reduce DRAM bandwidth requirements

GPU Memory Types

- Each thread is allocated private local memory
 - Used for stack frame, spilling registers
- Shared memory on each multithreaded processor
 - Not shared between processors
 - Dynamically allocated to thread blocks on creation
 - Can be used for communication between threads
- Global GPU memory available across GPU
 - Also available to the host / system processor
- Also constant and texture memories

GPU Memory Types

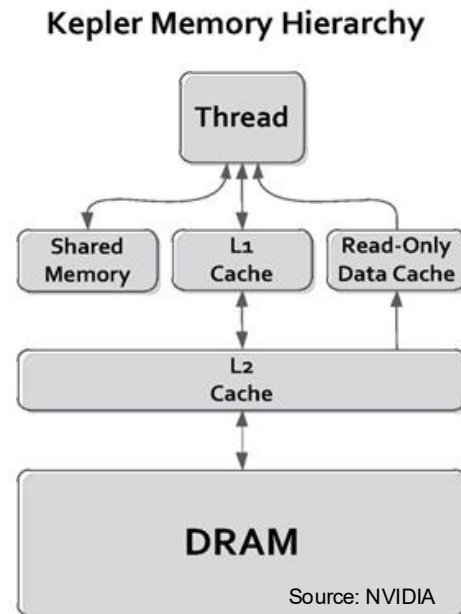


GPU Memory Locations

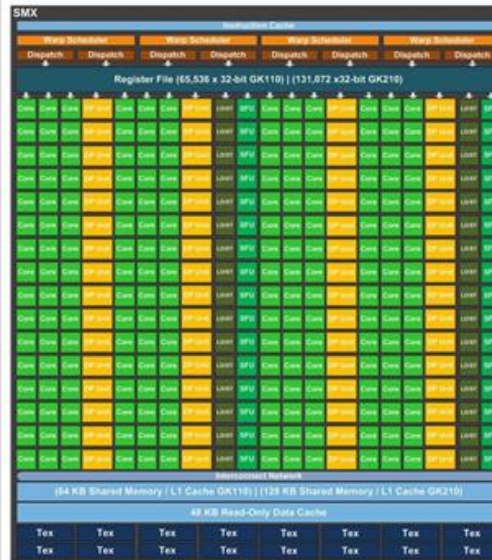
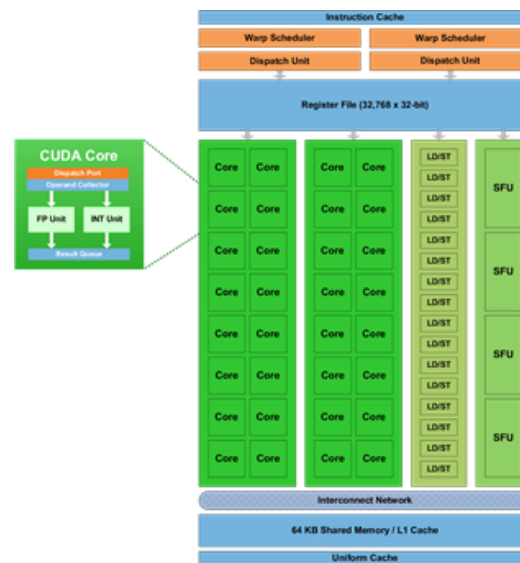
- Local memory
 - In external DRAM (so it can be large); can be cached
- Shared memory
 - Within each multithreaded processor; high bandwidth
- Global memory
 - In external DRAM; can be cached
- Constant and texture memories
 - In external DRAM; cached in specialised caches (e.g. constant cache broadcasts scalar values)

Example – Kepler Memory

- L1 cache and shared memory per Streaming Multiprocessor (SM)
 - Configurable 64KB
 - E.g. 16KB / 48KB split
- Read-only data cache
 - High bandwidth
 - Reduces main L1 cache pressure
- ECC on registers, caches and DRAM

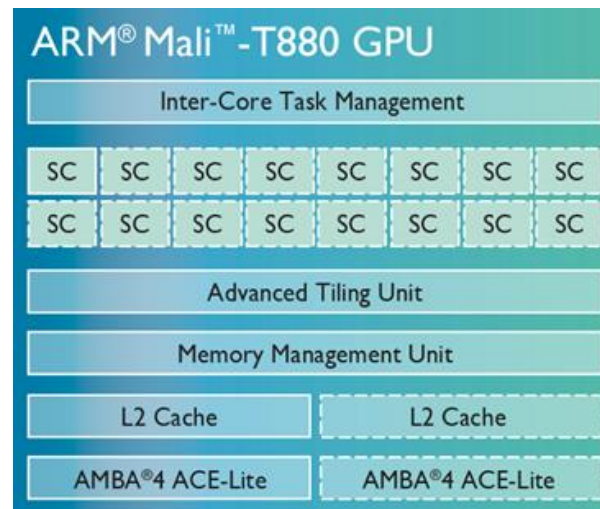


Fermi vs. Kepler



Source: NVIDIA

A Mobile GPU



Source: Arm

Programming GPUs

- So now we've looked at the underlying hardware, how do we program them?
- We can use high-level languages
 - To make things portable across GPUs
 - To abstract away the GPU internals

Programming GPUs

- Several well-known languages
 - OpenGL, DirectX, BrookGPU, CUDA, OpenCL
- We're going to focus on CUDA first
 - Because it's widely used
 - Because it's mature
- Then OpenCL
 - Another common language
 - Focus on support for heterogeneous architectures

CUDA Overview

- Compute Unified Device Architecture
- Developed by NVIDIA
- It's a C-like language
 - But allows C++ constructs too (e.g. templates)
- Programmer identifies the code to run on the CPU and that for the GPU

CUDA Terminology

CUDA Term	Description
Kernel	A program or function, designed to be executed in parallel
CUDA Thread	A single stream of instructions from a computation kernel
Warp	A group of threads that are executed together
Thread Block	A set of threads that execute the same kernel and can cooperate
Grid	A set of thread blocks that execute the same kernel

■ Consider our DAXPY loop

```
for (i=0; i<1024; ++i) {  
    Z[i] = a*X[i] + Y[i]  
}
```

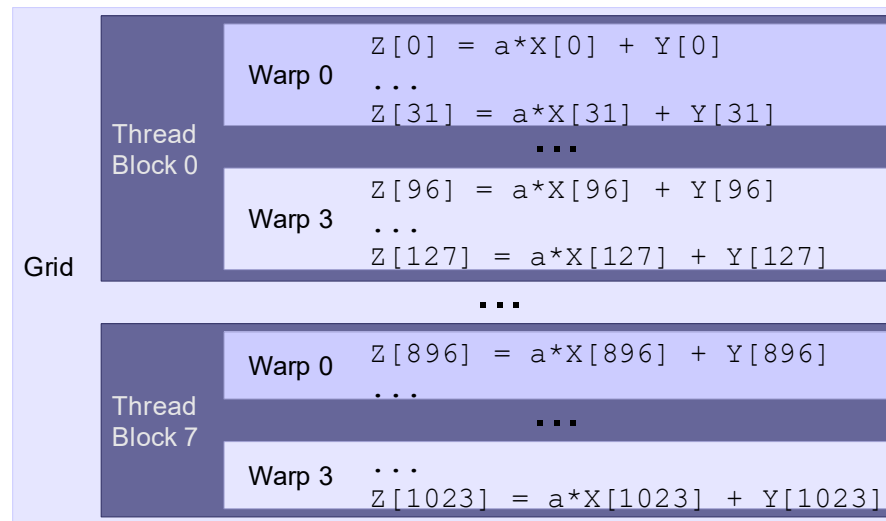
Original Loop

```
for (i=0; i<1024; i+=128) {  
    for (j=i; j<i+128; ++j) {  
        Z[j] = a*X[j] + Y[j]  
    }  
}
```

Strip-Mined / Tiled Loop

CUDA Terminology

```
for (i=0; i<1024; i+=128)
  for (j=i; j<i+128; ++j)
    Z[j] = a*X[j] + Y[j]
```



CUDA Overview

- Distinguish between GPU and CPU functions
 - `__device__` or `__global__` vs `__host__`
- Variables in GPU funcs stored in GPU memory
 - Accessible across all GPU multiprocessors
- Must call GPU functions with code dimensions
 - `func<<<dimGrid, dimBlock>>>(params)`
 - These specify the number of blocks in a grid
 - And the number of threads in a block

DAXPY in C

- DAXPY in C could be this:

```
void daxpy(int n, double a, double *X, double *Y) {  
    for (i=0; i<n; ++i) {  
        Y[i] = a*X[i] + Y[i];  
    }  
}
```

- And called like this:

```
daxpy(n, 2.0, X, Y);
```

DAXPY in CUDA

- DAXPY equivalent in CUDA:

```
__device__  
void daxpy(int n, double a, double *X, double *Y) {  
    int i = blockIdx.x * blockDim.x + threadIdx.x;  
    if (i < n) Y[i] = a*X[i] + Y[i];  
}
```

- Called using 256 threads per thread block:

```
__host__  
int nblocks = (n + 255) / 256;  
daxpy<<<nblocks, 256>>>(n, 2.0, X, Y);
```

OpenCL

- The Open Compute Language
- Designed to support heterogeneous computing



OpenCL

OpenCL

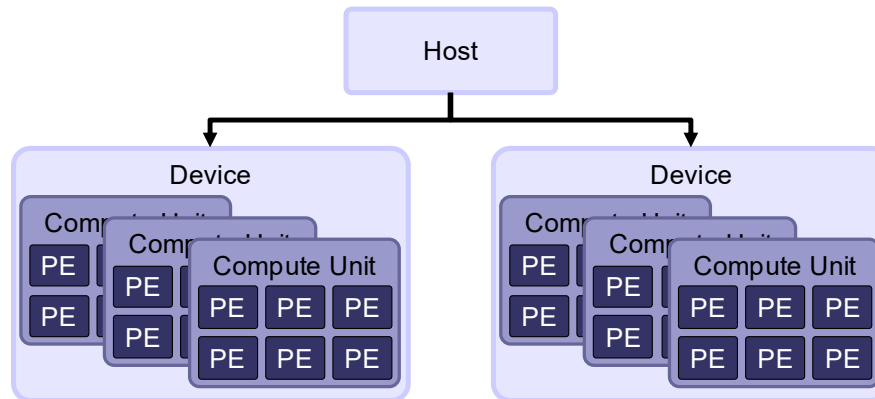
- A heterogeneous programming framework
 - CPUs, GPUs, DSPs, FPGAs, etc
- Managed by the Khronos Group
 - A non-profit technology consortium
- Current release is 3.0 dating from August 2020
 - Version 2 introduced support for (amongst others)
 - Nested parallelism
 - Shared virtual memory
 - Pipe memory objects
 - C11 atomics

OpenCL Models

- OpenCL's specification defines four models
- Platform model
 - Specifies one host and multiple devices
- Execution model
 - Defines how the host interacts with devices
- Kernel programming model
 - Defines how concurrency is mapped
- Memory model
 - Defines the abstract memory that kernels use

OpenCL Platform Model

- An OpenCL platform is a host with 1+ devices
- Devices are divided into compute units
 - And then divided into processing elements



OpenCL Platform Model

- A system could have multiple platforms
 - E.g. an Intel platform and an AMD platform
 - Each could have different characteristics
- Provides portability
 - A platform abstracts away the vendor-specific runtime
 - Apps use an API to choose the platform / device
 - Also provides an abstract device architecture
 - Vendors map this down to the actual hardware
- GPU is a device, streaming multiprocessor is a compute unit, threads are PEs

OpenCL Platform Model

- Discover the set of available platforms
 - Call twice: first get the number
 - Then allocate memory
 - Then call again to populate the array

```
clGetPlatformIDs(entries, *platforms, *num)
```

- Query the devices available on a platform
 - Similarly call twice

```
clGetDeviceIDs(platform, device_type, ...)
```

OpenCL Execution Model

- This describes how execution is actually set up
- For this we need a context
 - An abstract environment for the execution
 - Manages memory objects
 - Manages interaction between host and device
 - Keeps track of programs and kernels on each device

```
clCreateContext(*properties, num, *devices, ...)  
clCreateContextFromType(*properties, dev_type, ...)
```

OpenCL Execution Model

- Command queues allow the host to communicate with the devices
 - One queue per device; commands sent from host
 - For example, data transfer, running a kernel
- Queues can be in-order or out-of-order
 - In-order: FIFO queue
 - OoO: commands can be rearranged for efficiency
- Barriers used to synchronise queues

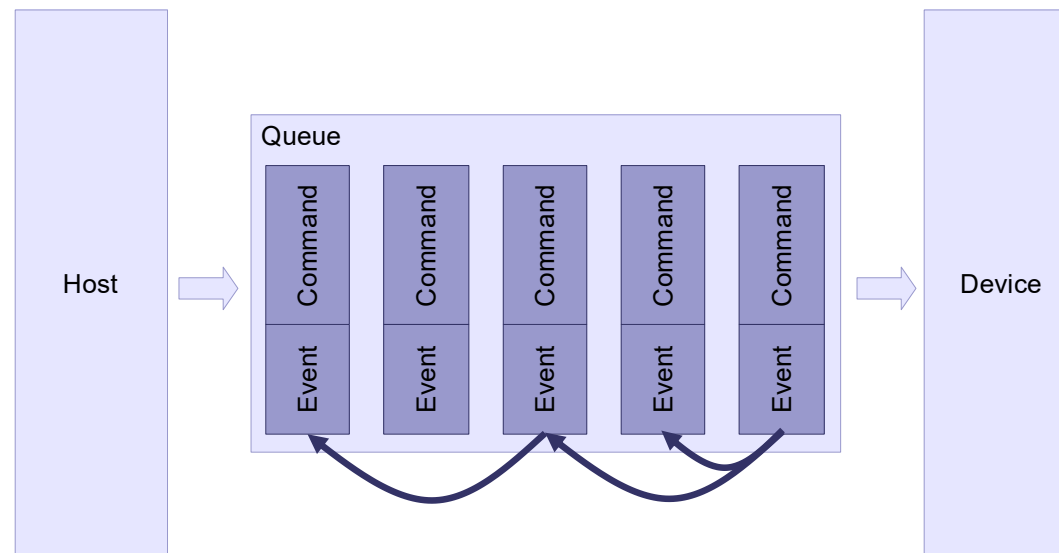
`clFlush(queue)`

`clFinish(queue)`

OpenCL Execution Model

- Event objects specify command dependences
- Each command has a wait list
 - Contains the events that this command depends on
- Each command has its own event
 - To link dependent commands with this
- Events contain the state of the command
 - Queued
 - Submitted
 - Ready
 - Running
 - Ended
 - Complete

OpenCL Execution Model



OpenCL Programming Model

- Kernels are what actually run on the device
 - Syntactically like a C function
 - Each kernel contains the body of a loop
- Kernels express parallelism at a fine granularity
 - Allows efficient mapping to a wide range of hardware
- Work-items are the basic unit of concurrency
 - Multiple work-items created to execute each kernel
 - Each work-item executes one iteration of a loop
 - Corresponds to a CUDA thread

OpenCL Programming Model

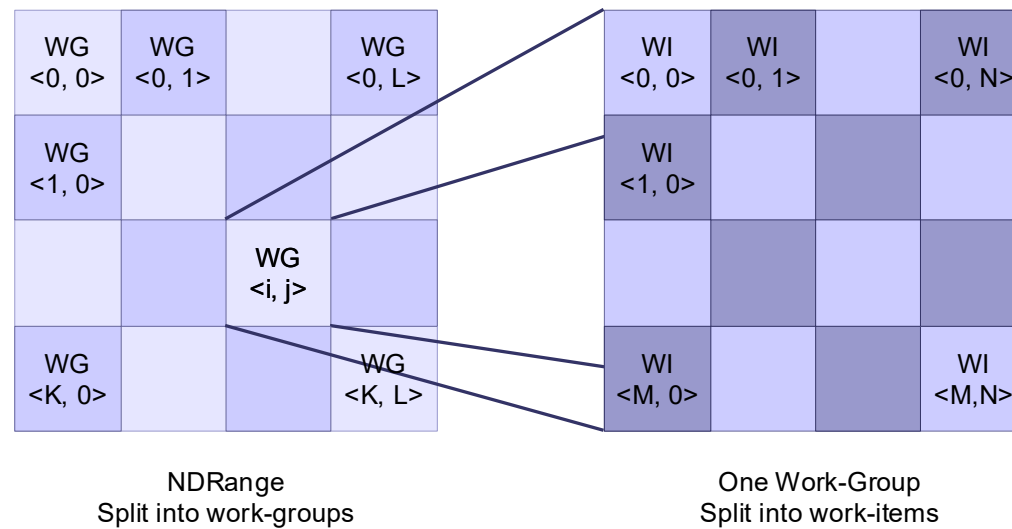
- The number of work-items created is expressed as an n-dimensional range (NDRange)
 - Either 1, 2 or 3 dimensions
 - Usually corresponds to the input / output data
 - Called an index space
- Work-items within an NDRange are grouped into smaller units called work-groups
 - Same dimensions as the index space
 - Share a memory address space
 - Can synchronise on barriers with the group

OpenCL Programming Model

```
for (i=0; i<1024; i+=32)
  for (j=i; j<i+32; ++j)
    Z[j] = a*X[j] + Y[j]
```

NDRange	Work	Z[0] = a*X[0] + Y[0]
	Group 0	...
		Z[31] = a*X[31] + Y[31]
		...
Work		Z[896] = a*X[896] + Y[896]
	Group 7	...
		Z[1023] = a*X[1023] + Y[1023]

OpenCL Programming Model



OpenCL Programming Model

- Kernels are compiled at runtime
- This allows optimisation for a specific device
- Development and execution on different systems
 - Not all systems need to know about all devices
 - New devices easily incorporated into a system
- Start with kernel code as a char array in memory

```
clCreateProgramWithSource(context 1, source, ...)  
clBuildProgram(program, 1, device, ...)  
clCreateKernel(program, "mykernel", status)  
clEnqueueNDRangeKernel(queue, kernel, ...)
```

OpenCL Memory Model

- An abstract memory model
 - Because memory subsystems vary between devices
 - Provides an independent target for programmers
 - Vendors map their hardware to this model
- Defines what happens to each memory operation
 - What values are read
 - In what order the operations occur
 - I.e. memory consistency

OpenCL Memory Model

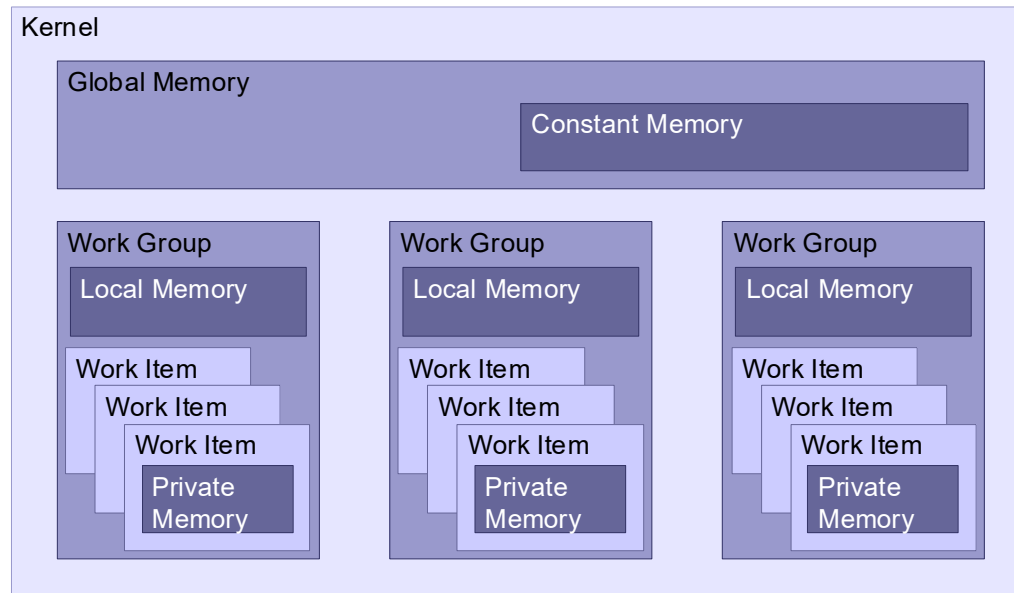
- Memory objects encapsulate data for devices
- Buffers
 - Equivalent to C arrays (contiguous elements)
- Images
 - Abstract storage, cannot be directly referenced
- Pipes
 - Ordered sequences of data as a FIFO

```
clCreateBuffer(context, flags, size, ...)  
clCreateImage(context, flags, format, desc, ...)  
clCreatePipe(context, flags, pkt_size, max_pkts, ...)
```

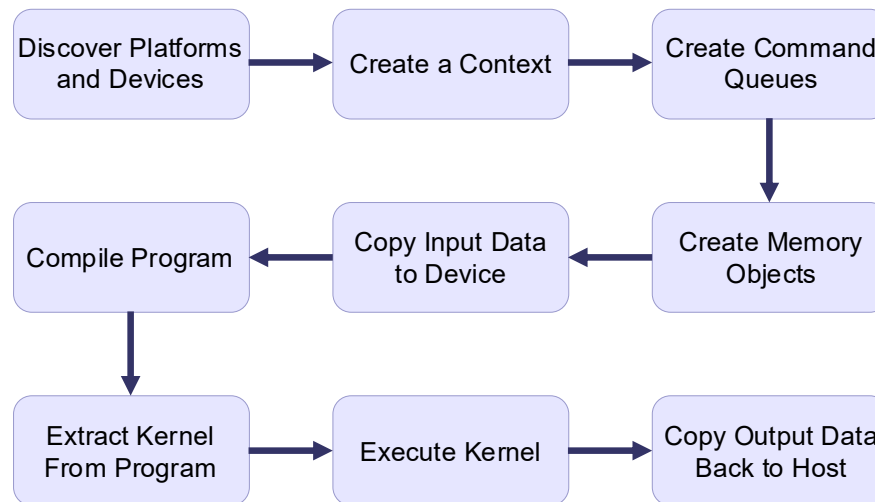
OpenCL Memory Model

- Memory is either host memory or device memory
- Host memory defined outside OpenCL
- Device memory is accessible to kernels
 - Global memory visible to all work-items
 - Constant memory visible to all for simultaneous access
 - Local memory is shared between work-items within a work-group
 - Private memory only visible to each work-item

OpenCL Memory Model



OpenCL Overview



DAXPY in OpenCL

- DAXPY equivalent in OpenCL:

```
__kernel
void daxpy(__global double a, __global double *X,
           __global double *Y) {
    int tid = get_global_id(0);
    Y[tid] = a*X[tid] + Y[tid];
}
```

- Called using 256 work-items per work-group:

```
cl_kernel k = clCreateKernel(p, "daxpy", &status);
s = clSetKernelArg(k, 0, sizeof(cl_mem), a);
i[0] = 4096; w[0] = 256;
s = clEnqueueNDRangeKernel(q, k, 1, NULL, i, w, ...);
```

CUDA vs OpenCL

- OpenCL is more verbose
 - Platform discovery performed at runtime
 - Compilation performed at runtime
- CUDA only targets NVIDIA's GPUs
 - So one platform and static compilation to PTX
- OpenCL must target CPUs, GPUs, FPGAs, etc
 - So ISA or intermediate language not known statically
 - And support multiple vendors too

GPU Summary

- Modern GPUs are multithreaded multiprocessors
 - SIMT enables energy-efficient execution
 - Multithreading hides stalls
- Programming languages developed to help code
- CUDA is NVIDIA's common approach
 - This has enabled GPUs to break away from just the graphics domain into GP compute
- OpenCL is more broad
 - Heterogeneous compute for all types of device

Introduction to Computer Architecture

Lecture 16

Future Directions

Prof. Simon W. Moore

Thanks to Dr Timothy Jones for the original version of these slides

Overview

- Where is computer architecture heading?
- A look at the challenges facing us
- Some of the solutions
 - Some proposed by industry
 - Some developed in academia

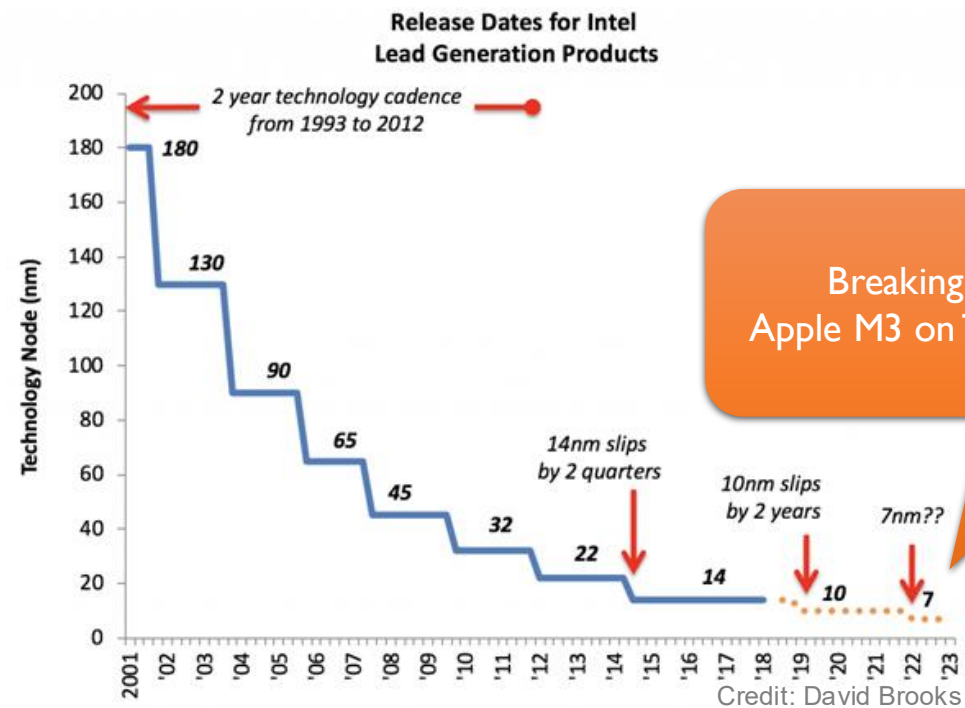
What's Industry's Opinion

- “Energy efficiency is the new fundamental limiter of processor performance, way beyond numbers of processors.”
Borkar and Chien. The Future of Microprocessors, CACM 54(5), 2011
 - Shekhar Borkar is an Intel Fellow and Director of Microprocessor Technology Lab
 - Andrew Chien is former vice president of research at Intel (and editor-in-chief of CACM)

What's Industry's Opinion?



Is Moore's Law Slowing?



Challenges

- There are several challenges facing industry
- Energy efficiency: how to keep power consumption down
- Reliability: keeping things working (correctly?)
- Performance: how to make things go as fast as required
- Security: trusting and securing computation
- Often these things are intertwined

Energy-Efficiency Challenge

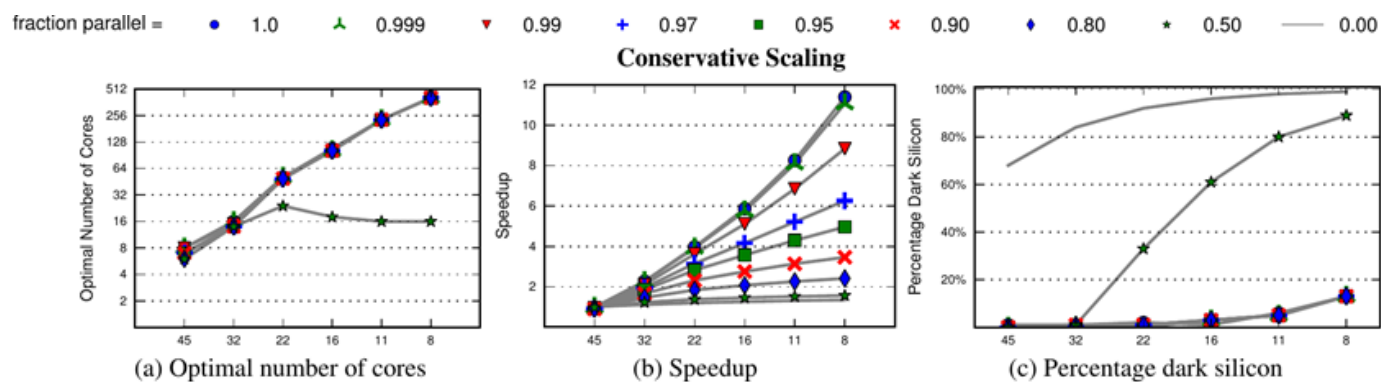
- Energy-efficiency a major challenge
 - We can squeeze billions of transistors onto a chip
 - But we simply can't power them all on at once
 - Hence the term 'dark silicon'
- So what can we do with all these transistors?
 - Since the mid 2000s we started building multicores
 - Move from complex single core to simple multicore
 - Dial down clock frequency and save power

A Problem

“Neither CPU-like nor GPU-like multicore designs are sufficient to achieve the expected performance speedup levels. Radical microarchitectural innovations are necessary to alter the power/performance Pareto frontier to deliver speed-ups commensurate with Moore’s Law.”

Esmailzadeh et al. *Dark Silicon and the End of Multicore Scaling*. ISCA 2011

Multicore Scaling



Source: Esmailzadeh et al.

- Uses a model to determine future speedups
 - Parallel app with 512 cores is only 12x speedup
 - With 50% parallel there's 90% dark silicon at 8nm
 - (We're currently building 5nm cores)

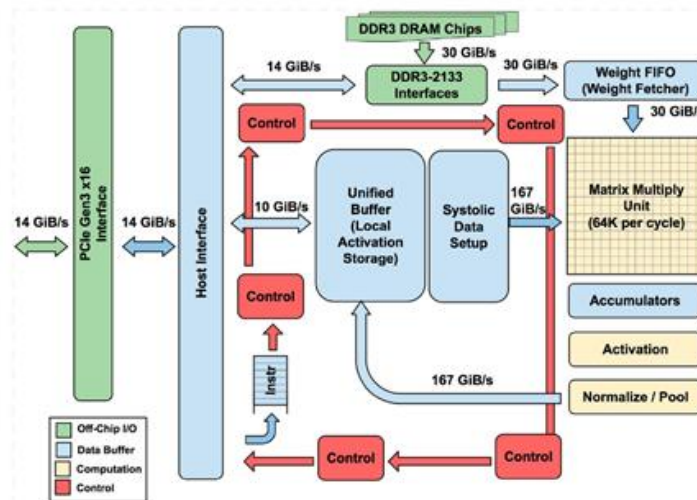
Achieving Energy Efficiency

- So simply adding cores doesn't look promising
- Need to fundamentally alter the cores themselves
 - Make them more efficient by eliminating overheads
- Some people have studied this
- “The ASIC is 500x more energy efficient than our original four processor CMP.”
Hameed et al. Understanding Sources of Inefficiency in General-Purpose Chips. ISCA 2010

Achieving Energy Efficiency

- So one option is to specialise our multicore chips
 - Don't simply have homogeneous cores
 - Make the cores heterogeneous
- We can also go further
 - Remove the core altogether for some algorithms
 - By creating accelerators for certain tasks

Google's TPU



Jouppi et al. *In-Datacenter Performance Analysis of a Tensor Processing Unit*. ISCA 2017.

Tensor Processing Unit



Google's TPU

- A custom ASIC that's been in operation in Google's data centres since 2015
- Runs neural network inference applications
- Performs 15x – 30x faster than CPU or GPU
- Performance/Watt 30x – 80x higher
- More efficient because it lacks out-of-order hardware, caches, multithreading, etc
- Designed for deterministic latency

Incorporating Reconfiguration

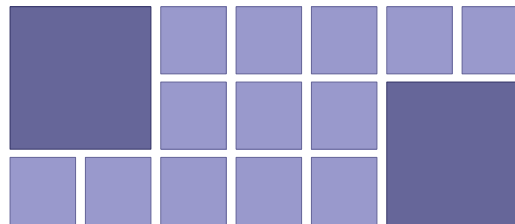


Putnam et al. *A Reconfigurable Fabric for Accelerating Large-Scale Datacenter Services*. ISCA 2014.

- Using FPGAs, Microsoft report 95% improvement in latency with only 10% extra power for Bing search

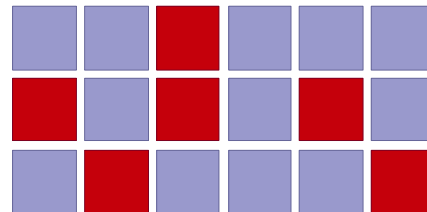
Performance Challenge

- In the mid 2000s multicores became mainstream
 - Before this point we mainly had a single core
 - Multiprocessors were high-end, specialist machines
- Now performance comes from parallelism
 - But can we beat Amdahl's Law?!



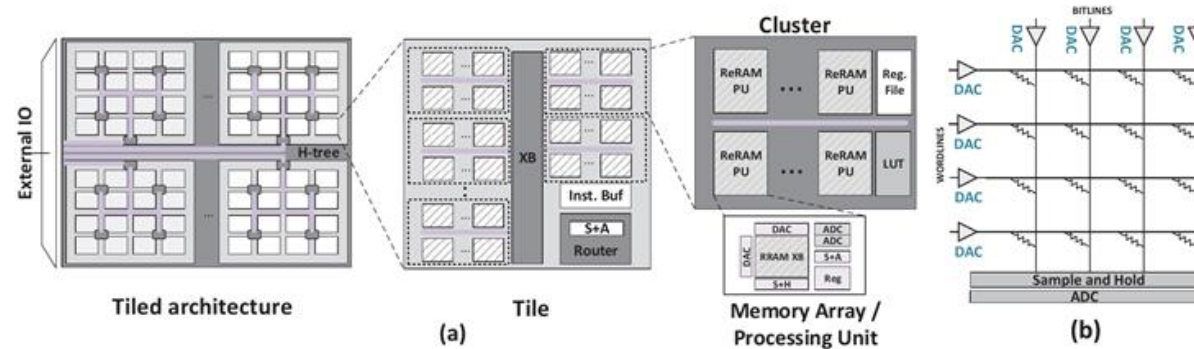
Speeding Sequential Code

- Or temporarily exceeding our power budget
- Allow some cores to run faster (and hotter)
 - E.g. based on requirements for immediate responsiveness in mobile applications
 - Partly supported by dynamic voltage/frequency scaling on commercial cores
- See Computational Sprinting, U. Michigan



Processing-In-Memory

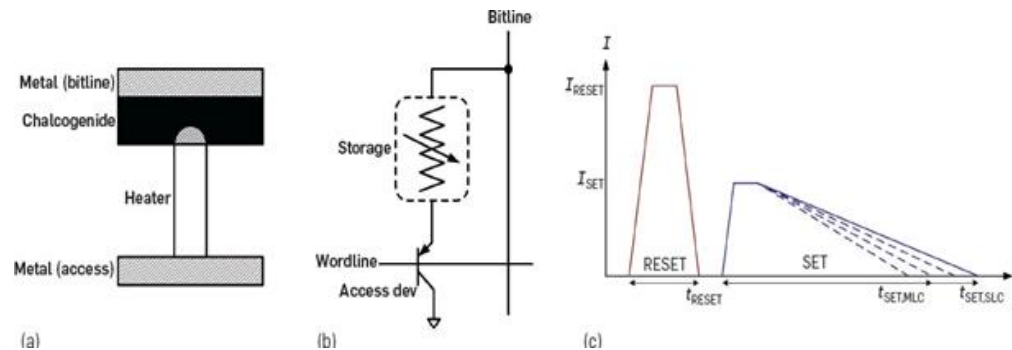
- Data movement is expensive in terms of energy
- Processing in memory is becoming fashionable
- E.g. in ReRAM memory for massive parallelism



Fujiki et al. *In-Memory Data Parallel Processor*. ASPLOS 2018.

New Memory Technologies

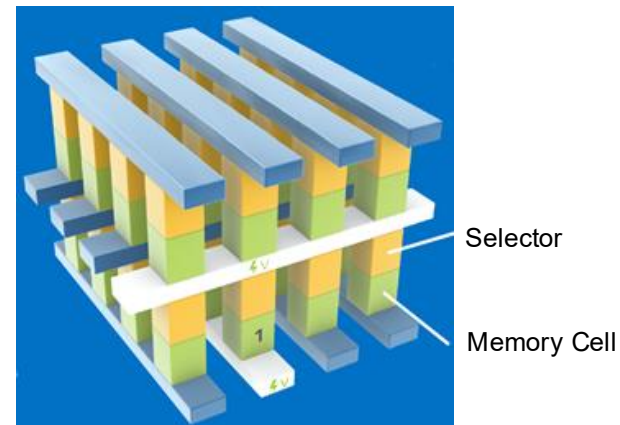
- Other new memory technologies are being developed and will change the system trade-offs
- Phase-change memory (PCM) is non-volatile but with slower writes than DRAM



Lee et al. *Architecting Phase Change Memory as a Scalable DRAM Alternative*. ISCA 2009.

3D Xpoint

- ReRAM developed by Intel and Micron
- Shipping in certain systems (SSD caches), could be alongside DRAM
- Latency of 10s of nanoseconds
 - Slower than DRAM
 - But significantly faster than flash



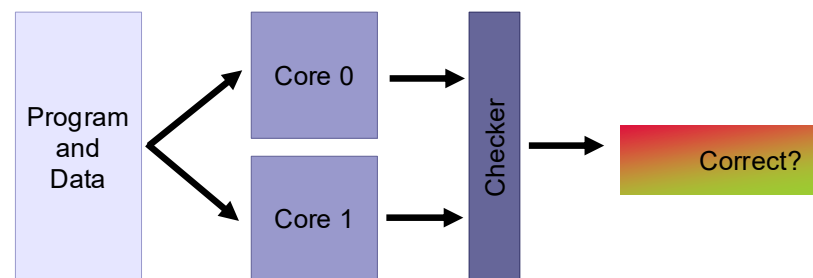
Credit: Intel / Micron

Reliability Challenge

- Reliability is also becoming a significant issue
- At manufacture, process variation is a concern
 - This is where transistors on the chip have different characteristics; some operate slower than others
- During the processor's lifetime, failures are likely
 - Transistors wear out quickly
 - Permanent faults develop that have to be addressed

Dual-Core Lockstep

- In safety-critical applications, two separate processors often run the same code
 - Some checking logic ensures results are the same
 - Mandated by standards for some sectors
 - E.g. ASIL-D for automotive



Poor Reliability – Embrace It!

- Many applications don't require 100% accuracy
- Graphics, video, audio, sensor analysis, games, etc



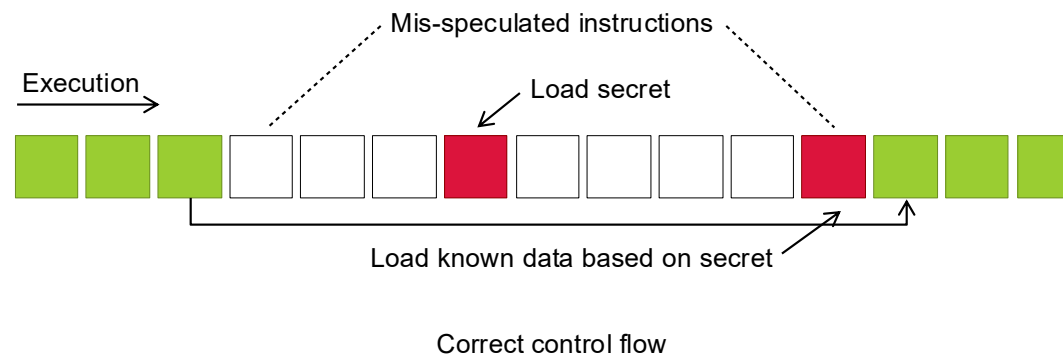
- Mishra et al. *iACT: A Software-Hardware Framework for Understanding the Scope of Approximate Computing*. WACAS 2014.

Security Challenge

- Increasingly our systems need to be secure
 - We may be running on untrusted services
 - E.g. somewhere in the cloud
 - We may be computing with sensitive information
- Programming languages can help
 - E.g. to prevent buffer overflow
 - But there's an awful lot of legacy code out there
- Recent attacks have highlighted vulnerabilities

Spectre

- Spectre relies on speculative execution
 - Trick victim to load secret data
 - Cache side channels to leak information via timing: see if the load hit in the cache or not



Spectre

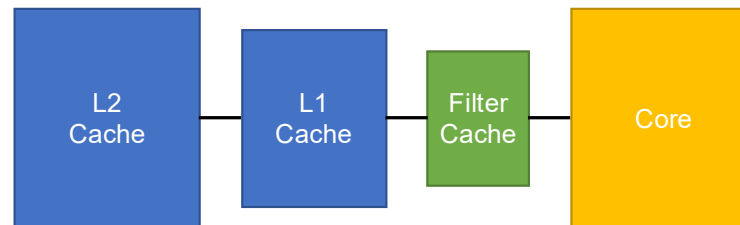
- Speculation comes from branch prediction
 - As seen in lecture 7
- Although the execution is rolled back, effects remain in the cache
 - E.g. some data may be cached that shouldn't be

```
uint8 A[10];
uint8 B[256*64];
void victim(size_t addr) {
    if (addr < 10) {          // mispredicted branch
        uint8 val = A[addr]; // secret is accessed
        ...      = B[64*val]; // secret is transmitted
    }
}
```

Yu et al. *Speculative Taint Tracking (STT): A Comprehensive Protection for Speculatively Accessed Data*. MICRO 2019

Dealing with Spectre

- Most research to deal with Spectre try to hide any speculative state
 - InvisiSpec, SpecShield, SafeSpec
- Computer Architecture Group research: MuonTrap



Security Challenge

- More generally we want to
 - Prevent leakage of (secret) information
 - In other words, ensure the confidentiality of information
 - Maintain the integrity of information
 - I.e. ensure that it has not had any unauthorised modifications
- To do this we must protect against existing attacks
 - By, for example, patching known vulnerabilities
- We can also protect against whole classes of attack
 - E.g. CHERI secure processor work from Lecture 10
- But also make systems resilient to future attacks
 - I.e. enabling them to be patched in-the-field

lowRISC

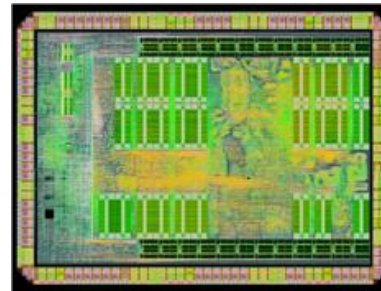
- “Create an open-source SoC capable of running Linux well”
- Make it real to encourage contributions and grow community
 - Volume silicon manufacture
 - Ability to purchase in small quantities
 - Low-cost development board
 - Regular updates to SoC
- Remove constraints on use of processor IP
- Free from commercial influences & release cycles



Slides courtesy of Robert Mullins

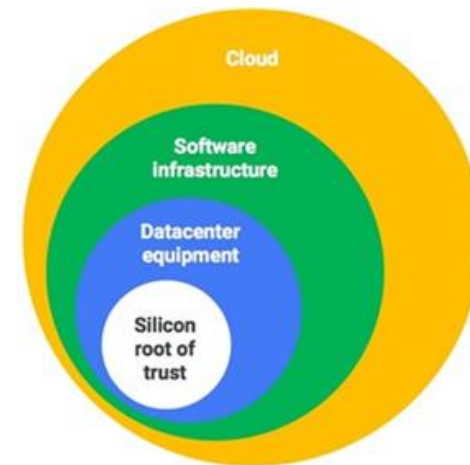
lowRISC

- RISC-V ISA from UC Berkeley
 - Aim to create open ISA standard for industry
 - Explicitly designed to be extensible
 - Simple base integer ISA (~40 instructions)
- Rocket SoC: cores, L1, L2 cache, interconnect
- Silicon in 45nm and 28nm



lowRISC

- Part of the Silicon Transparency Working Group
 - Cambridge working with Google and ETH Zurich
- Aim to build an open, transparent implementation of a secure root-of-trust
- Trust the silicon
 - Securely identify components
 - Confidence in integrity of whole system



Summary

- Computer architecture is changing rapidly
 - Multicores, many-cores and heterogeneity
 - Rise of acceleration and specialised compute
- Lots of good solutions; many more to find
 - How to extract parallelism and deal with unreliability
 - Security a major headache that needs addressing
 - Fundamental changes to the way computers operate
- “A New Golden Age for Computer Architecture”
 - Hennessy and Patterson, Turing Lecture 2018